

IT Business Analyst — Từ Zero Đến Thành Thạo

Dành cho người chưa biết gì về BA — kèm bộ tài liệu mẫu điền sẵn

Mục lục

Phần 0 — Nhập môn & định hướng nghề BA

- Chương 1. BA là ai, làm gì? Vai trò cầu nối giữa nghiệp vụ và kỹ thuật
- Chương 2. Tổng quan SDLC: các giai đoạn, vì sao dự án phần mềm cần quy trình
- Chương 3. Hệ sinh thái một dự án phần mềm: BA phối hợp với ai, khi nào
- Chương 4. Định hướng nghề nghiệp BA: lộ trình, chứng chỉ IIBA/CBAP, CCBA, PMI-PBA
- Chương 5. Bộ kỹ năng & công cụ của BA: kỹ năng cứng, kỹ năng mềm, công cụ phổ biến

Phần 1 — Quy trình phát triển & quản lý phần mềm

- Chương 6. Waterfall: đặc điểm, các giai đoạn tuần tự, ưu/nhược điểm
- Chương 7. Agile: Agile Manifesto, 12 nguyên tắc, tư duy Agile khác Waterfall ở đâu
- Chương 8. Scrum chi tiết: vai trò, sự kiện, artifact
- Chương 9. Extreme Programming (XP): thực hành kỹ thuật và vai trò BA
- Chương 10. Kanban & so sánh các mô hình: Waterfall vs Scrum vs Kanban vs XP
- Chương 11. Vai trò và trách nhiệm cụ thể của BA trong từng mô hình

Phần 2 — Thu thập & phân tích yêu cầu

- Chương 12. Kỹ thuật Elicitation: interview, workshop, survey, observation
- Chương 13. Stakeholder Analysis & ma trận RACI
- Chương 14. Phân loại yêu cầu: Business, Stakeholder, Solution, Transition
- Chương 15. Mô hình hoá yêu cầu: Use Case, Activity, Sequence Diagram, BPMN cơ bản
- Chương 16. Requirements Traceability Matrix (RTM)
- Chương 17. Requirements Validation & Verification
- Chương 18. Quản lý thay đổi yêu cầu: scope creep, Change Request

Phần 3 — Tài liệu BA cốt lõi (kèm mẫu điền sẵn)

- Chương 19. BRD & PRD: khác nhau ở đâu, cấu trúc, mẫu đầy đủ
- Chương 20. SRS theo chuẩn IEEE 830: cấu trúc, cách viết, mẫu đầy đủ
- Chương 21. FRD & yêu cầu phi chức năng (performance, security, usability)
- Chương 22. Use Case Specification: actor, flow chính/phụ, exception, mẫu
- Chương 23. User Story & Acceptance Criteria: INVEST, Gherkin, mẫu
- Chương 24. Epic, Feature, User Story: phân rã backlog, mẫu backlog
- Chương 25. Wireframe & Prototype cơ bản cho BA

Phần 4 — Quản lý dự án & giao hàng

- Chương 26. Ưu tiên hoá Backlog: MoSCoW, WSJF, mô hình Kano

Chương 27. Sprint Planning & vai trò của BA trong sprint
Chương 28. Incremental & Iterative Delivery: khái niệm, phân biệt
Chương 29. Release Plan & Delivery Document: mẫu Incremental Delivery Plan
Chương 30. Estimation cơ bản cho BA: story points, T-shirt sizing
Chương 31. RAID Log: quản lý Risk, Assumption, Issue, Dependency

Phần 5 — Giao tiếp, kiểm thử & nghiệm thu

Chương 32. Giao tiếp hiệu quả của BA: email, họp, trình bày với stakeholder
Chương 33. Facilitation: điều phối workshop, requirement review meeting
Chương 34. BA & QA: viết test case cơ bản, UAT — mẫu UAT checklist
Chương 35. Sign-off & nghiệm thu: quy trình ký duyệt, mẫu Sign-off Document
Chương 36. Đo lường thành công sau go-live: KPI, OKR, thu thập feedback

Phần 6 — Công cụ & kỹ năng nâng cao

Chương 37. Jira & Confluence cho BA: quản lý backlog, viết & liên kết tài liệu
Chương 38. Data & SQL cơ bản cho BA: đọc hiểu mô hình dữ liệu, SELECT query
Chương 39. Domain knowledge: BA trong Banking, E-commerce, Insurance
Chương 40. Case study tổng hợp "FoodNow": nhìn lại toàn bộ vòng đời dự án

Phụ lục

Phụ lục A — Mẫu BRD & PRD đầy đủ (dự án FoodNow)
Phụ lục B — Mẫu SRS đầy đủ theo IEEE 830 (dự án FoodNow)
Phụ lục C — Mẫu Epic + User Story backlog đầy đủ (dự án FoodNow)
Phụ lục D — Mẫu Incremental Delivery Plan đầy đủ (dự án FoodNow)
Phụ lục E — Bảng thuật ngữ BA (Glossary) tra cứu nhanh
Phụ lục F — Tài liệu tham khảo & nguồn học thêm

Phần 0 — Nhập môn & định hướng nghề BA

Chương 1: BA là ai, làm gì? Vai trò cầu nối giữa nghiệp vụ và kỹ thuật

Mục tiêu học

- Giải thích được BA (Business Analyst) là ai, làm công việc gì trong một dự án phần mềm — bằng ví dụ cụ thể, không phải định nghĩa hàn lâm.
- Phân biệt được BA với các vai trò dễ nhầm: PM (Project Manager), PO (Product Owner), BSA (Business System Analyst), Data Analyst.
- Hiểu vì sao BA tồn tại: vấn đề gì xảy ra nếu một dự án phần mềm **không có** BA.
- Làm quen với case study xuyên suốt cả cuốn sách: dự án "FoodNow".

1.1 Vấn đề mà BA giải quyết

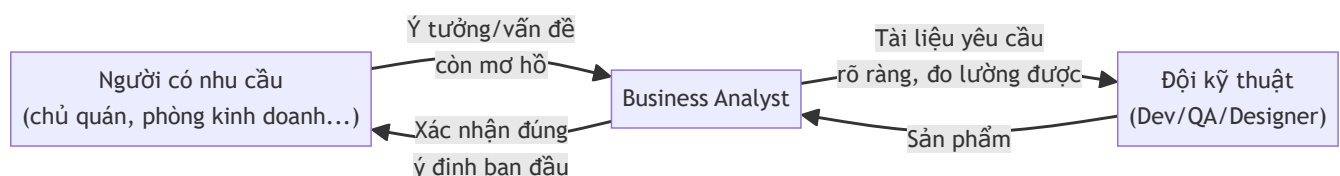
Hãy tưởng tượng một tình huống rất thật: Giám đốc kinh doanh của một chuỗi nhà hàng nói với đội kỹ thuật: *"Tôi muốn có một app để khách đặt đồ ăn online."* Chỉ với một câu đó, đội Dev bắt tay vào code luôn thì gần như chắc chắn sẽ làm sai — vì câu nói đó **thiếu quá nhiều thông tin**:

- Khách đặt đồ ăn để **giao tận nơi**, hay chỉ để **đặt trước rồi đến lấy** tại quán?
- Thanh toán online hay chỉ nhận tiền mặt khi giao?
- Một nhà hàng hay áp dụng cho toàn bộ chuỗi nhiều chi nhánh?
- "Thành công" của app này được đo bằng gì — tăng doanh thu? Giảm thời gian chờ? Giảm nhân viên trực điện thoại nhận order?

Nếu Dev tự đoán rồi code, khả năng cao sẽ ra một sản phẩm **đúng theo hiểu của Dev, sai theo ý định thật của người đưa ra yêu cầu** — và phát hiện ra điều đó sau khi đã tốn vài tháng code là quá muộn, quá tốn kém để sửa.

Business Analyst (BA) là người đứng giữa, có nhiệm vụ:

- Hỏi đúng câu hỏi để làm rõ **vấn đề nghiệp vụ thật sự** đằng sau một yêu cầu mơ hồ.
- Chuyển hoá câu trả lời thành **tài liệu yêu cầu** mà cả người làm nghiệp vụ (đọc hiểu được) lẫn đội kỹ thuật (đủ chi tiết để code đúng) đều dùng chung được.
- Theo sát suốt quá trình xây dựng để đảm bảo sản phẩm cuối cùng **giải quyết đúng vấn đề ban đầu** — không chỉ "chạy được" mà còn "đúng cái người ta cần".



Nói ngắn gọn: **BA không viết code, không thiết kế giao diện, không quản lý deadline** — việc của BA là đảm bảo **cả đội hiểu đúng và hiểu giống nhau** về việc cần làm, trước và trong suốt quá trình làm.

1.2 Một ngày làm việc điển hình của BA

Tuỳ công ty và giai đoạn dự án, nhưng thường xoay quanh:

- Họp với người dùng/khách hàng (stakeholder) để hỏi và làm rõ yêu cầu.
- Viết hoặc cập nhật tài liệu: user story, use case, sơ đồ quy trình nghiệp vụ.
- Làm việc với Dev để giải thích yêu cầu, trả lời câu hỏi phát sinh khi code.
- Làm việc với QA để đảm bảo test case bao phủ đúng ý định nghiệp vụ, không chỉ đúng theo mô tả kỹ thuật.
- Tham gia demo/nghiệm thu, xác nhận tính năng làm ra đúng như mong đợi.
- Theo dõi số liệu sau khi tính năng lên production, xem có đạt mục tiêu nghiệp vụ đề ra không.

Bạn sẽ thấy BA xuất hiện ở **mọi giai đoạn** trong vòng đời dự án (Chương 2 sẽ nói kỹ về SDLC) — không chỉ ở đầu dự án lúc "thu thập yêu cầu" như nhiều người lầm tưởng.

1.3 Phân biệt BA với các vai trò dễ nhầm

Vai trò	Trọng tâm công việc	Khác BA ở điểm nào
Project Manager (PM)	Quản lý tiến độ, ngân sách, rủi ro, nhân sự dự án	PM quan tâm "làm đúng hạn, đúng ngân sách"; BA quan tâm "làm đúng cái cần làm"
Product Owner (PO)	Đại diện chủ sản phẩm, quyết định ưu tiên trong backlog (thường trong Scrum)	PO có quyền quyết định ưu tiên/đánh đổi; BA phân tích và đề xuất , thường không phải người quyết cuối cùng. Ở nhiều công ty nhỏ, một người kiêm cả hai vai trò
Business System Analyst (BSA)	Thiên về phân tích hệ thống kỹ thuật đang có, thiết kế giải pháp tích hợp	BSA cần hiểu sâu kiến trúc hệ thống hơn; BA thuần thiên về nghiệp vụ hơn. Ranh giới này khác nhau tuỳ công ty
Data Analyst	Phân tích số liệu, xây dashboard, tìm insight từ dữ liệu có sẵn	Data Analyst làm việc với dữ liệu đã có; BA làm việc với yêu cầu về hệ thống/sản phẩm chưa tồn tại

Trong thực tế, ranh giới giữa các vai trò này khá mờ và khác nhau giữa các công ty — công ty nhỏ thường gộp BA + PO, thậm chí BA + PM làm một người. Sách này tập trung vào **bản chất công việc** của BA, để bạn áp dụng được dù chức danh thực tế ở công ty bạn có gọi khác đi.

1.4 Case study xuyên suốt: dự án "FoodNow"

Để mọi khái niệm trong sách không chỉ là lý thuyết suông, từ chương này trở đi chúng ta sẽ theo sát **một dự án giả định duy nhất**, dùng lại xuyên suốt 40 chương và toàn bộ mẫu tài liệu ở phần Phụ lục:

Bối cảnh dự án FoodNow: Chuỗi nhà hàng "FoodNow" có 12 chi nhánh tại TP.HCM, hiện nhận đặt hàng qua điện thoại và các app giao đồ ăn bên thứ ba (mất 20-25% phí hoa hồng mỗi đơn). Ban giám đốc quyết định xây dựng **app đặt đồ ăn riêng của FoodNow** để: (1) giảm phụ thuộc vào app bên thứ ba, (2) xây dựng dữ liệu khách hàng riêng để làm chương trình khách hàng thân thiết, (3) tăng biên lợi nhuận trên mỗi đơn hàng.

Bạn sẽ thấy dự án FoodNow xuất hiện lại trong ví dụ ở hầu hết các chương sau — từ lúc còn là một câu nói mơ hồ của ban giám đốc (như ví dụ ở mục 1.1) cho đến khi có đầy đủ BRD, SRS, backlog, delivery plan, và cả tài liệu sau go-live (Chương 40 và Phụ lục A-D).

Bài tập

1. Nghĩ về một sản phẩm/app bạn dùng hằng ngày (ví dụ app ngân hàng, app giao đồ ăn). Thử liệt kê 3 câu hỏi mà bạn nghĩ một BA đã phải hỏi trước khi đội kỹ thuật bắt tay code tính năng đó.
2. Với bối cảnh dự án FoodNow ở mục 1.4, viết ra 5 câu hỏi bạn sẽ hỏi ban giám đốc FoodNow trước khi bắt đầu phân tích sâu hơn (gợi ý: ai dùng app, dùng để làm gì, đo thành công bằng gì...).

Sai lầm thường gặp

- **Nhầm BA là người "ghi chép lại yêu cầu":** BA không chỉ chép lại nguyên văn điều stakeholder nói — việc quan trọng hơn là **phát hiện ra điều họ chưa nói nhưng cần thiết**, và phát hiện mâu thuẫn giữa lời nói của các stakeholder khác nhau.
- **Nghĩ BA chỉ làm việc ở đầu dự án:** nhiều BA mới vào nghề nghĩ xong "giai đoạn thu thập yêu cầu" là hết việc — thực tế BA cần theo sát đến tận sau khi sản phẩm lên production (xem Chương 2).
- **Tự ý quyết định thay vì xác nhận lại với stakeholder:** khi gặp yêu cầu mơ hồ, BA mới thường tự suy đoán ý người dùng để "cho nhanh" thay vì hỏi lại — dễ dẫn tới sai lệch được phát hiện quá trễ.

Tóm tắt & tiếp theo

BA là người đảm bảo cả đội — từ người có ý tưởng đến người code — hiểu đúng và hiểu giống nhau về vấn đề cần giải quyết, xuyên suốt vòng đời dự án chứ không chỉ ở bước đầu. Chương 2 sẽ đặt công việc của BA vào bức tranh lớn hơn: vòng đời phát triển phần mềm (SDLC) — để thấy rõ BA xuất hiện ở giai đoạn nào, làm gì ở mỗi giai đoạn.

Chương 2: Tổng quan SDLC: các giai đoạn, vì sao dự án phần mềm cần quy trình

Mục tiêu học

- Gọi tên và giải thích được các giai đoạn chuẩn của SDLC (Software Development Life Cycle).
- Hiểu vì sao dự án phần mềm cần một quy trình có cấu trúc, thay vì "nghĩ gì làm nấy".
- Xác định được BA tham gia vào giai đoạn nào của SDLC, và đóng góp gì ở từng giai đoạn.
- Phân biệt được "mô hình SDLC" (Waterfall, Agile...) với "các giai đoạn SDLC" — hai khái niệm hay bị nhầm là một.

2.1 SDLC là gì?

SDLC (Software Development Life Cycle — Vòng đời phát triển phần mềm) là tập hợp các giai đoạn mà một sản phẩm phần mềm phải trải qua, từ lúc còn là một ý tưởng/vấn đề cần giải quyết, đến lúc phần mềm được xây dựng, kiểm thử, triển khai, vận hành, và cuối cùng có thể bị thay thế/ngừng sử dụng.

Có 6 giai đoạn kinh điển, gần như mọi mô hình phát triển phần mềm (Waterfall, Agile, Scrum...) đều là một cách **sắp xếp lại thứ tự và tần suất lặp lại** của 6 giai đoạn này — không mô hình nào bỏ hẳn một giai đoạn:



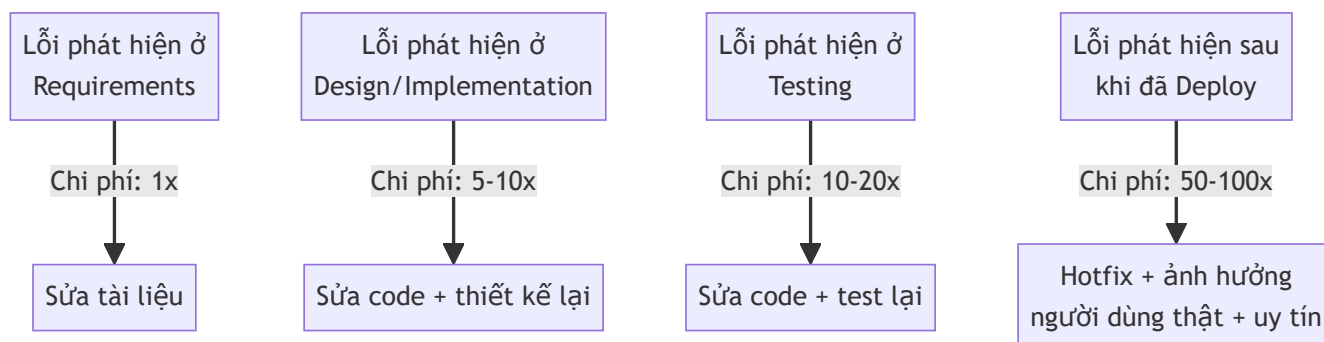
Giai đoạn	Câu hỏi chính	Ai chủ trì	BA làm gì
1. Planning	Vấn đề nghiệp vụ là gì? Có đáng đầu tư không?	Ban lãnh đạo, PM	Hỗ trợ phân tích sơ bộ, ước tính giá trị nghiệp vụ, viết Business Case
2. Requirements	Cần xây cái gì, cho ai, để làm gì?	BA	Chủ trì elicitation, viết BRD/PRD/SRS (Chương 12-24)
3. Design	Xây như thế nào (kiến trúc, giao diện, dữ liệu)?	Kiến trúc sư, Designer, Dev lead	Giải thích yêu cầu, phản hồi thiết kế có đúng ý nghiệp vụ không
4. Implementation	Code	Dev	Trả lời câu hỏi phát sinh, làm rõ yêu cầu chưa rõ khi code
5. Testing	Có đúng yêu cầu không? Có lỗi không?	QA	Viết/duyet test case dựa trên acceptance criteria, tham gia UAT (Chương 34)
6. Deployment & Maintenance	Vận hành có ổn không? Có đạt mục tiêu nghiệp vụ không?	DevOps/Ops, PM	Theo dõi KPI sau go-live, thu thập feedback, đề xuất cải tiến (Chương 36)

Điểm mấu chốt: **BA xuất hiện xuyên suốt cả 6 giai đoạn**, không chỉ ở giai đoạn 2. Đây là lý do Chương 1 nói BA không phải "người ghi chép yêu cầu lúc đầu dự án rồi thôi".

2.2 Vì sao cần quy trình, không "nghĩ gì làm nấy"?

Không có SDLC, một dự án phần mềm thường gặp các vấn đề sau — tất cả đều có thể minh họa bằng dự án FoodNow (Chương 1):

- **Xây sai thứ cần xây:** Dev code app đặt hàng nhưng không có tính năng thanh toán online vì không ai hỏi rõ ràng từ đầu — đến lúc demo mới phát hiện, phải làm lại.
- **Chi phí sửa lỗi tăng theo cấp số nhân:** một yêu cầu hiểu sai được phát hiện ở giai đoạn Requirements chỉ tốn vài giờ viết lại tài liệu; phát hiện ở giai đoạn Testing tốn vài ngày sửa code; phát hiện **sau khi đã lên production** có thể tốn hàng tuần và ảnh hưởng uy tín thương hiệu.
- **Không ai chịu trách nhiệm rõ ràng:** khi có vấn đề, không biết lỗi ở khâu nào — thu thập yêu cầu sai, thiết kế sai, hay code sai.
- **Không đo lường được tiến độ thực sự:** "sắp xong rồi" là một câu vô nghĩa nếu không có các cột mốc rõ ràng gắn với từng giai đoạn.



Con số 1x/5x/10x/50x ở trên là ước lượng tương đối mang tính minh họa (dựa trên nguyên lý "chi phí sửa lỗi tăng theo giai đoạn phát hiện" được nhắc nhiều trong tài liệu kỹ thuật phần mềm), không phải số liệu đo được cố định cho mọi dự án — điều quan trọng cần nhớ là **xu hướng tăng**, không phải con số chính xác.

Đây chính là lý do công việc của BA — làm rõ yêu cầu **càng sớm càng tốt, càng kỹ càng tốt** — có giá trị kinh tế trực tiếp, không chỉ là "thủ tục giấy tờ".

2.3 SDLC không phải là "làm tuần tự từ trên xuống"

Một hiểu lầm phổ biến: nhiều người mới nghĩ SDLC nghĩa là làm xong hẳn giai đoạn 1 mới sang giai đoạn 2, làm xong hẳn giai đoạn 2 mới sang giai đoạn 3... Đó là cách làm của **một mô hình cụ thể** gọi là Waterfall (Chương 6) — không phải bản chất của SDLC.

Trong các mô hình Agile (Chương 7-10), 6 giai đoạn này vẫn tồn tại đầy đủ, nhưng được **lặp lại nhiều lần** trong các chu kỳ ngắn (sprint) — mỗi sprint đều có một chút Requirements, Design, Implementation, Testing cho một phần nhỏ sản phẩm. Phần 1 của sách (Chương 6-11) sẽ đi sâu vào từng mô hình cụ thể.

Bài tập

1. Với dự án FoodNow, liệt kê 1 rủi ro cụ thể có thể xảy ra nếu bỏ qua hẳn giai đoạn "Testing" trước khi phát hành app cho khách hàng dùng thật.
2. Giả sử đội Dev của FoodNow phát hiện ra tính năng "thanh toán online" chưa được làm rõ **ngay khi đang code** (giai đoạn Implementation) thay vì lúc thu thập yêu cầu. Theo sơ đồ chi phí ở mục 2.2, điều này có ý nghĩa gì về mặt chi phí so với phát hiện sớm hơn?

Sai lầm thường gặp

- **Nhầm "SDLC" với "Waterfall"**: SDLC là khái niệm bao trùm (các giai đoạn phải trải qua), Waterfall chỉ là một trong nhiều cách sắp xếp các giai đoạn đó.
- **Nghĩ BA chỉ cần giỏi ở giai đoạn Requirements**: như bảng ở mục 2.1, BA có vai trò cụ thể ở mọi giai đoạn — bỏ qua Design/Testing/Maintenance là bỏ lỡ phần lớn giá trị BA có thể mang lại.
- **Coi nhẹ giai đoạn Maintenance**: nhiều BA mới chỉ tập trung "ra mắt được sản phẩm" mà quên rằng phần lớn vòng đời thực tế của một phần mềm nằm ở giai đoạn vận hành sau go-live.

Tóm tắt & tiếp theo

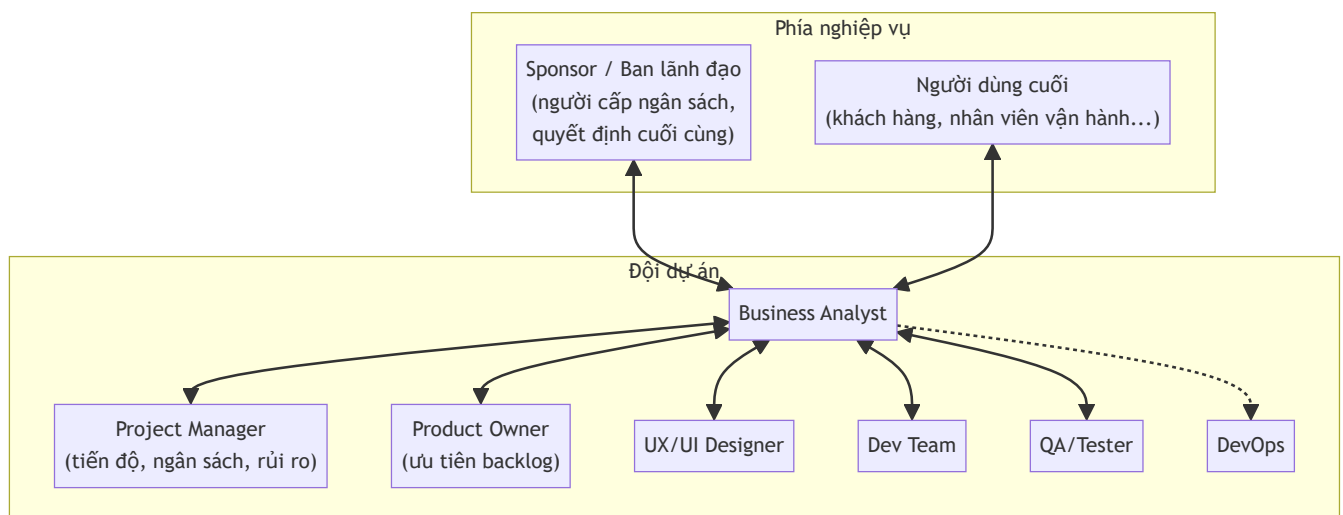
SDLC gồm 6 giai đoạn cố định (Planning, Requirements, Design, Implementation, Testing, Deployment & Maintenance) mà mọi dự án phần mềm đều trải qua dưới hình thức này hay hình thức khác — BA có vai trò xuyên suốt cả 6 giai đoạn. Chương 3 sẽ mở rộng góc nhìn ra toàn bộ **hệ sinh thái con người** trong một dự án phần mềm: BA cần phối hợp với những vai trò nào, ở đâu, khi nào.

Chương 3: Hệ sinh thái một dự án phần mềm: BA phối hợp với ai, khi nào

Mục tiêu học

- Gọi tên được các vai trò chính trong một dự án phần mềm và trách nhiệm cốt lõi của từng vai trò.
- Biết BA cần trao đổi với vai trò nào ở giai đoạn nào, và trao đổi về nội dung gì.
- Phân biệt hai nhóm stakeholder: **nội bộ đội dự án** và **bên ngoài đội dự án** (khách hàng, người dùng cuối, đối tác, cơ quan quản lý...).
- Hiểu vì sao BA cần "nói được ngôn ngữ" của cả hai phía nghiệp vụ và kỹ thuật.

3.1 Sơ đồ tổng quan các vai trò



BA gần như là vai trò **giao tiếp với nhiều đầu mối nhất** trong một dự án — đây vừa là điểm thú vị, vừa là thử thách lớn nhất của nghề: phải "nói được nhiều ngôn ngữ" khác nhau tùy người đang trao đổi cùng.

3.2 Từng vai trò — trách nhiệm và điểm chạm với BA

Sponsor / Ban lãnh đạo

Người cấp ngân sách, có quyền quyết định dự án có tiếp tục hay không. Trong ví dụ FoodNow, đây là Ban giám đốc chuỗi nhà hàng. BA làm việc với Sponsor chủ yếu ở giai đoạn đầu (làm rõ mục tiêu nghiệp vụ, Business Case — Chương 19) và cuối (báo cáo kết quả sau go-live — Chương 36).

Nguyên tắc khi làm việc với Sponsor: nói bằng ngôn ngữ **kết quả kinh doanh** (doanh thu, chi phí, thời gian), không phải ngôn ngữ kỹ thuật. Sponsor không cần biết "API là gì", họ cần biết "tính năng này giúp giảm bao nhiêu % chi phí hoa hồng cho app bên thứ ba".

Người dùng cuối (End user)

Người thực sự dùng sản phẩm hằng ngày — trong FoodNow là khách đặt đồ ăn, và nhân viên nhà hàng xử lý đơn hàng. Đây là nguồn thông tin **quan trọng nhất nhưng dễ bị bỏ qua nhất** — nhiều dự án chỉ hỏi ý kiến Sponsor mà quên hỏi người dùng cuối, dẫn đến sản phẩm "đúng ý sếp, sai ý người dùng".

Project Manager (PM)

PM quan tâm: dự án có đúng tiến độ, đúng ngân sách, rủi ro được kiểm soát không. BA cung cấp cho PM: ước lượng độ phức tạp của yêu cầu (để PM lập kế hoạch), cảnh báo sớm khi phát hiện yêu cầu mới phát sinh có thể ảnh hưởng tiến độ (liên quan Change Request — Chương 18).

Product Owner (PO)

Ở các đội theo Scrum, PO là người quyết định thứ tự ưu tiên trong backlog. BA hỗ trợ PO bằng cách phân tích và làm rõ yêu cầu **trước khi** đưa vào backlog để PO ưu tiên — gọi là "backlog refinement" (Chương 27). Ở nhiều công ty vừa và nhỏ, một người kiêm cả BA và PO.

UX/UI Designer

Designer biến yêu cầu thành giao diện. BA cần giải thích rõ **luồng nghiệp vụ** (business flow) đằng sau mỗi màn hình để Designer thiết kế đúng — ví dụ: màn hình "chọn món" của FoodNow cần biết rõ có bao nhiêu bước trước khi thanh toán, có cho phép sửa đơn sau khi đặt không.

Dev Team

Đội kỹ thuật hiện thực hoá yêu cầu thành sản phẩm. BA là người mà Dev hỏi lại khi gặp trường hợp tài liệu chưa mô tả rõ (ví dụ: "Nếu khách huỷ đơn sau khi nhà hàng đã bắt đầu nấu thì xử lý sao?"). BA cần **có mặt và phản hồi nhanh** trong suốt giai đoạn Implementation, không chỉ bàn giao tài liệu rồi biến mất.

QA/Tester

QA viết test case để kiểm tra sản phẩm có đúng yêu cầu không. BA hỗ trợ QA bằng **Acceptance Criteria** rõ ràng (Chương 23) và tham gia UAT (Chương 34) để xác nhận kết quả test đúng phản ánh ý định nghiệp vụ ban đầu — không chỉ đúng theo mô tả kỹ thuật khô cứng.

DevOps

Ít trao đổi trực tiếp với BA hơn các vai trò khác, nhưng BA cần biết kế hoạch triển khai (deploy theo giai đoạn, có rollback không) để tư vấn đúng cho Sponsor về rủi ro và mốc thời gian ra mắt (liên quan Chương 28-29 về Incremental Delivery).

3.3 Vì sao BA cần "song ngữ" nghiệp vụ — kỹ thuật

Cùng một thông tin, BA cần diễn đạt khác nhau tùy người nghe:

Thông tin gốc	Nói với Sponsor	Nói với Dev
Khách hàng cần theo dõi trạng thái đơn hàng real-time	"Giảm số cuộc gọi hỏi 'đơn tôi tới đâu rồi', tăng trải nghiệm khách hàng"	"Cần cơ chế push notification hoặc polling để cập nhật trạng thái đơn hàng, cập nhật tối thiểu 4 trạng thái: đã nhận, đang chuẩn bị, đang giao, đã giao"

Khả năng "dịch" hai chiều này chính là giá trị cốt lõi của BA — không phải kỹ năng viết tài liệu đẹp, mà là khả năng đảm bảo **thông tin không bị méo** khi đi từ đầu này sang đầu kia.

Bài tập

- Với dự án FoodNow, liệt kê 3 stakeholder cụ thể (có thể đặt tên chức danh giả định) mà bạn sẽ cần phỏng vấn trước khi viết tài liệu yêu cầu đầu tiên.
- Chọn một yêu cầu bất kỳ của FoodNow (ví dụ: "khách có thể đặt lại đơn hàng cũ chỉ với 1 chạm"), viết hai phiên bản diễn đạt: một cho Sponsor, một cho Dev, theo mẫu ở bảng mục 3.3.

Sai lầm thường gặp

- Chỉ làm việc với 1-2 stakeholder quen thuộc:** dễ bỏ sót góc nhìn của các vai trò khác — ví dụ chỉ hỏi Sponsor mà quên hỏi nhân viên vận hành thực tế sẽ dùng hệ thống hằng ngày.
- Dùng thuật ngữ kỹ thuật khi nói với người nghiệp vụ (hoặc ngược lại):** gây hiểu lầm hoặc làm người nghe cảm thấy bị bỏ rơi khỏi cuộc trò chuyện.
- Coi PM và PO như nhau:** ở công ty có tách bạch hai vai trò, nhằm lẫn trách nhiệm dễ dẫn đến BA báo cáo sai người, sai thời điểm.

Tóm tắt & tiếp theo

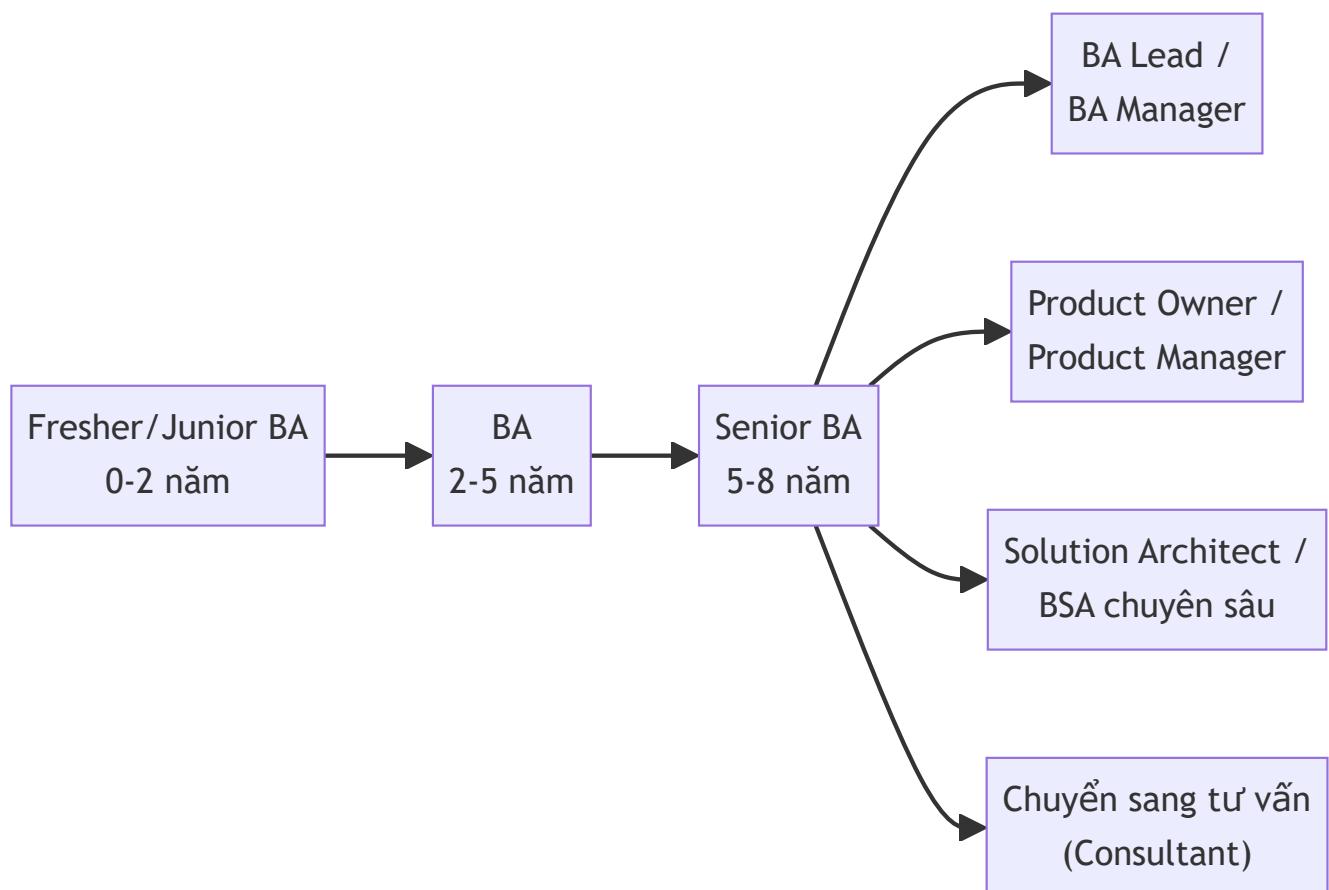
BA là vai trò kết nối nhiều đầu mối nhất trong dự án — từ Sponsor, người dùng cuối, đến PM, PO, Designer, Dev, QA, DevOps — và cần điều chỉnh cách diễn đạt phù hợp với từng đối tượng. Chương 4 sẽ nhìn xa hơn: nếu bạn muốn theo nghề BA lâu dài, lộ trình phát triển sự nghiệp trông như thế nào, cần học gì, thi chứng chỉ nào.

Chương 4: Định hướng nghề nghiệp BA: lộ trình, chứng chỉ IIBA/CBAP, CCBA, PMI-PBA

Mục tiêu học

- Hình dung được lộ trình phát triển sự nghiệp BA từ mới vào nghề đến cấp cao.
- Biết các chứng chỉ phổ biến trong nghề BA, chứng chỉ nào phù hợp ở giai đoạn nào.
- Biết BA có thể rẽ nhánh sang những vai trò nào khi đã có kinh nghiệm.
- Tự đánh giá được mình đang ở đâu trên lộ trình, và bước tiếp theo cần chuẩn bị gì.

4.1 Lộ trình sự nghiệp điển hình



Không có một con đường "đúng duy nhất" — BA là nghề có nhiều điểm rẽ nhánh vì kỹ năng cốt lõi (phân tích, giao tiếp, hiểu nghiệp vụ) chuyển hoá tốt sang nhiều vai trò khác.

Fresher/Junior BA (0-2 năm)

Công việc chủ yếu: hỗ trợ BA senior thu thập yêu cầu, viết user story dưới sự hướng dẫn, tham gia họp ghi chép (meeting minutes), làm quen công cụ (Jira, Confluence). Trọng tâm học: nắm vững các kỹ thuật elicitation (Chương 12), cách viết tài liệu chuẩn (Chương 19-24), và **học lắng nghe** — kỹ năng bị đánh giá thấp nhưng quan trọng nhất ở giai đoạn này.

BA (2-5 năm)

Có thể tự chủ trì thu thập yêu cầu cho một module/tính năng, tự viết BRD/SRS mà không cần review sát sao. Bắt đầu tham gia sâu vào ước lượng (Chương 30), quản lý thay đổi yêu cầu (Chương 18). Đây là giai đoạn nên cân nhắc học sâu một domain cụ thể (Banking, E-commerce... — Chương 39) để tạo lợi thế cạnh tranh.

Senior BA (5-8 năm)

Chủ trì toàn bộ dự án về mặt yêu cầu, tư vấn ngược lại cho Sponsor về tính khả thi/rủi ro nghiệp vụ, mentor BA junior. Bắt đầu cần kỹ năng lãnh đạo không chính thức (influence without authority) vì thường không có quyền quản lý trực tiếp nhưng vẫn cần điều phối nhiều bên.

Các hướng rẽ sau Senior BA

- **BA Lead/Manager:** quản lý một đội BA, phân bổ người vào các dự án, chuẩn hoá quy trình BA trong tổ chức.
- **Product Owner/Product Manager:** dịch chuyển từ "phân tích yêu cầu" sang "quyết định sản phẩm" — cần thêm tư duy về chiến lược sản phẩm, thị trường, không chỉ phân tích yêu cầu đơn thuần.
- **Solution Architect/BSA chuyên sâu:** đi sâu hơn vào khía cạnh kỹ thuật, thiết kế giải pháp tích hợp hệ thống — phù hợp với BA có nền tảng kỹ thuật tốt.
- **Tư vấn (Consultant):** làm việc độc lập hoặc tại công ty tư vấn, mang kinh nghiệm domain đi làm nhiều dự án khác nhau thay vì gắn với một sản phẩm/công ty.

4.2 Các chứng chỉ phổ biến

Chứng chỉ	Tổ chức cấp	Phù hợp giai đoạn	Đặc điểm
ECBA (Entry Certificate in Business Analysis)	IIBA	Mới vào nghề, chưa có/ít kinh nghiệm	Không yêu cầu kinh nghiệm làm việc, kiểm tra kiến thức nền tảng theo BABOK
CCBA (Certification of Competency in Business Analysis)	IIBA	2-3 năm kinh nghiệm	Yêu cầu một số giờ kinh nghiệm thực tế theo BABOK, mức trung cấp
CBAP (Certified Business Analysis Professional)	IIBA	5+ năm kinh nghiệm	Chứng chỉ được công nhận rộng rãi nhất cho BA senior, yêu cầu nhiều giờ kinh nghiệm và giờ học liên tục (CDU)
PMI-PBA (Professional in Business Analysis)	PMI	Có kinh nghiệm, thường kết hợp làm việc gần PM	Nhấn mạnh liên kết giữa business analysis và project management

BABOK (Business Analysis Body of Knowledge) là tài liệu nền tảng của IIBA, tổng hợp các kỹ thuật và khái niệm chuẩn cho nghề BA — nhiều khái niệm trong sách này (elicitation, requirements classification, traceability...) bắt nguồn từ BABOK, dù được trình bày lại theo cách thực chiến hơn, ít hàn lâm hơn.

Chứng chỉ có cần thiết không? Không bắt buộc để làm nghề BA — nhiều BA giỏi không có chứng chỉ nào. Giá trị của chứng chỉ nằm ở: (1) hệ thống hoá kiến thức nếu bạn tự học phân tán, (2) có lợi khi ứng tuyển ở công ty nước ngoài/tập đoàn lớn coi trọng bằng cấp, (3) là mốc để tự đánh giá quá trình học. Kinh nghiệm thực chiến và khả năng tạo ra tài liệu/kết quả thật vẫn là yếu tố quyết định nhất.

4.3 Tự đánh giá vị trí hiện tại

Dùng bảng sau để tự chấm điểm (1 = chưa làm được, 5 = thành thạo) sau khi hoàn thành cuốn sách này:

Kỹ năng	Chương liên quan	Tự đánh giá (1-5)
Thu thập yêu cầu (elicitation)	12	
Viết BRD/PRD/SRS	19-21	
Viết User Story/Use Case	22-24	
Quản lý backlog, ước lượng	26, 30	
Giao tiếp với stakeholder cấp cao	32	
Điều phối workshop	33	
Hiểu ít nhất 1 domain nghiệp vụ	39	

Quay lại bảng này sau khi đọc xong sách (Chương 40) để thấy sự tiến bộ, và biết chương nào cần đọc lại/thực hành thêm.

Bài tập

1. Tự chấm điểm bảng ở mục 4.3 **ngay bây giờ, trước khi đọc các chương sau** — dùng làm mốc so sánh sau này.
2. Tìm hiểu xem công ty (hoặc công ty bạn muốn ứng tuyển) có yêu cầu/khuyến khích chứng chỉ BA nào không. Nếu có, ghi chú lại yêu cầu kinh nghiệm tối thiểu để thi chứng chỉ đó.

Sai lầm thường gặp

- **Chờ có chứng chỉ mới dám nhận việc BA:** kinh nghiệm thực tế (kể cả từ dự án cá nhân, dự án tình nguyện) giá trị hơn nhiều so với chỉ có chứng chỉ mà chưa thực hành.
- **Không đầu tư vào domain knowledge:** BA giỏi kỹ thuật phân tích nhưng không hiểu ngành (banking, bán lẻ...) thường bị giới hạn ở các dự án đơn giản — xem thêm Chương 39.
- **Nghĩ BA là nghề "an toàn, không cần học thêm":** công cụ, phương pháp luận (Agile, AI hỗ trợ phân tích...) thay đổi liên tục — BA cần cập nhật liên tục như mọi nghề khác trong ngành IT.

Tóm tắt & tiếp theo

Ngành BA có lộ trình rõ ràng từ Junior đến Senior, với nhiều hướng rẽ (Lead, PO, Architect, Tư vấn) và một số chứng chỉ hỗ trợ (ECBA/CCBA/CBAP từ IIBA, PMI-PBA từ PMI) — không bắt buộc nhưng hữu ích. Chương 5 sẽ đi vào chi tiết bộ kỹ năng và công cụ cụ thể mà một BA cần trang bị để làm việc hiệu quả từ ngày đầu tiên.

Chương 5: Bộ kỹ năng & công cụ của BA: kỹ năng cứng, kỹ năng mềm, công cụ phổ biến

Mục tiêu học

- Liệt kê được các nhóm kỹ năng cứng và kỹ năng mềm cốt lõi của BA.
- Biết công cụ nào dùng để làm gì trong công việc BA hằng ngày, ở mức đủ để bắt đầu dùng được.
- Hiểu vì sao kỹ năng mềm thường quan trọng hơn kỹ năng cứng đối với BA — khác với nhiều vai trò kỹ thuật khác.

5.1 Kỹ năng cứng (Hard skills)

Nhóm kỹ năng	Cụ thể	Học ở chương nào
Elicitation	Kỹ thuật phỏng vấn, workshop, quan sát	12
Modeling	Vẽ Use Case Diagram, Activity Diagram, BPMN	15
Viết tài liệu	BRD, PRD, SRS, User Story, Use Case Spec	19-24
Quản lý yêu cầu	Traceability Matrix, Change Request	16, 18
Ước lượng & ưu tiên hoá	Story point, MoSCoW, WSJF	26, 30
Đọc hiểu dữ liệu	SQL cơ bản, đọc mô hình dữ liệu (ERD)	38

Lưu ý: BA **không cần biết lập trình**. Kỹ năng "đọc hiểu dữ liệu" ở Chương 38 dừng ở mức viết được câu SELECT đơn giản để tự tra cứu, không phải kỹ năng lập trình ứng dụng.

5.2 Kỹ năng mềm (Soft skills)

Đây là nhóm kỹ năng **quyết định BA giỏi hay không**, nhiều hơn kỹ năng cứng — vì bản chất công việc BA là làm việc với con người, không phải làm việc với máy:

- **Lắng nghe chủ động (active listening)**: nghe để hiểu ý định thật, không chỉ nghe để ghi chép từ ngữ. Ví dụ: khi Sponsor nói "tôi muốn app nhanh hơn", BA giỏi sẽ hỏi tiếp "nhanh hơn ở bước nào cụ thể, và nhanh hơn để giải quyết vấn đề gì" thay vì ghi thẳng "yêu cầu: tăng tốc độ app".
- **Đặt câu hỏi có mục đích**: biết hỏi câu nào để mở rộng thông tin (open question), câu nào để xác nhận/thu hẹp (closed question) — xem chi tiết kỹ thuật ở Chương 12.
- **Giao tiếp thích ứng theo đối tượng**: đã nói ở Chương 3 — nói khác nhau với Sponsor và với Dev.

- **Giải quyết xung đột:** các stakeholder thường có yêu cầu mâu thuẫn nhau (ví dụ: phòng Marketing muốn thêm nhiều bước thu thập thông tin khách hàng, phòng UX muốn quy trình đặt hàng càng ngắn càng tốt) — BA cần điều hoà, không né tránh.
- **Tư duy phản biện (critical thinking):** không chấp nhận yêu cầu ở mức bề mặt, luôn hỏi "tại sao cần cái này" để tìm ra vấn đề gốc rễ (root cause), tránh việc chỉ giải quyết triệu chứng.
- **Kiên nhẫn & chịu được sự mơ hồ:** giai đoạn đầu dự án luôn mơ hồ, thông tin không đầy đủ — BA cần thoải mái làm việc trong trạng thái đó thay vì đòi hỏi mọi thứ phải rõ ràng ngay từ đầu.

5.3 Công cụ phổ biến

Công cụ	Dùng để làm gì	Mức độ cần biết
Jira	Quản lý backlog, epic/story, theo dõi tiến độ sprint	Thành thạo (Chương 37)
Confluence	Viết và lưu trữ tài liệu (BRD, SRS, meeting notes), liên kết với Jira	Thành thạo (Chương 37)
draw.io / Lucidchart / Visio	Vẽ sơ đồ quy trình, use case diagram, wireframe đơn giản	Cơ bản, đủ vẽ sơ đồ trong tài liệu
Figma	Xem/góp ý thiết kế UI do Designer làm, đôi khi tự vẽ wireframe nhanh	Cơ bản — BA thường chỉ cần đọc/comment, không cần tự thiết kế
Excel/Google Sheets	Traceability Matrix, RAID log, so sánh phương án	Thành thạo — công cụ "vạn năng" bị đánh giá thấp nhưng dùng liên tục
Draw diagrams as code (Mermaid, PlantUML)	Vẽ sơ đồ dạng text, dễ version control, dễ nhúng vào tài liệu Markdown/Confluence	Cơ bản, ngày càng phổ biến ở các đội kỹ thuật hiện đại

Sách này không đi sâu hướng dẫn cài đặt/thao tác từng công cụ như một khoá học phần mềm riêng — Chương 37 giới thiệu cách **dùng Jira/Confluence trong quy trình làm việc của BA** (workflow), không phải hướng dẫn admin/cấu hình hệ thống (đó là việc của PM/IT Admin).

5.4 Ma trận ưu tiên học: nên học gì trước?

Nếu bạn hoàn toàn mới, thứ tự ưu tiên hợp lý:

1. Kỹ năng mềm (lắng nghe, đặt câu hỏi) — nền tảng cho mọi thứ khác, học qua thực hành là chính.
2. Cách viết User Story và Acceptance Criteria (Chương 23) — kỹ năng dùng hằng ngày ở hầu hết công ty làm Agile hiện nay.
3. Cách vẽ Use Case/Activity Diagram cơ bản (Chương 15) — công cụ tư duy giúp phát hiện lỗi hổng trong yêu cầu trước khi trình bày với người khác.

4. Jira/Confluence ở mức dùng được (Chương 37) — hầu hết nơi tuyển dụng đòi hỏi biết công cụ này.
 5. BRD/SRS đầy đủ (Chương 19-20) — cần thiết hơn khi làm ở các dự án lớn, tổ chức theo Waterfall hoặc yêu cầu tài liệu hoá chặt chẽ (ví dụ ngành Banking, Insurance).
-

Bài tập

1. Tự đánh giá bản thân theo 6 kỹ năng mềm ở mục 5.2 (thang 1-5). Chọn ra 1 kỹ năng yếu nhất, viết một hành động cụ thể để cải thiện trong tuần tới (ví dụ: "trong buổi họp tới, chủ động hỏi lại ít nhất 2 câu làm rõ trước khi ghi chú yêu cầu").
 2. Nếu chưa dùng Jira/Confluence bao giờ, tạo một tài khoản dùng thử (bản miễn phí cho cá nhân) và thử tạo một Epic + 2 User Story cho một tính năng bất kỳ của FoodNow.
-

Sai lầm thường gặp

- **Học công cụ trước, bỏ qua kỹ năng mềm:** biết dùng Jira thành thạo không giúp ích nếu không biết hỏi đúng câu hỏi để có nội dung đưa vào Jira.
 - **Nghĩ phải giỏi vẽ đẹp mới vẽ được sơ đồ:** giá trị của sơ đồ nằm ở việc **làm rõ tư duy và phát hiện lỗi hỏng logic**, không nằm ở tính thẩm mỹ — sơ đồ vẽ tay trên giấy trong buổi họp vẫn có giá trị như sơ đồ vẽ bằng công cụ chuyên nghiệp.
 - **Ôm đồm học hết mọi công cụ cùng lúc:** nên học công cụ theo nhu cầu thực tế của dự án đang làm, thay vì học dàn trải nhiều công cụ chưa dùng đến.
-

Tóm tắt & tiếp theo

BA cần cả kỹ năng cứng (viết tài liệu, modeling, quản lý yêu cầu) lẫn kỹ năng mềm (lắng nghe, đặt câu hỏi, giao tiếp thích ứng) — trong đó kỹ năng mềm thường là yếu tố phân biệt BA giỏi với BA trung bình. Phần 0 đến đây đã cho bạn bức tranh tổng quan về nghề BA. Từ Chương 6, sách sẽ đi sâu vào Phần 1: các mô hình quản lý dự án phần mềm cụ thể (Waterfall, Agile, Scrum, XP, Kanban) và vai trò chi tiết của BA trong từng mô hình.

Phần 1 — Quy trình phát triển & quản lý phần mềm

Chương 6: Waterfall: đặc điểm, các giai đoạn tuần tự, ưu/nhược điểm

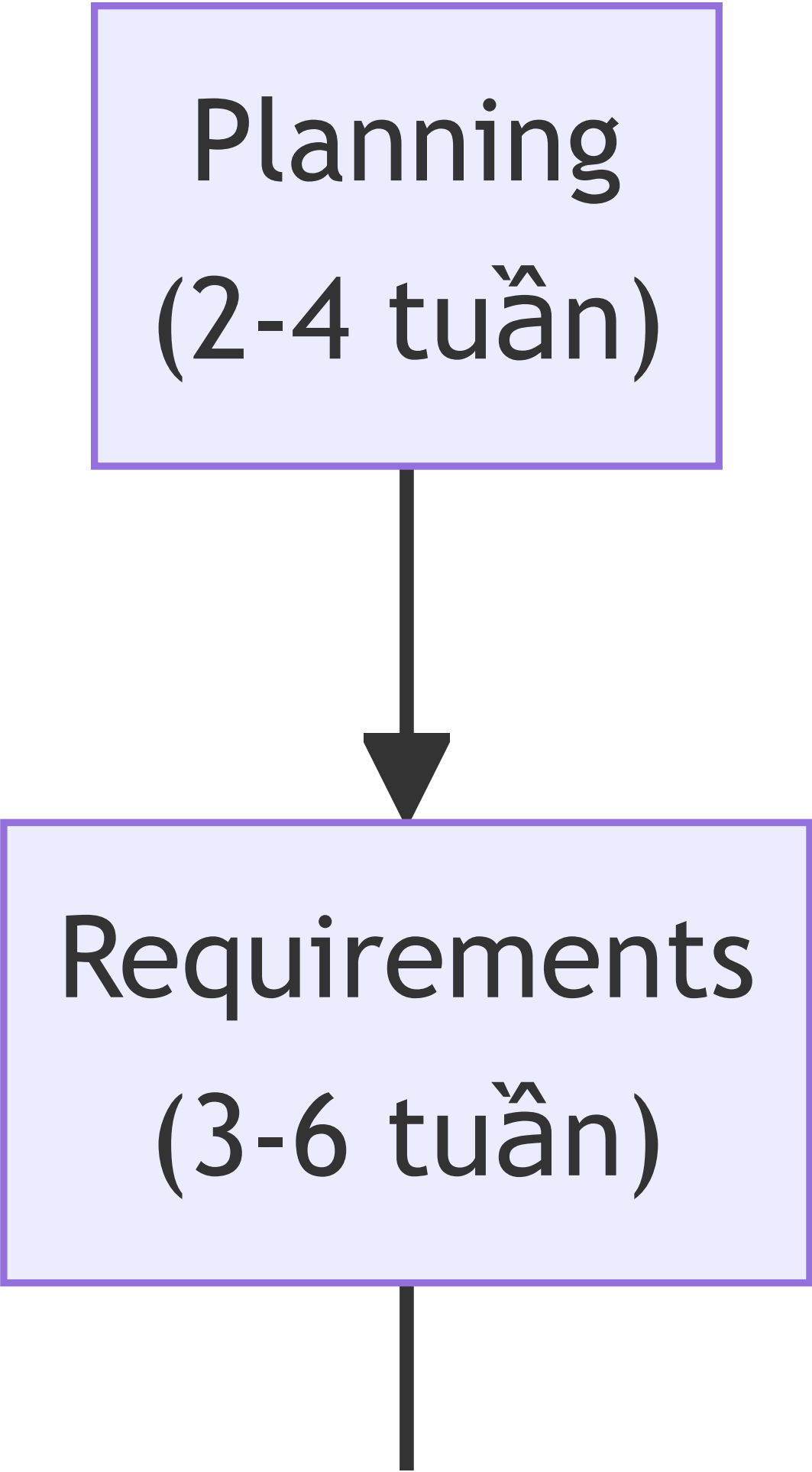
Mục tiêu học

- Mô tả được đặc điểm cốt lõi của mô hình Waterfall.
 - Vẽ và giải thích được sơ đồ các giai đoạn Waterfall áp dụng cho một dự án cụ thể.
 - Liệt kê ưu và nhược điểm của Waterfall, và biết trong tình huống nào Waterfall vẫn là lựa chọn hợp lý — không phải mô hình "lỗi thời" như nhiều tài liệu Agile vẫn ngầm ám chỉ.
-

6.1 Waterfall là gì?

Waterfall (mô hình thác nước) áp dụng đúng 6 giai đoạn SDLC (Chương 2) theo thứ tự **tuần tự, hoàn thành hẳn giai đoạn trước mới sang giai đoạn sau**, giống nước chảy từ bậc thang cao xuống thấp — không chảy ngược lại.

Planning
(2-4 tuần)

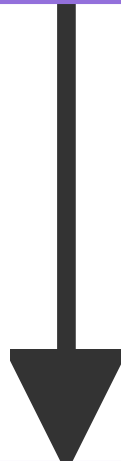


```
graph TD; A[Planning<br/>(2-4 tuần)] --> B[Requirements<br/>(3-6 tuần)];
```

Requirements
(3-6 tuần)

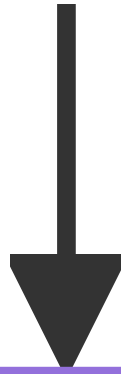


Design
(3-6 tuần)

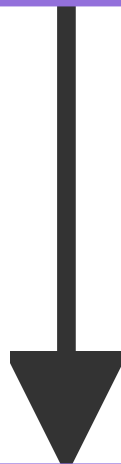


Implementation
(nhiều tháng)





Testing
(3-6 tuần)



Deployment
(1-2 tuần)

Đặc điểm cốt lõi: **tài liệu yêu cầu (SRS — Chương 20) được "chốt" (sign-off) trước khi bắt đầu Design**, và Design được chốt trước khi bắt đầu code. Mọi thay đổi sau khi chốt phải đi qua quy trình Change Request chính thức (Chương 18) — thường tốn thời gian và cần phê duyệt lại từ Sponsor.

6.2 Ví dụ Waterfall áp dụng cho FoodNow

Nếu FoodNow chọn làm theo Waterfall:

1. **Planning**: Ban giám đốc phê duyệt ngân sách và mục tiêu tổng thể (giảm phụ thuộc app bên thứ ba, xây dữ liệu khách hàng riêng).
2. **Requirements**: BA phỏng vấn toàn bộ stakeholder (ban giám đốc, quản lý chi nhánh, nhân viên giao hàng), viết BRD và SRS đầy đủ cho **toàn bộ phạm vi app** (đặt hàng, thanh toán, theo dõi đơn, chương trình khách hàng thân thiết) — tài liệu này được review và **ký duyệt** trước khi sang bước tiếp theo.
3. **Design**: kiến trúc sư thiết kế toàn bộ hệ thống, Designer thiết kế toàn bộ giao diện dựa trên SRS đã chốt.
4. **Implementation**: Dev code toàn bộ tính năng trong nhiều tháng, không có tính năng nào ra mắt cho người dùng thật cho đến khi xong hết.
5. **Testing**: QA kiểm thử toàn bộ hệ thống một lượt.
6. **Deployment**: ra mắt app hoàn chỉnh một lần duy nhất.

Điểm quan trọng: nếu ở tháng thứ 4 (đang Implementation), ban giám đốc **đổi ý** muốn thêm tính năng đặt bàn tại quán (một nhu cầu không có trong SRS ban đầu) — yêu cầu này phải qua Change Request, có thể làm trễ toàn bộ tiến độ vì nhiều phần thiết kế/code đã dựa trên phạm vi cũ.

6.3 Ưu điểm

- **Rõ ràng, dễ lập kế hoạch và ước tính ngân sách** trước khi bắt đầu — phù hợp khi cần cam kết ngân sách/thời gian cố định với khách hàng (hợp đồng fixed-price).
- **Tài liệu hoá đầy đủ**: hữu ích cho các ngành yêu cầu tuân thủ quy định nghiêm ngặt (Banking, Y tế, Hàng không) — cần hồ sơ đầy đủ cho kiểm toán/audit.
- **Phù hợp khi yêu cầu đã ổn định, ít khả năng thay đổi**: ví dụ dự án thay thế một hệ thống cũ bằng công nghệ mới nhưng nghiệp vụ giữ nguyên y hệt.
- **Dễ quản lý với đội phân tán nhiều nhà thầu phụ**: mỗi giai đoạn có bàn giao (deliverable) rõ ràng, dễ phân định trách nhiệm giữa các bên.

6.4 Nhược điểm

- **Phát hiện sai sót quá muộn:** như sơ đồ chi phí ở Chương 2, nếu SRS hiểu sai một yêu cầu quan trọng, phải đến giai đoạn Testing (sau nhiều tháng) mới phát hiện ra.
- **Không linh hoạt với thay đổi:** thị trường/nghịệp vụ thay đổi trong lúc dự án đang chạy (điều rất phổ biến với sản phẩm mới) sẽ khiến SRS "lỗi thời" trước khi kịp ra mắt.
- **Không có giá trị nghiệp vụ nào được tạo ra cho đến khi xong toàn bộ:** FoodNow phải chờ hết cả dự án (có thể 6-9 tháng) mới có bất kỳ tính năng nào dùng được, thay vì có thể ra mắt sớm tính năng đặt hàng cơ bản trước.
- **Áp lực lớn lên giai đoạn Requirements:** vì mọi thứ dựa trên SRS đã chốt, BA chịu áp lực phải "đúng ngay từ đầu" — điều gần như không tương với sản phẩm hoàn toàn mới, thị trường chưa rõ.

6.5 Khi nào Waterfall vẫn là lựa chọn hợp lý?

Trái với quan niệm phổ biến "Waterfall đã lỗi thời, Agile luôn tốt hơn", Waterfall vẫn phù hợp khi:

- Yêu cầu **đã rõ ràng và ổn định** (ví dụ: hệ thống thay thế phần mềm kế toán cũ bằng công nghệ mới, nghiệp vụ kế toán không đổi).
- Hợp đồng dạng **fixed-price, fixed-scope** với bên thứ ba — cần cam kết phạm vi/chi phí rõ ràng ngay từ đầu.
- Ngành có **yêu cầu tuân thủ pháp lý nghiêm ngặt** đòi hỏi tài liệu hoá đầy đủ trước khi triển khai (một số hệ thống trong ngành tài chính, dược phẩm, hàng không).
- Dự án **quy mô nhỏ, rủi ro thấp**, không cần lặp lại nhiều vòng phản hồi.

Chương 10 sẽ có bảng so sánh đầy đủ hơn giữa Waterfall và các mô hình Agile để giúp bạn (hoặc đội dự án) chọn mô hình phù hợp cho từng tình huống cụ thể.

Bài tập

1. Với dự án FoodNow, giả sử ban giám đốc yêu cầu cam kết ngân sách và thời gian cố định trước khi ký hợp đồng với đội phát triển thuê ngoài. Waterfall có phù hợp trong tình huống này không? Giải thích bằng 2 lý do dựa trên mục 6.5.
2. Liệt kê 2 rủi ro cụ thể nếu FoodNow chọn Waterfall nhưng thị trường giao đồ ăn thay đổi nhanh (đối thủ ra tính năng mới) trong lúc dự án đang ở giai đoạn Implementation.

Sai lầm thường gặp

- **Coi Waterfall là "làm không có kế hoạch gì thêm sau khi ký SRS"**: Waterfall vẫn cần quy trình Change Request để xử lý thay đổi phát sinh — không phải "đóng băng hoàn toàn" mà là "thay đổi có kiểm soát, có phê duyệt" (Chương 18).
- **Áp dụng Waterfall cho sản phẩm hoàn toàn mới, thị trường chưa rõ**: rủi ro rất cao vì không có cơ chế học hỏi và điều chỉnh sớm — nên cân nhắc Agile (Chương 7) trong tình huống này.
- **Nghĩ BA hết việc sau khi SRS được ký**: BA vẫn cần theo sát Design/Implementation/Testing để xử lý các câu hỏi làm rõ phát sinh, dù phạm vi đã chốt.

Tóm tắt & tiếp theo

Waterfall làm tuần tự, hoàn thành hẳn từng giai đoạn SDLC trước khi sang giai đoạn tiếp theo — rõ ràng, dễ lập kế hoạch, nhưng phát hiện sai sót muộn và kém linh hoạt. Phù hợp khi yêu cầu ổn định hoặc cần cam kết phạm vi/chi phí cố định. Chương 7 sẽ giới thiệu triết lý hoàn toàn khác: Agile — ra đời chính là để giải quyết những nhược điểm của Waterfall.

Chương 7: Agile: Agile Manifesto, 12 nguyên tắc, tư duy Agile khác Waterfall ở đâu

Mục tiêu học

- Trình bày được 4 giá trị cốt lõi của Agile Manifesto và ý nghĩa thực tế của từng giá trị.
- Giải thích được Agile **không phải là một quy trình cụ thể** mà là một tư duy/triết lý — Scrum, Kanban, XP (Chương 8-10) là các **framework cụ thể hoá** tư duy đó.
- So sánh được sự khác biệt gốc rễ giữa tư duy Agile và tư duy Waterfall.

7.1 Bối cảnh ra đời

Năm 2001, 17 chuyên gia phần mềm (đại diện nhiều phương pháp nhẹ khác nhau như Scrum, XP, Crystal...) họp lại và ký bản **Agile Manifesto** — không phải để tạo ra một quy trình mới, mà để phát biểu chung một **hệ giá trị** mà họ nhận thấy tạo ra phần mềm tốt hơn cách làm Waterfall truyền thống, vốn đang gây nhiều thất bại dự án vào thời điểm đó.

7.2 Bốn giá trị cốt lõi

Chúng tôi đánh giá cao:

- **Cá nhân và sự tương tác** hơn là quy trình và công cụ
- **Phần mềm chạy được** hơn là tài liệu đầy đủ
- **Hợp tác với khách hàng** hơn là đàm phán hợp đồng
- **Phản hồi với thay đổi** hơn là bám sát kế hoạch

Mặc dù các giá trị bên phải vẫn có ý nghĩa, chúng tôi đánh giá cao hơn các giá trị bên trái.

Điểm quan trọng thường bị hiểu sai: "**hơn là**" **không có nghĩa là "bỏ hẳn"**. Agile vẫn cần tài liệu, vẫn cần quy trình, vẫn cần hợp đồng — chỉ là khi phải đánh đổi, ưu tiên về bên trái. Đây là lý do BA làm Agile **vẫn viết tài liệu** (Chương 19-24) — chỉ là tài liệu gọn hơn, tập trung vào việc đủ dùng để đội hiểu và làm đúng, thay vì tài liệu hoá cho mục đích hồ sơ/kiểm toán.

Giá trị	Ý nghĩa với công việc BA
Cá nhân & tương tác hơn quy trình & công cụ	BA nên ưu tiên trao đổi trực tiếp với Dev/stakeholder hơn là chỉ gửi tài liệu rồi chờ phản hồi qua email
Phần mềm chạy được hơn tài liệu đầy đủ	BA viết User Story đủ để Dev làm đúng (Chương 23), không cần SRS 100 trang cho mọi tính năng nhỏ
Hợp tác với khách hàng hơn đàm phán hợp đồng	BA khuyến khích Sponsor/người dùng tham gia liên tục (demo mỗi sprint), không chỉ ký duyệt một lần rồi biến mất
Phản hồi với thay đổi hơn bám sát kế hoạch	BA xem yêu cầu thay đổi giữa dự án là bình thường, thậm chí tốt (nghĩa là đang học được điều mới), không phải sự cố cần né tránh

7.3 12 nguyên tắc (tóm tắt theo nhóm)

Agile Manifesto đi kèm 12 nguyên tắc chi tiết hơn — nhóm lại theo chủ đề để dễ nhớ:

Nhóm giao hàng liên tục & sớm:

1. Ưu tiên cao nhất là làm hài lòng khách hàng qua việc giao phần mềm có giá trị **sớm và liên tục**.
2. Giao phần mềm chạy được thường xuyên, chu kỳ càng ngắn càng tốt (vài tuần thay vì vài tháng).

Nhóm chào đón thay đổi: 3. Chào đón thay đổi yêu cầu, kể cả ở giai đoạn muộn — Agile khai thác thay đổi để tạo lợi thế cạnh tranh cho khách hàng, thay vì coi đó là điều cần tránh.

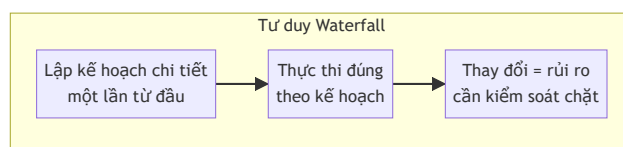
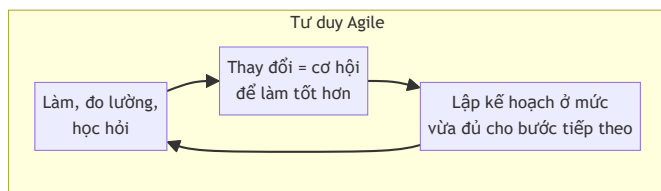
Nhóm con người & hợp tác: 4. Người làm nghiệp vụ và Dev phải làm việc cùng nhau hằng ngày suốt dự án. 5. Xây dựng dự án quanh những cá nhân có động lực, tin tưởng họ hoàn thành công việc. 6. Cách truyền đạt thông tin hiệu quả nhất là trò chuyện trực tiếp (face-to-face).

Nhóm đo lường & bền vững: 7. Phần mềm chạy được là thước đo tiến độ chính. 8. Agile khuyến khích nhịp độ phát triển bền vững, duy trì được mãi mãi — không chạy nước rút kiệt sức.

Nhóm kỹ thuật & đơn giản hoá: 9. Liên tục chú ý đến kỹ thuật xuất sắc và thiết kế tốt (xem thêm XP — Chương 9). 10. Đơn giản — nghệ thuật tối đa hoá lượng công việc **không cần làm** — là căn bản.

Nhóm tự tổ chức & cải tiến: 11. Kiến trúc, yêu cầu, thiết kế tốt nhất đến từ đội **tự tổ chức**. 12. Đội thường xuyên suy ngẫm cách làm việc hiệu quả hơn, rồi điều chỉnh hành vi cho phù hợp (đây là gốc rễ của buổi Retrospective trong Scrum — Chương 8).

7.4 Agile khác Waterfall ở đâu, về bản chất?



Khác biệt gốc rễ nằm ở **niềm tin về việc có thể biết trước mọi thứ hay không**:

- Waterfall giả định: nếu phân tích đủ kỹ ở đầu dự án, có thể **biết trước gần như toàn bộ** yêu cầu — nên đầu tư mạnh vào lập kế hoạch/phân tích chi tiết trước khi làm.
- Agile giả định: với sản phẩm phần mềm phức tạp, đặc biệt sản phẩm mới, **không thể biết trước hết mọi thứ** dù phân tích kỹ đến đâu — nên thay vì cố đoán đúng ngay từ đầu, hãy làm từng phần nhỏ, học từ phản hồi thực tế, rồi điều chỉnh liên tục.

Điều này thay đổi căn bản vai trò của BA: trong Agile, BA **không cố viết một tài liệu hoàn hảo ngay từ đầu**, mà làm việc liên tục theo từng chu kỳ ngắn, liên tục làm rõ và tinh chỉnh yêu cầu dựa trên phản hồi thực tế từ mỗi lần giao hàng.

Bài tập

1. Với dự án FoodNow, nếu ban giám đốc chưa chắc chắn khách hàng có sẵn sàng trả thêm phí giao hàng nhanh hay không, mô hình nào (Waterfall hay Agile) giúp FoodNow **kiểm chứng giả thuyết này với chi phí thấp hơn**? Giải thích dựa trên nguyên tắc 1-3 ở mục 7.3.
2. Chọn 3 trong 12 nguyên tắc Agile mà bạn thấy khó áp dụng nhất trong môi trường công ty truyền thống (nếu bạn từng làm việc ở môi trường như vậy), giải thích vì sao khó.

Sai lầm thường gặp

- **Nghĩ Agile nghĩa là "không cần tài liệu, không cần kế hoạch"**: sai — Agile vẫn cần cả hai, chỉ ưu tiên về bên trái của 4 giá trị khi phải đánh đổi, không phải bỏ hẳn về bên phải.
- **Nghĩ Agile luôn nhanh hơn Waterfall**: Agile không nhất thiết nhanh hơn về tổng thời gian — giá trị chính là **giảm rủi ro và có phản hồi sớm**, không phải "làm nhanh hơn" theo nghĩa đơn giản.
- **Áp dụng nửa vời**: gọi là "Agile" nhưng vẫn giữ nguyên tư duy "lập kế hoạch chi tiết một lần, không chấp nhận thay đổi" — đây là một dạng "Waterfall trá hình" khá phổ biến trong thực tế.

Tóm tắt & tiếp theo

Agile là một tư duy/triết lý (4 giá trị, 12 nguyên tắc), không phải một quy trình cụ thể — khác biệt gốc rễ với Waterfall nằm ở niềm tin "có thể biết trước mọi thứ hay không". Chương 8 sẽ đi vào **Scrum** — framework Agile phổ biến nhất, cụ thể hoá tư duy này thành vai trò, sự kiện, và artifact rõ ràng mà BA cần nắm vững.

Chương 8: Scrum chi tiết: vai trò, sự kiện, artifact

Mục tiêu học

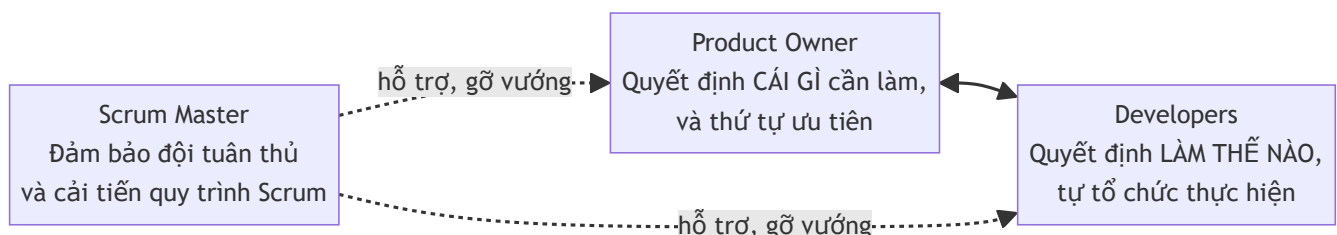
- Gọi tên và giải thích được 3 vai trò, 5 sự kiện, 3 artifact của Scrum.
- Mô tả được một sprint điển hình diễn ra như thế nào từ đầu đến cuối.
- Xác định rõ BA tham gia vào sự kiện nào, đóng góp gì — kể cả khi BA không phải một vai trò "chính thức" được định nghĩa trong Scrum Guide.

8.1 Scrum là gì?

Scrum là framework Agile phổ biến nhất, tổ chức công việc thành các chu kỳ lặp cố định gọi là **Sprint** (thường 1-4 tuần, phổ biến nhất là 2 tuần). Mỗi sprint là một vòng lặp đầy đủ các giai đoạn SDLC thu nhỏ (Requirements → Design → Implementation → Testing) cho một phần nhỏ sản phẩm.

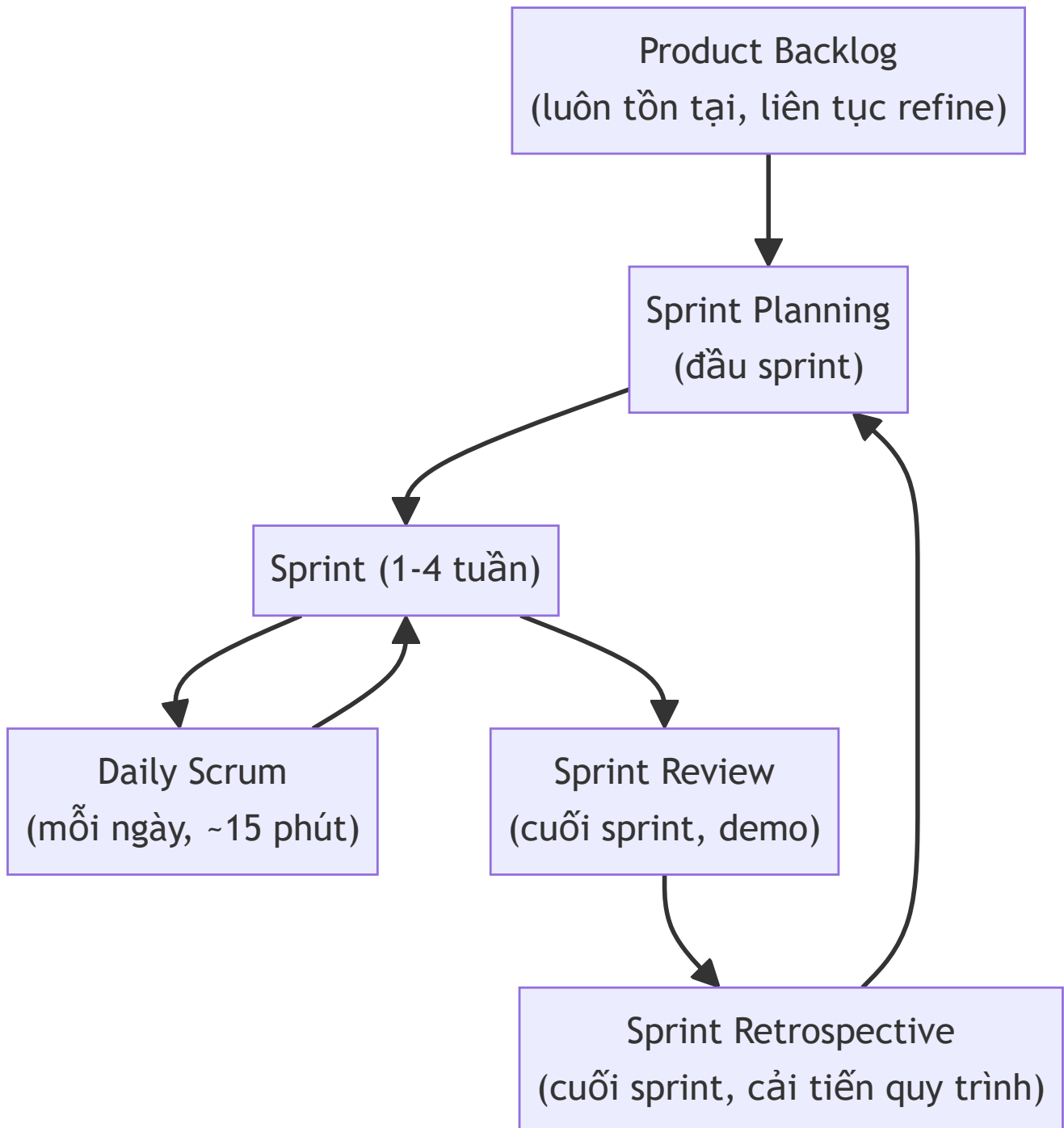
Lưu ý quan trọng: **Scrum không định nghĩa vai trò "Business Analyst"** trong bộ tài liệu chính thức (Scrum Guide) — chỉ có Product Owner, Scrum Master, Developers. Trong thực tế, công việc BA vẫn tồn tại và cần thiết, thường được một người đảm nhận dưới danh nghĩa: chính PO kiêm luôn, một BA riêng hỗ trợ PO, hoặc một Developer kiêm phân tích. Sách này gọi chung người làm công việc đó là "BA" dù chức danh chính thức trong đội có thể khác.

8.2 Ba vai trò



- **Product Owner (PO):** chịu trách nhiệm tối đa hoá giá trị sản phẩm, quản lý Product Backlog, quyết định thứ tự ưu tiên. BA thường làm việc **rất sát** với PO — cung cấp phân tích để PO ra quyết định đúng (Chương 27).
- **Scrum Master (SM):** không phải "quản lý dự án" theo nghĩa truyền thống — vai trò của SM là đảm bảo đội hiểu và áp dụng đúng Scrum, loại bỏ trở ngại (impediment) cho đội.
- **Developers:** bao gồm mọi người trực tiếp tạo ra sản phẩm (coder, tester, designer trong một số đội) — tự tổ chức, tự quyết định cách hoàn thành công việc trong sprint.

8.3 Năm sự kiện (Events)



Sự kiện	Mục đích	BA tham gia thế nào
Sprint	"Chiếc hộp" chứa mọi sự kiện khác, có độ dài cố định	BA làm việc liên tục trong suốt sprint, không chỉ ở đầu/cuối
Sprint Planning	Đội chọn các hạng mục từ Product Backlog đưa vào Sprint Backlog, thống nhất mục tiêu sprint	BA giải thích chi tiết yêu cầu, trả lời câu hỏi để đội ước lượng chính xác (Chương 27, 30)
Daily Scrum	Đội đồng bộ tiến độ hằng ngày trong 15 phút	BA tham dự để nắm vấn đề phát sinh sớm, sẵn sàng làm rõ yêu cầu ngay khi Dev cần
Sprint Review	Demo sản phẩm đã hoàn thành cho stakeholder, thu thập phản hồi	BA chuẩn bị nội dung demo cùng đội, ghi nhận phản hồi để đưa vào backlog tiếp theo
Sprint Retrospective	Đội tự nhìn lại cách làm việc, đề xuất cải tiến	BA đóng góp góc nhìn về quy trình làm rõ yêu cầu có hiệu quả không

8.4 Ba Artifact (sản phẩm/tài liệu)

Artifact	Là gì	Liên quan đến BA
Product Backlog	Danh sách toàn bộ công việc cần làm cho sản phẩm, sắp theo ưu tiên, luôn thay đổi	BA/PO liên tục "refine" (làm rõ, chia nhỏ) các hạng mục trong backlog (Chương 24, 27)
Sprint Backlog	Tập con của Product Backlog được chọn cho sprint hiện tại, cộng với kế hoạch để hoàn thành	BA đảm bảo các hạng mục trong Sprint Backlog đã đủ rõ ràng để Dev bắt tay làm ngay
Increment	Tổng các phần đã hoàn thành, đạt "Definition of Done", có thể release được	BA xác nhận Increment có đúng đáp ứng ý định nghiệp vụ ban đầu không (liên hệ Chương 34 - UAT)

8.5 Một sprint điển hình với FoodNow

Giả sử đội FoodNow làm Scrum, sprint 2 tuần, đang phát triển tính năng "theo dõi đơn hàng real-time":

- Sprint Planning (ngày 1):** PO đưa User Story "Là khách hàng, tôi muốn xem trạng thái đơn hàng theo thời gian thực để biết khi nào đồ ăn tới" vào sprint. BA giải thích rõ 4 trạng thái đơn hàng cần hiển thị, Dev ước lượng story point.
- Trong sprint:** Dev code, gặp câu hỏi "nếu nhà hàng huỷ đơn giữa chừng thì hiển thị trạng thái gì?" — hỏi trực tiếp BA thay vì tự đoán, BA trả lời trong Daily Scrum hoặc trao đổi riêng.
- Sprint Review (cuối sprint):** Demo tính năng cho đại diện quản lý chi nhánh (stakeholder), ghi nhận phản hồi: "cần thêm âm thanh thông báo khi đơn được giao xong" — đưa vào backlog cho sprint sau.
- Sprint Retrospective:** Đội nhận ra việc làm rõ yêu cầu về trạng thái đơn hàng lẽ ra nên làm kỹ hơn **trước** Sprint Planning (ở bước refinement) để tránh phát sinh câu hỏi giữa sprint.

Bài tập

1. Vẽ lại timeline một sprint 2 tuần cho FoodNow theo mẫu ở mục 8.5, nhưng với một tính năng khác: "khách hàng đặt lại đơn hàng cũ chỉ với 1 chạm". Ghi rõ BA làm gì ở mỗi sự kiện.
2. Giải thích vì sao Daily Scrum giới hạn 15 phút lại quan trọng — điều gì xảy ra nếu buổi họp này kéo dài thành 1 tiếng thảo luận sâu mỗi ngày?

Sai lầm thường gặp

- **BA vắng mặt trong Daily Scrum/Sprint Planning vì nghĩ "không phải việc của mình"**: dẫn đến Dev phải tự đoán yêu cầu khi BA không có mặt để hỏi ngay.
- **Nhồi nhét quá nhiều làm rõ yêu cầu vào Sprint Planning**: nên làm ở bước refinement/backlog grooming **trước** Sprint Planning (Chương 27), để Sprint Planning chỉ còn việc thống nhất phạm vi và ước lượng, không phải ngồi phân tích yêu cầu từ đầu.
- **Coi Sprint Review là buổi "nghiệm thu chính thức"**: Sprint Review là để lấy phản hồi sớm, tinh chỉnh hướng đi — không thay thế cho UAT chính thức trước khi release rộng rãi (Chương 34).

Tóm tắt & tiếp theo

Scrum tổ chức công việc qua Sprint với 3 vai trò (PO, SM, Developers), 5 sự kiện (Sprint, Planning, Daily Scrum, Review, Retrospective) và 3 artifact (Product Backlog, Sprint Backlog, Increment) — dù không có vai trò BA chính thức, công việc phân tích vẫn cần thiết xuyên suốt. Chương 9 sẽ giới thiệu Extreme Programming (XP) — framework Agile tập trung vào **thực hành kỹ thuật**, thường được kết hợp cùng Scrum.

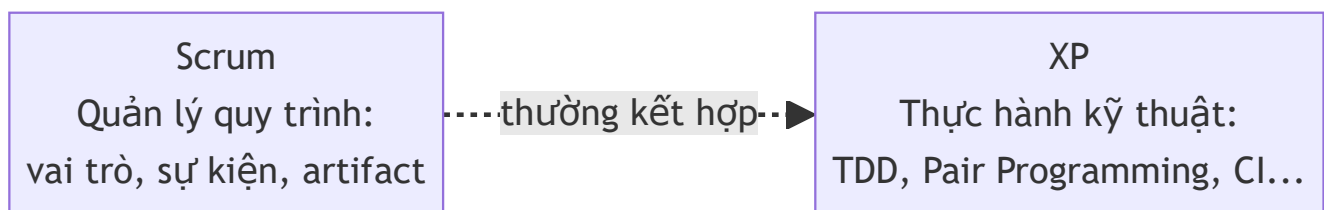
Chương 9: Extreme Programming (XP): thực hành kỹ thuật và vai trò BA

Mục tiêu học

- Giải thích được XP là gì, khác Scrum ở điểm nào (Scrum tập trung quản lý quy trình, XP tập trung thực hành kỹ thuật).
- Liệt kê và giải thích được các thực hành kỹ thuật cốt lõi của XP: TDD, Pair Programming, Continuous Integration, Refactoring, Simple Design, Collective Code Ownership.
- Hiểu vai trò của BA khi làm việc cùng một đội áp dụng XP — khác gì so với đội chỉ làm Scrum thuần túy.

9.1 XP là gì, vì sao ra đời?

Extreme Programming (XP) ra đời cùng thời điểm với Scrum (giữa/cuối thập niên 1990), cũng là một trong những framework góp phần hình thành Agile Manifesto. Nếu Scrum trả lời câu hỏi "**tổ chức công việc và con người thế nào**", thì XP trả lời câu hỏi "**làm sao viết code chất lượng cao, thích ứng nhanh với thay đổi**". Hai framework này **không loại trừ nhau** — nhiều đội áp dụng Scrum để quản lý quy trình, đồng thời áp dụng các thực hành kỹ thuật của XP bên trong mỗi sprint.



9.2 Các thực hành kỹ thuật cốt lõi

Thực hành	Mô tả ngắn	Ảnh hưởng đến BA
Test-Driven Development (TDD)	Viết test trước khi viết code hiện thực tính năng	Acceptance Criteria (Chương 23) của BA cần đủ rõ ràng, cụ thể để Dev chuyển thành test case ngay được
Pair Programming	Hai lập trình viên cùng làm việc trên một máy, một người code, một người review trực tiếp	Không ảnh hưởng trực tiếp đến BA, nhưng giúp giảm lỗi hiểu sai yêu cầu vì có 2 người cùng đọc hiểu tài liệu
Continuous Integration (CI)	Tích hợp code vào nhánh chính nhiều lần mỗi ngày, tự động build & test	Cho phép giao hàng nhỏ, thường xuyên hơn — phù hợp để BA tổ chức demo/lấy phản hồi liên tục
Refactoring liên tục	Cải thiện cấu trúc code mà không đổi hành vi bên ngoài	BA cần hiểu: thời gian "refactor" không tạo ra tính năng mới ngay, nhưng cần thiết để duy trì tốc độ phát triển dài hạn — cần giải thích điều này với Sponsor khi họ hỏi "sao sprint này không ra tính năng gì mới"
Simple Design	Làm giải pháp đơn giản nhất đáp ứng đúng yêu cầu hiện tại, không thiết kế thừa cho tương lai chưa chắc xảy ra	Khuyến khích BA viết yêu cầu tập trung vào giá trị cần ngay , tránh yêu cầu "phòng khi sau này cần" (gold plating)
Collective Code Ownership	Bất kỳ ai trong đội cũng có thể sửa bất kỳ phần code nào	Không ảnh hưởng trực tiếp BA, nhưng nghĩa là BA có thể trao đổi yêu cầu với bất kỳ Dev nào trong đội, không chỉ người "phụ trách module đó"
On-site Customer	Có đại diện khách hàng/nghiệp vụ làm việc cùng đội hằng ngày, trả lời câu hỏi ngay lập tức	Đây chính là vai trò gần nhất với BA trong XP nguyên bản — XP kỳ vọng có người đóng vai "khách hàng tại chỗ" để làm rõ yêu cầu ngay khi Dev cần, không phải chờ qua nhiều lớp trung gian

9.3 "On-site Customer" — vai trò gần nhất với BA trong XP

Đây là khái niệm quan trọng nhất của XP đối với công việc BA: XP giả định có một người **luôn có mặt, luôn sẵn sàng trả lời câu hỏi về yêu cầu ngay lập tức** khi Dev cần, thay vì Dev phải đợi email/hộp theo lịch mới được giải đáp. Trong thực tế hiện đại, vai trò "on-site customer" này thường do chính BA đảm nhận (vì đại diện khách hàng thật hiếm khi có thể ngồi cùng đội hằng ngày).

Điều này đặt ra yêu cầu cụ thể cho BA làm việc trong đội theo XP:

- Cần **sẵn sàng phản hồi nhanh** trong giờ làm việc — không chỉ có mặt trong các buổi họp chính thức.
- Cần viết **Acceptance Criteria đủ cụ thể** để Dev có thể tự chuyển thành test case (liên hệ TDD) mà không cần hỏi lại quá nhiều — nhưng vẫn chấp nhận sẽ có câu hỏi phát sinh và cần trả lời ngay.

- Cần hiểu đủ về mặt kỹ thuật để tham gia thảo luận về đánh đổi (trade-off) khi Dev đề xuất đơn giản hoá một yêu cầu vì lý do kỹ thuật (liên hệ Simple Design).

9.4 Ví dụ: BA làm việc với đội FoodNow áp dụng XP

Đội FoodNow áp dụng Scrum + các thực hành XP. Khi BA viết Acceptance Criteria cho tính năng "áp dụng mã giảm giá khi thanh toán":

- Vì đội áp dụng TDD, BA viết Acceptance Criteria theo cú pháp Given-When-Then (Chương 23) đủ cụ thể để Dev viết test tự động trực tiếp từ đó — ví dụ: "Given giỏ hàng có tổng 200.000đ, When nhập mã giảm giá 'FOODNOW10' hợp lệ, Then tổng tiền giảm còn 180.000đ".
- Trong lúc code, Dev phát hiện trường hợp chưa rõ: "nếu khách nhập 2 mã giảm giá cùng lúc thì sao?" — vì BA đóng vai trò gần với "on-site customer", Dev nhắn hỏi trực tiếp và nhận câu trả lời ngay trong ngày, thay vì phải chờ đến buổi họp tuần sau.
- Vì đội áp dụng Simple Design, Dev đề xuất: thay vì xây dựng hệ thống áp dụng nhiều mã giảm giá cùng lúc (phức tạp, chưa chắc cần), chỉ cho phép áp dụng 1 mã/đơn hàng ở phiên bản đầu — BA đồng ý vì đúng tinh thần "chưa cần thiết ngay", ghi nhận làm cải tiến tương lai nếu có nhu cầu thật.

Bài tập

1. Viết một Acceptance Criteria theo Given-When-Then cho tính năng "khách hàng huỷ đơn trong vòng 2 phút sau khi đặt" của FoodNow — đủ cụ thể để Dev có thể viết test tự động trực tiếp từ đó.
2. Giải thích bằng lời của bạn: vì sao "on-site customer" lại là khái niệm quan trọng với BA hơn là với PM hay Designer.

Sai lầm thường gặp

- **BA chỉ giao tài liệu rồi biến mất, không sẵn sàng trả lời câu hỏi nhanh:** đi ngược tinh thần "on-site customer" của XP, khiến Dev phải tự đoán và dễ làm sai ý.
- **Viết Acceptance Criteria quá mơ hồ, không đủ cụ thể để chuyển thành test case:** làm giảm hiệu quả của TDD, Dev phải đoán ý hoặc quay lại hỏi nhiều lần.
- **Từ chối mọi đề xuất đơn giản hoá từ Dev vì "không đúng như tài liệu":** nên cân nhắc tinh thần Simple Design — nếu đơn giản hoá không ảnh hưởng giá trị nghiệp vụ cốt lõi, nên linh hoạt.

Tóm tắt & tiếp theo

XP bổ sung các thực hành kỹ thuật (TDD, Pair Programming, CI, Refactoring, Simple Design) cho Agile, và khái niệm "on-site customer" đặt ra kỳ vọng cụ thể về việc BA cần sẵn sàng, phản hồi nhanh, viết yêu cầu đủ cụ thể để chuyển hoá thành test. Chương 10 sẽ giới thiệu Kanban — một framework Agile khác, đơn giản hơn Scrum về mặt cấu trúc sự kiện, và tổng hợp bảng so sánh toàn bộ các mô hình đã học (Waterfall, Scrum, XP, Kanban).

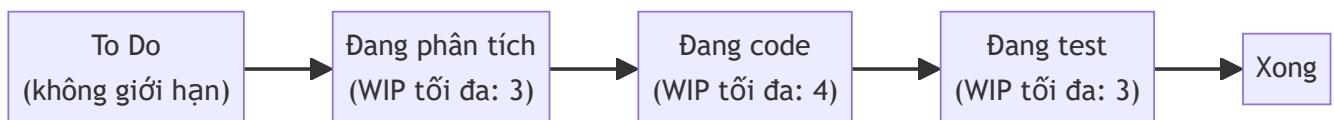
Chương 10: Kanban & so sánh các mô hình: Waterfall vs Scrum vs Kanban vs XP

Mục tiêu học

- Giải thích được nguyên lý cốt lõi của Kanban: luồng liên tục (continuous flow) và giới hạn công việc đang làm (WIP limit).
- Phân biệt được Kanban với Scrum — điểm khác biệt quan trọng nhất là gì.
- Dùng được bảng tiêu chí để chọn mô hình phù hợp cho một dự án cụ thể, thay vì chọn theo trào lưu.

10.1 Kanban là gì?

Kanban (tiếng Nhật nghĩa là "bảng hiệu"/"thẻ trực quan") là phương pháp quản lý công việc dựa trên **luồng liên tục**, khác với Scrum tổ chức công việc theo các chu kỳ cố định (sprint). Công việc di chuyển qua các cột trạng thái (ví dụ: To Do → Đang phân tích → Đang làm → Đang test → Xong) trên một **bảng Kanban**, mỗi cột có thể giới hạn số lượng công việc tối đa được phép nằm trong đó cùng lúc — gọi là **WIP limit (Work In Progress limit)**.



Vì sao cần giới hạn WIP? Nguyên lý cốt lõi: làm nhiều việc cùng lúc (đa nhiệm) khiến mỗi việc đều chậm lại do chi phí chuyển đổi ngữ cảnh (context switching). Giới hạn WIP buộc đội **hoàn thành việc đang làm trước khi nhận việc mới** — thường tạo ra thời gian hoàn thành (lead time) ngắn hơn cho toàn hệ thống, dù có vẻ nghịch lý.

10.2 Kanban khác Scrum ở đâu?

Tiêu chí	Scrum	Kanban
Nhịp độ	Cố định (sprint 1-4 tuần)	Liên tục, không chia chu kỳ cố định
Vai trò	Định nghĩa rõ (PO, SM, Developers)	Không định nghĩa vai trò bắt buộc
Cách giới hạn công việc	Giới hạn theo Sprint Backlog (số lượng công việc chọn cho sprint)	Giới hạn WIP theo từng cột trên bảng
Thay đổi giữa chu kỳ	Hạn chế thay đổi Sprint Backlog giữa sprint đang chạy	Có thể thêm việc mới bất cứ lúc nào miễn còn chỗ trống theo WIP limit
Đo lường	Velocity (số story point hoàn thành mỗi sprint)	Lead time, Cycle time (thời gian một hạng mục đi từ đầu đến cuối bảng)
Phù hợp nhất khi	Công việc có thể nhóm thành các đợt giao hàng rõ ràng	Công việc đến liên tục, không đều, khó nhóm thành đợt cố định (ví dụ: đội support, đội vận hành)

Ví dụ thực tế: đội phát triển tính năng mới cho FoodNow (công việc có kế hoạch, có thể nhóm theo tính năng) thường hợp với Scrum hơn. Đội vận hành/bảo trì hệ thống FoodNow sau go-live (nhận yêu cầu sửa lỗi/hỗ trợ liên tục, không đều, khó đoán trước) thường hợp với Kanban hơn.

10.3 Vai trò BA trong Kanban

Vì Kanban không định nghĩa chu kỳ cố định, công việc của BA cũng liên tục hơn: thay vì có một giai đoạn "refinement trước sprint" rõ rệt như Scrum, BA cần đảm bảo luôn có sẵn một lượng hạng mục **đã được làm rõ đầy đủ** ("ready") trong cột To Do, để đội không bao giờ bị đói việc rõ ràng để làm. Kỹ thuật MoSCoW/ưu tiên hoá (Chương 26) vẫn áp dụng được, chỉ khác ở chỗ ưu tiên được xem xét liên tục thay vì "chốt" một lần đầu mỗi sprint.

10.4 Bảng so sánh tổng hợp 4 mô hình

Tiêu chí	Waterfall	Scrum	Kanban	XP
Loại	Tuần tự	Lặp (iterative), theo chu kỳ	Luồng liên tục	Tập thực hành kỹ thuật (thường kết hợp Scrum/Kanban)
Tài liệu yêu cầu	Đầy đủ, chốt trước khi code (SRS)	Gọn, tinh chỉnh liên tục (User Story)	Gọn, tinh chỉnh liên tục	Acceptance Criteria đủ cụ thể để viết test
Chấp nhận thay đổi	Thấp — cần Change Request chính thức	Trung bình — hạn chế đổi trong sprint đang chạy	Cao — có thể đổi bất cứ lúc nào (trong giới hạn WIP)	Cao, đi kèm Simple Design để dễ thích ứng
Giao hàng	Một lần, ở cuối dự án	Từng sprint (1-4 tuần)	Liên tục, ngay khi hạng mục hoàn thành	Thường xuyên nhờ CI
Phù hợp khi	Yêu cầu ổn định, cần cam kết phạm vi/chi phí cố định	Sản phẩm mới, cần phản hồi định kỳ theo đợt	Công việc đến liên tục, không đều (support/vận hành)	Cần chất lượng kỹ thuật cao, thay đổi thường xuyên
Vai trò BA	Rõ ràng theo giai đoạn, áp lực cao ở Requirements	Làm việc sát PO, refine backlog liên tục	Đảm bảo luôn có backlog "ready", làm việc liên tục	Gần vai trò "on-site customer", phản hồi tức thời

10.5 Cách chọn mô hình cho một dự án cụ thể

Không có mô hình nào "tốt nhất tuyệt đối" — câu hỏi cần đặt ra:

- Yêu cầu đã ổn định chưa, hay còn nhiều điều chưa biết?** → Chưa ổn định: nghiêng về Agile (Scrum/Kanban/XP). Đã ổn định + cần cam kết chi phí cố định: cân nhắc Waterfall.
- Công việc có thể nhóm thành đợt rõ ràng không, hay đến liên tục ngẫu nhiên?** → Nhóm được thành đợt: Scrum. Đến liên tục, không đều: Kanban.
- Có cần chất lượng kỹ thuật/tốc độ thích ứng rất cao không (sản phẩm cạnh tranh gắt)?** → Cân nhắc bổ sung thực hành XP bên trong Scrum/Kanban.
- Ràng buộc pháp lý/hợp đồng có đòi hỏi tài liệu hoá đầy đủ trước khi code không?** → Nghiêng Waterfall hoặc mô hình lai (Chương 11 sẽ nói thêm về mô hình Hybrid).

Với FoodNow: giai đoạn phát triển tính năng mới (chưa chắc chắn khách hàng phản ứng ra sao) hợp Scrum; giai đoạn vận hành sau go-live (xử lý bug, yêu cầu nhỏ liên tục) hợp Kanban.

Bài tập

- Với đội vận hành hệ thống FoodNow sau go-live (nhận báo lỗi và yêu cầu nhỏ liên tục từ các chi nhánh), vẽ một bảng Kanban đơn giản (tối thiểu 4 cột) và đặt WIP limit hợp lý cho từng cột.

2. Dùng 4 câu hỏi ở mục 10.5 để tự chọn mô hình phù hợp cho một dự án bạn biết hoặc đang làm.

Sai lầm thường gặp

- **Nghĩ Kanban là "Scrum không có sprint" — đơn giản hơn nên dễ làm hơn:** Kanban đòi hỏi kỷ luật cao về giới hạn WIP, không có sự "ép buộc" bởi ranh giới sprint như Scrum — nhiều đội áp dụng sai vì không tuân thủ WIP limit, biến bảng Kanban thành danh sách việc không giới hạn.
 - **Chọn mô hình theo trào lưu ("ai cũng làm Scrum nên mình cũng làm"):** nên chọn dựa trên đặc điểm công việc thực tế, theo 4 câu hỏi ở mục 10.5.
 - **Trộn lẫn Scrum và Kanban mà không hiểu rõ đang đánh đổi gì:** một số đội dùng "Scrumban" (lai giữa hai mô hình) hợp lý, nhưng cần hiểu rõ lý do, không phải trộn ngẫu nhiên vì không quyết được.
-

Tóm tắt & tiếp theo

Kanban dựa trên luồng liên tục và giới hạn WIP, khác Scrum ở việc không có chu kỳ cố định. Không mô hình nào (Waterfall/Scrum/Kanban/XP) tốt tuyệt đối — việc chọn mô hình cần dựa trên đặc điểm công việc thực tế. Chương 11 sẽ tổng kết Phần 1 bằng cách đi sâu vào **vai trò và trách nhiệm cụ thể của BA** trong từng mô hình, giúp bạn biết chính xác cần làm gì khi vào một đội đã chọn sẵn một mô hình cụ thể.

Chương 11: Vai trò và trách nhiệm cụ thể của BA trong từng mô hình

Mục tiêu học

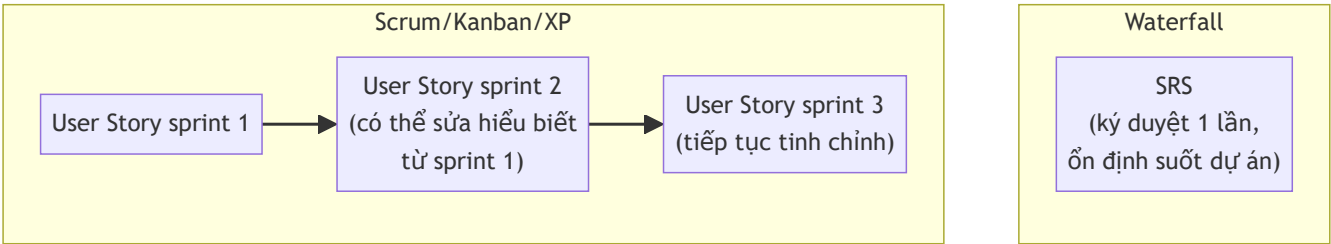
- Tổng hợp lại, so sánh trực tiếp deliverable và nhịp độ làm việc của BA trong Waterfall, Scrum, Kanban, XP.
- Biết cách thích ứng nhanh khi chuyển từ một đội theo mô hình này sang đội theo mô hình khác.
- Hiểu khái niệm mô hình Hybrid (lai) — thực tế phổ biến hơn "Agile thuần" hay "Waterfall thuần".

11.1 Bảng tổng hợp deliverable của BA theo từng mô hình

Mô hình	Tài liệu chính BA tạo ra	Nhịp độ	Khi nào tài liệu được xem là "xong"
Waterfall	BRD, SRS đầy đủ (Chương 19-20)	Một lần, đầu dự án	Khi được Sponsor ký duyệt chính thức (sign-off), gần như không đổi sau đó trừ Change Request
Scrum	User Story, Acceptance Criteria, backlog được refine liên tục (Chương 23-24, 27)	Liên tục theo sprint	Khi đạt "Definition of Ready" trước khi vào sprint, và "Definition of Done" sau khi hoàn thành
Kanban	Tương tự Scrum nhưng không gắn với ranh giới sprint	Liên tục, không theo chu kỳ cố định	Khi hạng mục đạt tiêu chí "ready" để vào cột tiếp theo trên bảng
XP	Acceptance Criteria rất cụ thể (đủ để viết test tự động), phản hồi tức thời	Liên tục, tức thời (on-site customer)	Khi test tự động dựa trên Acceptance Criteria pass

11.2 Sự khác biệt về "độ chắc chắn" của tài liệu

Khác biệt quan trọng nhất giữa Waterfall và các mô hình Agile không phải là "có tài liệu hay không" — mà là **tài liệu có được xem là "cố định" hay "sống, liên tục thay đổi"**:



BA chuyển từ Waterfall sang Agile thường gặp khó khăn tâm lý ban đầu: cảm giác "chưa chắc chắn, chưa hoàn chỉnh" khi viết User Story ngắn gọn cho sprint hiện tại mà chưa biết chi tiết toàn bộ sản phẩm sẽ ra sao — đây là điều **bình thường và có chủ đích** trong Agile, không phải làm việc cẩu thả.

11.3 Thích ứng khi chuyển mô hình

Từ Waterfall sang Scrum/Kanban — điều BA cần thay đổi tư duy:

- Ngừng cố viết tài liệu "hoàn chỉnh một lần" — chấp nhận viết vừa đủ cho bước tiếp theo, tinh chỉnh dần dựa trên phản hồi.
- Tăng tần suất giao tiếp với Dev — không còn chuyển giao tài liệu một lần rồi biến mất, mà đồng hành liên tục.
- Học cách chia nhỏ yêu cầu lớn (Epic) thành các User Story độc lập, có thể giao hàng riêng lẻ (Chương 24) — kỹ năng gần như không cần thiết trong Waterfall thuần.

Từ Scrum sang Kanban — điều BA cần thay đổi tư duy:

- Bỏ nhịp "refine theo sprint" cố định, chuyển sang duy trì liên tục một hàng đợi backlog "ready".
- Theo dõi cycle time/lead time thay vì velocity để đánh giá hiệu quả làm rõ yêu cầu.

Vào một đội áp dụng XP — điều BA cần chuẩn bị:

- Sẵn sàng tinh thần "on-site customer": phản hồi nhanh, viết Acceptance Criteria rất cụ thể.
- Hiểu đủ kỹ thuật để tham gia thảo luận đánh đổi khi Dev đề xuất Simple Design.

11.4 Mô hình Hybrid (lai) — thực tế phổ biến hơn "thuần"

Trong thực tế, rất ít công ty áp dụng một mô hình "thuần" 100% theo đúng sách vở. Phổ biến hơn là các mô hình lai, ví dụ:

- **Water-Scrum-Fall**: giai đoạn đầu dự án (thu thập yêu cầu tổng thể, ký hợp đồng, ngân sách) làm theo kiểu Waterfall vì cần cam kết với Sponsor/khách hàng; giai đoạn triển khai thực tế làm theo Scrum để linh hoạt; giai đoạn cuối (release, đào tạo người dùng, nghiệm thu chính thức) lại quay về phong cách Waterfall vì cần tài liệu hoá đầy đủ cho bàn giao.
- **Scrumban**: kết hợp nhịp sprint của Scrum với bảng trực quan và giới hạn WIP của Kanban.
- **SAFe, LeSS** (Scaled Agile): các framework mở rộng Agile cho tổ chức lớn nhiều đội cùng phối hợp — nằm ngoài phạm vi sách này, nhưng người đọc nên biết tên để tự tìm hiểu thêm khi làm việc ở tổ chức quy mô lớn.

BA giỏi không phải người "thuộc lòng lý thuyết một mô hình", mà là người **nhận diện được mô hình thực tế** đội mình đang áp dụng (dù có tên chính thức hay không) và điều chỉnh cách làm tài liệu, nhịp độ giao tiếp cho phù hợp.

11.5 Checklist nhanh: "Tôi vừa vào một đội mới, cần làm gì trước tiên?"

1. Hỏi rõ: đội đang theo mô hình nào (tên gọi chính thức, nhưng quan trọng hơn là **cách làm thực tế** — có sprint không, có ký duyệt SRS chính thức không, có bảng Kanban không).
 2. Tìm hiểu deliverable BA trước đó của đội trông như thế nào (xem lại tài liệu cũ trong Confluence/Jira nếu có) để theo đúng chuẩn/định dạng đội đang quen dùng.
 3. Xác định nhịp độ giao tiếp kỳ vọng: có Daily Scrum không, có kỳ vọng phản hồi tức thời như XP không, hay có thể làm việc theo lịch họp cố định như Waterfall.
 4. Xác định ai có quyền "chốt"/ký duyệt yêu cầu — Sponsor (Waterfall) hay PO (Scrum/Kanban).
-

Bài tập

1. Giả sử bạn được chuyển từ một dự án FoodNow làm theo Waterfall (đã có SRS đầy đủ) sang một dự án mới làm theo Scrum. Liệt kê 3 thói quen làm việc bạn cần thay đổi ngay, dựa trên mục 11.3.
 2. Dùng checklist ở mục 11.5 để lên danh sách câu hỏi bạn sẽ hỏi trong ngày đầu tiên nếu gia nhập một đội dự án hoàn toàn mới.
-

Sai lầm thường gặp

- **Áp dụng cứng nhắc lý thuyết sách vở mà không quan sát thực tế đội đang làm gì:** mô hình lai (Hybrid) rất phổ biến — cần linh hoạt thay vì đòi hỏi đội phải làm "đúng chuẩn" theo lý thuyết.
 - **Giữ nguyên thói quen cũ khi chuyển mô hình** (ví dụ vẫn cố viết SRS 50 trang khi đội đã chuyển sang Scrum) — gây lãng phí thời gian và không phù hợp nhịp độ đội đang cần.
 - **Không hỏi rõ ai có quyền quyết định cuối cùng khi vào đội mới:** dẫn đến trình bày tài liệu sai người, sai quy trình phê duyệt.
-

Tóm tắt & tiếp theo

BA cần điều chỉnh deliverable và nhịp độ làm việc theo mô hình đội đang áp dụng — từ tài liệu "cố định, ký duyệt một lần" của Waterfall đến tài liệu "sống, liên tục tinh chỉnh" của Scrum/Kanban/XP, và cần nhận diện linh hoạt các mô hình lai (Hybrid) phổ biến trong thực tế. Đây cũng là điểm kết thúc Phần 1. Từ Chương 12, sách bước vào Phần 2 — đi sâu vào kỹ năng cốt lõi nhất của nghề BA: thu thập và phân tích yêu cầu, bắt đầu với các kỹ thuật Elicitation.

Phần 2 — Thu thập & phân tích yêu cầu

Chương 12: Kỹ thuật Elicitation: interview, workshop, survey, observation

Mục tiêu học

- Liệt kê được các kỹ thuật elicitation phổ biến và biết khi nào nên dùng kỹ thuật nào.
 - Chuẩn bị và thực hiện được một buổi phỏng vấn thu thập yêu cầu hiệu quả.
 - Phân biệt câu hỏi mở (open question) và câu hỏi đóng (closed question), biết dùng đúng lúc.
 - Tránh được các lỗi phổ biến khiến elicitation thu về thông tin sai hoặc thiếu.
-

12.1 Elicitation là gì?

Elicitation (thu thập/khơi gợi yêu cầu) là quá trình **chủ động lấy thông tin** từ stakeholder về nhu cầu, vấn đề, kỳ vọng của họ — khác với việc chỉ "chờ" họ tự nói ra. Từ "khơi gợi" quan trọng: nhiều stakeholder không tự biết diễn đạt rõ nhu cầu của mình (như ví dụ ban giám đốc FoodNow ở Chương 1: "tôi muốn có app để khách đặt đồ ăn online" — một câu quá mơ hồ để bắt tay làm ngay). Việc của BA là dùng đúng kỹ thuật để **giúp họ diễn đạt rõ ràng hơn** điều họ thực sự cần.

12.2 Các kỹ thuật elicitation phổ biến

Kỹ thuật	Mô tả	Khi nào dùng	Ưu điểm	Hạn chế
Interview (Phỏng vấn 1-1)	Trò chuyện trực tiếp với một stakeholder	Cần đào sâu ý kiến cá nhân, chủ đề nhạy cảm	Đào sâu được chi tiết, xây dựng quan hệ tin cậy	Tốn thời gian, chỉ có 1 góc nhìn mỗi lần
Workshop (Hội thảo nhóm)	Tập hợp nhiều stakeholder cùng thảo luận	Cần thống nhất ý kiến giữa nhiều bên có góc nhìn khác nhau	Phát hiện mâu thuẫn giữa các bên ngay tại chỗ, tiết kiệm thời gian so với phỏng vấn từng người	Cần điều phối tốt (Chương 33), dễ bị người nói nhiều lấn át người ít nói
Survey/Questionnaire (Khảo sát)	Bảng câu hỏi gửi cho số đông	Cần thu thập ý kiến từ số lượng lớn người (ví dụ hàng trăm khách hàng)	Thu thập được số lượng lớn, dễ định lượng	Không đào sâu được, câu hỏi thiết kế sai sẽ thu về dữ liệu vô nghĩa
Observation (Quan sát)	Quan sát người dùng thực hiện công việc hiện tại	Người dùng khó diễn đạt bằng lời việc họ làm hằng ngày (thao tác đã thành thói quen)	Phát hiện được vấn đề thật mà chính người dùng cũng không nhận ra là "vấn đề"	Tốn thời gian, người bị quan sát có thể thay đổi hành vi vì biết đang bị quan sát
Document Analysis (Phân tích tài liệu có sẵn)	Đọc quy trình, biểu mẫu, báo cáo hiện có	Có hệ thống/quy trình cũ cần tìm hiểu trước khi thay thế	Không tốn thời gian của stakeholder, có sẵn dữ liệu khách quan	Tài liệu có thể đã lỗi thời, không phản ánh cách làm thực tế hiện tại
Brainstorming	Cả nhóm cùng đưa ý tưởng tự do, chưa đánh giá đúng/sai ngay	Giai đoạn đầu, cần nhiều ý tưởng đa dạng trước khi thu hẹp	Khuyến khích sáng tạo, không giới hạn tư duy sớm	Dễ lan man nếu không điều phối tốt, cần bước lọc ý tưởng sau đó
Focus Group	Nhóm nhỏ người dùng đại diện thảo luận có định hướng	Cần hiểu cảm nhận/thái độ của một nhóm người dùng cụ thể	Thu được góc nhìn đa dạng hơn phỏng vấn 1-1, sâu hơn survey	Cần người điều phối có kỹ năng, kết quả có thể bị ảnh hưởng bởi tâm lý đám đông

12.3 Ví dụ áp dụng cho FoodNow

Với dự án FoodNow, BA có thể kết hợp nhiều kỹ thuật cho từng nhóm stakeholder:

- **Interview** với Ban giám đốc: làm rõ mục tiêu chiến lược (giảm phụ thuộc app bên thứ ba, tăng biên lợi nhuận) — chủ đề mang tính chiến lược, cần trao đổi riêng, sâu.

- **Workshop** với quản lý các chi nhánh: thống nhất quy trình xử lý đơn hàng chung, vì mỗi chi nhánh hiện có thể đang làm hơi khác nhau — cần cả nhóm ngồi lại thống nhất một lần.
- **Survey** gửi cho khách hàng thân thiết hiện tại: hỏi họ có sẵn sàng dùng app riêng của FoodNow thay vì app bên thứ ba không, lý do gì khiến họ vẫn dùng app khác.
- **Observation** tại một chi nhánh: quan sát trực tiếp nhân viên nhận và xử lý đơn hàng qua điện thoại hiện tại — có thể phát hiện ra vấn đề mà nhân viên không tự nhận ra (ví dụ: mất trung bình 2 phút để ghi chép thủ công một đơn hàng phức tạp, một điểm nghẽn rõ ràng chưa ai từng nêu ra).
- **Document Analysis**: xem lại mẫu hoá đơn giấy hiện tại, báo cáo doanh thu theo chi nhánh — để hiểu cấu trúc dữ liệu cần có trong hệ thống mới.

12.4 Kỹ thuật đặt câu hỏi trong phỏng vấn

Câu hỏi mở (open question): khuyến khích người trả lời diễn giải, cung cấp thông tin mới bạn chưa nghĩ tới. Dùng ở đầu buổi phỏng vấn hoặc khi cần khám phá.

Ví dụ: "Anh/chị mô tả giúp em quy trình xử lý một đơn hàng từ lúc khách gọi điện đến lúc giao xong hiện nay?"

Câu hỏi đóng (closed question): giới hạn câu trả lời (có/không, chọn 1 trong các lựa chọn). Dùng để xác nhận lại thông tin, thu hẹp phạm vi, hoặc khi thời gian có hạn.

Ví dụ: "Vậy có nghĩa là hiện tại chưa có cách nào để khách theo dõi trạng thái đơn hàng, đúng không ạ?"

Kỹ thuật "5 Whys" (5 lần hỏi vì sao): khi nhận một yêu cầu ở mức bề mặt, tiếp tục hỏi "vì sao" để tìm ra vấn đề gốc rễ:

Yêu cầu ban đầu:
'Tôi muốn có app đặt đồ ăn'

Vì sao?

Vì khách hàng đang
đặt qua app bên thứ ba

Vì sao đó là vấn đề?

Vì mất 20-25%
phí hoa hồng mỗi đơn

Vì sao quan trọng?

Vì biên lợi nhuận
đang bị bào mòn

Vậy vấn đề gốc là gì?

Cần kênh bán hàng
trực tiếp, chi phí thấp hơn
— app riêng là **MỘT** giải pháp,
không phải **MỤC TIÊU**

Kỹ thuật này giúp BA tránh sai lầm nghiêm trọng: nhận yêu cầu "xây app" như một mục tiêu tự thân, trong khi mục tiêu thật là "giảm chi phí hoa hồng" — biết điều này giúp BA đề xuất và đánh giá các giải pháp thay thế (ví dụ: đàm phán lại phí với app bên thứ ba) một cách khách quan hơn, thay vì mặc định "xây app riêng" là câu trả lời duy nhất.

Bài tập

1. Chọn 3 kỹ thuật elicitation phù hợp nhất để tìm hiểu nhu cầu của **nhân viên giao hàng** của FoodNow (một nhóm stakeholder chưa được đề cập ở ví dụ 12.3) — giải thích vì sao chọn từng kỹ thuật.
2. Áp dụng kỹ thuật "5 Whys" cho yêu cầu: "Tôi muốn app có tính năng chat trực tiếp với nhà hàng." Viết ra chuỗi câu hỏi "vì sao" và vấn đề gốc rễ bạn tìm ra được (có thể tự giả định câu trả lời hợp lý).

Sai lầm thường gặp

- **Chỉ dùng một kỹ thuật duy nhất cho mọi tình huống** (ví dụ chỉ phỏng vấn, không bao giờ quan sát thực tế) — bỏ lỡ loại thông tin mà kỹ thuật đó không thu được.
- **Đặt toàn câu hỏi đóng ngay từ đầu buổi phỏng vấn**: giới hạn thông tin thu được, bỏ lỡ những điều bạn còn chưa biết để hỏi.
- **Dừng lại ở yêu cầu bề mặt, không hỏi "vì sao"**: dẫn đến xây dựng cái được yêu cầu nhưng sai vấn đề gốc rễ cần giải quyết.
- **Thiết kế survey với câu hỏi dẫn dắt (leading question)**: ví dụ "Anh/chị có đồng ý rằng app hiện tại quá chậm không?" — ép người trả lời theo hướng có sẵn, làm sai lệch kết quả.

Tóm tắt & tiếp theo

Elicitation có nhiều kỹ thuật (interview, workshop, survey, observation, document analysis, brainstorming, focus group) — mỗi kỹ thuật phù hợp với một tình huống khác nhau, và kỹ thuật "5 Whys" giúp tìm ra vấn đề gốc rễ thay vì dừng ở yêu cầu bề mặt. Chương 13 sẽ học cách xác định **nên hỏi ai, khi nào** một cách có hệ thống — thông qua Stakeholder Analysis và ma trận RACI.

Chương 13: Stakeholder Analysis & ma trận RACI

Mục tiêu học

- Xác định và phân loại được stakeholder của một dự án theo mức độ quyền lực và mức độ quan tâm.
 - Xây dựng được ma trận RACI cho một hoạt động/tài liệu cụ thể.
 - Tránh được sai lầm phổ biến: bỏ sót stakeholder quan trọng hoặc tốn quá nhiều thời gian cho stakeholder ít ảnh hưởng.
-

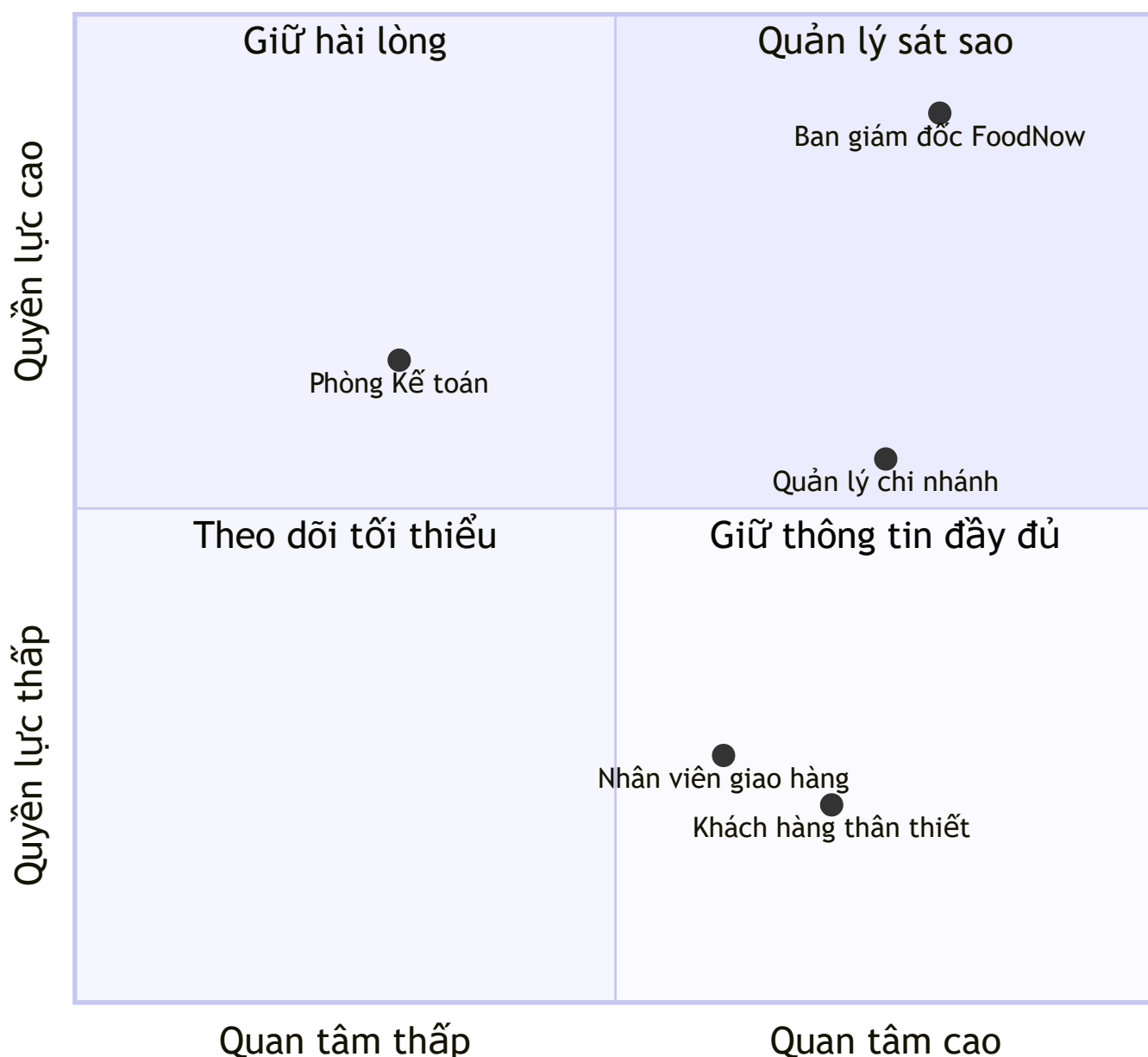
13.1 Vì sao cần phân tích stakeholder trước khi elicitation?

Chương 12 giới thiệu các kỹ thuật thu thập yêu cầu — nhưng trước khi chọn kỹ thuật, cần trả lời: **hỏi ai, và dành bao nhiêu thời gian/công sức cho mỗi người?** Một dự án có thể có hàng chục stakeholder, nhưng không phải ai cũng cần được đầu tư thời gian như nhau. **Stakeholder Analysis** giúp BA phân bổ nguồn lực hợp lý.

13.2 Ma trận Quyền lực — Quan tâm (Power-Interest Grid)

Công cụ kinh điển để phân loại stakeholder theo 2 trục: **mức độ quyền lực/ảnh hưởng** đến dự án, và **mức độ quan tâm** đến kết quả dự án.

Ma trận Quyền lực - Quan tâm



Nhóm	Chiến lược
Quyền lực cao, quan tâm cao (Ban giám đốc)	Quản lý sát sao: cập nhật thường xuyên, thu hút tham gia sâu vào quyết định quan trọng
Quyền lực cao, quan tâm thấp (ví dụ: một cổ đông không trực tiếp điều hành)	Giữ hài lòng: cập nhật định kỳ, không cần chi tiết, nhưng đảm bảo họ không bất ngờ khi có quyết định lớn
Quyền lực thấp, quan tâm cao (Phòng Kế toán — cần hệ thống mới tương thích báo cáo)	Giữ thông tin đầy đủ: lắng nghe ý kiến, cập nhật thường xuyên dù họ không có quyền quyết định cuối cùng
Quyền lực thấp, quan tâm thấp	Theo dõi tối thiểu: cập nhật ở mức tổng quan, không cần đầu tư nhiều thời gian

Lưu ý quan trọng: "**quyền lực thấp**" không có nghĩa là "**không quan trọng để hỏi ý kiến nghiệp vụ**". Nhân viên giao hàng của FoodNow có quyền lực thấp trong việc quyết định dự án, nhưng vẫn là nguồn thông tin elicitation quan trọng (Chương 12) vì họ hiểu rõ nhất vấn đề vận hành thực tế. Ma trận này dùng để quyết định **chiến lược giao tiếp và quản lý kỳ vọng**, không phải để quyết định "ai đáng để lắng nghe yêu cầu".

13.3 Ma trận RACI

RACI là công cụ làm rõ **ai chịu trách nhiệm gì** cho một hoạt động hoặc tài liệu cụ thể — tránh tình trạng "tưởng người khác làm" hoặc nhiều người cùng tưởng mình có quyền quyết định.

- **R — Responsible (Người thực hiện)**: người trực tiếp làm công việc.
- **A — Accountable (Người chịu trách nhiệm cuối cùng)**: người có quyền quyết định cuối cùng, chịu trách nhiệm giải trình về kết quả. **Chỉ nên có một người A cho mỗi hoạt động** — nhiều người "A" cùng lúc dẫn đến không ai thực sự chịu trách nhiệm.
- **C — Consulted (Người được tham vấn)**: người cần được hỏi ý kiến trước khi quyết định, có thông tin đầu vào quan trọng nhưng không quyết định.
- **I — Informed (Người được thông báo)**: người cần biết kết quả sau khi quyết định/hoàn thành, nhưng không cần tham gia trước đó.

Ví dụ: RACI cho việc viết và duyệt tài liệu SRS của FoodNow

Hoạt động	Ban giám đốc	BA	Trưởng nhóm Dev	Quản lý chi nhánh	QA Lead
Thu thập yêu cầu ban đầu	C	R	I	C	I
Viết bản nháp SRS	I	R	C	I	C
Duyệt và ký SRS chính thức	A	R	C	I	I
Review tính khả thi kỹ thuật	I	C	R/A	I	I
Xác nhận SRS khớp quy trình vận hành thực tế	I	R	I	A	I

Đọc bảng này: BA là **Responsible** (người thực hiện) cho việc viết SRS, nhưng **Accountable** (quyền quyết định cuối) về việc **ký duyệt chính thức** thuộc về Ban giám đốc — BA không tự ý "chốt" tài liệu mà không có sự phê duyệt của người có thẩm quyền.

13.4 Cách xây dựng RACI cho dự án của bạn

1. Liệt kê các hoạt động/tài liệu chính cần làm rõ trách nhiệm (thường là các mốc quan trọng: elicitation, viết tài liệu, duyệt tài liệu, review thiết kế, UAT, sign-off).
 2. Liệt kê tất cả vai trò/cá nhân liên quan.
 3. Với mỗi ô giao giữa hoạt động và vai trò, gán đúng một chữ cái R/A/C/I (một hoạt động có thể có nhiều R, nhiều C, nhiều I — nhưng **chỉ nên có đúng 1 A**).
 4. Rà soát lại: có hoạt động nào thiếu "A" không (không ai chịu trách nhiệm cuối)? Có hoạt động nào có nhiều hơn 1 "A" không (dễ gây tranh cãi quyền quyết định)?
-

Bài tập

1. Vẽ ma trận Quyền lực — Quan tâm cho dự án FoodNow, thêm ít nhất 2 stakeholder chưa xuất hiện ở ví dụ mục 13.2 (ví dụ: đối tác giao hàng thuê ngoài, bộ phận Marketing).
 2. Xây dựng bảng RACI cho hoạt động "Nghiệm thu UAT trước khi go-live" (Chương 34-35 sẽ nói chi tiết về UAT/Sign-off) — với các vai trò: Ban giám đốc, BA, QA Lead, Quản lý chi nhánh, Trưởng nhóm Dev.
-

Sai lầm thường gặp

- **Gán nhiều người cùng làm "A" cho một hoạt động:** dẫn đến khi có vấn đề, không ai thực sự chịu trách nhiệm giải trình — mỗi người đều nghĩ "người kia sẽ quyết".
 - **Bỏ sót stakeholder "quyền lực thấp, quan tâm cao":** nhóm này thường có thông tin quý giá và dễ trở thành người phản đối gay gắt nếu cảm thấy bị bỏ rơi trong quá trình.
 - **Nhầm lẫn RACI với sơ đồ tổ chức (org chart):** RACI mô tả trách nhiệm theo **từng hoạt động cụ thể**, một người có thể là "A" ở hoạt động này nhưng chỉ là "C" ở hoạt động khác — không phải một bảng phân cấp chức vụ cố định.
-

Tóm tắt & tiếp theo

Stakeholder Analysis (qua ma trận Quyền lực — Quan tâm) giúp BA phân bổ thời gian giao tiếp hợp lý, còn ma trận RACI làm rõ ai thực hiện, ai chịu trách nhiệm cuối, ai được tham vấn, ai được thông báo cho từng hoạt động cụ thể. Chương 14 sẽ đi vào phân loại các **loại yêu cầu** khác nhau mà BA thu thập được — vì không phải mọi yêu cầu đều cùng một "loại", và việc phân loại đúng ảnh hưởng đến cách tài liệu hoá chúng.

Chương 14: Phân loại yêu cầu: Business, Stakeholder, Solution, Transition

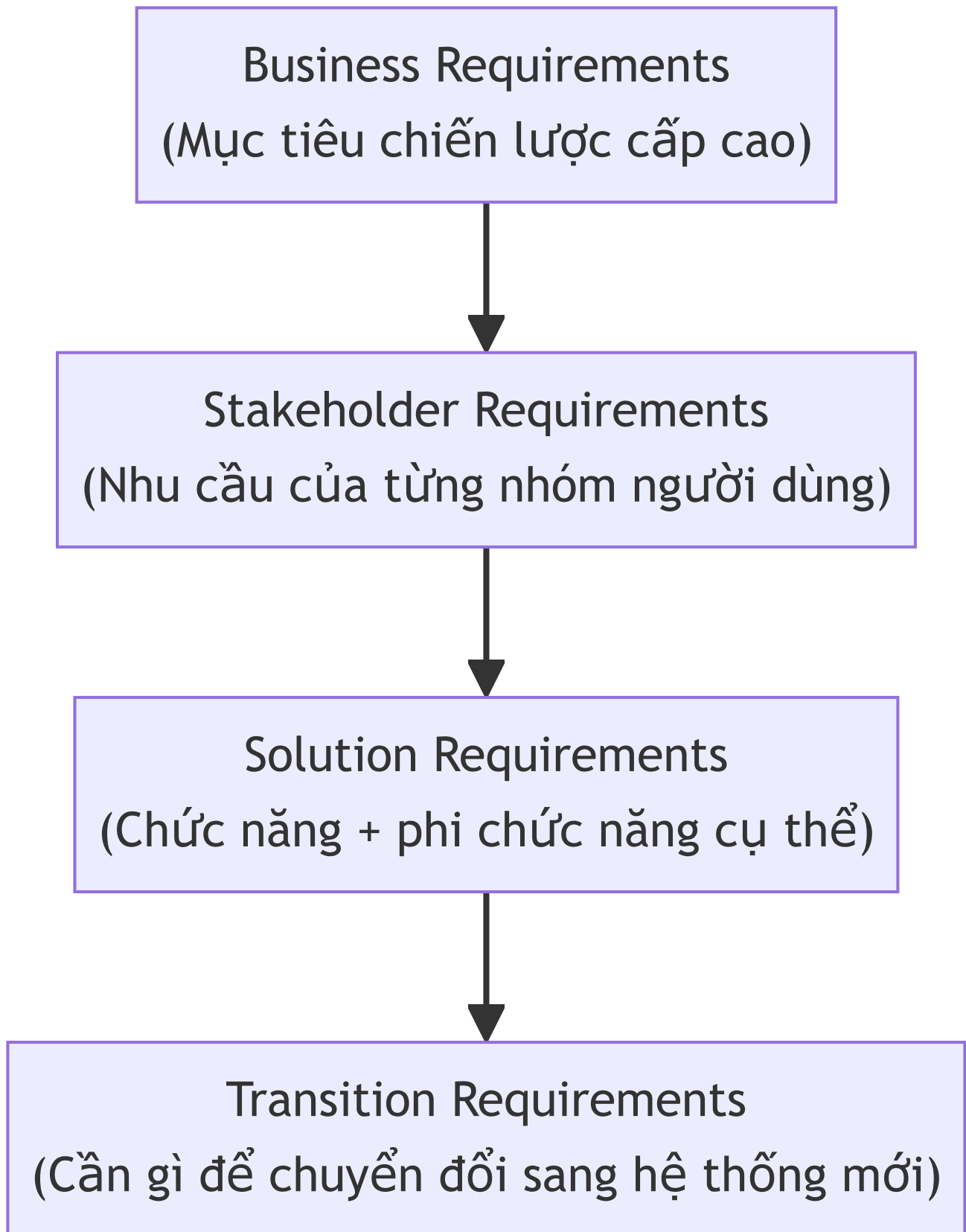
Mục tiêu học

- Phân biệt được 4 loại yêu cầu theo phân loại chuẩn của BABOK: Business, Stakeholder, Solution (Functional/Non-functional), Transition Requirements.
 - Biết mỗi loại yêu cầu nên xuất hiện trong tài liệu nào (liên hệ Chương 19-21).
 - Tránh lỗi phổ biến: trộn lẫn các loại yêu cầu trong cùng một danh sách phẳng, gây khó theo dõi.
-

14.1 Vì sao cần phân loại yêu cầu?

Một danh sách yêu cầu không phân loại thường trộn lẫn nhiều tầng khác nhau — từ mục tiêu chiến lược cấp cao đến chi tiết kỹ thuật cụ thể — khiến người đọc khó biết yêu cầu nào quan trọng hơn, yêu cầu nào phục vụ yêu cầu nào. Phân loại giúp tổ chức tài liệu rõ ràng và giúp BA kiểm tra được **tính đầy đủ**: có yêu cầu Solution nào không bắt nguồn từ một yêu cầu Business/Stakeholder nào không (nếu có, cần đặt câu hỏi "yêu cầu này để làm gì, phục vụ mục tiêu nào?").

14.2 Bốn loại yêu cầu



Business Requirements (Yêu cầu nghiệp vụ)

Mục tiêu chiến lược cấp cao nhất, thường xuất phát từ Sponsor/ban lãnh đạo — trả lời câu hỏi **"vì sao dự án này tồn tại"**.

Ví dụ FoodNow: "Giảm phụ thuộc vào các app giao đồ ăn bên thứ ba, tăng biên lợi nhuận trên mỗi đơn hàng thêm tối thiểu 15% trong vòng 12 tháng sau khi ra mắt app riêng."

Đặc điểm: đo lường được bằng chỉ số kinh doanh (doanh thu, chi phí, thời gian), không mô tả chi tiết tính năng cụ thể. Ghi trong **BRD** (Chương 19).

Stakeholder Requirements (Yêu cầu của các bên liên quan)

Nhu cầu cụ thể của **từng nhóm người dùng/stakeholder**, là cầu nối giữa mục tiêu chiến lược và chi tiết kỹ thuật — trả lời câu hỏi **"nhóm người này cần gì để công việc/trải nghiệm của họ tốt hơn"**.

Ví dụ FoodNow:

- Khách hàng: "Tôi muốn đặt món và thanh toán trong vòng dưới 2 phút."
- Nhân viên chi nhánh: "Tôi muốn nhận đơn hàng mới ngay lập tức mà không cần liên tục kiểm tra điện thoại/màn hình."

Ghi trong **BRD** hoặc phần đầu của **PRD** (Chương 19).

Solution Requirements (Yêu cầu giải pháp)

Chi tiết cụ thể mà hệ thống/sản phẩm phải có để đáp ứng Stakeholder Requirements — chia làm 2 nhánh:

- **Functional Requirements (Yêu cầu chức năng):** hệ thống phải **làm được gì**.

Ví dụ: "Hệ thống phải cho phép khách hàng thêm/xoá/sửa số lượng món trong giỏ hàng trước khi thanh toán."

- **Non-functional Requirements (Yêu cầu phi chức năng):** hệ thống phải **làm tốt đến mức nào** (chất lượng, hiệu năng, bảo mật...) — chi tiết ở Chương 21.

Ví dụ: "Trang xác nhận đơn hàng phải tải xong trong vòng dưới 2 giây với kết nối 4G."

Ghi chi tiết trong **SRS** (Chương 20) và **FRD** (Chương 21).

Transition Requirements (Yêu cầu chuyển đổi)

Những gì cần thiết **chỉ trong giai đoạn chuyển đổi** từ trạng thái cũ sang hệ thống mới, không còn cần thiết sau khi chuyển đổi hoàn tất — trả lời câu hỏi **"làm sao đi từ đây đến đó an toàn"**.

Ví dụ FoodNow: "Cần import toàn bộ dữ liệu khách hàng thân thiết hiện có (đang lưu trên Excel ở từng chi nhánh) vào hệ thống mới trước ngày go-live." hoặc "Cần đào tạo nhân viên 12 chi nhánh sử dụng hệ thống xử lý đơn hàng mới trong vòng 2 tuần trước khi ngừng hệ thống cũ."

Đặc điểm dễ nhận biết: có "ngày hết hạn" tự nhiên — sau khi chuyển đổi xong, yêu cầu này không còn ý nghĩa nữa (khác với Solution Requirements vẫn tồn tại mãi trong sản phẩm).

14.3 Bảng tổng hợp: từ mục tiêu đến chi tiết

Loại yêu cầu	Câu hỏi trả lời	Ví dụ FoodNow	Ghi ở tài liệu nào
Business	Vì sao làm dự án này?	Tăng biên lợi nhuận 15%, giảm phụ thuộc app bên thứ ba	BRD
Stakeholder	Từng nhóm người cần gì?	Khách cần đặt hàng nhanh; nhân viên cần nhận đơn tức thời	BRD/PRD
Solution — Functional	Hệ thống phải làm được gì?	Cho phép sửa giỏ hàng trước khi thanh toán	SRS/FRD
Solution — Non-functional	Hệ thống phải tốt đến mức nào?	Trang xác nhận tải dưới 2 giây	SRS/FRD
Transition	Cần gì để chuyển đổi an toàn?	Import dữ liệu khách hàng cũ, đào tạo nhân viên	Kế hoạch triển khai riêng

14.4 Kiểm tra tính đầy đủ bằng cách "truy ngược"

Một kỹ thuật hữu ích: với mỗi Solution Requirement, thử **truy ngược lên** xem nó phục vụ Stakeholder Requirement nào, và Stakeholder Requirement đó phục vụ Business Requirement nào. Nếu không truy ngược lên được — đây có thể là dấu hiệu của **"gold plating"** (thêm tính năng không ai thực sự cần, thường do Dev hoặc chính BA tự thêm vào vì nghĩ "sẽ hay") hoặc yêu cầu bị lạc mất bối cảnh nghiệp vụ ban đầu.

Ví dụ: nếu trong SRS của FoodNow có yêu cầu "hệ thống hỗ trợ 15 ngôn ngữ khác nhau" nhưng không Stakeholder Requirement/Business Requirement nào nhắc đến việc mở rộng ra thị trường quốc tế — đây là dấu hiệu cần đặt câu hỏi lại: yêu cầu này từ đâu ra, phục vụ mục tiêu gì?

Kỹ thuật truy ngược này chính là nền tảng của **Requirements Traceability Matrix** — công cụ sẽ học chi tiết ở Chương 16.

Bài tập

1. Với dự án FoodNow, viết 1 ví dụ cho mỗi loại trong 4 loại yêu cầu (Business, Stakeholder, Solution-Functional, Solution-Non-functional, Transition) — khác với các ví dụ đã có trong chương này.
 2. Cho yêu cầu sau: "Hệ thống phải cho phép khách hàng đăng nhập bằng vân tay." Thử truy ngược: yêu cầu này có thể phục vụ Stakeholder Requirement nào? Stakeholder Requirement đó phục vụ Business Requirement nào? Nếu không truy ngược được hợp lý, đề xuất câu hỏi bạn sẽ hỏi lại stakeholder.
-

Sai lầm thường gặp

- **Viết Business Requirement quá chi tiết kỹ thuật** (ví dụ: "hệ thống dùng database PostgreSQL") — đây là chi tiết Solution, không phải mục tiêu nghiệp vụ, và ràng buộc kỹ thuật không cần thiết quá sớm.
 - **Viết Solution Requirement không truy ngược được lên Stakeholder/Business Requirement nào** — dấu hiệu của gold plating hoặc yêu cầu bị hiểu sai/thêm ngẫu hứng.
 - **Nhét Transition Requirements chung với Solution Requirements trong SRS**: gây nhầm lẫn vì Transition Requirements có "ngày hết hạn", không nên nằm trong tài liệu mô tả sản phẩm lâu dài.
-

Tóm tắt & tiếp theo

Bốn loại yêu cầu (Business, Stakeholder, Solution, Transition) tạo thành một chuỗi phân cấp từ mục tiêu chiến lược đến chi tiết kỹ thuật cụ thể, và mỗi Solution Requirement nên truy ngược được lên yêu cầu cấp cao hơn mà nó phục vụ. Chương 15 sẽ học cách **mô hình hoá trực quan** các yêu cầu này bằng sơ đồ: Use Case Diagram, Activity Diagram, Sequence Diagram, và BPMN cơ bản.

Chương 15: Mô hình hoá yêu cầu: Use Case, Activity, Sequence Diagram, BPMN cơ bản

Mục tiêu học

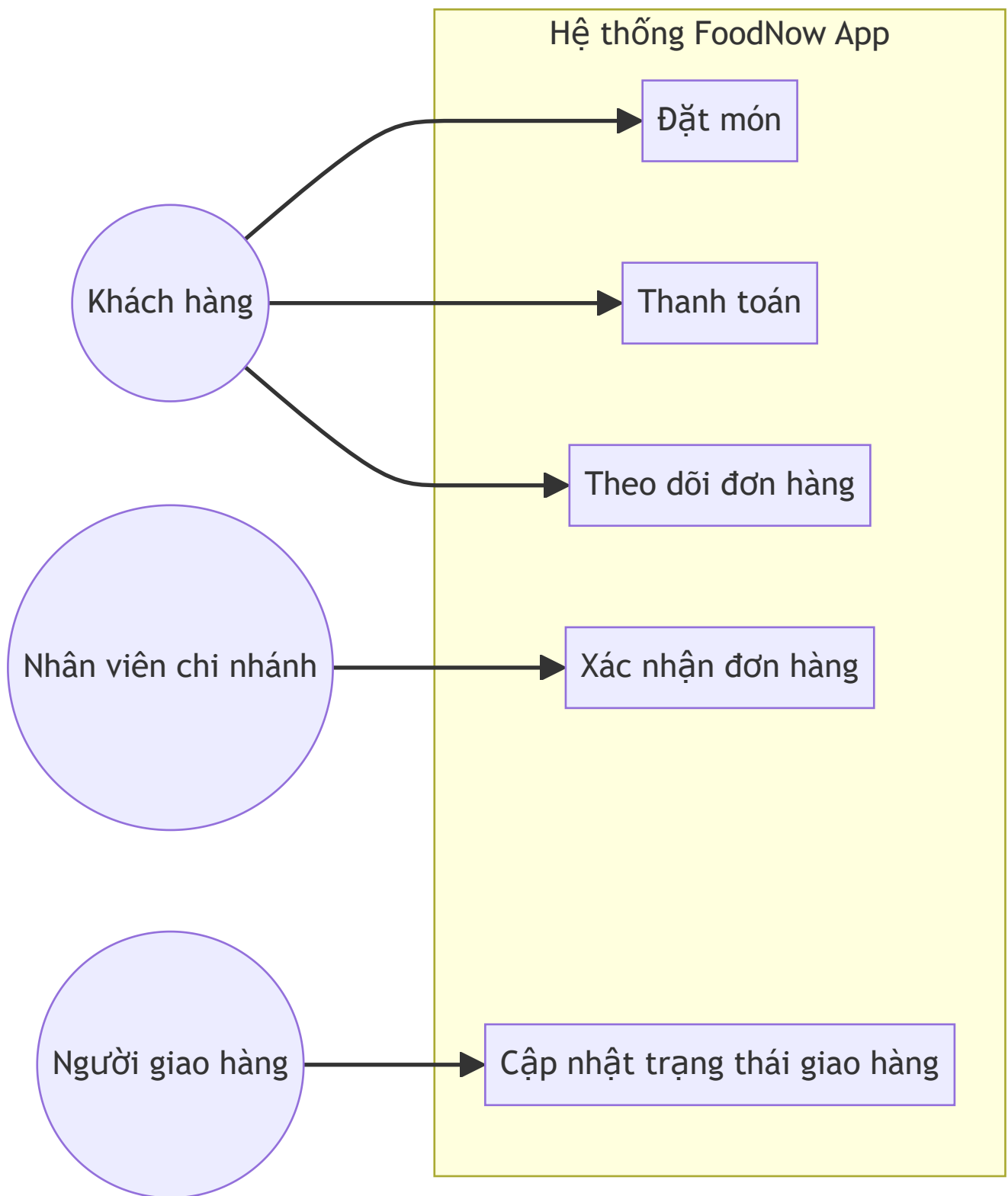
- Vẽ được Use Case Diagram để mô tả tổng quan chức năng hệ thống và actor tương tác.
 - Vẽ được Activity Diagram để mô tả luồng nghiệp vụ có rẽ nhánh, song song.
 - Đọc hiểu được Sequence Diagram mô tả tương tác giữa các thành phần theo thời gian.
 - Vẽ được sơ đồ BPMN cơ bản để mô tả quy trình nghiệp vụ liên phòng ban.
 - Biết chọn đúng loại sơ đồ cho đúng mục đích — lỗi phổ biến nhất của người mới là dùng sai loại sơ đồ cho tình huống không phù hợp.
-

15.1 Vì sao cần mô hình hoá bằng sơ đồ?

Văn bản thuần (như tài liệu Chương 14) tốt để mô tả **chi tiết**, nhưng kém hiệu quả để thể hiện **cấu trúc tổng thể, luồng đi, hoặc tương tác giữa nhiều bên**. Não người xử lý thông tin trực quan nhanh hơn nhiều so với đọc văn bản dài — một sơ đồ tốt giúp stakeholder (kể cả người không rành kỹ thuật) nhìn thấy ngay lỗi hổng logic mà đọc văn bản khó phát hiện ra.

15.2 Use Case Diagram — tổng quan chức năng và actor

Dùng để trả lời: "**hệ thống có những chức năng gì, ai (actor) tương tác với chức năng nào**" — phù hợp cho tài liệu ở mức tổng quan (BRD, phần đầu SRS), trước khi đi vào chi tiết từng use case (Chương 22).



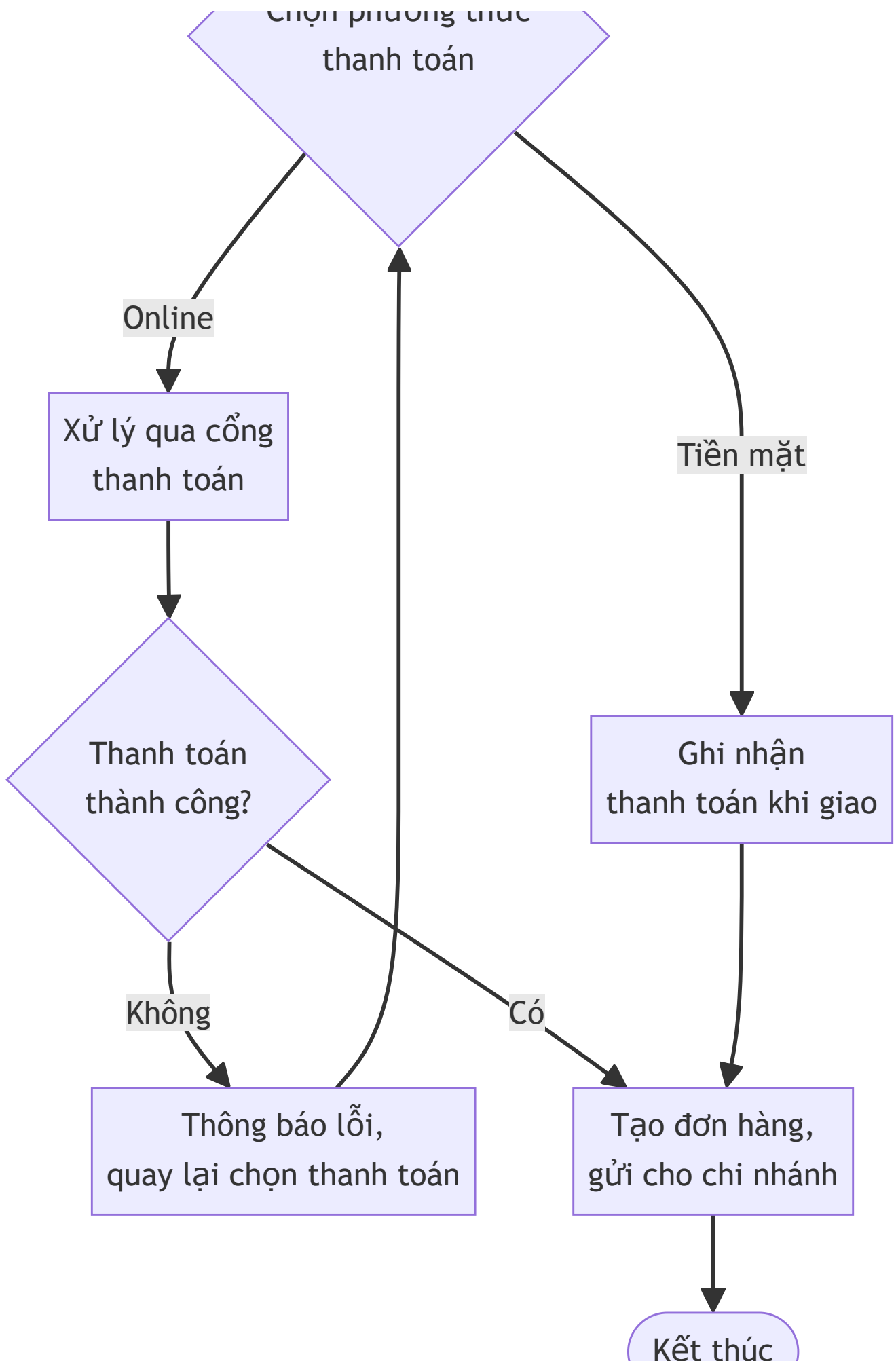
Lưu ý: sơ đồ trên dùng ký hiệu đơn giản hoá (flowchart) thay vì ký hiệu UML Use Case chuẩn (hình que biểu diễn actor, hình elip biểu diễn use case) vì công cụ Mermaid dùng trong sách chưa hỗ trợ đầy đủ ký hiệu UML Use Case. Khi làm việc thực tế, bạn có thể dùng draw.io/Lucidchart/Visio để vẽ đúng ký hiệu UML chuẩn nếu công ty yêu cầu — nhưng **giá trị nằm ở nội dung (actor nào, chức năng nào), không nằm ở việc tuân thủ ký hiệu 100% đúng chuẩn UML.**

Actor là bất kỳ ai/hệ thống nào tương tác với hệ thống từ bên ngoài — có thể là người (Khách hàng, Nhân viên) hoặc hệ thống khác (ví dụ: cổng thanh toán VNPay, hệ thống SMS gửi mã OTP).

15.3 Activity Diagram — luồng nghiệp vụ có rẽ nhánh

Dùng để mô tả **các bước tuần tự, có điều kiện rẽ nhánh, có thể có nhánh song song** — phù hợp mô tả một quy trình nghiệp vụ cụ thể, chi tiết hơn Use Case Diagram.

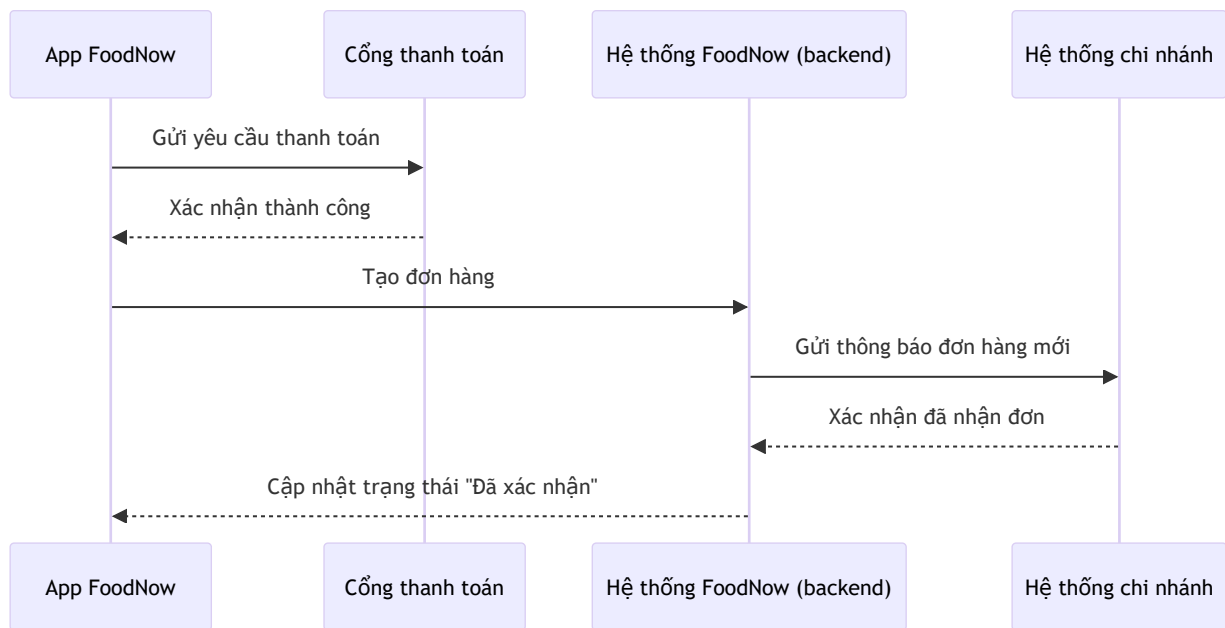




Activity Diagram đặc biệt hữu ích để **phát hiện các trường hợp ngoại lệ (exception) chưa được tính đến** — ví dụ, khi vẽ sơ đồ trên, BA buộc phải trả lời câu hỏi "nếu thanh toán online thất bại thì sao?" — một câu hỏi dễ bị bỏ sót nếu chỉ mô tả bằng văn bản liệt kê các bước "thành công".

15.4 Sequence Diagram — tương tác giữa các thành phần theo thời gian

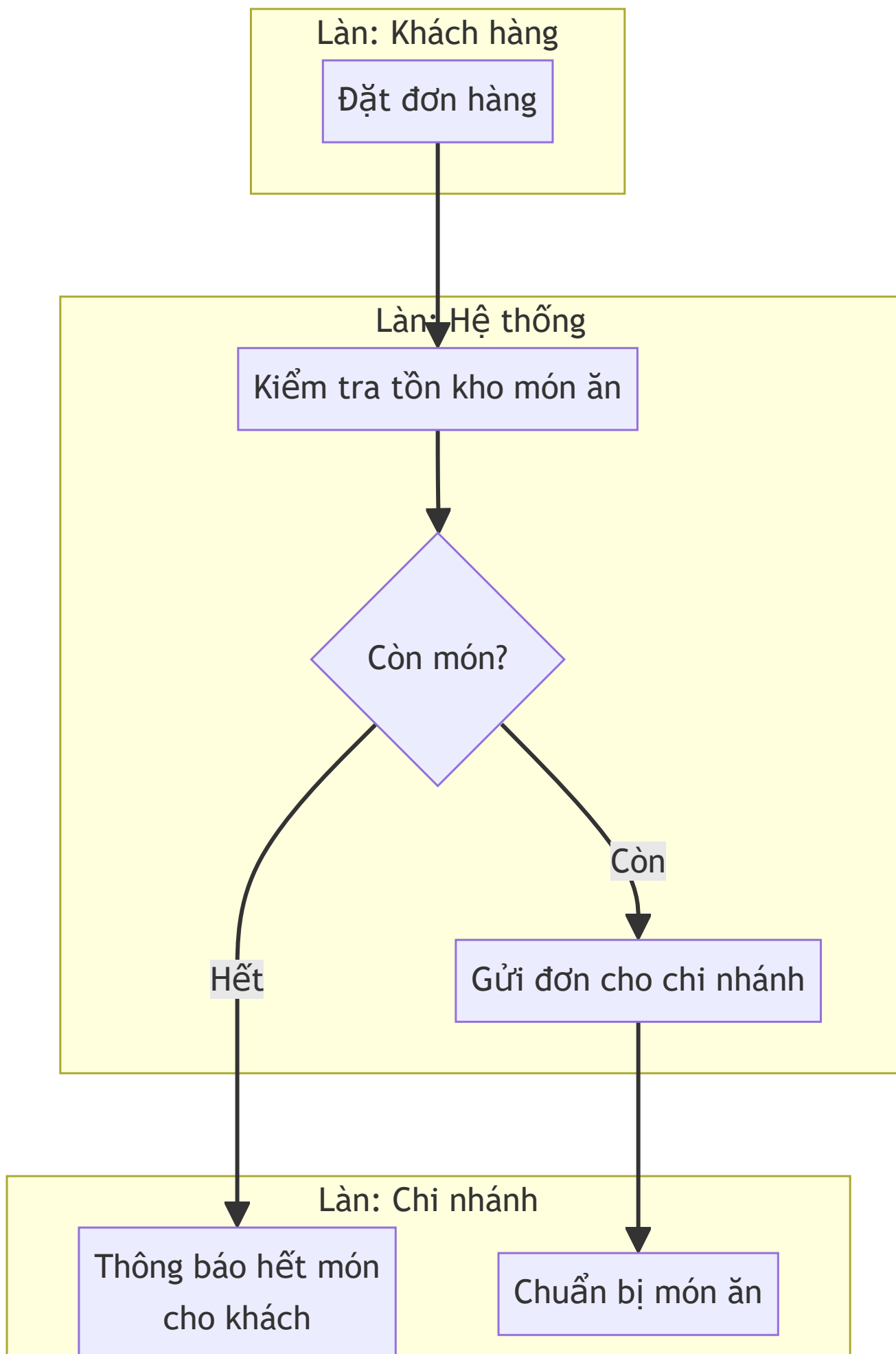
Dùng để mô tả **thứ tự trao đổi thông điệp (message) giữa các đối tượng/hệ thống theo thực thời gian** — hữu ích khi cần làm rõ tương tác giữa nhiều hệ thống (ví dụ: app FoodNow, cổng thanh toán, hệ thống quản lý chi nhánh).



Sequence Diagram thường được BA dùng khi làm việc gần với đội kỹ thuật hơn (ví dụ khi viết SRS chi tiết — Chương 20), vì nó thể hiện rõ **ai gọi ai, theo thứ tự nào** — thông tin quan trọng để Dev thiết kế API/tích hợp hệ thống.

15.5 BPMN cơ bản — quy trình nghiệp vụ liên phòng ban

BPMN (Business Process Model and Notation) là chuẩn ký hiệu chuyên dùng để mô tả quy trình nghiệp vụ, đặc biệt khi quy trình đi **qua nhiều phòng ban/vai trò khác nhau** — dùng "làn bơi" (swimlane) để phân tách trách nhiệm của từng bên.



Khác biệt với Activity Diagram thông thường: BPMN **nhấn mạnh ai làm gì** qua các làn bơi, phù hợp khi cần trình bày cho nhiều phòng ban thấy rõ ranh giới trách nhiệm của mình trong một quy trình chung — rất hữu ích trong workshop có nhiều phòng ban tham gia (Chương 33).

15.6 Chọn đúng loại sơ đồ cho đúng mục đích

Câu hỏi cần trả lời	Dùng sơ đồ nào
Hệ thống có những chức năng gì, ai dùng?	Use Case Diagram
Luồng thao tác chi tiết, có rẽ nhánh điều kiện?	Activity Diagram
Thứ tự trao đổi thông tin giữa các hệ thống/thành phần?	Sequence Diagram
Quy trình đi qua nhiều phòng ban, cần rõ ai chịu trách nhiệm đoạn nào?	BPMN (swimlane)

Bài tập

- Vẽ Use Case Diagram cho tính năng "Quản lý chương trình khách hàng thân thiết" của FoodNow, với ít nhất 2 actor và 3 use case.
- Vẽ Activity Diagram mô tả luồng "khách hàng huỷ đơn hàng" — bao gồm ít nhất một điều kiện rẽ nhánh (ví dụ: huỷ được hay không tùy trạng thái đơn hàng hiện tại).

Sai lầm thường gặp

- Dùng Activity Diagram khi cần thể hiện trách nhiệm liên phòng ban:** nên dùng BPMN với swimlane để rõ ràng hơn về ai làm gì.
- Vẽ sơ đồ quá chi tiết, nhồi nhét mọi trường hợp ngoại lệ vào một sơ đồ duy nhất:** làm sơ đồ rối, khó đọc — nên tách thành nhiều sơ đồ nhỏ theo từng luồng chính/luồng phụ.
- Coi việc vẽ đúng ký hiệu UML/BPMN chuẩn quan trọng hơn nội dung:** với hầu hết đội dự án thực tế, sơ đồ đơn giản hoá để hiểu quan trọng hơn tuân thủ 100% ký hiệu chuẩn — trừ khi công ty có yêu cầu nghiêm ngặt về chuẩn tài liệu.

Tóm tắt & tiếp theo

Bốn loại sơ đồ (Use Case, Activity, Sequence, BPMN) phục vụ các mục đích khác nhau — chọn đúng loại sơ đồ giúp giao tiếp hiệu quả hơn văn bản thuần, và giúp BA tự phát hiện lỗi hổng logic khi vẽ. Chương 16 sẽ học cách dùng **Requirements Traceability Matrix (RTM)** để theo dõi mọi yêu cầu đã thu thập và mô hình hoá được, đảm bảo không yêu cầu nào "biến mất" trong quá trình dự án.

Chương 16: Requirements Traceability Matrix (RTM)

Mục tiêu học

- Giải thích được RTM là gì và giải quyết vấn đề gì.
 - Xây dựng được một RTM đơn giản, liên kết từ Business Requirement đến test case.
 - Dùng RTM để trả lời 2 câu hỏi quan trọng: "yêu cầu này có bị bỏ sót không?" và "nếu thay đổi yêu cầu này, những gì khác bị ảnh hưởng?"
-

16.1 RTM là gì, giải quyết vấn đề gì?

Requirements Traceability Matrix (RTM) là một bảng theo dõi, liên kết mỗi yêu cầu với các "đối tượng" liên quan xuyên suốt vòng đời dự án: yêu cầu đó bắt nguồn từ đâu (nguồn), được thiết kế ở đâu, được code ở phần nào, được test bởi test case nào. Nói cách khác, RTM là hiện thực hoá kỹ thuật "truy ngược" đã giới thiệu ở Chương 14, nhưng mở rộng theo cả 2 chiều: **truy ngược lên** (yêu cầu này từ đâu ra) và **truy xuôi xuống** (yêu cầu này ảnh hưởng đến gì ở các giai đoạn sau).

RTM giải quyết 2 vấn đề lớn:

1. **Không bỏ sót yêu cầu:** mọi yêu cầu trong BRD/SRS đều phải xuất hiện trong RTM, và cuối dự án phải có ít nhất một test case xác nhận yêu cầu đó đã được hiện thực đúng — nếu một yêu cầu không có dòng nào trong RTM khớp với test case, đó là dấu hiệu yêu cầu đó **có thể đã bị quên**.
2. **Quản lý ảnh hưởng khi có thay đổi (impact analysis):** khi một yêu cầu thay đổi (Chương 18), RTM giúp trả lời nhanh "những use case, thiết kế, đoạn code, test case nào cần xem lại?" thay vì phải dò tìm thủ công trong toàn bộ tài liệu.

16.2 Cấu trúc một RTM đơn giản

ID yêu cầu	Mô tả yêu cầu	Nguồn (Business/Stakeholder)	Use Case liên quan	Thiết kế/Module	Test Case ID	Trạng thái
BO-01	Giảm phụ thuộc app bên thứ ba, tăng biên lợi nhuận	BRD, mục 2 (Ban giám đốc)	—	—	—	Mục tiêu tổng thể
BR-01	Khách hàng đặt và thanh toán trong dưới 2 phút	BO-01	UC-01 Đặt món	Module Giỏ hàng	TC-01, TC-02	Đã hiện thực
FR-CART-03	Hệ thống cho phép sửa số lượng hoặc xoá món trong giỏ hàng	BR-01	UC-01 Đặt món	Module Giỏ hàng	TC-03	Đã hiện thực
FR-PAY-04	Hệ thống cho phép áp dụng mã giảm giá khi thanh toán	BR-01	UC-01 Đặt món	Module Thanh toán	TC-05, TC-06	Đang phát triển
NFR-01	Màn hình thực đơn tải xong dưới 2 giây (4G)	BR-01	UC-01 Đặt món	Module Giỏ hàng	TC-04	Chưa test hiệu năng

Đọc theo hàng ngang, mỗi dòng cho biết **toàn bộ hành trình** của một yêu cầu — từ mục tiêu kinh doanh (cột "Nguồn"), qua use case, module thiết kế, đến test case xác nhận. Đọc theo cột dọc, "Trạng thái" giúp nhanh chóng thấy yêu cầu nào còn thiếu bước nào (ví dụ NFR-01 "chưa test hiệu năng" dù đã có thiết kế và ID test case — cần theo dõi tiếp).

(ID dùng trong bảng trên khớp với ID chính thức sẽ dùng lại ở BRD — Chương 19 và SRS — Chương 20, để bạn thấy rõ cùng một yêu cầu được truy vết xuyên suốt nhiều tài liệu như thế nào.)

16.3 Ví dụ dùng RTM để phát hiện yêu cầu bị bỏ sót

Giả sử BRD của FoodNow có thêm một yêu cầu chưa được liệt kê ở ví dụ Chương 19: "BR-06: Nhân viên chi nhánh cần in được hoá đơn cho khách khi đến lấy tại quán (không giao hàng)." Khi rà soát RTM ở cuối giai đoạn Design, BA phát hiện **không có dòng Functional Requirement nào truy ngược về BR-06** — đây là dấu hiệu rõ ràng: yêu cầu này đã bị bỏ sót khi viết SRS chi tiết, cần bổ sung ngay trước khi Dev bắt đầu code, thay vì để đến giai đoạn Testing mới phát hiện thiếu (nhớ lại chi phí sửa lỗi tăng theo giai đoạn ở Chương 2).

16.4 Ví dụ dùng RTM để phân tích ảnh hưởng khi thay đổi

Giả sử giữa dự án, Ban giám đốc FoodNow yêu cầu thay đổi: "Không giới hạn chỉ 1 mã giảm giá/đơn hàng nữa — cho phép áp dụng tối đa 2 mã cùng lúc." Đây là thay đổi trực tiếp lên FR-PAY-04. BA tra RTM, lọc theo Module "Thanh toán", thấy ngay: thay đổi này ảnh hưởng đến UC-01 (Đặt món), các Test Case TC-05, TC-06 cần viết lại, và cần rà soát thêm các yêu cầu phi chức năng liên quan đến hiệu năng module Thanh toán (nhóm NFR trong SRS — Chương 20) vì logic tính giá phức tạp hơn có thể ảnh hưởng thời gian phản hồi. Nhờ RTM, BA đưa ra được đánh giá ảnh hưởng nhanh và đầy đủ hơn nhiều so với việc dò trí nhớ hoặc đọc lại toàn bộ tài liệu.

16.5 Khi nào cần RTM đầy đủ, khi nào không cần?

RTM đầy đủ, chi tiết như bảng ở mục 16.2 phù hợp nhất với:

- Dự án theo Waterfall (Chương 6), nơi tài liệu được chốt và cần theo dõi chặt chẽ suốt dự án dài.
- Ngành có yêu cầu tuân thủ pháp lý nghiêm ngặt (Banking, Y tế, Hàng không) — cần bằng chứng mọi yêu cầu đều được kiểm thử đầy đủ cho mục đích audit.
- Dự án quy mô lớn, nhiều module, nhiều người tham gia qua thời gian dài — trí nhớ cá nhân không đủ để theo dõi toàn bộ mối liên hệ.

Với dự án Agile nhỏ, nhịp độ nhanh (Chương 7-11), RTM đầy đủ như trên có thể **quá nặng nề** — công cụ như Jira thường tự động tạo liên kết truy vết ở mức nhẹ hơn (Epic → Story → Task → Test linked) mà không cần một bảng Excel riêng biệt (xem thêm Chương 37).

Bài tập

1. Xây dựng một RTM (theo mẫu bảng mục 16.2) cho 3 yêu cầu bất kỳ của tính năng "Chương trình khách hàng thân thiết" của FoodNow — từ Business Requirement đến Test Case.
 2. Giả sử FoodNow quyết định bỏ hẳn tính năng thanh toán tiền mặt khi giao hàng (COD), chỉ cho thanh toán online. Dùng tư duy RTM, liệt kê các use case/module/test case có khả năng bị ảnh hưởng bởi thay đổi này.
-

Sai lầm thường gặp

- **Xây RTM rồi không cập nhật khi có thay đổi:** RTM lỗi thời gây hại hơn không có RTM, vì tạo cảm giác an toàn giả (tưởng đã theo dõi đầy đủ nhưng thực ra dữ liệu đã cũ).
- **Áp dụng RTM đầy đủ cho mọi dự án bất kể quy mô:** gây lãng phí thời gian cho dự án nhỏ, nhịp độ nhanh — cần cân nhắc mức độ chi tiết phù hợp (mục 16.5).
- **Chỉ dùng RTM một chiều (chỉ truy xuôi hoặc chỉ truy ngược):** mất đi một nửa giá trị — cần dùng cả hai chiều để vừa kiểm tra tính đầy đủ, vừa phân tích ảnh hưởng khi thay đổi.

Tóm tắt & tiếp theo

RTM là bảng liên kết yêu cầu xuyên suốt vòng đời dự án, giúp phát hiện yêu cầu bị bỏ sót và phân tích ảnh hưởng khi có thay đổi — mức độ chi tiết cần điều chỉnh theo quy mô và mô hình dự án. Chương 17 sẽ học cách xác nhận yêu cầu đã thu thập là **"đúng"** và **"đủ"** trước khi coi là hoàn thành, thông qua Requirements Validation & Verification.

Chương 17: Requirements Validation & Verification

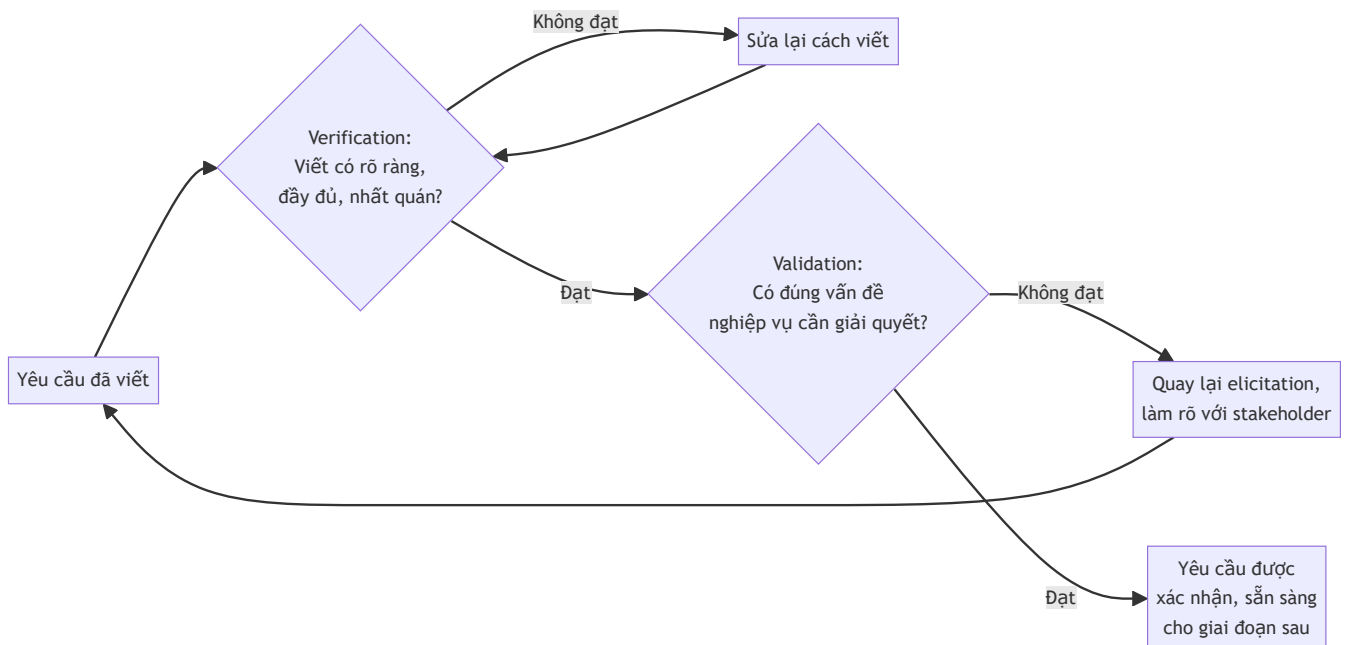
Mục tiêu học

- Phân biệt rõ ràng Validation ("có đúng vấn đề cần giải quyết không") và Verification ("tài liệu có viết đúng chuẩn, đủ rõ ràng không") — hai khái niệm rất dễ nhầm vì tên gọi gần giống nhau.
- Áp dụng được bộ tiêu chí kiểm tra chất lượng một yêu cầu tốt.
- Biết cách tổ chức một buổi review yêu cầu hiệu quả.

17.1 Validation vs Verification — khác nhau ở đâu?

Đây là cặp khái niệm gây nhầm lẫn nhiều nhất trong BABOK vì tên tiếng Anh gần giống nhau. Cách nhớ đơn giản nhất, dùng hai câu hỏi:

- **Verification** — "**Are we building it right?**" (Chúng ta có đang xây **đúng cách** không?): kiểm tra yêu cầu có được viết rõ ràng, đầy đủ, nhất quán, không mâu thuẫn — kiểm tra **chất lượng của bản thân tài liệu**, không quan tâm nó có giải quyết đúng vấn đề nghiệp vụ hay không.
- **Validation** — "**Are we building the right thing?**" (Chúng ta có đang xây **đúng thứ** không?): kiểm tra yêu cầu (dù viết rõ ràng hay không) có thực sự giải quyết đúng vấn đề nghiệp vụ, đúng mong đợi của stakeholder hay không.



Ví dụ minh họa khác biệt: yêu cầu "Hệ thống phải cho phép khách hàng huỷ đơn hàng bất cứ lúc nào, kể cả khi đã giao xong" — yêu cầu này có thể **verify đạt** (viết rõ ràng, không mơ hồ, dùng đúng thuật ngữ) nhưng **validate không đạt** (về mặt nghiệp vụ, cho phép huỷ đơn sau khi đã giao xong là vô lý, gây thiệt hại cho nhà hàng — rõ ràng đây là lỗi hiểu sai ý định thật của stakeholder, cần quay lại elicitation làm rõ).

17.2 Bộ tiêu chí Verification — thế nào là một yêu cầu viết tốt?

Tiêu chí	Giải thích	Ví dụ VI PHẠM	Sửa lại
Rõ ràng (Unambiguous)	Chỉ hiểu theo một nghĩa duy nhất	"Hệ thống phải nhanh"	"Trang chủ tải xong trong dưới 2 giây với kết nối 4G"
Đầy đủ (Complete)	Không thiếu thông tin cần thiết để hiện thực	"Gửi email xác nhận đơn hàng" (gửi khi nào? nội dung gì?)	"Ngay sau khi thanh toán thành công, gửi email xác nhận gồm: mã đơn hàng, danh sách món, tổng tiền, thời gian giao dự kiến"
Nhất quán (Consistent)	Không mâu thuẫn với yêu cầu khác trong cùng tài liệu	Yêu cầu A nói "tối đa 5 món/đơn", yêu cầu B (cùng tài liệu) nói "không giới hạn số món"	Rà soát và thống nhất lại với stakeholder trước khi chốt
Khả thi (Feasible)	Có thể hiện thực được trong ràng buộc thời gian/ngân sách/công nghệ	"Giao hàng trong vòng 5 phút mọi lúc, mọi nơi"	"Giao hàng trong vòng 30 phút trong bán kính 5km, với các đơn đặt trong giờ hoạt động"
Đo lường được (Measurable/Testable)	Có thể viết được test case để xác nhận đạt/không đạt	"Giao diện phải đẹp"	"Giao diện tuân thủ bộ nhận diện thương hiệu FoodNow (màu, font đính kèm ở Phụ lục), đã được Designer duyệt"
Có thể truy vết (Traceable)	Có thể liên kết lên nguồn gốc và xuống đến thiết kế/test (liên hệ Chương 16)	Yêu cầu xuất hiện đột ngột, không rõ ai yêu cầu, để làm gì	Ghi rõ nguồn: "Theo yêu cầu của Phòng Marketing trong workshop ngày X"

17.3 Kỹ thuật kiểm tra Validation

Verification có thể tự BA làm một mình (đọc lại, đối chiếu tiêu chí). Validation **bắt buộc cần có sự tham gia của stakeholder** — không thể tự BA "tự xác nhận" yêu cầu là đúng vấn đề nghiệp vụ:

- **Review trực tiếp với stakeholder:** trình bày lại yêu cầu bằng ngôn ngữ của họ (không phải ngôn ngữ kỹ thuật), hỏi "đây có đúng ý anh/chị không?"
- **Prototype/Wireframe** (Chương 25): cho stakeholder xem hình ảnh cụ thể thay vì chỉ đọc văn bản — nhiều người validate chính xác hơn khi nhìn thấy giao diện thay vì đọc mô tả trừu tượng.
- **Walkthrough kịch bản cụ thể:** đi qua từng bước một tình huống thực tế ("giả sử khách đặt 3 món, 1 món hết hàng giữa chừng, thì...") để kiểm tra yêu cầu có xử lý đúng các trường hợp thật.

17.4 Tổ chức một buổi review yêu cầu hiệu quả

1. **Gửi tài liệu trước ít nhất 1-2 ngày:** để người tham dự có thời gian đọc trước, buổi họp dùng để thảo luận sâu chứ không phải đọc lần đầu tại chỗ.
2. **Xác định rõ mục tiêu buổi review:** verification (rà soát cách viết) hay validation (xác nhận đúng vấn đề nghiệp vụ) — hai mục tiêu này cần cách tổ chức khác nhau, verification có thể làm qua checklist nhanh, validation cần thảo luận sâu với đúng stakeholder có thẩm quyền.
3. **Dùng checklist tiêu chí (mục 17.2)** làm khung thảo luận, tránh review lan man không có cấu trúc.
4. **Ghi nhận mọi phản hồi, kể cả chưa thống nhất được ngay:** đưa vào danh sách vấn đề mở (open items), phân công người theo dõi, hạn chót.
5. **Không rời phòng họp mà không có quyết định rõ ràng:** mỗi yêu cầu được review nên có 1 trong 3 trạng thái cuối: Chấp nhận, Cần sửa (có ghi rõ sửa gì), hoặc Cần làm rõ thêm (có ghi rõ ai sẽ làm rõ, khi nào).

Bài tập

1. Cho yêu cầu: "Hệ thống phải bảo mật tốt." Áp dụng bộ tiêu chí Verification ở mục 17.2, chỉ ra yêu cầu này vi phạm tiêu chí nào, và viết lại thành một yêu cầu đạt chuẩn.
2. Với yêu cầu (đã viết rõ ràng): "Khách hàng có thể đặt tối đa 20 đơn hàng cùng lúc trong một ngày." — yêu cầu này **verify đạt** nhưng có khả năng **validate không đạt**. Giải thích vì sao, và đề xuất câu hỏi bạn sẽ hỏi lại stakeholder để làm rõ ý định thật.

Sai lầm thường gặp

- **Chỉ làm Verification, bỏ qua Validation:** tài liệu viết rất chuyên nghiệp, rõ ràng, đầy đủ — nhưng giải quyết sai vấn đề, vì không ai xác nhận lại với đúng stakeholder có thẩm quyền.
- **Tự mình "validate" mà không có stakeholder tham gia:** BA tự cho là đúng theo hiểu biết cá nhân — đây thực chất vẫn chỉ là verification (kiểm tra logic nội tại), không phải validation thật sự.
- **Review yêu cầu quá muộn, sau khi Design/Implementation đã bắt đầu:** mất đi lợi ích lớn nhất của validation — phát hiện sai sót càng sớm, chi phí sửa càng thấp (nhắc lại nguyên lý Chương 2).

Tóm tắt & tiếp theo

Verification kiểm tra "viết có đúng cách không" (rõ ràng, đầy đủ, nhất quán, khả thi, đo lường được, truy vết được), Validation kiểm tra "có đúng vấn đề nghiệp vụ không" — cần phân biệt rõ và làm cả hai, không thể thay thế cho nhau. Chương 18 sẽ học cách quản lý khi yêu cầu **thay đổi** giữa dự án — điều gần như chắc chắn sẽ xảy ra, dù dự án làm theo mô hình nào.

Chương 18: Quản lý thay đổi yêu cầu: scope creep, Change Request

Mục tiêu học

- Định nghĩa được "scope creep" và phân biệt với thay đổi yêu cầu hợp lý, có kiểm soát.
- Thực hiện được quy trình Change Request đầy đủ: đề xuất, đánh giá tác động, phê duyệt.
- Biết cách xử lý thay đổi yêu cầu khác nhau giữa dự án Waterfall và Agile.

18.1 Scope creep là gì?

Scope creep ("phạm vi phình to không kiểm soát") là hiện tượng phạm vi dự án tăng dần theo thời gian mà **không qua đánh giá tác động và phê duyệt chính thức** — thường bắt đầu từ những yêu cầu nhỏ, có vẻ "tiện thể làm luôn" nhưng cộng dồn lại gây trễ tiến độ, vượt ngân sách nghiêm trọng.

Ví dụ FoodNow: trong lúc demo tính năng đặt món, một quản lý chi nhánh buột miệng "tiện thể làm luôn tính năng đặt bàn trước cho khách muốn ăn tại quán nhé, chắc không mất nhiều thời gian đâu." Nếu BA/Dev đồng ý làm ngay mà không đánh giá tác động, không cập nhật kế hoạch/ngân sách, không có ai phê duyệt chính thức — đây chính là scope creep, dù bản thân yêu cầu đó có thể hợp lý.

Điểm mấu chốt: **vấn đề không nằm ở việc thay đổi yêu cầu** (thay đổi là bình thường, đặc biệt trong Agile — Chương 7) — vấn đề nằm ở việc thay đổi **không qua kiểm soát, không ai đánh giá tác động, không ai chính thức phê duyệt đánh đổi**.

18.2 Quy trình Change Request (CR)

1. Đề xuất thay đổi
(ai cũng có thể đề xuất)



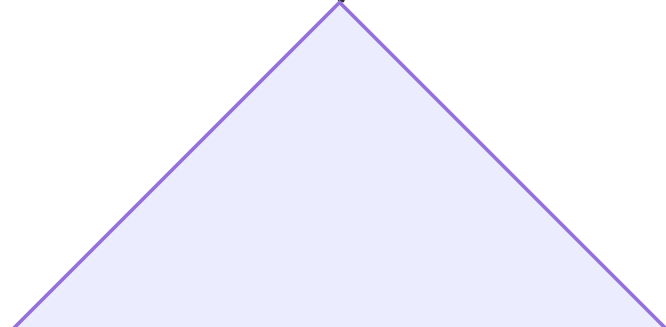
2. Ghi nhận vào
Change Request Log

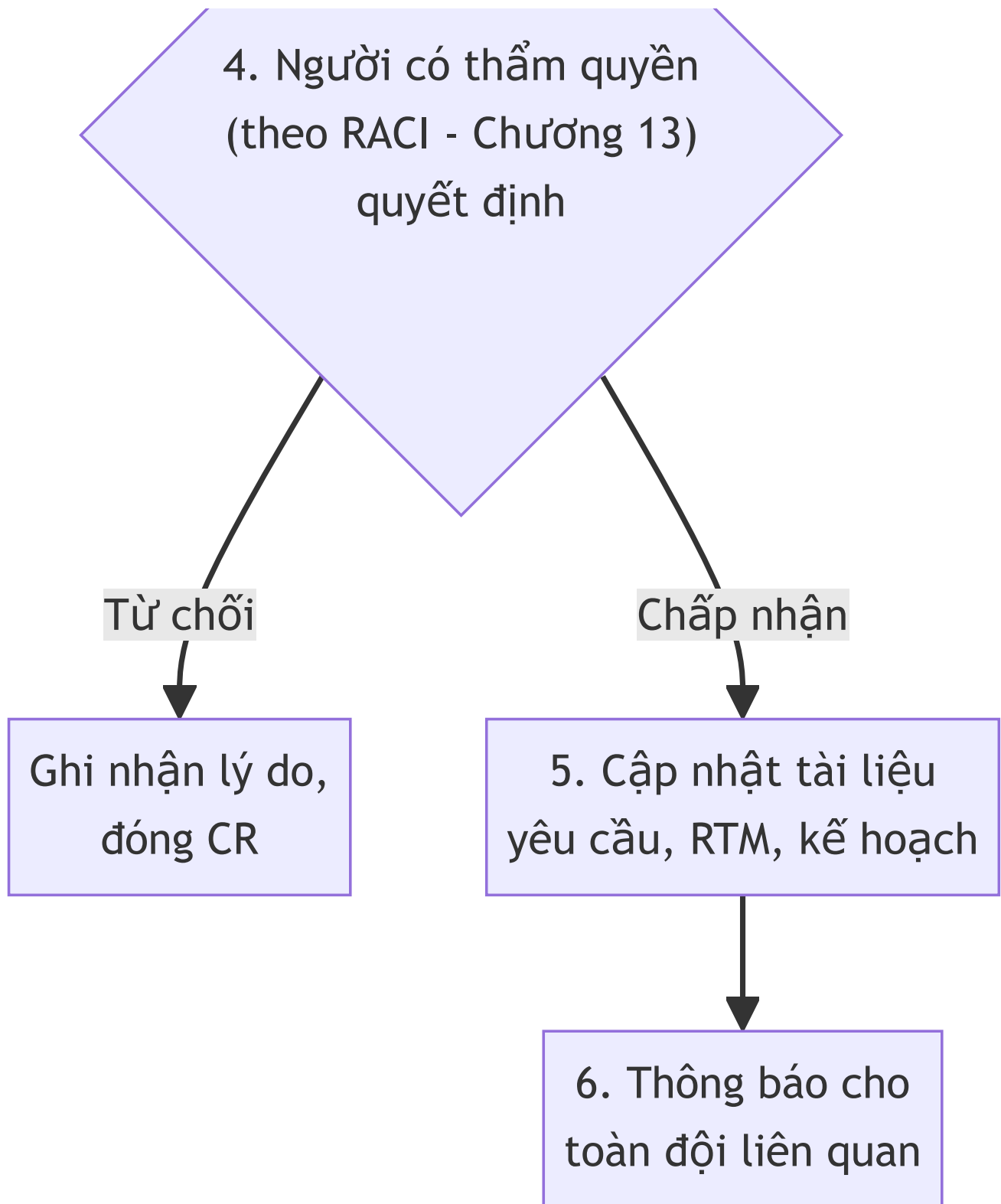


3. BA phân tích tác động
(dùng RTM - Chương 16)



Tác động: phạm vi,
thời gian, chi phí, rủi ro





Bước 1-2: Đề xuất và ghi nhận

Bất kỳ ai (Sponsor, Dev, người dùng cuối) đều có thể đề xuất thay đổi — nhưng **mọi đề xuất đều phải được ghi nhận chính thức** vào một "Change Request Log" (có thể đơn giản là một bảng Excel hoặc một loại issue riêng trong Jira), không xử lý miệng, không quyết định ngẫu hứng tại chỗ.

Bước 3: Phân tích tác động (Impact Analysis)

Đây là bước quan trọng nhất và là nơi RTM (Chương 16) phát huy giá trị lớn nhất. BA cần trả lời:

- **Phạm vi:** thay đổi này ảnh hưởng đến những use case/module nào khác?
- **Thời gian:** cần thêm bao lâu để hiện thực thay đổi này?
- **Chi phí:** có cần thêm nguồn lực không?
- **Rủi ro:** thay đổi này có xung đột với yêu cầu nào đã có không? Có ảnh hưởng đến các cam kết đã đưa ra với stakeholder khác không?

Bước 4: Phê duyệt

Ai có quyền phê duyệt phụ thuộc vào RACI đã xác định (Chương 13) — thường là Sponsor cho thay đổi lớn ảnh hưởng ngân sách/tiến độ đáng kể, hoặc PO cho thay đổi trong phạm vi backlog ở dự án Agile.

Bước 5-6: Cập nhật và thông báo

Sau khi được phê duyệt, cần cập nhật **toàn bộ tài liệu liên quan** (BRD/SRS, RTM, kế hoạch dự án) và thông báo rõ ràng cho tất cả các bên bị ảnh hưởng — tránh tình trạng một phần đội vẫn làm theo yêu cầu cũ vì không biết đã có thay đổi.

18.3 Change Request Log — mẫu đơn giản

CR ID	Ngày đề xuất	Người đề xuất	Mô tả thay đổi	Tác động (tóm tắt)	Người phê duyệt	Trạng thái
CR-01	2024-03-10	Quản lý chi nhánh Q1	Thêm tính năng đặt bàn trước	+2 tuần, +1 Dev, không ảnh hưởng module hiện có	Ban giám đốc	Đã phê duyệt
CR-02	2024-03-15	Phòng Marketing	Đổi giao diện trang chủ theo bộ nhận diện mới	+3 ngày, cần Designer làm lại	PO	Đang đánh giá

18.4 Xử lý thay đổi khác nhau giữa Waterfall và Agile

	Waterfall	Agile (Scrum/Kanban)
Tần suất thay đổi được chấp nhận	Thấp, cần lý do chính đáng vì ảnh hưởng SRS đã ký	Cao hơn — coi thay đổi là bình thường (Chương 7)
Ai phê duyệt	Thường cần Sponsor/Ban chỉ đạo dự án chính thức	Thường PO đủ thẩm quyền cho thay đổi trong phạm vi sản phẩm
Khi nào áp dụng thay đổi	Ngay khi phê duyệt, có thể ảnh hưởng công việc đang làm dở	Đưa vào backlog, ưu tiên và lên kế hoạch cho sprint tiếp theo — hạn chế đổi Sprint Backlog đang chạy
Tài liệu cần cập nhật	SRS, RTM, kế hoạch dự án đầy đủ	User Story mới/cập nhật trong backlog, ít khi cần "ký duyệt" hình thức

Trong Agile, quy trình CR thường **nhẹ hơn nhiều** — nhiều đội không gọi đó là "Change Request" mà chỉ đơn giản là "thêm một User Story mới vào backlog, PO sắp xếp độ ưu tiên." Tuy nhiên, bản chất vẫn cần đi qua các bước tương tự (ghi nhận, đánh giá tác động, quyết định có thẩm quyền) — chỉ là hình thức nhẹ nhàng, nhanh hơn.

Bài tập

- Xử lý tình huống ở mục 18.1 (yêu cầu đặt bàn trước) theo đúng quy trình 6 bước ở mục 18.2 — viết ra nội dung cụ thể cho từng bước (giả định hợp lý các con số về thời gian/chi phí).
- Giải thích bằng lời của bạn: vì sao "thay đổi yêu cầu" trong Agile không bị coi là scope creep, trong khi thay đổi tương tự trong Waterfall (nếu không qua CR chính thức) lại là scope creep?

Sai lầm thường gặp

- **Từ chối mọi thay đổi một cách cứng nhắc:** đi ngược tinh thần thích ứng cần thiết, đặc biệt trong Agile — không phải mọi thay đổi đều xấu, chỉ thay đổi **không kiểm soát** mới là vấn đề.
- **Chấp nhận thay đổi ngay tại chỗ mà không phân tích tác động:** nguyên nhân phổ biến nhất của scope creep — như ví dụ "tiện thể làm luôn" ở mục 18.1.
- **Không cập nhật tài liệu sau khi CR được phê duyệt:** dẫn đến tài liệu (SRS, RTM) không còn phản ánh đúng thực tế, gây nhầm lẫn về sau.

Tóm tắt & tiếp theo

Scope creep xảy ra khi thay đổi yêu cầu không qua đánh giá tác động và phê duyệt chính thức — quy trình Change Request (đề xuất, ghi nhận, phân tích tác động dùng RTM, phê duyệt, cập nhật, thông báo) giúp kiểm soát điều này, dù mức độ hình thức khác nhau giữa Waterfall và Agile. Đây là chương cuối của Phần 2. Từ Chương 19, sách bước vào Phần 3 — đi sâu vào **cách viết từng loại tài liệu BA cốt lõi**, kèm mẫu tài liệu thật cho dự án FoodNow, bắt đầu với BRD và PRD.

Phần 3 — Tài liệu BA cốt lõi (kèm mẫu điền sẵn)

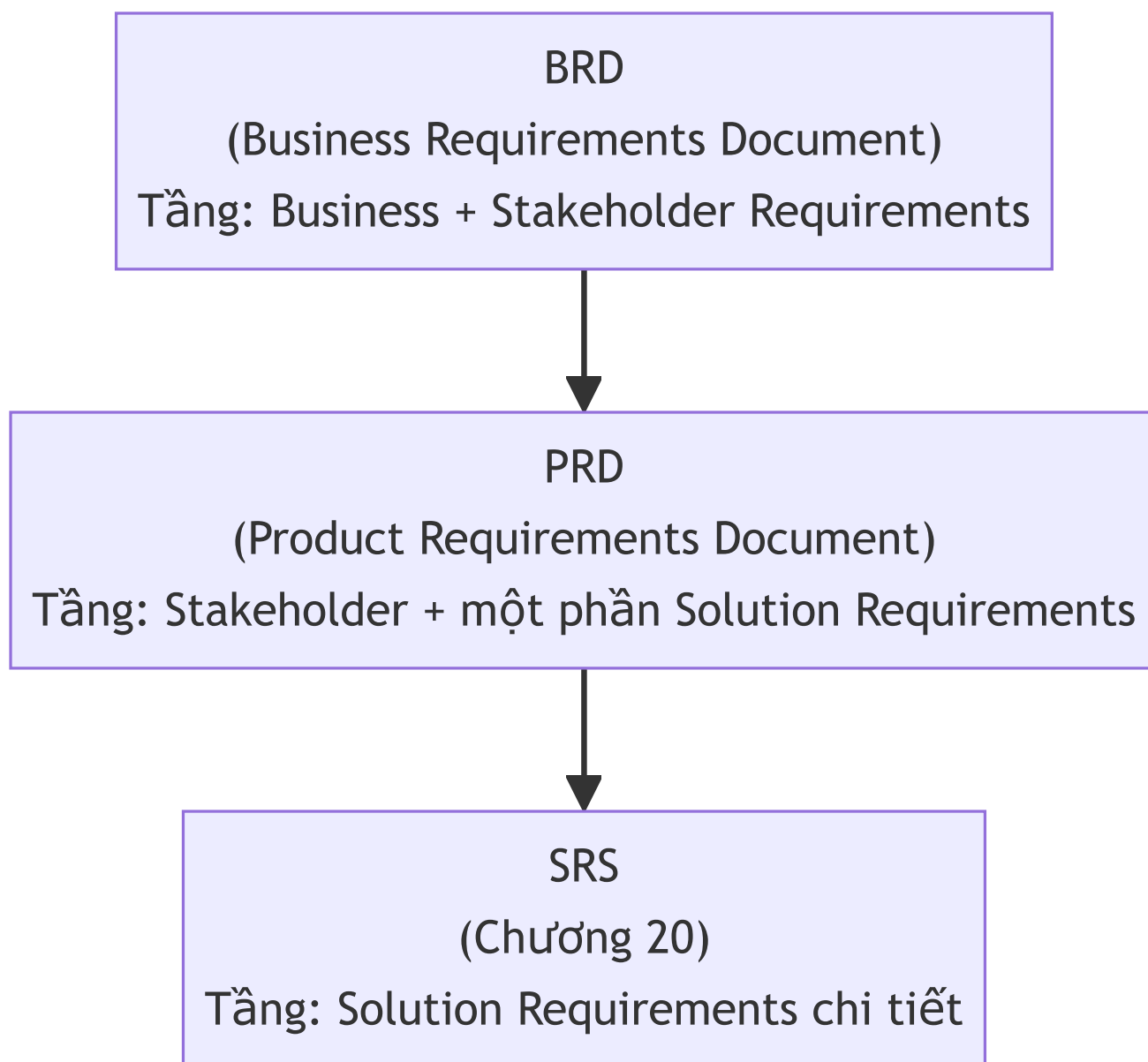
Chương 19: BRD & PRD: khác nhau ở đâu, cấu trúc, mẫu đầy đủ

Mục tiêu học

- Phân biệt rõ BRD (Business Requirements Document) và PRD (Product Requirements Document) — hai tài liệu hay bị dùng lẫn lộn hoặc coi là một.
 - Nắm được cấu trúc chuẩn của từng loại tài liệu.
 - Biết khi nào cần viết cả hai, khi nào chỉ cần một.
 - Có sẵn mẫu tài liệu đầy đủ trong `templates/brd-prd/` để tham khảo và tái sử dụng.
-

19.1 BRD và PRD khác nhau ở đâu?

Đây là cặp tài liệu gây nhầm lẫn phổ biến — nhiều công ty dùng tên gọi khác nhau cho tài liệu có nội dung tương tự, hoặc gộp chung thành một tài liệu. Về bản chất, hai tài liệu này nằm ở hai tầng khác nhau trong chuỗi phân loại yêu cầu đã học ở Chương 14:



	BRD	PRD
Trả lời câu hỏi	Vì sao dự án này tồn tại? Mục tiêu kinh doanh là gì?	Sản phẩm cần có những gì để đạt mục tiêu đó?
Đối tượng đọc chính	Sponsor, Ban lãnh đạo, các phòng ban liên quan	Product Owner, Designer, Dev lead
Mức độ chi tiết kỹ thuật	Rất thấp — tập trung giá trị kinh doanh	Trung bình — mô tả tính năng ở mức khái niệm, chưa đi sâu chi tiết kỹ thuật (đó là việc của SRS)
Ai thường viết	BA (đôi khi cùng PM)	BA hoặc Product Manager/Product Owner
Độ ổn định	Ít thay đổi (mục tiêu kinh doanh thường ổn định)	Có thể điều chỉnh khi hiểu thêm về sản phẩm/thị trường

Cách nhớ đơn giản: BRD trả lời "TẠI SAO", PRD trả lời "CÁI GÌ" (ở mức tính năng/khái niệm), SRS (Chương 20) trả lời "NHƯ THẾ NÀO" (ở mức chi tiết đủ để code).

Trong thực tế, nhiều công ty nhỏ/vừa gộp BRD và PRD thành một tài liệu duy nhất (thường gọi chung là BRD hoặc PRD tùy thói quen công ty) — điều này hoàn toàn hợp lý cho dự án nhỏ. Sách này trình bày tách biệt để bạn hiểu rõ bản chất từng phần, và biết cách gộp/tách linh hoạt tùy tình huống thực tế.

19.2 Cấu trúc chuẩn của BRD

1. **Thông tin chung:** tên dự án, người soạn thảo, ngày, phiên bản, danh sách phê duyệt.
2. **Bối cảnh & vấn đề nghiệp vụ (Business Problem):** vấn đề gì đang tồn tại, vì sao cần giải quyết.
3. **Mục tiêu kinh doanh (Business Objectives):** đo lường được, gắn với chỉ số cụ thể (liên hệ Business Requirements — Chương 14).
4. **Phạm vi (Scope):** trong phạm vi (in scope) và ngoài phạm vi (out of scope) — phần "ngoài phạm vi" quan trọng không kém, tránh hiểu lầm và scope creep (Chương 18).
5. **Đối tượng sử dụng (Stakeholders):** các nhóm liên quan và nhu cầu tổng quan của từng nhóm (liên hệ Stakeholder Analysis — Chương 13).
6. **Yêu cầu nghiệp vụ chi tiết:** danh sách Business + Stakeholder Requirements, có ID để truy vết (liên hệ RTM — Chương 16).
7. **Giả định & ràng buộc (Assumptions & Constraints):** những điều được giả định là đúng, và giới hạn (ngân sách, thời gian, công nghệ bắt buộc).
8. **Rủi ro sơ bộ:** các rủi ro nghiệp vụ đã nhận diện được (chi tiết hơn ở RAID Log — Chương 31).
9. **Tiêu chí thành công (Success Criteria):** làm sao biết dự án đã thành công (liên hệ KPI/OKR — Chương 36).
10. **Phê duyệt (Sign-off):** chữ ký/xác nhận của người có thẩm quyền (Chương 35).

19.3 Cấu trúc chuẩn của PRD

1. **Thông tin chung:** tương tự BRD.
2. **Tóm tắt sản phẩm (Product Overview):** sản phẩm là gì, cho ai, giải quyết vấn đề gì (liên kết ngược lại BRD).
3. **Đối tượng người dùng & Persona:** mô tả cụ thể hơn BRD, có thể có persona chi tiết cho từng nhóm người dùng chính.
4. **Danh sách tính năng (Feature List):** liệt kê các tính năng ở mức khái niệm, thường nhóm theo Epic (liên hệ Chương 24).
5. **Yêu cầu phi chức năng ở mức tổng quan:** hiệu năng, bảo mật, khả năng mở rộng — mức khái quát, chi tiết đầy đủ nằm ở SRS (Chương 20-21).
6. **Ưu tiên hoá tính năng:** MoSCoW hoặc tương tự (Chương 26) — tính năng nào bắt buộc có ở phiên bản đầu, tính năng nào có thể làm sau.
7. **Lộ trình phát hành sơ bộ (Release Roadmap):** liên hệ Incremental Delivery (Chương 28-29).

8. **Các chỉ số đo lường thành công của sản phẩm:** tương tự Success Criteria của BRD nhưng cụ thể hơn ở mức sản phẩm (ví dụ: tỷ lệ chuyển đổi, tỷ lệ giữ chân người dùng).

19.4 Mẫu tài liệu đầy đủ cho FoodNow

Một bộ BRD + PRD đầy đủ, điền sẵn nội dung cho dự án FoodNow, được lưu tại:

`templates/brd-prd/FoodNow-BRD.md` và `templates/brd-prd/FoodNow-PRD.md`

Hai file này dùng được ngay làm mẫu tham khảo — sao chép và điều chỉnh nội dung theo dự án thật của bạn. Phụ lục A (cuối sách) cũng in lại đầy đủ nội dung này để tiện tra cứu khi đọc trọn bộ.

Bài tập

1. Mở file `templates/brd-prd/FoodNow-BRD.md`, đọc kỹ mục "Phạm vi" — thử bổ sung thêm 2 mục vào danh sách "ngoài phạm vi" mà bạn nghĩ là hợp lý cho phiên bản đầu tiên của app FoodNow.
2. Dựa theo cấu trúc PRD ở mục 19.3, viết phần "Danh sách tính năng" cho một sản phẩm bạn tự chọn (có thể là một app/website bạn quen thuộc), nhóm tính năng theo 2-3 Epic.

Sai lầm thường gặp

- **Nhét chi tiết kỹ thuật (API, database) vào BRD:** BRD nên giữ ở mức nghiệp vụ thuần túy, tránh ràng buộc giải pháp kỹ thuật quá sớm — điều này thuộc về SRS (Chương 20).
- **Bỏ qua mục "Ngoài phạm vi" (Out of scope):** đây là nguồn gốc phổ biến của tranh cãi và scope creep sau này — nêu rõ những gì KHÔNG làm cũng quan trọng như nêu những gì SẼ làm.
- **Viết PRD trước khi có BRD:** dễ dẫn đến sản phẩm không truy ngược được lên mục tiêu kinh doanh rõ ràng — nên xác định "vì sao" (BRD) trước khi xác định "cái gì" (PRD), dù trong thực tế hai tài liệu này có thể phát triển song song và tinh chỉnh lẫn nhau.

Tóm tắt & tiếp theo

BRD trả lời "vì sao", PRD trả lời "cái gì" — BRD hướng đến Sponsor/lãnh đạo, PRD hướng đến đội sản phẩm/thiết kế; nhiều công ty nhỏ gộp chung thành một tài liệu. Chương 20 sẽ đi vào tài liệu chi tiết nhất trong bộ ba này: SRS theo chuẩn IEEE 830 — nơi yêu cầu được đặc tả đủ chi tiết để Dev hiện thực chính xác.

Chương 20: SRS theo chuẩn IEEE 830: cấu trúc, cách viết, mẫu đầy đủ

Mục tiêu học

- Hiểu vị trí của SRS trong chuỗi tài liệu BRD → PRD → SRS.
- Nắm được cấu trúc chuẩn IEEE 830 (và biến thể thực tế thường dùng) cho SRS.
- Viết được yêu cầu chức năng chi tiết đủ để Dev hiện thực chính xác, không cần đoán thêm.
- Có sẵn mẫu SRS đầy đủ trong `templates/srs/` để tham khảo.

20.1 SRS là gì, vì sao cần chuẩn hoá?

SRS (Software Requirements Specification) là tài liệu đặc tả **chi tiết nhất** về mặt chức năng và phi chức năng của phần mềm — đây là tài liệu mà Dev dựa vào để code, và QA dựa vào để viết test case. Nếu BRD/PRD (Chương 19) nói "vì sao" và "cái gì ở mức khái niệm", SRS nói "**cụ thể như thế nào**" — đủ chi tiết để hai người Dev khác nhau đọc SRS và hiểu ra **cùng một cách**, không có khoảng trống để tự suy đoán khác nhau.

IEEE 830 là chuẩn công nghiệp lâu đời (từ IEEE — Institute of Electrical and Electronics Engineers) định nghĩa cấu trúc và tiêu chí chất lượng cho SRS. Dù ra đời từ thời Waterfall thịnh hành, cấu trúc này vẫn hữu ích làm khung tham khảo, kể cả khi đội áp dụng Agile chỉ dùng một phần rút gọn của nó.

20.2 Cấu trúc chuẩn IEEE 830 (rút gọn, thực chiến)

1. Giới thiệu (Introduction)

- Mục đích tài liệu, phạm vi sản phẩm, định nghĩa thuật ngữ viết tắt, tài liệu tham chiếu (liên kết ngược về BRD/PRD).

2. Mô tả tổng quan (Overall Description)

- Bối cảnh sản phẩm, các chức năng chính, đặc điểm người dùng, các ràng buộc chung, giả định.

3. Yêu cầu cụ thể (Specific Requirements) — phần quan trọng và dài nhất:

- **Yêu cầu chức năng (Functional Requirements):** mô tả từng chức năng, thường tổ chức theo use case hoặc theo module — mỗi yêu cầu có ID riêng để truy vết (Chương 16).
- **Yêu cầu giao diện bên ngoài (External Interface Requirements):** giao diện người dùng, giao diện phần cứng (nếu có), giao diện phần mềm khác (API bên thứ ba như cổng thanh toán).
- **Yêu cầu phi chức năng (Non-functional Requirements):** hiệu năng, bảo mật, độ tin cậy, khả năng bảo trì (chi tiết đầy đủ ở Chương 21).

4. Phụ lục: mô hình dữ liệu sơ bộ, sơ đồ hỗ trợ (liên hệ Chương 15), bảng thuật ngữ.

20.3 Nguyên tắc viết yêu cầu chức năng chất lượng cao

Áp dụng lại bộ tiêu chí Verification đã học ở Chương 17 (rõ ràng, đầy đủ, nhất quán, khả thi, đo lường được, truy vết được), cộng thêm các nguyên tắc riêng cho SRS:

- **Mỗi yêu cầu có ID duy nhất:** ví dụ `FR-CART-01` (Functional Requirement, module Cart, số 01) — giúp truy vết trong RTM và tham chiếu khi trao đổi (Dev có thể hỏi "FR-CART-03 nghĩa là gì?" thay vì mô tả dài dòng lại).
- **Một yêu cầu, một hành vi:** tránh gộp nhiều hành vi vào một câu bằng chữ "và" — nếu một trong hai phần thay đổi/cần test riêng, việc gộp chung gây khó theo dõi.
- **Ưu tiên câu chủ động, chủ ngữ rõ ràng:** "Hệ thống **PHẢI** gửi email xác nhận..." rõ ràng hơn "Email xác nhận sẽ được gửi..." (câu bị động dễ làm mất chủ thể chịu trách nhiệm).
- **Nêu rõ điều kiện và trường hợp ngoại lệ:** không chỉ mô tả "đường đi thành công" (happy path) mà cả các trường hợp lỗi/ngoại lệ (liên hệ Activity Diagram — Chương 15).

Ví dụ: yêu cầu viết kém vs viết tốt

Viết kém: "Khách hàng có thể thanh toán online."

Viết tốt (FR-PAY-01): "Hệ thống **PHẢI** cho phép khách hàng thanh toán bằng thẻ nội địa/quốc tế hoặc ví điện tử (MoMo, ZaloPay) thông qua cổng thanh toán tích hợp. Nếu giao dịch thất bại, hệ thống **PHẢI** hiển thị thông báo lỗi rõ ràng và cho phép khách hàng thử lại hoặc chọn phương thức thanh toán khác, không được huỷ giỏ hàng đã chọn."

20.4 Mối quan hệ SRS — Use Case — User Story

Nhiều người mới nhầm lẫn SRS với Use Case Specification (Chương 22) hoặc User Story (Chương 23). Phân biệt:

	SRS	Use Case Spec	User Story
Mức chi tiết	Đặc tả đầy đủ cả module/hệ thống	Đặc tả một luồng tương tác cụ thể (actor - hệ thống)	Mô tả ngắn gọn một nhu cầu, dưới góc nhìn người dùng
Định dạng	Văn bản đặc tả có cấu trúc, ID hoá	Các bước tương tác (main flow, alternate flow)	"Là [vai trò], tôi muốn [nhu cầu], để [lợi ích]"
Phù hợp mô hình	Waterfall, dự án lớn cần tài liệu hoá đầy đủ	Cả hai, thường đi kèm SRS ở dự án Waterfall	Agile (Scrum/Kanban)

Trong thực tế, một SRS đầy đủ **có thể chứa nhiều Use Case Specification** làm chi tiết bổ sung cho từng luồng nghiệp vụ phức tạp.

20.5 Mẫu tài liệu đầy đủ cho FoodNow

Một bản SRS đầy đủ theo cấu trúc IEEE 830, điền sẵn cho module "Đặt món & Thanh toán" của FoodNow App, được lưu tại:

```
templates/srs/FoodNow-SRS.md
```

Xem thêm bản đầy đủ tại Phụ lục B (cuối sách).

Bài tập

- Viết lại yêu cầu sau theo đúng nguyên tắc ở mục 20.3 (có ID, rõ ràng, có trường hợp ngoại lệ):
"Khách hàng nhận thông báo khi đơn hàng được giao."
 - Mở `templates/srs/FoodNow-SRS.md`, chọn 1 yêu cầu chức năng bất kỳ, thử viết 2 test case (Chương 34 sẽ nói kỹ hơn về viết test case) để xác nhận yêu cầu đó đã được hiện thực đúng.
-

Sai lầm thường gặp

- **Viết SRS như một bản sao chép PRD, chỉ dài hơn:** SRS cần đi sâu đến mức chi tiết Dev không cần hỏi lại — nếu chỉ lặp lại ý PRD với câu chữ dài hơn, không tạo thêm giá trị.
 - **Bỏ qua yêu cầu giao diện bên ngoài (External Interface):** quên mô tả rõ tích hợp với hệ thống bên thứ ba (cổng thanh toán, SMS OTP) — nguồn gốc phổ biến của lỗi tích hợp phát hiện muộn.
 - **Không đánh ID cho từng yêu cầu:** khiến việc truy vết (RTM — Chương 16) và trao đổi giữa các bên trở nên khó khăn, dễ nhầm lẫn đang nói về yêu cầu nào.
-

Tóm tắt & tiếp theo

SRS đặc tả chi tiết nhất về chức năng và phi chức năng của phần mềm theo chuẩn IEEE 830 (hoặc biến thể rút gọn), là tài liệu Dev/QA dựa vào trực tiếp để làm việc — cần viết rõ ràng, có ID, có trường hợp ngoại lệ. Chương 21 sẽ đi sâu vào phần dễ bị viết sơ sài nhất của SRS: **yêu cầu phi chức năng (Non-functional Requirements)** và FRD.

Chương 21: FRD & yêu cầu phi chức năng (performance, security, usability)

Mục tiêu học

- Phân biệt FRD (Functional Requirements Document) với SRS — và biết khi nào cần tách riêng.
 - Hiểu được các nhóm yêu cầu phi chức năng phổ biến và biết viết mỗi nhóm sao cho đo lường được.
 - Tránh lỗi phổ biến nhất về yêu cầu phi chức năng: viết chung chung, không kiểm tra được.
-

21.1 FRD là gì, khác SRS ở đâu?

FRD (Functional Requirements Document) đôi khi được dùng như một tài liệu **tách riêng phần chức năng** ra khỏi SRS đầy đủ (SRS bao gồm cả chức năng lẫn phi chức năng, giao diện bên ngoài...). Ở nhiều công ty, FRD và SRS được dùng thay thế lẫn nhau (không phân biệt), hoặc FRD được coi là "phiên bản rút gọn, chỉ tập trung chức năng" của SRS đầy đủ.

Khi nào nên tách FRD riêng khỏi SRS?

- Khi đối tượng đọc khác nhau: FRD cho Dev tập trung code chức năng, SRS đầy đủ (bao gồm phi chức năng, ràng buộc hệ thống) dành cho kiến trúc sư/đội hạ tầng.
- Khi dự án lớn, cần tài liệu phi chức năng dùng chung cho **nhiều module** (không lặp lại yêu cầu bảo mật/hiệu năng ở từng SRS riêng lẻ của từng module).

Với dự án quy mô vừa/nhỏ như FoodNow, gộp chung trong một SRS (như mẫu ở Chương 20) là đủ và hợp lý — không cần tách FRD riêng.

21.2 Vì sao yêu cầu phi chức năng hay bị viết sơ sài?

Yêu cầu chức năng ("hệ thống phải làm được gì") thường dễ hình dung và dễ được stakeholder nói ra rõ ràng khi elicitation (Chương 12) — vì nó gắn trực tiếp với thao tác cụ thể. Yêu cầu phi chức năng ("hệ thống phải làm tốt đến mức nào") lại thường **bị coi là hiển nhiên** ("tất nhiên phải nhanh, tất nhiên phải bảo mật rồi") nên ít khi được hỏi rõ ràng — dẫn đến viết chung chung như "hệ thống phải nhanh", "hệ thống phải bảo mật", những câu **không thể verify được** (Chương 17) và không ai biết khi nào là "đủ nhanh, đủ bảo mật".

21.3 Các nhóm yêu cầu phi chức năng phổ biến

Nhóm	Câu hỏi cần hỏi khi elicitation	Ví dụ viết đạt chuẩn (đo lường được)
Performance (Hiệu năng)	Thao tác này cần nhanh đến mức nào? Bao nhiêu người dùng cùng lúc?	"Trang xác nhận đơn hàng phải tải xong trong dưới 2 giây với 95% lượt truy cập, đo trên kết nối 4G"
Security (Bảo mật)	Dữ liệu nào nhạy cảm? Ai được phép truy cập gì?	"Mật khẩu người dùng phải được băm (hash) bằng thuật toán bcrypt, không lưu dạng plain text"
Usability (Khả năng sử dụng)	Người dùng mục tiêu có đặc điểm gì (tuổi, kinh nghiệm công nghệ)?	"Người dùng lần đầu phải hoàn tất được một đơn hàng mà không cần hướng dẫn, đo qua kiểm thử usability với tối thiểu 5 người dùng thử nghiệm"
Reliability (Độ tin cậy)	Hệ thống được phép "chết" (downtime) bao lâu?	"Hệ thống đặt hàng phải đạt độ khả dụng (uptime) tối thiểu 99.5% mỗi tháng"
Scalability (Khả năng mở rộng)	Trong 1-2 năm tới, tải sẽ tăng đến mức nào?	"Hệ thống phải chịu tải gấp 3 lần hiện tại mà không cần thay đổi kiến trúc, khi FoodNow mở thêm chi nhánh"
Maintainability (Khả năng bảo trì)	Ai sẽ bảo trì hệ thống sau này, cần tài liệu gì?	"Toàn bộ API tích hợp phải có tài liệu Swagger/OpenAPI cập nhật"
Compatibility (Khả năng tương thích)	Chạy trên thiết bị/nền tảng nào?	"Ứng dụng phải chạy được trên iOS 14+ và Android 8+"
Compliance (Tuân thủ pháp lý)	Có quy định pháp luật/ngành nào cần tuân thủ?	"Dữ liệu cá nhân khách hàng phải được xử lý theo quy định bảo vệ dữ liệu cá nhân hiện hành"

21.4 Kỹ thuật viết yêu cầu phi chức năng đo lường được

Công thức chung: **[Đối tượng] phải [hành vi đo lường được] trong [điều kiện cụ thể], đo bằng [phương pháp/công cụ đo].**

Ví dụ áp dụng công thức: **"[Trang thanh toán] phải [phản hồi kết quả giao dịch trong dưới 3 giây] trong [điều kiện tải bình thường, dưới 100 giao dịch/phút], đo bằng [công cụ kiểm thử hiệu năng JMeter trong môi trường staging]."**

So sánh với câu mơ hồ ban đầu "hệ thống phải nhanh" — câu viết theo công thức trên cho QA đủ thông tin để viết test case cụ thể (liên hệ Chương 34), và cho Dev/kiến trúc sư biết chính xác mục tiêu kỹ thuật cần đạt được.

21.5 Kỹ thuật elicitation riêng cho yêu cầu phi chức năng

Vì stakeholder ít khi tự nói ra yêu cầu phi chức năng rõ ràng, BA cần **chủ động hỏi**, gợi ý bằng kịch bản cụ thể thay vì hỏi trừu tượng:

- Thay vì hỏi "Anh/chị cần hệ thống bảo mật đến mức nào?" (quá trừu tượng, khó trả lời) → hỏi "Nếu có người cố đăng nhập sai mật khẩu 10 lần liên tiếp vào tài khoản khách hàng, anh/chị mong muốn hệ thống xử lý thế nào?"
- Thay vì hỏi "Hệ thống cần nhanh đến mức nào?" → hỏi "Trong giờ cao điểm trưa, ước tính có bao nhiêu khách đặt hàng cùng lúc? Nếu app load chậm 5 giây vào lúc đó, ảnh hưởng ra sao đến doanh thu?"

Bài tập

1. Viết lại yêu cầu "Ứng dụng phải dễ dùng" thành một yêu cầu Usability đo lường được, theo công thức ở mục 21.4.
2. Với dự án FoodNow, hệ thống cần lưu trữ ảnh món ăn (dung lượng lớn, nhiều chi nhánh upload liên tục). Viết một yêu cầu Scalability liên quan đến việc lưu trữ này, đo lường được.

Sai lầm thường gặp

- **Chỉ hỏi yêu cầu chức năng khi elicitation, quên hẳn phi chức năng:** dẫn đến hệ thống "chạy đúng" nhưng chậm, dễ bị tấn công, hoặc không mở rộng được khi cần.
- **Viết yêu cầu phi chức năng nhưng không nêu cách đo:** "hệ thống phải bảo mật" không đo được — luôn cần gắn với con số/phương pháp đo cụ thể như công thức mục 21.4.
- **Đặt mục tiêu phi chức năng phi thực tế** (ví dụ "uptime 100%, không bao giờ được lỗi") — không hệ thống nào đạt 100% tuyệt đối; nên đặt mục tiêu khả thi, có thể đàm phán dựa trên chi phí/kỹ thuật thực tế (liên hệ tiêu chí "Feasible" ở Chương 17).

Tóm tắt & tiếp theo

FRD (nếu tách riêng) tập trung phần chức năng của SRS; yêu cầu phi chức năng (hiệu năng, bảo mật, khả dụng, mở rộng, bảo trì, tương thích, tuân thủ) cần được viết đo lường được theo công thức "đối tượng - hành vi - điều kiện - cách đo", và BA cần chủ động hỏi vì stakeholder ít khi tự nói ra rõ ràng. Chương 22 sẽ học cách viết **Use Case Specification** — đặc tả chi tiết từng luồng tương tác cụ thể giữa actor và hệ thống.

Chương 22: Use Case Specification: actor, flow chính/phụ, exception, mẫu

Mục tiêu học

- Nắm được cấu trúc chuẩn của một Use Case Specification.
- Viết được Main Flow (luồng chính), Alternate Flow (luồng phụ), và Exception Flow (luồng ngoại lệ) cho một use case cụ thể.
- Biết mức độ chi tiết phù hợp — không quá sơ sài, không quá rườm rà.

22.1 Use Case Specification là gì?

Nếu Use Case Diagram (Chương 15) cho cái nhìn tổng quan "có những chức năng gì, ai dùng", **Use Case Specification** đi sâu mô tả **từng bước tương tác cụ thể** giữa actor và hệ thống cho **MỘT** use case, bao gồm cả những gì xảy ra khi mọi thứ suôn sẻ (main flow) và khi có vấn đề phát sinh (alternate/exception flow).

22.2 Cấu trúc chuẩn

Mục	Nội dung
Tên Use Case	Ngắn gọn, dạng động từ + tân ngữ (ví dụ: "Đặt món")
ID	Để truy vết (liên hệ RTM — Chương 16)
Actor chính	Ai khởi tạo use case này
Actor phụ	Hệ thống/vai trò khác tham gia (nếu có)
Mô tả ngắn	1-2 câu tóm tắt mục đích
Điều kiện tiên quyết (Precondition)	Trạng thái hệ thống phải có trước khi use case bắt đầu
Điều kiện kết thúc (Postcondition)	Trạng thái hệ thống sau khi use case hoàn tất thành công
Luồng chính (Main Flow)	Các bước khi mọi thứ diễn ra suôn sẻ (happy path)
Luồng phụ (Alternate Flow)	Các nhánh khác vẫn dẫn đến kết quả thành công, nhưng đi đường khác
Luồng ngoại lệ (Exception Flow)	Các trường hợp lỗi, dẫn đến use case không hoàn thành như mong đợi
Yêu cầu đặc biệt	Ràng buộc phi chức năng riêng cho use case này (nếu có)

22.3 Ví dụ đầy đủ: Use Case "Đặt món và thanh toán" (FoodNow)

UC-01: Đặt món và thanh toán

Actor chính: Khách hàng **Actor phụ:** Cổng thanh toán VNPay, Hệ thống quản trị chi nhánh

Mô tả ngắn: Khách hàng chọn món, thêm vào giỏ hàng, chọn hình thức giao nhận, và hoàn tất thanh toán để tạo đơn hàng.

Điều kiện tiên quyết: Khách hàng đã đăng nhập hoặc đặt hàng dưới dạng khách vắng lai (guest).

Điều kiện kết thúc: Đơn hàng được tạo thành công và gửi đến hệ thống quản trị chi nhánh.

Luồng chính (Main Flow):

1. Khách hàng mở app, xem danh sách món theo danh mục.
2. Khách hàng chọn món, thêm vào giỏ hàng với số lượng mong muốn.
3. Khách hàng xem lại giỏ hàng, xác nhận tiếp tục.
4. Khách hàng chọn hình thức giao nhận: giao tận nơi hoặc lấy tại quán.
5. Nếu giao tận nơi, khách hàng nhập/chọn địa chỉ.
6. Khách hàng chọn phương thức thanh toán (online hoặc COD).
7. Nếu online, hệ thống chuyển hướng đến cổng thanh toán VNPay, khách hàng hoàn tất giao dịch.
8. Hệ thống xác nhận thanh toán thành công, tạo đơn hàng.
9. Hệ thống gửi đơn hàng đến hệ thống quản trị chi nhánh tương ứng.
10. Hệ thống hiển thị màn hình xác nhận đơn hàng thành công cho khách hàng.

Luồng phụ (Alternate Flow):

- **3a. Khách hàng muốn sửa giỏ hàng:** quay lại bước 2, sau khi sửa xong tiếp tục từ bước 3.
- **6a. Khách hàng nhập mã giảm giá hợp lệ trước khi thanh toán:** hệ thống áp dụng giảm giá vào tổng tiền, tiếp tục từ bước 6.
- **6b. Khách hàng chọn thanh toán COD:** bỏ qua bước 7 (không cần cổng thanh toán), chuyển thẳng đến bước 8, đơn hàng được tạo với trạng thái "Chờ thanh toán khi giao".

Luồng ngoại lệ (Exception Flow):

- **4a. Một món trong giỏ hàng hết hàng tại chi nhánh được chọn:** hệ thống thông báo, yêu cầu khách hàng xóa hoặc thay thế món đó trước khi tiếp tục (liên hệ FR-CART-04, Chương 20).
- **5a. Địa chỉ giao hàng nằm ngoài bán kính phục vụ của mọi chi nhánh:** hệ thống thông báo không thể giao đến địa chỉ này, không cho phép tiếp tục (liên hệ FR-DELIVERY-02).
- **7a. Giao dịch thanh toán online thất bại:** hệ thống hiển thị lỗi, giữ nguyên giỏ hàng, quay lại bước 6 để khách hàng thử lại hoặc chọn phương thức khác (liên hệ FR-PAY-03).

Yêu cầu đặc biệt: Toàn bộ luồng từ bước 1 đến bước 10 (trường hợp happy path) phải hoàn tất trong dưới 2 phút theo thao tác thông thường của người dùng (liên hệ BR-01, Chương 19).

22.4 Mức độ chi tiết phù hợp

Use Case Specification không cần chi tiết đến mức mô tả từng pixel giao diện (đó là việc của wireframe — Chương 25) — tập trung vào **luồng logic tương tác và các quyết định rẽ nhánh**. Một dấu hiệu use case viết vừa đủ chi tiết: một Dev mới, chưa từng nghe về dự án, đọc xong có thể tự tin liệt kê được toàn bộ các trường hợp cần code (kể cả các trường hợp ngoại lệ) mà không cần hỏi lại thêm.

Bài tập

1. Viết đầy đủ Use Case Specification cho "UC-03: Theo dõi đơn hàng" của FoodNow, theo cấu trúc ở mục 22.2 — bao gồm ít nhất 1 luồng phụ và 1 luồng ngoại lệ.
2. Với Use Case "UC-01: Đặt món và thanh toán" ở mục 22.3, thử tìm thêm ít nhất 1 luồng ngoại lệ chưa được liệt kê (gợi ý: điều gì xảy ra nếu khách hàng thoát app giữa chừng khi đang thanh toán?).

Sai lầm thường gặp

- **Chỉ viết luồng chính, bỏ qua luồng ngoại lệ:** đây là nguyên nhân phổ biến nhất của lỗi phát sinh khi test (Chương 34) — Dev không được thông báo trước nên phải tự đoán xử lý ra sao.
- **Viết use case quá chi tiết đến mức giao diện cụ thể:** làm tài liệu cồng kềnh, khó bảo trì khi giao diện thay đổi (mà logic nghiệp vụ không đổi) — tách riêng phần giao diện sang wireframe.
- **Không liên kết use case với yêu cầu chức năng cụ thể trong SRS:** làm mất khả năng truy vết — luôn ghi rõ ID yêu cầu liên quan như ví dụ ở mục 22.3.

Tóm tắt & tiếp theo

Use Case Specification mô tả chi tiết từng bước tương tác actor-hệ thống cho một use case cụ thể, gồm luồng chính, luồng phụ, và luồng ngoại lệ — quan trọng nhất là không bỏ sót các trường hợp ngoại lệ. Chương 23 sẽ chuyển sang định dạng phổ biến hơn trong các dự án Agile: **User Story và Acceptance Criteria**, nhẹ hơn Use Case Specification nhưng vẫn cần đủ rõ ràng để Dev hiện thực đúng.

Chương 23: User Story & Acceptance Criteria: INVEST, Gherkin, mẫu

Mục tiêu học

- Viết được User Story đúng cấu trúc chuẩn, thể hiện rõ vai trò, nhu cầu, và lợi ích.
- Áp dụng bộ tiêu chí INVEST để đánh giá chất lượng một User Story.
- Viết được Acceptance Criteria theo cú pháp Gherkin (Given-When-Then) đủ cụ thể để Dev/QA dùng trực tiếp.

23.1 User Story là gì?

User Story là định dạng mô tả yêu cầu ngắn gọn, phổ biến trong Agile (Scrum/Kanban — Chương 8-10), viết theo góc nhìn người dùng thay vì mô tả kỹ thuật:

Là [vai trò], tôi muốn [nhu cầu/hành động], để [lợi ích/mục đích].

Ví dụ FoodNow: "Là khách hàng, tôi muốn xem trạng thái đơn hàng theo thời gian thực, để tôi biết chính xác khi nào đồ ăn sẽ đến."

Ba thành phần đều quan trọng: **vai trò** (ai cần), **nhu cầu** (cần gì), **lợi ích** (vì sao cần) — phần "lợi ích" thường bị bỏ qua bởi người mới, nhưng lại là phần quan trọng nhất: nó giúp Dev hiểu **mục đích thật sự**, từ đó có thể tự đề xuất giải pháp tốt hơn nếu nhu cầu ban đầu chưa phải cách tối ưu nhất để đạt lợi ích đó.

23.2 Bộ tiêu chí INVEST

Một User Story chất lượng nên đạt các tiêu chí sau (viết tắt INVEST):

Chữ cái	Tiêu chí	Giải thích
I	Independent (Độc lập)	Có thể phát triển và giao hàng độc lập, không phụ thuộc chặt vào story khác phải làm trước
N	Negotiable (Có thể thương lượng)	Không phải hợp đồng cứng nhắc — chi tiết cách hiện thực có thể thảo luận giữa BA/PO và Dev
V	Valuable (Có giá trị)	Mang lại giá trị rõ ràng cho người dùng hoặc nghiệp vụ — không phải "task kỹ thuật" trá hình
E	Estimable (Ước lượng được)	Đội đủ hiểu để ước lượng được độ phức tạp/thời gian (Chương 30)
S	Small (Đủ nhỏ)	Đủ nhỏ để hoàn thành trong một sprint, tốt nhất là trong vài ngày
T	Testable (Kiểm thử được)	Có Acceptance Criteria rõ ràng để xác nhận hoàn thành đúng

Ví dụ: User Story không đạt INVEST và cách sửa

Không đạt (quá lớn, không độc lập): "Là khách hàng, tôi muốn có toàn bộ tính năng đặt hàng và thanh toán." → Vi phạm **S** (quá lớn, cần nhiều sprint) và **I** (gộp nhiều luồng phụ thuộc nhau).

Sửa lại — chia nhỏ:

- "Là khách hàng, tôi muốn thêm/xoá món trong giỏ hàng, để tôi kiểm soát đúng đơn hàng trước khi thanh toán."
- "Là khách hàng, tôi muốn thanh toán bằng thẻ hoặc ví điện tử, để tôi không cần chuẩn bị tiền mặt."
- "Là khách hàng, tôi muốn áp dụng mã giảm giá khi thanh toán, để tôi tiết kiệm chi phí."

Mỗi story nhỏ hơn giờ có thể ước lượng, phát triển, và giao hàng độc lập hơn nhiều so với story gốc.

23.3 Acceptance Criteria — làm rõ "khi nào coi là hoàn thành"

Acceptance Criteria (AC) là danh sách điều kiện cụ thể để xác nhận một User Story đã được hiện thực đúng — đây là cầu nối giữa User Story (mô tả nhu cầu ở mức khái quát) và test case (Chương 34). Định dạng phổ biến nhất: **Gherkin (Given-When-Then)**.

- **Given** (Bối cảnh): trạng thái hệ thống/dữ liệu trước khi hành động xảy ra.
- **When** (Hành động): người dùng thực hiện hành động gì.
- **Then** (Kết quả mong đợi): hệ thống phải phản hồi như thế nào.

Ví dụ Acceptance Criteria cho User Story "Áp dụng mã giảm giá"

User Story: Là khách hàng, tôi muốn áp dụng mã giảm giá khi thanh toán, để tôi tiết kiệm chi phí.

Acceptance Criteria:

Scenario 1: Áp dụng mã giảm giá hợp lệ

Given giỏ hàng có tổng giá trị 200.000đ

And mã giảm giá "FOODNOW10" đang có hiệu lực, giảm 10%

When khách hàng nhập mã "FOODNOW10" và bấm Áp dụng

Then tổng tiền thanh toán giảm còn 180.000đ

And hệ thống hiển thị thông báo "Đã áp dụng mã giảm giá FOODNOW10"

Scenario 2: Nhập mã giảm giá đã hết hạn

Given mã giảm giá "TET2024" đã hết hạn sử dụng

When khách hàng nhập mã "TET2024" và bấm Áp dụng

Then hệ thống hiển thị thông báo lỗi "Mã giảm giá đã hết hạn"

And tổng tiền thanh toán không thay đổi

Scenario 3: Nhập mã giảm giá thứ hai sau khi đã áp dụng một mã

Given khách hàng đã áp dụng thành công mã "FOODNOW10"

When khách hàng nhập thêm mã "SUMMER5" và bấm Áp dụng

Then hệ thống hiển thị thông báo "Chỉ được áp dụng 1 mã giảm giá cho mỗi đơn hàng"

And mã "FOODNOW10" vẫn được giữ nguyên hiệu lực

Nhận xét: mỗi Scenario tương ứng gần như trực tiếp với một test case (liên hệ TDD — Chương 9, và UAT — Chương 34) — đây chính là giá trị lớn nhất của Gherkin: giảm khoảng cách giữa "yêu cầu" và "kiểm thử được".

23.4 Mẫu tài liệu

Xem thêm ví dụ User Story + Acceptance Criteria đầy đủ hơn cho nhiều tính năng khác của FoodNow tại:

`templates/user-story-epic/FoodNow-UserStories.md`

Bài tập

1. Viết một User Story cho tính năng "khách hàng huỷ đơn hàng trong vòng 2 phút sau khi đặt", kèm tối thiểu 2 Acceptance Criteria theo cú pháp Gherkin (1 trường hợp thành công, 1 trường hợp thất bại — ví dụ huỷ sau khi đã quá 2 phút).
2. Áp dụng bộ tiêu chí INVEST để đánh giá User Story bạn vừa viết ở câu 1 — nếu vi phạm tiêu chí nào, hãy sửa lại.

Sai lầm thường gặp

- **Viết User Story thiếu phần "để [lợi ích]":** mất đi ngữ cảnh quan trọng nhất giúp Dev hiểu mục đích thật, dễ dẫn đến hiện thực đúng chữ nhưng sai tinh thần.
- **Acceptance Criteria chỉ mô tả happy path, không có trường hợp lỗi:** giống lỗi phổ biến ở Use Case (Chương 22) — luôn cần ít nhất 1 kịch bản thất bại/ngoại lệ.
- **User Story quá lớn (không đạt tiêu chí "Small"):** thường là dấu hiệu cần tách thành Epic (Chương 24) chứa nhiều User Story nhỏ hơn, thay vì cố nhồi tất cả vào một story.

Tóm tắt & tiếp theo

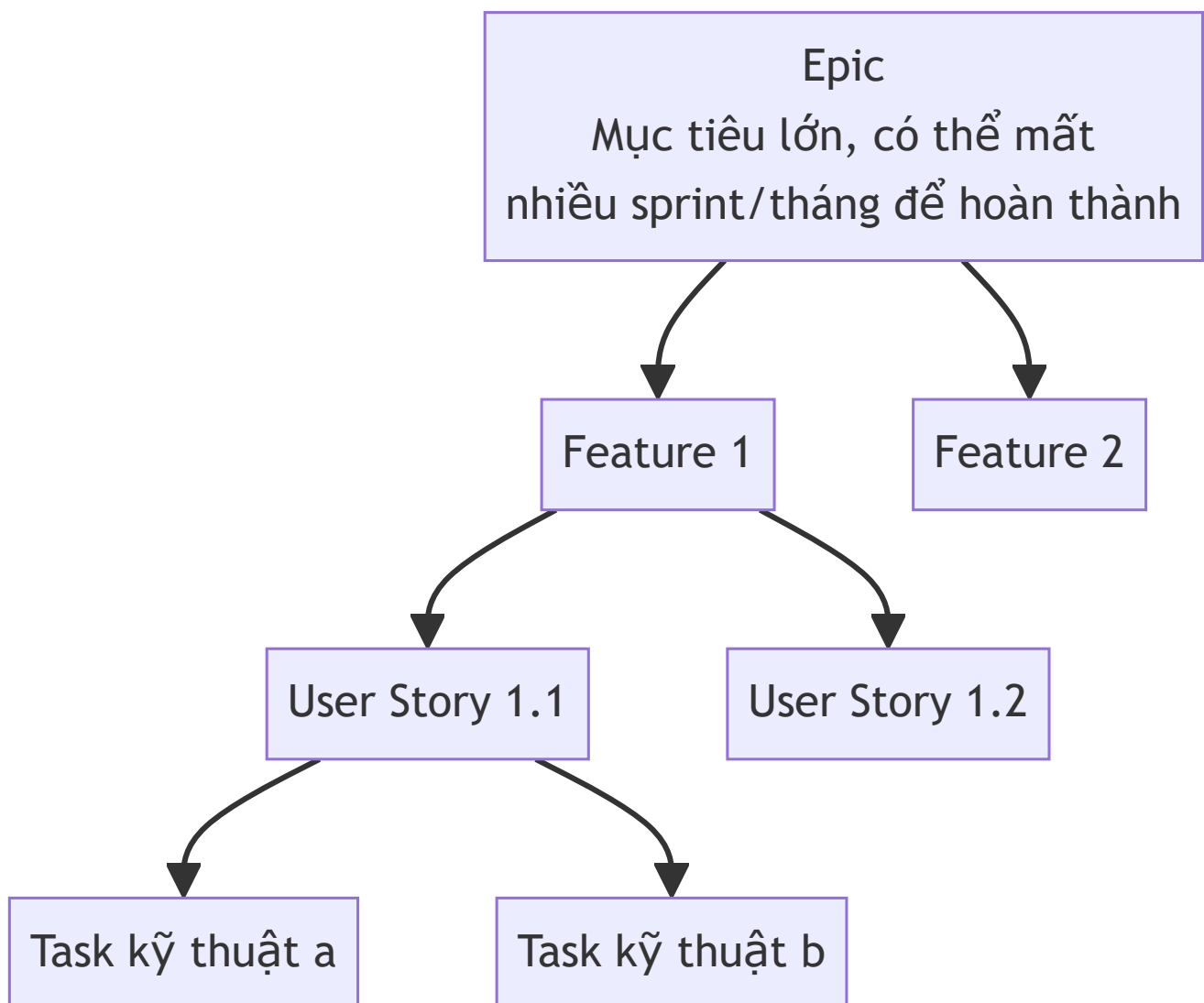
User Story mô tả nhu cầu ngắn gọn theo góc nhìn người dùng (vai trò - nhu cầu - lợi ích), đánh giá chất lượng bằng bộ tiêu chí INVEST, và đi kèm Acceptance Criteria viết theo Gherkin (Given-When-Then) để làm rõ ràng "khi nào coi là hoàn thành". Chương 24 sẽ học cách tổ chức nhiều User Story liên quan thành **Epic**, và kỹ thuật phân rã (decomposition) một yêu cầu lớn thành backlog có thể quản lý được.

Chương 24: Epic, Feature, User Story: phân rã backlog, mẫu backlog

Mục tiêu học

- Phân biệt được Epic, Feature, User Story, Task — 4 cấp độ phân rã công việc phổ biến trong Agile.
- Thực hành kỹ thuật phân rã (decomposition) một Epic lớn thành các User Story nhỏ đạt tiêu chí INVEST (Chương 23).
- Xây dựng được một backlog có cấu trúc, sẵn sàng đưa vào công cụ như Jira (Chương 37).

24.1 Bốn cấp độ phân rã



Cấp độ	Quy mô	Ví dụ FoodNow
Epic	Lớn, nhiều sprint/tháng	"Chương trình khách hàng thân thiết"
Feature	Vừa, có thể là 1-2 sprint	"Tích điểm theo giá trị đơn hàng"
User Story	Nhỏ, hoàn thành trong vài ngày (Chương 23)	"Là khách hàng, tôi muốn xem số điểm hiện có, để biết mình còn bao nhiêu điểm"
Task	Chi tiết kỹ thuật, do Dev tự chia nhỏ	"Tạo API GET /loyalty-points/{userId}"

Lưu ý: **Task kỹ thuật thường do Dev tự chia nhỏ và quản lý**, không phải công việc chính của BA — BA tập trung ở 3 cấp độ Epic/Feature/User Story, nơi ngôn ngữ vẫn là ngôn ngữ nghiệp vụ chứ chưa đi vào chi tiết kỹ thuật thuần túy.

24.2 Kỹ thuật phân rã Epic thành User Story

Có nhiều cách phân rã một Epic — không có cách "đúng duy nhất", nhưng các trục phân rã phổ biến:

1. **Theo luồng nghiệp vụ (workflow steps)**: chia theo các bước tuần tự trong một quy trình.
2. **Theo quy tắc nghiệp vụ (business rules)**: tách riêng từng biến thể/quy tắc phức tạp.
3. **Theo loại dữ liệu/đối tượng (data variations)**: ví dụ tách theo loại người dùng khác nhau.
4. **Theo mức độ ưu tiên (happy path trước, edge case sau)**: làm trước phần lõi tạo giá trị chính, xử lý edge case ở story riêng, làm sau nếu cần.

Ví dụ: phân rã Epic "Chương trình khách hàng thân thiết" (FoodNow)

Epic: Chương trình khách hàng thân thiết — khách hàng tích điểm theo giá trị đơn hàng, có thể đổi điểm lấy ưu đãi.

Phân rã theo luồng nghiệp vụ (tích điểm → xem điểm → đổi điểm):

ID	User Story	Ưu tiên (MoSCoW — Chương 26)
US-401	Là khách hàng, tôi muốn được tự động tích điểm sau mỗi đơn hàng thành công, để tôi tích lũy dần mà không cần thao tác thêm	Must have
US-402	Là khách hàng, tôi muốn xem số điểm hiện có trên màn hình tài khoản, để tôi biết mình còn bao nhiêu điểm	Must have
US-403	Là khách hàng, tôi muốn xem lịch sử tích/đổi điểm, để tôi kiểm tra lại các giao dịch điểm của mình	Should have
US-404	Là khách hàng, tôi muốn đổi điểm lấy mã giảm giá, để tôi tiết kiệm chi phí cho đơn hàng tiếp theo	Must have
US-405	Là khách hàng, tôi muốn nhận thông báo khi điểm sắp hết hạn, để tôi không bỏ lỡ cơ hội sử dụng	Could have
US-406	Là Ban giám đốc, tôi muốn xem báo cáo tổng số điểm đã phát hành/đã đổi theo tháng, để đánh giá hiệu quả chương trình	Should have

Mỗi User Story ở trên đều đạt tiêu chí **Independent** (có thể làm US-402 mà chưa cần US-404 xong) và **Small** (mỗi story có thể ước lượng và hoàn thành trong vài ngày).

24.3 Định nghĩa "Ready" và "Done" cho một User Story

Để backlog vận hành trơn tru, đội cần thống nhất trước 2 định nghĩa quan trọng:

- **Definition of Ready (DoR):** điều kiện để một User Story đủ điều kiện đưa vào Sprint Planning (Chương 8, 27) — thường gồm: đã có Acceptance Criteria rõ ràng, đã được ước lượng, không còn phụ thuộc chưa giải quyết.
- **Definition of Done (DoD):** điều kiện để một User Story được coi là hoàn thành thực sự — không chỉ "code xong" mà thường gồm: code đã review, đã pass test tự động, đã qua QA, đã demo được.

BA có vai trò quan trọng trong việc đảm bảo DoR được đáp ứng **trước khi** đưa story vào sprint — tránh tình trạng Dev nhận story mơ hồ, phải dừng lại giữa chừng để hỏi lại BA.

24.4 Mẫu backlog đầy đủ

Một backlog đầy đủ hơn cho toàn bộ các Epic của FoodNow (không chỉ chương trình khách hàng thân thiết) được lưu tại:

`templates/user-story-epic/FoodNow-UserStories.md`

Xem bản đầy đủ tại Phụ lục C (cuối sách).

Bài tập

1. Phân rã Epic "Theo dõi đơn hàng real-time" của FoodNow thành tối thiểu 4 User Story, theo một trong các trục phân rã ở mục 24.2 — ghi rõ bạn chọn trục nào.
 2. Với 4 User Story vừa viết, tự đề xuất mức ưu tiên MoSCoW cho từng story (sẽ học chi tiết kỹ thuật MoSCoW ở Chương 26, nhưng có thể tự ước lượng theo trực giác trước).
-

Sai lầm thường gặp

- **Epic không bao giờ được phân rã, cứ để nguyên trong backlog:** khiến đội không thể ước lượng hay lên kế hoạch sprint cụ thể — Epic cần được phân rã trước khi đưa vào sprint bất kỳ.
 - **Phân rã theo tầng kỹ thuật thay vì theo giá trị nghiệp vụ** (ví dụ: 1 story "làm frontend", 1 story "làm backend" cho cùng một tính năng) — vi phạm tiêu chí "Valuable" vì không story riêng lẻ nào tạo ra giá trị hoàn chỉnh cho người dùng.
 - **Không thống nhất Definition of Ready/Done trước khi bắt đầu:** dẫn đến tranh cãi liên tục về "story này đã xong chưa" giữa các thành viên đội.
-

Tóm tắt & tiếp theo

Epic → Feature → User Story → Task là 4 cấp độ phân rã phổ biến, BA chủ yếu làm việc ở 3 cấp độ đầu; kỹ thuật phân rã theo luồng nghiệp vụ, quy tắc, loại dữ liệu, hoặc mức ưu tiên giúp chia Epic lớn thành User Story đạt tiêu chí INVEST. Chương 25 sẽ khép lại Phần 3 với chủ đề wireframe/prototype cơ bản — công cụ trực quan hỗ trợ BA mô tả ý tưởng rõ ràng hơn văn bản thuần.

Chương 25: Wireframe & Prototype cơ bản cho BA

Mục tiêu học

- Hiểu vì sao BA cần biết vẽ wireframe ở mức cơ bản, dù không phải Designer.
- Phân biệt 3 mức độ "trung thực" (fidelity) của wireframe: low-fidelity, mid-fidelity, high-fidelity.
- Vẽ được một wireframe low-fidelity đơn giản để minh họa ý tưởng trong buổi họp/workshop.
- Biết ranh giới: việc gì BA nên tự làm, việc gì nên để Designer đảm nhận.

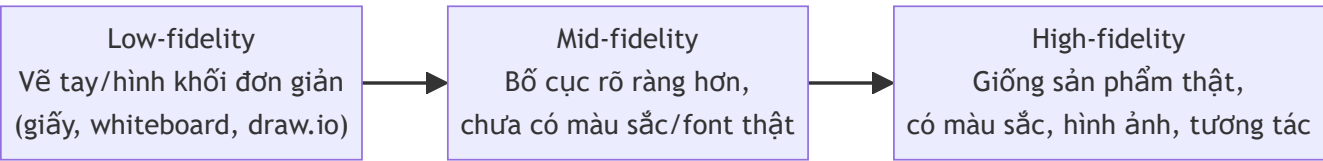
25.1 Vì sao BA cần biết wireframe, dù không phải Designer?

Chương 17 đã nói: **prototype/wireframe là một kỹ thuật Validation hiệu quả** — nhiều stakeholder validate chính xác hơn khi nhìn thấy hình ảnh cụ thể so với đọc mô tả bằng văn bản. Trong thực tế, BA thường cần vẽ nhanh một wireframe đơn giản ngay trong buổi họp/workshop để:

- Minh họa ý tưởng ngay lập tức, tránh hiểu lầm khi mô tả bằng lời quá trừu tượng.
- Làm cơ sở thảo luận cụ thể hơn khi elicitation (Chương 12) — nhiều stakeholder khó hình dung bằng lời, nhưng dễ dàng góp ý khi nhìn thấy hình vẽ, dù chỉ là hình vẽ tay đơn giản.
- Chuẩn bị tài liệu đầu vào rõ ràng hơn trước khi bàn giao cho Designer chính thức.

BA không cần biết dùng Figma thành thạo hay có gu thẩm mỹ như Designer — mục tiêu chỉ là truyền đạt được **cấu trúc và luồng thông tin**, không phải tạo ra giao diện đẹp.

25.2 Ba mức độ "trung thực" (fidelity)



Mức độ	Công cụ thường dùng	Ai thường làm	Khi nào dùng
Low-fidelity	Giấy, bút, whiteboard, draw.io	BA, PO, cả đội trong workshop	Giai đoạn ý tưởng ban đầu, thảo luận nhanh, không cần chuẩn xác
Mid-fidelity	Balsamiq, draw.io với template UI	BA (cơ bản) hoặc Designer	Làm rõ luồng và bố cục trước khi Designer đầu tư thời gian thiết kế chi tiết
High-fidelity	Figma, Adobe XD	Designer chuyên nghiệp	Thiết kế cuối cùng, gần với sản phẩm thật, dùng để Dev code theo

BA chủ yếu hoạt động ở mức **low-fidelity** và đôi khi **mid-fidelity** — high-fidelity là công việc chuyên môn của Designer.

25.3 Ví dụ: Wireframe low-fidelity cho màn hình giỏ hàng FoodNow

Dưới đây là một wireframe low-fidelity dạng khối đơn giản (mô tả bằng ký hiệu văn bản, tương đương những gì bạn có thể vẽ tay trên giấy hoặc bằng hình khối trong draw.io):

← Quay lại	Giỏ hàng
[Ảnh]	Cơm gà xối mỡ 35.000đ [- 2 +] [Xóa]
[Ảnh]	Trà đào cam sả 25.000đ [- 1 +] [Xóa]
Nhập mã giảm giá: [_____] [Áp dụng]	
Tạm tính:	95.000đ
Giảm giá:	-9.500đ
Tổng cộng:	85.500đ
[Tiếp tục thanh toán]	

Wireframe này đủ để trả lời các câu hỏi quan trọng khi thảo luận với stakeholder: Có hiển thị ảnh món không? Có cho sửa số lượng trực tiếp tại giỏ hàng không? Nhập mã giảm giá ở đâu trong luồng? — tất cả **không cần biết về đẹp**, chỉ cần đủ rõ để thảo luận đúng trọng tâm.

25.4 Ranh giới: việc của BA và việc của Designer

Việc BA nên làm	Việc nên để Designer làm
Vẽ nhanh low-fidelity để minh họa ý tưởng trong họp	Thiết kế high-fidelity cuối cùng
Xác định cấu trúc thông tin, luồng màn hình (Activity Diagram — Chương 15)	Chọn màu sắc, font, khoảng cách, hiệu ứng chuyển động
Góp ý về nội dung/luồng nghiệp vụ khi xem thiết kế Designer làm	Đảm bảo tuân thủ bộ nhận diện thương hiệu, nguyên tắc UX chuyên sâu
Đảm bảo wireframe phản ánh đúng yêu cầu nghiệp vụ (validate — Chương 17)	Kiểm tra khả năng tiếp cận (accessibility), responsive trên nhiều thiết bị

Nguyên tắc chung: BA tập trung vào **"thông tin gì cần có, theo thứ tự nào, luồng ra sao"** — Designer tập trung vào **"trình bày thông tin đó đẹp, dễ dùng, nhất quán thương hiệu như thế nào"**. Khi hai bên tôn trọng ranh giới này, tránh được tình trạng BA "chỉ đạo" Designer về mặt thẩm mỹ (vượt quá chuyên môn) hoặc Designer tự ý thay đổi luồng nghiệp vụ mà không hỏi lại BA.

Bài tập

1. Vẽ (bằng ký hiệu văn bản như ví dụ mục 25.3, hoặc vẽ tay/dùng draw.io) một wireframe low-fidelity cho màn hình "Theo dõi đơn hàng" của FoodNow — hiển thị được các trạng thái đơn hàng (liên hệ User Story US-201, Chương 24).
2. Với wireframe bạn vừa vẽ, liệt kê 3 câu hỏi bạn sẽ đặt ra cho stakeholder khi trình bày wireframe này để validate (liên hệ kỹ thuật Validation — Chương 17).

Sai lầm thường gặp

- **Đầu tư quá nhiều thời gian làm wireframe đẹp ở giai đoạn ý tưởng ban đầu:** lãng phí thời gian — ở giai đoạn cần thảo luận nhanh, low-fidelity là đủ, không cần mid/high-fidelity.
- **BA tự ý quyết định các chi tiết thẩm mỹ (màu sắc, font) trong wireframe rồi áp đặt cho Designer:** vượt quá ranh giới chuyên môn — nên để trống các quyết định này, tập trung vào cấu trúc/luồng.
- **Chỉ mô tả yêu cầu bằng văn bản, không bao giờ vẽ minh hoạ dù ý tưởng phức tạp:** bỏ lỡ công cụ validation hiệu quả, dễ gây hiểu lầm hơn nhiều so với có kèm hình minh hoạ dù đơn giản.

Tóm tắt & tiếp theo

BA nên biết vẽ wireframe low-fidelity (và đôi khi mid-fidelity) để minh hoạ ý tưởng nhanh, hỗ trợ validation hiệu quả hơn văn bản thuần — nhưng cần tôn trọng ranh giới, để lại phần thiết kế high-fidelity chuyên sâu cho Designer. Đây là chương khép lại Phần 3 (Tài liệu BA cốt lõi). Từ Chương 26, sách bước vào Phần 4 — Quản lý dự án & giao hàng, bắt đầu với kỹ thuật ưu tiên hoá backlog: MoSCoW, WSJF, và mô hình Kano.

Phần 4 — Quản lý dự án & giao hàng

Chương 26: Ưu tiên hoá Backlog: MoSCoW, WSJF, mô hình Kano

Mục tiêu học

- Áp dụng được kỹ thuật MoSCoW để phân loại độ ưu tiên của yêu cầu.
- Hiểu và tính toán được điểm WSJF ở mức cơ bản để so sánh độ ưu tiên giữa các hạng mục.
- Phân loại được tính năng theo mô hình Kano (Basic, Performance, Excitement) để hiểu tác động đến sự hài lòng của khách hàng.
- Biết chọn kỹ thuật phù hợp cho từng tình huống.

26.1 Vì sao cần ưu tiên hoá?

Backlog của bất kỳ dự án nào cũng gần như luôn có **những việc hơn thời gian/nguồn lực có thể làm ngay**. Ưu tiên hoá giúp trả lời câu hỏi quan trọng nhất trong quản lý sản phẩm: **"làm gì trước, làm gì sau, làm gì có thể không làm"** — dựa trên tiêu chí khách quan thay vì "ai la to nhất được làm trước".

26.2 MoSCoW

Kỹ thuật đơn giản và phổ biến nhất, phân loại mỗi hạng mục vào 1 trong 4 nhóm:

Nhóm	Ý nghĩa	Ví dụ FoodNow
M — Must have	Bắt buộc phải có, thiếu thì sản phẩm không dùng được/không đạt mục tiêu tối thiểu	Đặt món, thanh toán (US-101 đến US-104)
S — Should have	Quan trọng nhưng có thể tạm hoãn nếu thời gian gấp	Xem lịch sử đơn hàng (US-203)
C — Could have	Có thì tốt, không có cũng không ảnh hưởng lớn	Thông báo điểm sắp hết hạn (US-405)
W — Won't have (this time)	Đã cân nhắc nhưng quyết định KHÔNG làm ở phiên bản này	Đặt bàn trước tại quán (xem BRD, mục "Ngoài phạm vi" — Chương 19)

Nguyên tắc quan trọng: nhóm "Must have" nên chiếm **tối đa 60% tổng khối lượng công việc** — nếu mọi thứ đều được gắn nhãn "Must have", nhãn đó mất hết ý nghĩa (không còn phân biệt được gì với gì). Đây là lỗi rất phổ biến khi PO/Sponsor không quen làm việc với MoSCoW.

26.3 WSJF (Weighted Shortest Job First)

MoSCoW tốt để phân loại nhanh nhưng không giúp so sánh **định lượng** giữa các hạng mục cùng nhóm "Must have". **WSJF** giải quyết vấn đề này bằng công thức:

$$\text{WSJF} = \text{Cost of Delay (Chi phí trì hoãn)} \div \text{Job Size (Độ lớn công việc)}$$

Cost of Delay thường được ước lượng bằng tổng của 3 yếu tố (mỗi yếu tố chấm điểm tương đối, ví dụ theo thang Fibonacci 1-2-3-5-8-13):

- **Giá trị kinh doanh** (Business Value): mang lại lợi ích trực tiếp đến mức nào.
- **Mức độ khẩn cấp theo thời gian** (Time Criticality): trì hoãn càng lâu, giá trị mất đi càng nhanh hay không (ví dụ: tính năng phục vụ mùa Tết mất giá trị hoàn toàn nếu làm xong sau Tết).
- **Giảm rủi ro/tạo cơ hội** (Risk Reduction/Opportunity Enablement): làm sớm giúp giảm rủi ro hoặc mở ra cơ hội gì cho các hạng mục khác.

Ví dụ tính WSJF cho 2 hạng mục của FoodNow

Hạng mục	Giá trị kinh doanh	Khẩn cấp thời gian	Giảm rủi ro	Tổng Cost of Delay	Job Size	WSJF
Tích hợp cổng thanh toán VNPAY	8	5	8 (rủi ro kỹ thuật cao, nên làm sớm)	21	8	2.6
Thông báo điểm sắp hết hạn (US-405)	2	1	1	4	3	1.3

WSJF càng cao, càng nên ưu tiên làm trước — ở đây, tích hợp cổng thanh toán (2.6) nên làm trước thông báo điểm sắp hết hạn (1.3), phù hợp với trực giác nhưng giờ có con số cụ thể để giải thích quyết định với stakeholder, thay vì chỉ dựa vào cảm tính.

WSJF phổ biến trong các tổ chức áp dụng SAgile (Scaled Agile Framework — nhắc ở Chương 11), phù hợp khi cần so sánh định lượng giữa nhiều hạng mục lớn, đến từ nhiều đội khác nhau.

26.4 Mô hình Kano

MoSCoW và WSJF tập trung vào ưu tiên **thời điểm làm**, mô hình Kano tập trung vào **tác động đến sự hài lòng của khách hàng** — phân loại tính năng thành 3 nhóm:

Basic (Cơ bản) Không có: khách hàng rất khó chịu Có: khách hàng coi là hiển nhiên, không tăng hài lòng thêm	Performance (Hiệu năng) Càng tốt, khách hàng càng hài lòng – tỷ lệ thuận	Excitement (Bất ngờ thích thú) Không có: khách hàng không để ý Có: gây bất ngờ, tăng hài lòng vượt trội
--	--	--

Nhóm Kano	Ví dụ FoodNow	Hàm ý cho ưu tiên hoá
Basic	App phải đặt được hàng, thanh toán được — nếu lỗi, khách bỏ đi ngay	Bắt buộc làm tốt, nhưng làm tốt hơn cũng không "gây ấn tượng" thêm — đầu tư vừa đủ, không quá tay
Performance	Tốc độ tải trang, độ chính xác thời gian giao hàng dự kiến	Càng đầu tư tối ưu, khách hàng càng hài lòng hơn theo tỷ lệ — đáng đầu tư liên tục
Excitement	Gợi ý món ăn dựa trên lịch sử đặt hàng, quà bất ngờ vào sinh nhật khách hàng	Không bắt buộc nhưng tạo khác biệt cạnh tranh — cân nhắc làm sau khi đã vững phần Basic

Ứng dụng thực tế: nhiều đội mới làm sản phẩm mắc lỗi đầu tư quá nhiều vào tính năng "Excitement" (nghe hấp dẫn, dễ gây hứng thú khi brainstorm) trong khi tính năng "Basic" còn chưa ổn định — ví dụ FoodNow đầu tư tính năng gợi ý món ăn bằng AI trong khi app vẫn còn lỗi khi thanh toán — đây là sai lầm về thứ tự ưu tiên nghiêm trọng.

26.5 Kết hợp cả ba kỹ thuật trong thực tế

Quy trình thực chiến phổ biến:

- Dùng **Kano** để hiểu tác động của từng nhóm tính năng đến trải nghiệm khách hàng (làm 1 lần, mang tính định hướng chiến lược).
- Dùng **MoSCoW** để phân loại nhanh cho mỗi phiên bản/release cụ thể.
- Dùng **WSJF** khi cần so sánh định lượng giữa nhiều hạng mục cùng nằm trong nhóm "Must have" của MoSCoW, để quyết định thứ tự làm cụ thể.

Bài tập

- Phân loại 6 User Story của Epic C (Chương trình khách hàng thân thiết, xem `templates/user-story-epic/FoodNow-UserStories.md`) theo MoSCoW — so sánh với mức ưu tiên đã gợi ý sẵn trong bảng, giải thích nếu bạn có ý kiến khác.
- Chọn 2 tính năng của FoodNow, tự chấm điểm Cost of Delay (3 yếu tố) và Job Size, tính WSJF theo công thức mục 26.3.

Sai lầm thường gặp

- **Gắn nhãn "Must have" cho gần như mọi thứ trong backlog:** làm mất ý nghĩa phân loại — nên giới hạn nhóm Must have ở mức hợp lý (tối đa ~60%).
- **Chỉ dùng một kỹ thuật duy nhất cho mọi quyết định:** mỗi kỹ thuật trả lời một câu hỏi khác nhau (thời điểm làm vs tác động hài lòng khách hàng) — nên kết hợp như gợi ý ở mục 26.5.
- **Đầu tư quá sớm vào tính năng Excitement khi tính năng Basic chưa ổn định:** gây lãng phí nguồn lực và rủi ro mất khách hàng vì trải nghiệm cơ bản chưa tốt.

Tóm tắt & tiếp theo

MoSCoW phân loại nhanh 4 nhóm ưu tiên, WSJF định lượng so sánh dựa trên chi phí trì hoãn và độ lớn công việc, mô hình Kano phân loại tác động đến sự hài lòng khách hàng (Basic/Performance/Excitement) — nên kết hợp cả ba tùy mục đích. Chương 27 sẽ đi sâu vào vai trò cụ thể của BA trong Sprint Planning và hoạt động "backlog refinement" diễn ra trước đó.

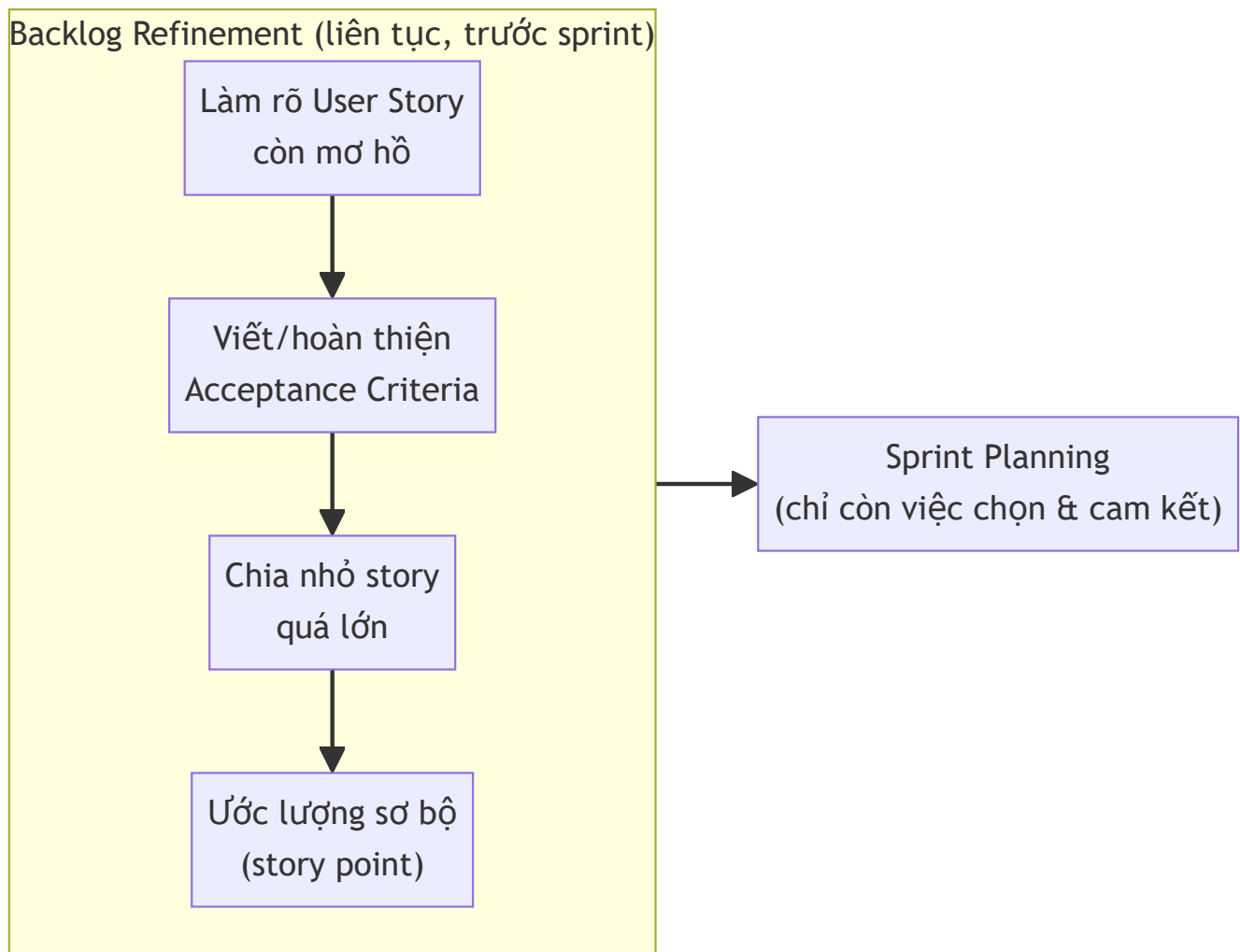
Chương 27: Sprint Planning & vai trò của BA trong sprint

Mục tiêu học

- Hiểu vai trò của hoạt động "Backlog Refinement" (còn gọi là Backlog Grooming) — diễn ra trước Sprint Planning, không phải trong sự kiện Sprint Planning chính thức.
- Biết BA cần chuẩn bị gì trước Sprint Planning để buổi họp diễn ra hiệu quả.
- Phân biệt công việc BA làm trong sprint đang chạy so với công việc chuẩn bị cho sprint tiếp theo.

27.1 Backlog Refinement — nơi phần lớn công việc BA diễn ra

Chương 8 đã giới thiệu 5 sự kiện chính thức của Scrum — nhưng **Backlog Refinement không phải một sự kiện chính thức** trong Scrum Guide, mà là một **hoạt động liên tục**, thường diễn ra định kỳ (ví dụ: 1-2 buổi/tuần, mỗi buổi 1 tiếng) để chuẩn bị backlog cho các sprint sắp tới.



Đây là nơi phần lớn thời gian và công sức của BA thực sự dành cho — **không phải trong buổi Sprint Planning**. Nếu backlog được refine tốt trước đó, Sprint Planning chỉ còn là việc PO trình bày độ ưu tiên, đội xác nhận hiểu đúng, và cam kết khối lượng công việc cho sprint — không cần dừng lại để phân tích yêu cầu từ đầu.

27.2 Checklist chuẩn bị của BA trước Sprint Planning

Với mỗi User Story dự kiến đưa vào sprint tiếp theo, BA cần đảm bảo đạt **Definition of Ready** (đã giới thiệu ở Chương 24):

- [] User Story viết đúng cấu trúc, có đủ "vai trò - nhu cầu - lợi ích" (Chương 23).
- [] Đạt tiêu chí INVEST, đặc biệt là **Small** (đủ nhỏ để hoàn thành trong 1 sprint) và **Estimable** (đủ hiểu để ước lượng).
- [] Có Acceptance Criteria viết theo Gherkin, bao phủ cả trường hợp thành công và thất bại.
- [] Không còn phụ thuộc (dependency) chưa giải quyết vào story/hệ thống khác.
- [] Đã trao đổi trước với Dev/QA nếu có điểm phức tạp về mặt kỹ thuật cần làm rõ sớm.

Nếu một story chưa đạt các tiêu chí trên, **không nên đưa vào Sprint Planning** — sẽ chỉ gây lãng phí thời gian buổi họp để phân tích thay vì lên kế hoạch.

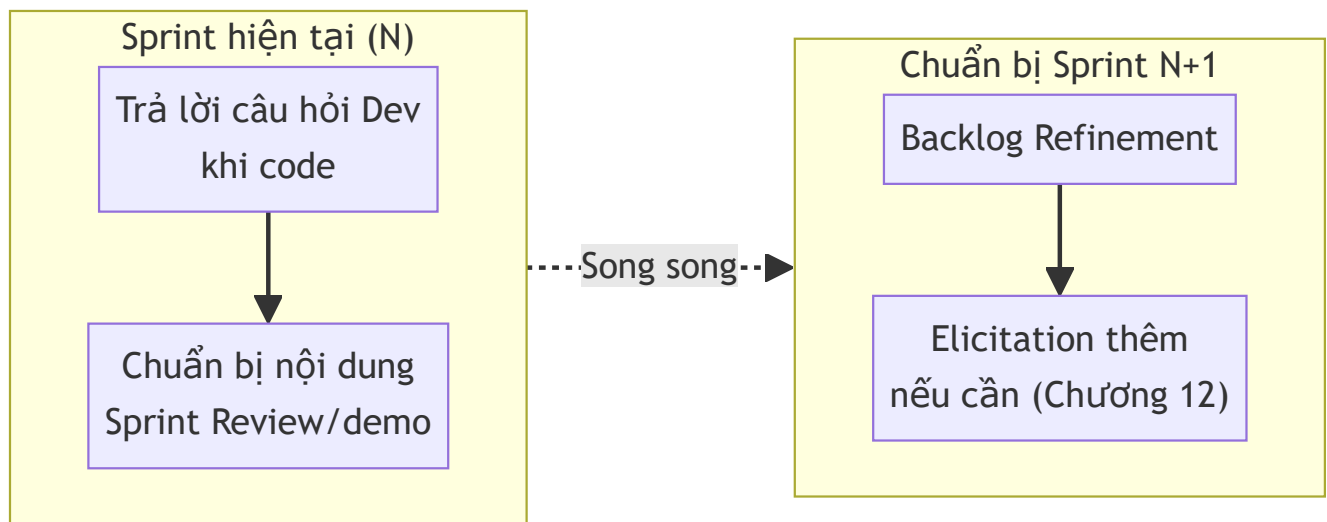
27.3 Vai trò của BA trong chính buổi Sprint Planning

Dù phần lớn công việc đã làm ở Refinement, BA vẫn có vai trò cụ thể trong buổi Sprint Planning:

- **Trả lời câu hỏi làm rõ phát sinh tại chỗ:** dù đã refine kỹ, Dev có thể vẫn có câu hỏi mới khi thảo luận chi tiết cách hiện thực.
 - **Xác nhận độ ưu tiên cùng PO:** đảm bảo thứ tự chọn story vào sprint phản ánh đúng giá trị nghiệp vụ (liên hệ MoSCoW/WSJF — Chương 26), không chỉ chọn theo độ dễ làm.
 - **Ghi nhận rủi ro/giả định mới phát sinh:** nếu trong lúc thảo luận, đội phát hiện một giả định quan trọng (ví dụ: "giả sử cổng thanh toán hỗ trợ hoàn tiền tự động") — BA cần ghi lại để xác minh sau, tránh giả định sai lệch lẫn trôi vào sprint mà không ai kiểm chứng (liên hệ RAID Log — Chương 31).
-

27.4 Công việc của BA trong sprint đang chạy — nhìn hai hướng cùng lúc

BA giỏi trong Scrum thường **làm việc song song hai luồng**:



Đây là điểm khác biệt lớn so với Waterfall (Chương 6), nơi BA thường làm xong hẳn giai đoạn Requirements rồi mới chuyển hẳn sang hỗ trợ Design/Implementation. Trong Scrum, BA luôn "sống" đồng thời ở hai thời điểm: **hỗ trợ sprint hiện tại**, và **chuẩn bị cho sprint tiếp theo** — nếu chỉ tập trung một trong hai, đội sẽ hoặc thiếu hỗ trợ tức thời, hoặc thiếu backlog sẵn sàng khi sprint tiếp theo bắt đầu.

27.5 Ví dụ thực tế với FoodNow

Trong tuần đang chạy Sprint 3 (đội đang code Epic B — Theo dõi đơn hàng), lịch làm việc điển hình của BA có thể như sau:

- **Thứ 2-3:** Trả lời câu hỏi Dev về US-202 (thông báo đầy) đang code trong sprint hiện tại; đồng thời tổ chức buổi Refinement 1 tiếng cho các story của Epic C (Chương trình khách hàng thân thiết) dự kiến vào Sprint 4.
- **Thứ 4:** Phỏng vấn nhanh (elicitation) với Phòng Marketing để làm rõ chi tiết US-405 (thông báo điểm sắp hết hạn) còn thiếu Acceptance Criteria.
- **Thứ 5:** Chuẩn bị kịch bản demo cho Sprint Review cuối tuần, xác nhận với QA rằng US-201, US-202 đã pass test.
- **Thứ 6:** Tham gia Sprint Review, ghi nhận phản hồi từ stakeholder; tham gia Retrospective.

Bài tập

1. Với 3 User Story chưa đạt Definition of Ready (tự chọn từ `templates/user-story-epic/FoodNow-UserStories.md`), giả định chúng còn thiếu Acceptance Criteria, viết ra kế hoạch cụ thể bạn sẽ làm trong buổi Backlog Refinement để đưa chúng đạt DoR.
2. Giải thích bằng lời của bạn: vì sao việc BA "làm việc song song hai luồng" (sprint hiện tại + chuẩn bị sprint tiếp theo) lại là một thách thức lớn hơn so với làm việc trong mô hình Waterfall.

Sai lầm thường gặp

- **Không tổ chức Backlog Refinement riêng, dồn hết việc phân tích vào Sprint Planning:** làm buổi Sprint Planning kéo dài quá lâu, đội mệt mỏi, giảm chất lượng ước lượng.
- **Chỉ tập trung sprint hiện tại, quên chuẩn bị cho sprint tiếp theo:** dẫn đến backlog "cạn kiệt" vào đầu sprint sau, đội phải chờ BA phân tích gấp, ảnh hưởng chất lượng.
- **Đưa story chưa đạt Definition of Ready vào Sprint Planning "cho đủ số lượng":** gây lãng phí thời gian buổi họp và rủi ro cam kết sai khối lượng công việc thực tế đội có thể làm.

Tóm tắt & tiếp theo

Phần lớn công việc BA trong Scrum diễn ra ở hoạt động Backlog Refinement (không phải sự kiện chính thức nhưng cực kỳ quan trọng) trước Sprint Planning, và BA cần làm việc song song hai luồng: hỗ trợ sprint hiện tại và chuẩn bị cho sprint tiếp theo. Chương 28 sẽ đi vào khái niệm cốt lõi của việc giao hàng theo Agile: **Incremental & Iterative Delivery**.

Chương 28: Incremental & Iterative Delivery: khái niệm, phân biệt

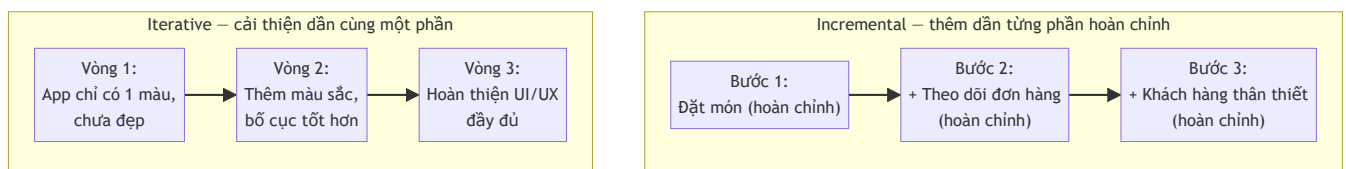
Mục tiêu học

- Phân biệt rõ ràng "Incremental" (tăng dần) và "Iterative" (lặp lại) — hai khái niệm hay bị dùng lẫn lộn dù ý nghĩa khác nhau.
- Giải thích được vì sao giao hàng từng phần giảm rủi ro so với giao hàng một lần (Waterfall).
- Xác định được chiến lược Incremental phù hợp cho một dự án cụ thể.

28.1 Incremental vs Iterative — khác nhau ở đâu?

Đây là cặp thuật ngữ dễ nhầm vì thường đi cùng nhau trong Agile, nhưng bản chất khác nhau:

- **Incremental (Tăng dần):** xây dựng sản phẩm bằng cách **thêm dần các phần mới, hoàn chỉnh** vào những gì đã có — mỗi phần thêm vào là một chức năng đầy đủ, dùng được, không cần sửa lại những gì đã làm trước đó.
- **Iterative (Lặp lại):** xây dựng sản phẩm bằng cách **làm đi làm lại, cải thiện dần** cùng một phần — mỗi vòng lặp làm cho phần đó tốt hơn, hoàn chỉnh hơn, không nhất thiết đã "dùng được" đầy đủ ngay từ vòng lặp đầu.



Trong thực tế, **Scrum kết hợp cả hai**: mỗi sprint vừa có thể thêm tính năng mới (incremental) vừa có thể cải thiện tính năng đã có dựa trên phản hồi (iterative) — đây là lý do 2 khái niệm hay bị gộp chung, dù về mặt lý thuyết là hai trục khác nhau.

28.2 Vì sao giao hàng từng phần giảm rủi ro?

So sánh hai chiến lược cho dự án FoodNow:

Chiến lược A — Giao hàng một lần (Waterfall thuần, Chương 6): phát triển toàn bộ Epic A, B, C, D trong 6 tháng, chỉ ra mắt khi hoàn thành 100%.

Chiến lược B — Incremental Delivery: ra mắt Epic A (Đặt món & Thanh toán) sau 2 tháng cho 2 chi nhánh thử nghiệm, thu thập phản hồi thật, rồi tiếp tục Epic B, C, D.

Chiến lược B: Incremental

Tháng 1-2: Epic A



Ra mắt thử nghiệm,
nhận phản hồi SỚM



Tháng 3-4: Epic B
(đã điều chỉnh theo
phản hồi Epic A)



Tháng 5-6: Epic C, D

Chiến lược A: Giao 1 lần

Tháng 1-6: Code toàn bộ



Tháng 6: Ra mắt
toàn bộ, **LẦN ĐẦU**
nhận phản hồi thật

Với Chiến lược A, nếu phát hiện vấn đề nghiêm trọng ở Epic A (ví dụ: khách hàng thấy luồng thanh toán khó hiểu) — vấn đề này chỉ được phát hiện ở tháng thứ 6, **sau khi đã lỡ xây cả Epic B, C, D** theo cùng một giả định sai. Với Chiến lược B, vấn đề được phát hiện ở tháng thứ 2, kịp thời điều chỉnh trước khi tiếp tục — đây chính là giá trị cốt lõi của Incremental Delivery: **biến những giả định chưa kiểm chứng thành bài học sớm, với chi phí thấp**, thay vì đặt cược toàn bộ dự án vào một lần ra mắt duy nhất.

28.3 Các chiến lược chia Incremental phổ biến

Chiến lược	Mô tả	Ví dụ FoodNow
Theo tính năng (Feature-based)	Chia theo Epic/Feature độc lập	Epic A trước, Epic B/C/D sau (như ví dụ 28.2)
Theo phân khúc người dùng (User segment)	Ra mắt cho một nhóm nhỏ trước, mở rộng dần	Thử nghiệm ở 2 chi nhánh trước, mở rộng 12 chi nhánh sau
Theo khu vực địa lý (Geographic)	Ra mắt ở một khu vực trước	Chỉ Quận 1, Quận 3 trước, mở rộng toàn TP.HCM sau
Theo mức độ hoàn chỉnh (Walking Skeleton)	Xây một luồng end-to-end tối giản trước (dù đơn giản), rồi bổ sung chi tiết dần	Luồng đặt-thanh toán-nhận đơn tối giản (chưa có UI đẹp, chưa có thông báo) chạy được trước, rồi hoàn thiện dần

"Walking Skeleton" đặc biệt hữu ích ở giai đoạn đầu dự án mới: thay vì hoàn thiện 100% một Epic rồi mới sang Epic khác, xây một luồng **mỏng nhưng xuyên suốt toàn bộ hệ thống** trước — giúp phát hiện sớm các vấn đề tích hợp giữa các thành phần (app, backend, cổng thanh toán, hệ thống chi nhánh) mà chỉ khi ráp nối thực tế mới lộ ra.

28.4 Thách thức khi áp dụng Incremental Delivery

- **Cần thiết kế kiến trúc đủ linh hoạt:** để thêm Epic mới không phải viết lại phần đã có — đây là lý do Simple Design và Refactoring (XP, Chương 9) quan trọng, giúp code luôn sẵn sàng mở rộng.
- **Quản lý kỳ vọng của Sponsor:** cần giải thích rõ ra mắt "một phần" không có nghĩa là "làm dở dang" — mỗi phần ra mắt đều hoàn chỉnh, chỉ là phạm vi nhỏ hơn (liên hệ Definition of Done — Chương 24).
- **Ảnh hưởng thương hiệu nếu ra mắt sớm nhưng có lỗi:** cần cân nhắc ra mắt giới hạn (như thử nghiệm 2 chi nhánh) trước khi mở rộng toàn bộ, để giảm rủi ro ảnh hưởng uy tín trên diện rộng.

Bài tập

1. Với dự án FoodNow, tự đề xuất một kế hoạch chia Incremental theo 1 trong 4 chiến lược ở mục 28.3 (khác với ví dụ đã có), giải thích lý do chọn chiến lược đó.
2. Giải thích bằng ví dụ cụ thể của riêng bạn (không dùng lại ví dụ FoodNow) sự khác biệt giữa "Incremental" và "Iterative".

Sai lầm thường gặp

- **Nhầm lẫn hoàn toàn Incremental và Iterative, dùng thay thế cho nhau:** dù thực tế thường đi cùng nhau, hiểu đúng bản chất giúp BA giải thích chiến lược giao hàng chính xác hơn với Sponsor.
- **Chia Incremental theo tầng kỹ thuật thay vì theo giá trị nghiệp vụ** (tương tự lỗi ở Chương 24) — ví dụ "increment 1: xong backend, increment 2: xong frontend" — không mang lại giá trị dùng được ở mỗi increment.
- **Không chuẩn bị tâm lý cho Sponsor về việc ra mắt từng phần:** nếu Sponsor kỳ vọng "ra mắt là phải đầy đủ mọi thứ ngay", cần giải thích rõ lợi ích của Incremental Delivery (mục 28.2) trước khi bắt đầu, tránh hiểu lầm giữa chừng dự án.

Tóm tắt & tiếp theo

Incremental (thêm dần từng phần hoàn chỉnh) và Iterative (cải thiện dần cùng một phần) là hai khái niệm khác nhau nhưng thường kết hợp trong Agile — giao hàng từng phần giúp phát hiện sớm vấn đề với chi phí thấp hơn nhiều so với đặt cược vào một lần ra mắt duy nhất. Chương 29 sẽ học cách biến chiến lược Incremental thành một **kế hoạch cụ thể, có tài liệu** — Release Plan và Incremental Delivery Plan, kèm mẫu đầy đủ.

Chương 29: Release Plan & Delivery Document: mẫu Incremental Delivery Plan

Mục tiêu học

- Nắm được cấu trúc của một Release Plan/Incremental Delivery Plan.
 - Biết cách trình bày kế hoạch giao hàng theo increment cho stakeholder không rành kỹ thuật.
 - Có sẵn mẫu Incremental Delivery Plan đầy đủ trong `templates/incremental-delivery/`.
-

29.1 Từ chiến lược đến kế hoạch cụ thể

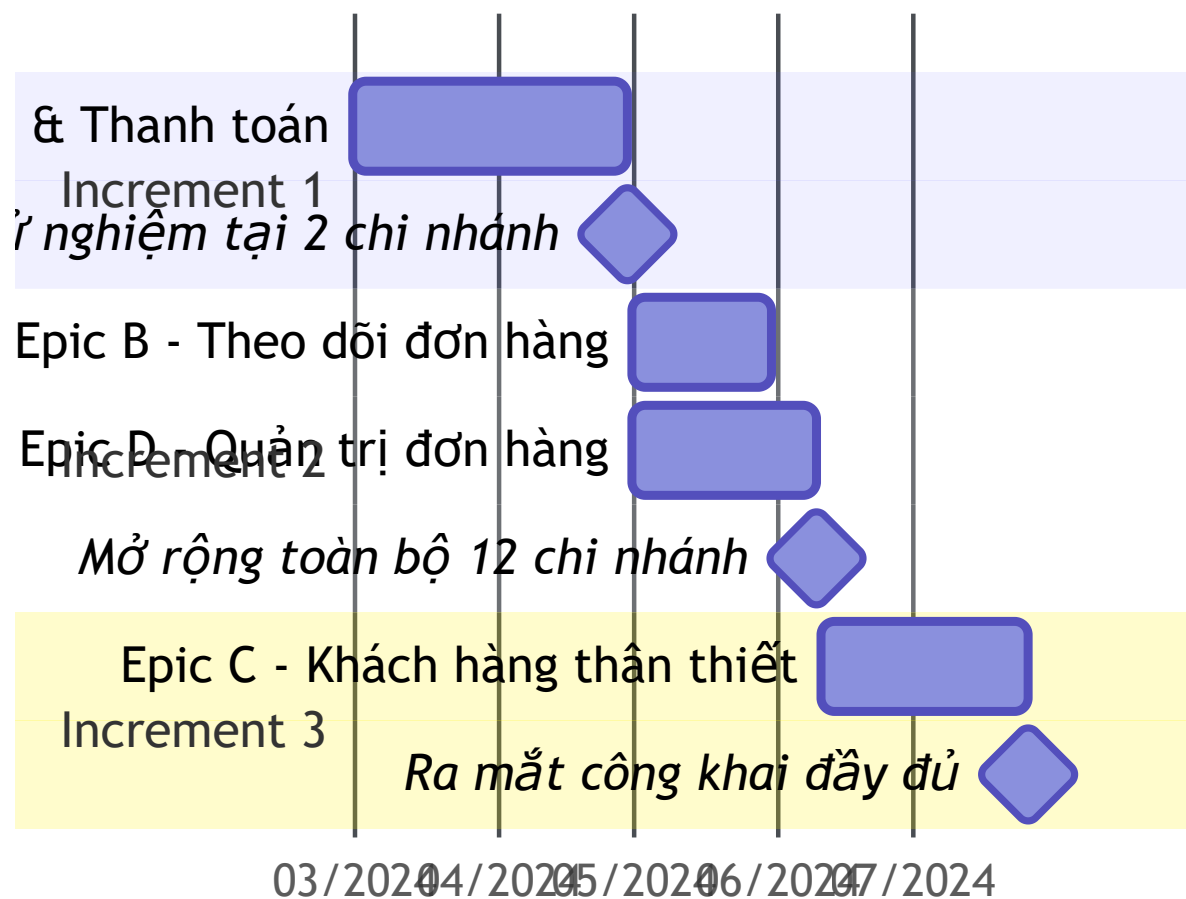
Chương 28 giới thiệu khái niệm và chiến lược Incremental Delivery — Chương này biến chiến lược đó thành một **tài liệu cụ thể**, giúp mọi stakeholder (kể cả người không rành kỹ thuật như Ban giám đốc) hiểu rõ: cái gì sẽ có, khi nào có, và làm sao đo được thành công của mỗi giai đoạn.

29.2 Cấu trúc một Incremental Delivery Plan

1. **Tổng quan:** liên kết ngược về mục tiêu kinh doanh (BRD — Chương 19) và chiến lược chia increment đã chọn (Chương 28).
2. **Bảng các Increment:** với mỗi increment, nêu rõ:
 - Nội dung/phạm vi (Epic/Feature nào).
 - Đối tượng nhận (toàn bộ người dùng hay nhóm thử nghiệm giới hạn).
 - Thời gian dự kiến.
 - Tiêu chí thành công/tiêu chí để chuyển sang increment tiếp theo.
 - Rủi ro chính của increment đó.
3. **Sơ đồ timeline trực quan:** giúp stakeholder hình dung nhanh (Gantt đơn giản hoặc timeline).
4. **Kế hoạch truyền thông/đào tạo đi kèm mỗi increment** (nếu cần) — liên hệ Transition Requirements (Chương 14).
5. **Điều kiện điều chỉnh kế hoạch:** nêu rõ ai có quyền quyết định thay đổi thứ tự/nội dung increment nếu có phát sinh (liên hệ Change Request — Chương 18).

29.3 Ví dụ timeline trực quan cho FoodNow

Incremental Delivery Plan - FoodNow AI



29.4 Mẫu tài liệu đầy đủ

Bản Incremental Delivery Plan đầy đủ (bao gồm bảng chi tiết từng increment, tiêu chí thành công, rủi ro, kế hoạch đào tạo) cho dự án FoodNow được lưu tại:

`templates/incremental-delivery/FoodNow-IncrementalDeliveryPlan.md`

Xem bản đầy đủ tại Phụ lục D (cuối sách).

29.5 Trình bày kế hoạch cho Sponsor không rành kỹ thuật

Khi trình bày Incremental Delivery Plan cho Ban giám đốc, tránh dùng thuật ngữ kỹ thuật ("sprint", "backlog", "story point") — nên diễn đạt theo hướng **giá trị kinh doanh của từng giai đoạn**:

Thay vì nói: "Increment 1 hoàn thành Epic A trong 2 sprint 30 ngày."

Nên nói: "Trong 2 tháng đầu, khách hàng tại 2 chi nhánh thí điểm đã có thể đặt món và thanh toán qua app riêng — đây là bước đầu để chúng ta bắt đầu thu thập dữ liệu thực tế và giảm dần chi phí hoa hồng cho app bên thứ ba tại 2 chi nhánh này, trước khi mở rộng."

Cách diễn đạt này áp dụng lại nguyên tắc "song ngữ nghiệp vụ - kỹ thuật" đã học ở Chương 3.

Bài tập

1. Mở `templates/incremental-delivery/FoodNow-IncrementalDeliveryPlan.md`, chọn 1 increment, thử viết lại phần "Nội dung/phạm vi" của increment đó theo ngôn ngữ dành cho Ban giám đốc (không dùng thuật ngữ kỹ thuật), theo mẫu ở mục 29.5.
 2. Vẽ một sơ đồ Gantt đơn giản (theo cú pháp Mermaid ở mục 29.3, hoặc vẽ tay) cho một kế hoạch chia increment khác mà bạn đề xuất ở bài tập Chương 28.
-

Sai lầm thường gặp

- **Coi Incremental Delivery Plan là cố định, không cập nhật khi có thay đổi:** kế hoạch này cần được xem là "sống", cập nhật khi có Change Request được phê duyệt (Chương 18) — không phải tài liệu chốt một lần như SRS trong Waterfall.
 - **Không nêu rõ tiêu chí để chuyển sang increment tiếp theo:** dẫn đến tình trạng "cứ đến ngày là chuyển giai đoạn" bất kể increment trước có đạt tiêu chí thành công hay không.
 - **Trình bày kế hoạch bằng thuật ngữ kỹ thuật cho Sponsor không rành kỹ thuật:** gây khó hiểu, giảm sự tin tưởng và tham gia của Sponsor vào quá trình theo dõi tiến độ.
-

Tóm tắt & tiếp theo

Incremental Delivery Plan biến chiến lược giao hàng từng phần thành tài liệu cụ thể với timeline, tiêu chí thành công, và rủi ro cho từng increment — cần trình bày bằng ngôn ngữ giá trị kinh doanh khi làm việc với Sponsor. Chương 30 sẽ học kỹ năng liên quan chặt chẽ: **ước lượng cơ bản** — làm sao BA phối hợp cùng Dev/QA ước lượng độ lớn công việc một cách hợp lý.

Chương 30: Estimation cơ bản cho BA: story points, T-shirt sizing

Mục tiêu học

- Hiểu vì sao ước lượng trong Agile dùng đơn vị tương đối (story point) thay vì thời gian tuyệt đối.
- Biết cách BA hỗ trợ (không thay thế) đội kỹ thuật trong quá trình ước lượng.
- Áp dụng được kỹ thuật T-shirt sizing và Planning Poker ở mức cơ bản.

30.1 Vì sao dùng đơn vị tương đối thay vì giờ/ngày?

Nhiều người mới thắc mắc: "Sao không ước lượng thẳng bằng số giờ/ngày cho dễ hiểu?" — vấn đề là con người **ước lượng thời gian tuyệt đối rất tệ** (ai cũng từng nói "chắc 2 tiếng là xong" rồi mất cả ngày), nhưng lại **so sánh độ phức tạp tương đối khá tốt** ("việc B khó gấp đôi việc A" để đánh giá hơn "việc B mất chính xác bao nhiêu giờ").

Story point là đơn vị đo độ phức tạp/công sức **tương đối**, không map trực tiếp sang giờ/ngày — thường dùng dãy số Fibonacci (1, 2, 3, 5, 8, 13, 21...) vì khoảng cách giữa các số tăng dần phản ánh đúng thực tế: càng việc lớn, càng khó ước lượng chính xác, nên không cần phân biệt "13 hay 14 điểm" — chỉ cần biết nó lớn hơn nhiều so với việc 3 điểm.

30.2 Vai trò của BA trong Estimation — hỗ trợ, không thay thế Dev

Nguyên tắc quan trọng nhất: BA không tự ước lượng thay Dev. Người ước lượng độ phức tạp kỹ thuật phải là người **trực tiếp làm việc đó** (Dev, QA) — BA đóng vai trò:

- Đảm bảo User Story đủ rõ ràng để đội ước lượng chính xác (Definition of Ready — Chương 24, 27).
- Trả lời câu hỏi làm rõ ngay khi đội đang thảo luận ước lượng (ví dụ: "có cần hỗ trợ cả trường hợp khách hàng chưa đăng nhập không?" — câu trả lời ảnh hưởng trực tiếp đến độ phức tạp).
- Giúp đội hiểu **giá trị nghiệp vụ** đằng sau, để phân biệt việc "phức tạp về nghiệp vụ" (nhiều quy tắc, nhiều trường hợp) và "phức tạp về kỹ thuật" (khó implement dù quy tắc đơn giản) — hai loại phức tạp này ảnh hưởng khác nhau đến ước lượng.

30.3 Planning Poker — kỹ thuật ước lượng theo nhóm

Planning Poker là kỹ thuật phổ biến nhất để ước lượng story point theo nhóm, tránh hiện tượng một người có tiếng nói ảnh hưởng đến tất cả người khác (anchoring bias):

1. BA trình bày User Story và Acceptance Criteria.
2. Mỗi thành viên đội (thường là Dev/QA) **đồng thời** chọn một con số (từ bộ thẻ Fibonacci) thể hiện ước lượng của riêng mình, không xem trước con số của người khác.
3. Tất cả lật thẻ cùng lúc.
4. Nếu con số khác nhau nhiều (ví dụ người chọn 3, người chọn 13) — người có số cao nhất và thấp nhất giải thích lý do, cả nhóm thảo luận ngắn.
5. Lập lại vòng chọn cho đến khi đạt đồng thuận tương đối.

Kỹ thuật này giúp lộ ra ngay **những hiểu biết khác nhau về cùng một yêu cầu** — nếu một Dev nghĩ task đơn giản (3 điểm) trong khi Dev khác nghĩ rất phức tạp (13 điểm), đây thường là dấu hiệu yêu cầu **chưa đủ rõ ràng**, cần BA làm rõ thêm trước khi chốt ước lượng — một lợi ích phụ quan trọng của Planning Poker ngoài việc ra con số cuối cùng.

30.4 T-shirt Sizing — ước lượng nhanh, ít chính xác hơn

Khi cần ước lượng **nhANH, ở mức rất tổng quan** (ví dụ để PO sắp xếp độ ưu tiên sơ bộ cho hàng chục Epic trước khi refine chi tiết), dùng **T-shirt Sizing**: XS, S, M, L, XL — không cần họp Planning Poker đầy đủ, chỉ cần đội cho ý kiến nhanh, tương đối.

Kích cỡ	Ý nghĩa	Ví dụ FoodNow (ở mức Epic)
XS	Rất nhỏ, vài giờ đến 1 ngày	Thêm 1 trường thông tin vào form đặt hàng
S	Nhỏ, 1-3 ngày	US-402: xem số điểm hiện có
M	Vừa, khoảng 1 sprint	US-106: áp dụng mã giảm giá
L	Lớn, 2-3 sprint	Epic B: Theo dõi đơn hàng real-time
XL	Rất lớn, cần phân rã thêm trước khi ước lượng chính xác	Epic C: Chương trình khách hàng thân thiết

Một Epic được xếp XL là tín hiệu rõ ràng: **cần phân rã (Chương 24) trước khi ước lượng chi tiết bằng story point** — T-shirt Sizing không thay thế Planning Poker, chỉ là bước lọc sơ bộ trước đó.

30.5 Velocity — dùng ước lượng để lập kế hoạch

Sau vài sprint, đội tích lũy được **Velocity** (tốc độ trung bình) — tổng story point hoàn thành mỗi sprint. BA/PO dùng Velocity để dự đoán: với backlog còn lại tổng cộng bao nhiêu điểm, còn cần khoảng bao nhiêu sprint nữa — hữu ích khi Sponsor hỏi "khi nào xong?"

Ví dụ: Đội FoodNow có Velocity trung bình 25 điểm/sprint (2 tuần). Backlog còn lại của Epic C là 60 điểm → ước tính cần khoảng $60/25 \approx 2.4$ sprint, tức khoảng 5 tuần để hoàn thành Epic C.

Lưu ý quan trọng: Velocity chỉ đáng tin cậy sau vài sprint ổn định của **cùng một đội** — không nên so sánh Velocity giữa các đội khác nhau (story point không phải đơn vị tuyệt đối, mỗi đội có "thang đo" riêng), và Velocity của một đội mới thành lập thường chưa ổn định.

Bài tập

1. Chọn 4 User Story từ `templates/user-story-epic/FoodNow-UserStories.md`, tự ước lượng T-shirt Size cho mỗi story (dựa trên trực giác, không cần chính xác tuyệt đối) — giải thích lý do.
 2. Giải thích bằng lời của bạn: nếu trong một buổi Planning Poker, 3 Dev chọn "3 điểm" còn 1 Dev chọn "13 điểm" cho cùng một story, BA nên làm gì tiếp theo?
-

Sai lầm thường gặp

- **BA tự ước lượng thay Dev vì "sợ mất thời gian họp":** vi phạm nguyên tắc cốt lõi — người làm việc đó phải là người ước lượng, BA chỉ hỗ trợ làm rõ yêu cầu.
 - **Quy đổi story point sang giờ một cách máy móc** (ví dụ "1 điểm = 4 giờ" cố định): đi ngược bản chất đơn vị tương đối của story point — quy đổi này chỉ đúng tạm thời với một đội cụ thể, không áp dụng chung cho mọi đội/mọi thời điểm.
 - **So sánh Velocity giữa các đội khác nhau để đánh giá "đội nào giỏi hơn":** sai lầm phổ biến của quản lý thiếu kinh nghiệm Agile — Velocity không phải thước đo năng suất tuyệt đối, chỉ có ý nghĩa để lập kế hoạch nội bộ của chính đội đó.
-

Tóm tắt & tiếp theo

Story point là đơn vị ước lượng tương đối, BA hỗ trợ (không thay thế) đội kỹ thuật ước lượng thông qua làm rõ yêu cầu; Planning Poker giúp ước lượng chi tiết theo nhóm, T-shirt Sizing giúp ước lượng nhanh ở mức tổng quan, và Velocity giúp lập kế hoạch dựa trên tốc độ thực tế của đội. Chương 31 sẽ học công cụ quản lý rủi ro/vấn đề xuyên suốt dự án: **RAID Log**.

Chương 31: RAID Log: quản lý Risk, Assumption, Issue, Dependency

Mục tiêu học

- Giải thích được 4 thành phần của RAID: Risk, Assumption, Issue, Dependency — và phân biệt rõ ràng giữa chúng (để nhầm lẫn Risk với Issue).
- Xây dựng và duy trì được một RAID Log cho dự án thực tế.
- Biết cách đánh giá mức độ ưu tiên xử lý rủi ro bằng ma trận Xác suất — Tác động.

31.1 Bốn thành phần của RAID

Thành phần	Định nghĩa	Điểm khác biệt chính
R — Risk (Rủi ro)	Sự kiện có thể xảy ra trong tương lai, gây ảnh hưởng xấu (hoặc đôi khi tốt) đến dự án	Chưa xảy ra — mang tính xác suất
A — Assumption (Giả định)	Điều được cho là đúng mà chưa được xác minh chắc chắn, dự án dựa vào đó để lập kế hoạch	Nếu giả định sai, có thể phát sinh rủi ro/vấn đề mới
I — Issue (Vấn đề)	Sự kiện xấu đã thực sự xảy ra , cần xử lý ngay	Đã xảy ra — không còn là xác suất
D — Dependency (Phụ thuộc)	Việc gì đó cần hoàn thành trước, bên ngoài tầm kiểm soát trực tiếp của đội	Không phải "xấu" hay "tốt", chỉ là một ràng buộc về trình tự

Cách nhớ dễ nhất phân biệt Risk và Issue: Risk là "có thể sẽ" xảy ra, Issue là "đã" xảy ra rồi — khi một Risk trở thành hiện thực, nó **chuyển thành** một Issue, cần được ghi nhận lại và xử lý theo hướng khác (không còn phòng ngừa được, chỉ có thể khắc phục).

31.2 Ví dụ minh họa 4 loại cho dự án FoodNow

- **Risk:** "Khách hàng có thể không sẵn sàng chuyển từ app bên thứ ba sang app riêng của FoodNow" (chưa xảy ra, chỉ là khả năng — liên hệ mục 7 trong BRD, Chương 19).
- **Assumption:** "Giả định toàn bộ 12 chi nhánh có kết nối Internet ổn định để dùng hệ thống quản trị" (giả định chưa xác minh chắc chắn — nếu sai, phát sinh rủi ro mới).
- **Issue:** "Chi nhánh Quận 7 hiện đang mất kết nối Internet thường xuyên, không thể dùng hệ thống quản trị mới trong giờ cao điểm" (đã thực sự xảy ra — đây là hệ quả nếu Assumption ở trên sai).
- **Dependency:** "Việc tích hợp cổng thanh toán VNPay phụ thuộc vào việc VNPay phê duyệt tài khoản merchant của FoodNow, dự kiến mất 2 tuần xử lý" (ràng buộc trình tự, ngoài tầm kiểm soát trực tiếp).

Lưu ý cách các mục này liên kết với nhau: Assumption sai → trở thành Issue thực tế; Dependency chưa hoàn thành đúng hạn → có thể trở thành Risk cho tiến độ chung.

31.3 Ma trận Xác suất — Tác động (Probability — Impact Matrix)

Không phải mọi Risk đều cần xử lý ngay — cần đánh giá theo 2 trục: **xác suất xảy ra** và **mức độ tác động** nếu xảy ra, để ưu tiên nguồn lực xử lý hợp lý.

Ma trận Xác suất - Tác động (Risk)



Risk nằm ở góc "Xác suất cao, Tác động cao" (ví dụ: chậm phê duyệt VNPAY merchant) cần **ưu tiên xử lý ngay** — có thể là bắt đầu thủ tục đăng ký merchant càng sớm càng tốt, ngay từ đầu dự án thay vì để đến gần lúc cần tích hợp mới bắt đầu.

31.4 Mẫu RAID Log

ID	Loại	Mô tả	Xác suất/Trạng thái	Tác động	Người phụ trách	Biện pháp xử lý	Ngày cập nhật
R-01	Risk	Khách hàng ngại chuyển từ app bên thứ ba sang app riêng	Cao	Cao	Phòng Marketing	Chương trình khuyến mãi ra mắt, ưu đãi độc quyền	2024-03-01
A-01	Assumption	Toàn bộ 12 chi nhánh có kết nối Internet ổn định	Chưa xác minh	Cao nếu sai	BA	Khảo sát thực tế từng chi nhánh trước Increment 2	2024-03-01
I-01	Issue	Chi nhánh Quận 7 mất kết nối Internet thường xuyên giờ cao điểm	Đã xảy ra	Trung bình	IT Admin	Lắp đặt đường truyền dự phòng (4G backup)	2024-03-15
D-01	Dependency	Tích hợp VNPay phụ thuộc phê duyệt merchant, dự kiến 2 tuần	Đang xử lý	Cao nếu trễ	BA + Trưởng nhóm Dev	Nộp hồ sơ đăng ký merchant ngay từ tuần đầu dự án	2024-02-05

31.5 Duy trì RAID Log — không phải "viết một lần rồi bỏ quên"

RAID Log cần được **rà soát định kỳ** (ví dụ: đầu mỗi Sprint Planning hoặc mỗi buổi họp tiến độ dự án), không phải tài liệu viết một lần khi bắt đầu dự án rồi không cập nhật:

- Risk có thể chuyển thành Issue (cần cập nhật loại, mức độ ưu tiên xử lý thay đổi hoàn toàn).
- Assumption có thể được xác minh đúng/sai (cần đóng lại hoặc chuyển thành Risk/Issue tương ứng).
- Dependency có thể đã hoàn thành (đóng lại) hoặc bị trễ (nâng mức độ rủi ro liên quan).

Bài tập

1. Thêm 2 mục mới vào RAID Log (mục 31.4) — mỗi mục một loại khác nhau trong 4 loại — dựa trên các yêu cầu/bối cảnh FoodNow đã học ở các chương trước.
2. Với Risk "Khách hàng ngại chuyển app" (R-01), giả sử sau 1 tháng ra mắt Increment 1, tỷ lệ chuyển đổi thực tế rất thấp — mô tả cách bạn sẽ cập nhật RAID Log để phản ánh Risk này đã trở thành Issue.

Sai lầm thường gặp

- **Nhầm lẫn Risk và Issue, ghi Issue nhưng gắn nhãn "Risk":** gây hiểu lầm về mức độ khẩn cấp — Issue cần xử lý ngay, Risk cần biện pháp phòng ngừa/theo dõi.
- **Không rà soát định kỳ RAID Log:** khiến tài liệu lỗi thời, mất tác dụng cảnh báo sớm.
- **Chỉ ghi nhận Risk/Issue mà bỏ qua Assumption và Dependency:** Assumption sai là nguồn gốc phổ biến của Issue phát sinh bất ngờ; Dependency bị trễ là nguyên nhân phổ biến của trễ tiến độ — cả 4 thành phần đều cần được theo dõi đầy đủ.

Tóm tắt & tiếp theo

RAID Log theo dõi 4 thành phần: Risk (chưa xảy ra), Assumption (chưa xác minh), Issue (đã xảy ra), Dependency (ràng buộc trình tự bên ngoài) — cần đánh giá theo ma trận Xác suất - Tác động và rà soát định kỳ, không viết một lần rồi bỏ quên. Đây là chương khép lại Phần 4. Từ Chương 32, sách bước vào Phần 5 — Giao tiếp, kiểm thử & nghiệm thu, bắt đầu với kỹ năng giao tiếp hiệu quả của BA với các cấp stakeholder khác nhau.

Phần 5 — Giao tiếp, kiểm thử & nghiệm thu

Chương 32: Giao tiếp hiệu quả của BA: email, họp, trình bày với stakeholder cấp cao

Mục tiêu học

- Viết được email yêu cầu/xác nhận thông tin rõ ràng, đi thẳng vào trọng tâm.
 - Chuẩn bị và điều hành được một buổi họp trao đổi yêu cầu hiệu quả.
 - Trình bày được nội dung phức tạp một cách ngắn gọn, thuyết phục với Ban giám đốc/Sponsor.
-

32.1 Nguyên tắc chung: viết cho người đọc, không viết cho bản thân

Sai lầm phổ biến nhất trong giao tiếp của BA mới vào nghề: viết/nói theo cách **dễ cho bản thân** (liệt kê đầy đủ mọi chi tiết mình biết) thay vì **dễ cho người nhận** (chỉ đưa thông tin họ cần, ở mức độ chi tiết phù hợp). Nguyên tắc "song ngữ nghiệp vụ - kỹ thuật" đã học ở Chương 3 áp dụng xuyên suốt chương này.

32.2 Viết email hiệu quả

Cấu trúc email hiệu quả cho BA — đặc biệt khi cần **xác nhận thông tin hoặc yêu cầu quyết định**:

1. **Tiêu đề rõ ràng, nêu rõ hành động cần:** "Cần xác nhận: giới hạn số món trong 1 đơn hàng" tốt hơn nhiều so với "Về vấn đề đặt hàng".
2. **Câu đầu tiên nêu ngay mục đích:** người bận rộn (như Sponsor) thường chỉ đọc 1-2 câu đầu.
3. **Nêu rõ điều cần từ người nhận:** xác nhận thông tin? Ra quyết định? Chỉ cần biết thông tin (FYI)? — nói rõ ngay, đừng để người đọc phải đoán.
4. **Đưa ra lựa chọn cụ thể thay vì câu hỏi mở** khi có thể: "Anh/chị chọn phương án A (giới hạn 20 món/đơn) hay B (không giới hạn)?" dễ trả lời nhanh hơn "Anh/chị nghĩ sao về giới hạn số món?"
5. **Nêu rõ hạn trả lời (deadline) nếu cần:** "Rất mong nhận phản hồi trước thứ Sáu (15/3) để kịp đưa vào Sprint Planning tuần sau."

Ví dụ email so sánh

Kém hiệu quả: "Chào anh, em đang làm phần giỏ hàng, có vài chỗ chưa rõ, không biết giới hạn số món thế nào, với lại giảm giá có áp dụng nhiều mã không, với cả có COD không ạ, anh xem giúp em."

Hiệu quả hơn:

Tiêu đề: Cần xác nhận 3 điểm về module Giỏ hàng — phản hồi trước Thứ Sáu 15/3

Chào anh,

Em cần anh xác nhận 3 điểm sau để hoàn thiện SRS module Giỏ hàng trước khi đội Dev bắt đầu Sprint 2:

1. Giới hạn số món/đơn: đề xuất tối đa 20 món/loại — anh xác nhận giúp em, hay có con số khác?
2. Mã giảm giá: chỉ cho áp dụng 1 mã/đơn (để đơn giản cho phiên bản đầu) — anh có đồng ý không?
3. Thanh toán COD: có áp dụng cho hình thức "lấy tại quán" không, hay chỉ áp dụng cho giao hàng tận nơi?

Rất mong nhận phản hồi trước Thứ Sáu (15/3) để kịp đưa vào Sprint Planning tuần sau.

Cảm ơn anh!

32.3 Điều hành một buổi họp trao đổi yêu cầu hiệu quả

- **Gửi agenda trước:** nêu rõ mục tiêu buổi họp, các điểm cần thảo luận, thời gian dự kiến cho mỗi điểm.
- **Bắt đầu bằng việc nêu lại mục tiêu:** "Hôm nay mình cần thống nhất 3 điểm về quy trình huỷ đơn hàng, dự kiến 30 phút."
- **Ghi chép công khai (share màn hình hoặc dùng bảng chung):** giúp mọi người xác nhận ngay tại chỗ nếu ghi sai ý, tránh hiểu lầm phát hiện muộn sau khi họp.
- **Chốt lại hành động cụ thể cuối buổi họp:** ai làm gì, đến khi nào — không kết thúc họp mà không có kết luận rõ ràng (liên hệ nguyên tắc ở Chương 17, mục 17.4 về review yêu cầu).
- **Gửi biên bản họp (meeting minutes) trong vòng 24 giờ:** khi thông tin còn mới, tránh để lâu gây quên chi tiết.

32.4 Trình bày với Ban giám đốc/Sponsor — nguyên tắc "Bottom Line Up Front"

Khi trình bày cho lãnh đạo cấp cao, thời gian thường rất hạn chế — áp dụng nguyên tắc "**Bottom Line Up Front**" (**BLUF**): nêu kết luận/đề xuất **ngay từ đầu**, sau đó mới giải thích chi tiết nếu họ muốn nghe thêm — ngược lại với thói quen kể chuyện theo trình tự thời gian (nêu bối cảnh → phân tích → kết luận ở cuối).

So sánh hai cách trình bày cùng một nội dung

Theo trình tự thời gian (không nên dùng với lãnh đạo bận rộn): "Tuần trước em có khảo sát 200 khách hàng, sau đó em phân tích dữ liệu, phát hiện ra là có tới 60% khách hàng nói họ sẽ dùng app riêng nếu có ưu đãi tốt hơn app hiện tại, nên em nghĩ mình nên làm chương trình khuyến mãi ra mắt..."

Theo BLUF (nên dùng): "Đề xuất: nên có chương trình khuyến mãi ra mắt để khuyến khích khách hàng chuyển sang app riêng. Lý do: khảo sát 200 khách hàng cho thấy 60% sẵn sàng chuyển nếu có ưu đãi tốt hơn hiện tại. Em cần anh/chị phê duyệt ngân sách khuyến mãi khoảng X đồng cho 2 tháng đầu."

Cấu trúc BLUF giúp lãnh đạo nắm được điều quan trọng nhất **ngay cả khi chỉ nghe 10 giây đầu** — rất phù hợp với đối tượng bận rộn, ít thời gian.

Bài tập

- Viết lại email sau theo cấu trúc hiệu quả ở mục 32.2: "Chào chị, em thấy phần thanh toán còn vài chỗ chưa chắc, không biết nếu lỗi mạng giữa chừng thì sao, chị xem giúp em với ạ."
- Chuẩn bị một đoạn trình bày ngắn (3-4 câu) theo nguyên tắc BLUF cho tình huống: bạn cần đề xuất trì hoãn Increment 2 của FoodNow thêm 1 tuần vì phát hiện rủi ro kỹ thuật mới.

Sai lầm thường gặp

- Viết email/trình bày quá dài, không nêu rõ hành động cần từ người nhận:** người bận rộn dễ bỏ qua hoặc trả lời chậm vì không rõ cần làm gì.
- Kể chuyện theo trình tự thời gian khi trình bày cho lãnh đạo cấp cao:** khiến người nghe mất kiên nhẫn trước khi nghe đến kết luận — luôn áp dụng BLUF trong tình huống thời gian hạn chế.
- Không gửi biên bản họp, để mọi người tự nhớ theo cách hiểu riêng:** dẫn đến tranh cãi về sau "lúc đó đâu có nói vậy" — luôn ghi chép và xác nhận công khai.

Tóm tắt & tiếp theo

Giao tiếp hiệu quả của BA cần viết/nói cho người đọc chứ không phải cho bản thân, dùng cấu trúc rõ ràng nêu hành động cần thiết, và áp dụng nguyên tắc BLUF khi trình bày với lãnh đạo cấp cao. Chương 33 sẽ đi sâu vào một kỹ năng liên quan chặt chẽ: **Facilitation** — điều phối một buổi workshop có nhiều bên tham gia, với các góc nhìn/lợi ích khác nhau.

Chương 33: Facilitation: điều phối workshop, requirement review meeting hiệu quả

Mục tiêu học

- Phân biệt vai trò "facilitator" (người điều phối) với vai trò người tham gia đóng góp ý kiến.
- Chuẩn bị và điều phối được một workshop có nhiều stakeholder với lợi ích/góc nhìn khác nhau.
- Xử lý được các tình huống khó thường gặp: người nói quá nhiều, người im lặng, mâu thuẫn gay gắt.

33.1 Facilitator khác gì với người tham gia thông thường?

Khi BA đóng vai trò **facilitator** trong một workshop, nhiệm vụ chính là **giúp cả nhóm đạt được mục tiêu chung** — không phải để tự mình đưa ra ý kiến/quan điểm cá nhân. Đây là điểm khác biệt quan trọng: một facilitator giỏi tập trung vào **quy trình thảo luận** (ai nói khi nào, làm sao đảm bảo mọi góc nhìn được lắng nghe, làm sao đi đến quyết định), trong khi nội dung/quan điểm là của những người tham gia khác đóng góp.

Điều này không có nghĩa BA hoàn toàn không được đóng góp ý kiến chuyên môn — chỉ là cần **tách bạch rõ ràng** hai vai trò trong đầu: lúc nào đang điều phối (giữ trung lập), lúc nào đang đóng góp ý kiến với tư cách BA có chuyên môn phân tích.

33.2 Chuẩn bị trước workshop

1. **Xác định rõ mục tiêu cụ thể:** "Thống nhất quy trình xử lý đơn hàng bị huỷ giữa các chi nhánh" cụ thể hơn nhiều so với "Bàn về vấn đề huỷ đơn".
2. **Xác định đúng người cần tham gia** (liên hệ Stakeholder Analysis — Chương 13): mời đủ người có thẩm quyền quyết định, tránh mời quá nhiều người không liên quan làm loãng buổi thảo luận.
3. **Chuẩn bị tài liệu/câu hỏi 미리 trước:** gửi trước để người tham gia có thời gian suy nghĩ, tránh mất thời gian "khởi động tư duy" ngay tại buổi họp.
4. **Chuẩn bị công cụ trực quan:** bảng trắng, sticky note (giấy nhớ), hoặc công cụ online (Miro, FigJam) để mọi người cùng nhìn thấy tiến trình thảo luận.
5. **Dự kiến trước các điểm có khả năng gây tranh cãi:** chuẩn bị sẵn cách dẫn dắt nếu xảy ra mâu thuẫn (ví dụ: giữa Phòng Marketing muốn thêm bước thu thập thông tin và UX muốn quy trình ngắn).

33.3 Kỹ thuật điều phối trong buổi workshop

Đảm bảo mọi góc nhìn được lắng nghe

- **Round-robin (lần lượt phát biểu):** mỗi người lần lượt nêu ý kiến ngắn gọn trước khi mở thảo luận tự do — tránh tình trạng người nói to/nói trước lấn át người khác.
- **Silent brainstorming trước khi thảo luận miệng:** mỗi người viết ý tưởng ra giấy nhớ trong 2-3 phút im lặng trước, sau đó mới dán lên bảng và thảo luận — giúp người hướng nội có cơ hội đóng góp bình đẳng với người hướng ngoại.

Xử lý người nói quá nhiều

- Nhẹ nhàng chuyển hướng: "Cảm ơn anh/chị, ý này rất hay, em ghi nhận lại. Bây giờ mình nghe thêm ý kiến từ phía chị B nhé."
- Nếu cần, đặt giới hạn thời gian rõ ràng cho mỗi lượt phát biểu ngay từ đầu buổi họp.

Xử lý người im lặng nhưng có thông tin quan trọng

- Chủ động hỏi trực tiếp: "Anh T bên vận hành chi nhánh, theo kinh nghiệm thực tế của anh, tình huống này có xảy ra thường xuyên không?"
- Dùng kỹ thuật silent brainstorming (ở trên) để những người ngại phát biểu trực tiếp vẫn đóng góp được ý kiến qua giấy nhớ.

Xử lý mâu thuẫn gay gắt giữa các bên

- **Không đứng về phe nào** — nhiệm vụ facilitator là giúp làm rõ **gốc rễ của mâu thuẫn** (thường là hai bên đang tối ưu cho hai mục tiêu khác nhau, cả hai đều hợp lý theo góc nhìn riêng).
- Dùng lại kỹ thuật "5 Whys" (Chương 12) để tìm hiểu vì sao mỗi bên coi trọng quan điểm của mình.
- Nếu không thể thống nhất ngay tại chỗ: ghi nhận là "vấn đề mở" (open item), phân công người quyết định cuối cùng theo RACI (Chương 13), hẹn thời gian cụ thể để giải quyết dứt điểm.

33.4 Ví dụ tình huống: workshop thống nhất quy trình xử lý đơn hàng bị huỷ

Bối cảnh: Phòng Marketing muốn cho phép khách hàng huỷ đơn bất cứ lúc nào (tăng trải nghiệm), trong khi Quản lý chi nhánh muốn giới hạn chặt (tránh lãng phí nguyên liệu đã chuẩn bị).

Cách facilitator xử lý:

1. Nêu lại mục tiêu chung: "Cả hai bên đều muốn khách hàng hài lòng và chi nhánh vận hành hiệu quả — mình cùng tìm phương án cân bằng cả hai."
2. Dùng "5 Whys" với Phòng Marketing: tại sao cần cho huỷ bất cứ lúc nào? → Vì khách hàng hay đặt nhầm, muốn có cơ hội sửa sai.

3. Dùng "5 Whys" với Quản lý chi nhánh: tại sao cần giới hạn chặt? → Vì nếu huỷ sau khi đã bắt đầu nấu, nguyên liệu lãng phí, ảnh hưởng chi phí.
 4. Từ 2 câu trả lời gốc rễ, đề xuất phương án cân bằng: cho phép huỷ **miễn phí trong 2 phút đầu** (đủ để sửa lỗi đặt nhầm), sau đó **không cho huỷ** (tránh lãng phí khi đã bắt đầu chuẩn bị) — đây chính là Use Case Exception Flow "7a" đã thấy ở Chương 22 và User Story US-204 (Chương 24)!
-

Bài tập

1. Với tình huống ở mục 33.4, giả sử Phòng Marketing vẫn không đồng ý với giới hạn 2 phút, muốn kéo dài thời gian huỷ lên 10 phút. Là facilitator, bạn sẽ dẫn dắt buổi thảo luận tiếp theo như thế nào để tìm phương án cả hai bên chấp nhận được?
 2. Liệt kê 2 kỹ thuật ở mục 33.3 mà bạn thấy phù hợp áp dụng cho một buổi họp nhóm bạn từng tham gia (trong học tập hoặc công việc) từng gặp vấn đề "một người nói quá nhiều, người khác im lặng".
-

Sai lầm thường gặp

- **Facilitator áp đặt quan điểm cá nhân thay vì giữ vai trò trung lập:** làm mất lòng tin của các bên, đặc biệt nếu quan điểm đó nghiêng về một phía có mâu thuẫn.
 - **Để buổi họp không có kết luận rõ ràng vì ngại làm mất lòng ai:** một facilitator giỏi vẫn cần thúc đẩy quyết định cuối cùng (hoặc ít nhất ghi nhận rõ "vấn đề mở" với người chịu trách nhiệm cụ thể), không để buổi họp kết thúc lấp lửng.
 - **Không chuẩn bị trước, ứng biến hoàn toàn tại chỗ:** dễ dẫn đến buổi họp lan man, không đạt mục tiêu — chuẩn bị kỹ (mục 33.2) là yếu tố quyết định thành công của workshop.
-

Tóm tắt & tiếp theo

Facilitation đòi hỏi BA giữ vai trò trung lập, tập trung vào quy trình thảo luận hơn là nội dung cá nhân, dùng các kỹ thuật cụ thể (round-robin, silent brainstorming, 5 Whys) để đảm bảo mọi góc nhìn được lắng nghe và xử lý mâu thuẫn một cách xây dựng. Chương 34 sẽ chuyển sang giai đoạn kiểm thử — mối quan hệ giữa BA và QA, cách viết test case cơ bản, và vai trò của BA trong UAT.

Chương 34: BA & QA: viết test case cơ bản, UAT — mẫu UAT checklist

Mục tiêu học

- Hiểu mối quan hệ hợp tác giữa BA và QA — không trùng lặp công việc, nhưng bổ sung cho nhau.
- Viết được test case cơ bản dựa trên Acceptance Criteria (Chương 23).
- Hiểu UAT (User Acceptance Testing) là gì, khác gì với các loại test khác, và vai trò cụ thể của BA trong UAT.
- Có sẵn mẫu UAT checklist trong `templates/uat/`.

34.1 BA và QA — ai làm gì, tránh trùng lặp

	BA	QA
Trọng tâm	Yêu cầu có đúng ý định nghiệp vụ không (Validation — Chương 17)	Sản phẩm có đúng theo đặc tả không (Verification), có lỗi kỹ thuật không
Loại kiểm tra chính	UAT (mục 34.3)	Unit test, Integration test, System test, Regression test
Nguồn thông tin	Acceptance Criteria, ý định nghiệp vụ thật	SRS, test case chi tiết kỹ thuật
Câu hỏi đặc trưng	"Đây có phải điều khách hàng thực sự cần không?"	"Chức năng này có hoạt động đúng như đặc tả trong mọi trường hợp không, kể cả các edge case kỹ thuật?"

BA và QA **không làm thay việc của nhau**, nhưng cần phối hợp chặt chẽ: BA cung cấp Acceptance Criteria rõ ràng để QA viết test case chính xác; QA phát hiện các trường hợp kỹ thuật mà BA có thể chưa nghĩ tới (ví dụ: điều gì xảy ra nếu mất kết nối mạng giữa chừng khi thanh toán) và hỏi lại BA để làm rõ ý định nghiệp vụ cho trường hợp đó.

34.2 Từ Acceptance Criteria đến Test Case

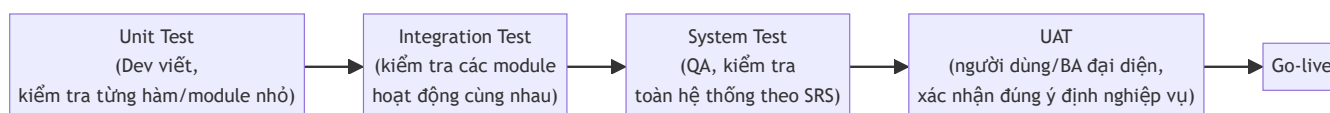
Chương 23 đã giới thiệu Acceptance Criteria theo Gherkin — đây gần như là "bản nháp" của test case. QA (hoặc BA hỗ trợ QA) chuyển hoá thành test case có cấu trúc chi tiết hơn:

Trường	Nội dung
Test Case ID	TC-PAY-05
Liên kết yêu cầu	FR-PAY-04, US-106 (liên hệ RTM — Chương 16)
Mô tả	Kiểm tra áp dụng mã giảm giá hợp lệ khi thanh toán
Điều kiện tiên quyết	Giỏ hàng có tổng 200.000đ; mã "FOODNOW10" đang hiệu lực, giảm 10%
Các bước thực hiện	1. Vào màn hình thanh toán. 2. Nhập mã "FOODNOW10". 3. Bấm "Áp dụng"
Kết quả mong đợi	Tổng tiền hiển thị giảm còn 180.000đ; thông báo "Đã áp dụng mã giảm giá FOODNOW10" xuất hiện
Kết quả thực tế	(QA điền khi thực hiện test)
Đạt/Không đạt	(QA điền khi thực hiện test)

Nhận thấy: Test Case TC-PAY-05 gần như là bản mở rộng trực tiếp từ "Scenario 1" trong Acceptance Criteria đã viết ở Chương 23 — đây chính là giá trị của việc viết Acceptance Criteria rõ ràng ngay từ đầu.

34.3 UAT (User Acceptance Testing) là gì?

UAT là vòng kiểm thử **cuối cùng**, thực hiện bởi (hoặc đại diện cho) **người dùng thực tế/ người có thẩm quyền nghiệp vụ** — không phải bởi Dev hay QA kỹ thuật — nhằm xác nhận sản phẩm sẵn sàng để đưa vào sử dụng thật. Đây là bước hiện thực hoá khái niệm **Validation** đã học ở Chương 17 ở giai đoạn cuối cùng trước khi go-live.



Khác biệt quan trọng nhất giữa System Test và UAT: System Test hỏi "hệ thống có đúng theo SRS không" (Verification); UAT hỏi "SRS (và sản phẩm dựa trên đó) có thực sự đúng ý định nghiệp vụ, dùng được trong thực tế không" (Validation) — một tính năng có thể **pass System Test hoàn toàn** (đúng 100% theo SRS) nhưng vẫn **fail UAT** nếu bản thân SRS đã hiểu sai ý định nghiệp vụ từ đầu.

34.4 Vai trò của BA trong UAT

- **Chuẩn bị kịch bản UAT:** dựa trên các use case/User Story quan trọng nhất (thường ưu tiên các luồng "Must have" theo MoSCoW — Chương 26), viết thành các kịch bản kiểm thử theo góc nhìn người dùng thực tế, không phải theo góc nhìn kỹ thuật.
- **Điều phối người tham gia UAT:** mời đúng đại diện người dùng thực tế (ví dụ: mời một vài nhân viên chi nhánh thực tế thử hệ thống quản trị đơn hàng, không chỉ để QA nội bộ tự test).

- **Tổng hợp phản hồi và phân loại:** phân biệt phản hồi nào là **lỗi thực sự cần sửa trước go-live** và phản hồi nào là **đề xuất cải tiến có thể làm sau** (liên hệ MoSCoW).
- **Xác nhận sign-off cuối cùng:** sau khi UAT đạt yêu cầu, BA phối hợp thu thập xác nhận chính thức từ người có thẩm quyền (Chương 35).

34.5 Mẫu UAT Checklist

Một mẫu UAT checklist đầy đủ cho Increment 1 của FoodNow (module Đặt món & Thanh toán) được lưu tại:

```
templates/uat/FoodNow-UAT-Checklist.md
```

Bài tập

1. Dựa vào Acceptance Criteria "Scenario 2" đã viết ở Chương 23 (mã giảm giá hết hạn), viết một Test Case đầy đủ theo cấu trúc ở mục 34.2.
2. Với module "Theo dõi đơn hàng" (Epic B), đề xuất 3 người/nhóm người bạn sẽ mời tham gia UAT, và giải thích vì sao chọn họ (không chỉ chọn người trong đội dự án).

Sai lầm thường gặp

- **Để chính đội Dev/QA đóng luôn vai trò UAT:** mất đi giá trị cốt lõi của UAT — cần góc nhìn của người dùng thực tế, những người chưa "quen mắt" với sản phẩm như đội phát triển.
- **Coi UAT chỉ là thủ tục hình thức, làm qua loa:** bỏ lỡ cơ hội cuối cùng để phát hiện sai lệch giữa yêu cầu và ý định nghiệp vụ thật — nhớ lại chi phí sửa lỗi tăng mạnh nếu phát hiện sau go-live (Chương 2).
- **Không phân loại rõ phản hồi UAT là "lỗi cần sửa ngay" hay "cải tiến làm sau":** dẫn đến trì hoãn go-live không cần thiết vì cố gắng sửa mọi phản hồi, kể cả những cải tiến không quan trọng.

Tóm tắt & tiếp theo

BA và QA bổ sung cho nhau: QA tập trung Verification (đúng đặc tả), BA tập trung Validation (đúng ý định nghiệp vụ) qua UAT — vòng kiểm thử cuối cùng với người dùng thực tế trước go-live. Chương 35 sẽ học bước tiếp theo sau khi UAT đạt yêu cầu: quy trình **Sign-off và nghiệm thu chính thức**.

Chương 35: Sign-off & nghiệm thu: quy trình ký duyệt, mẫu Sign-off Document

Mục tiêu học

- Hiểu Sign-off là gì, vì sao cần thiết dù dự án làm theo mô hình nào.
- Nắm được quy trình chuẩn để tổ chức một buổi nghiệm thu/sign-off.
- Biết xử lý tình huống khi có bên không đồng ý ký duyệt.
- Có sẵn mẫu Sign-off Document trong `templates/sign-off/`.

35.1 Sign-off là gì, vì sao cần thiết?

Sign-off là hành động **xác nhận chính thức** (thường bằng chữ ký hoặc xác nhận bằng văn bản) của người có thẩm quyền, thể hiện rằng một tài liệu/tính năng/giai đoạn dự án đã đạt yêu cầu và có thể tiến hành bước tiếp theo (ví dụ: go-live, chuyển sang giai đoạn kế tiếp, đóng dự án).

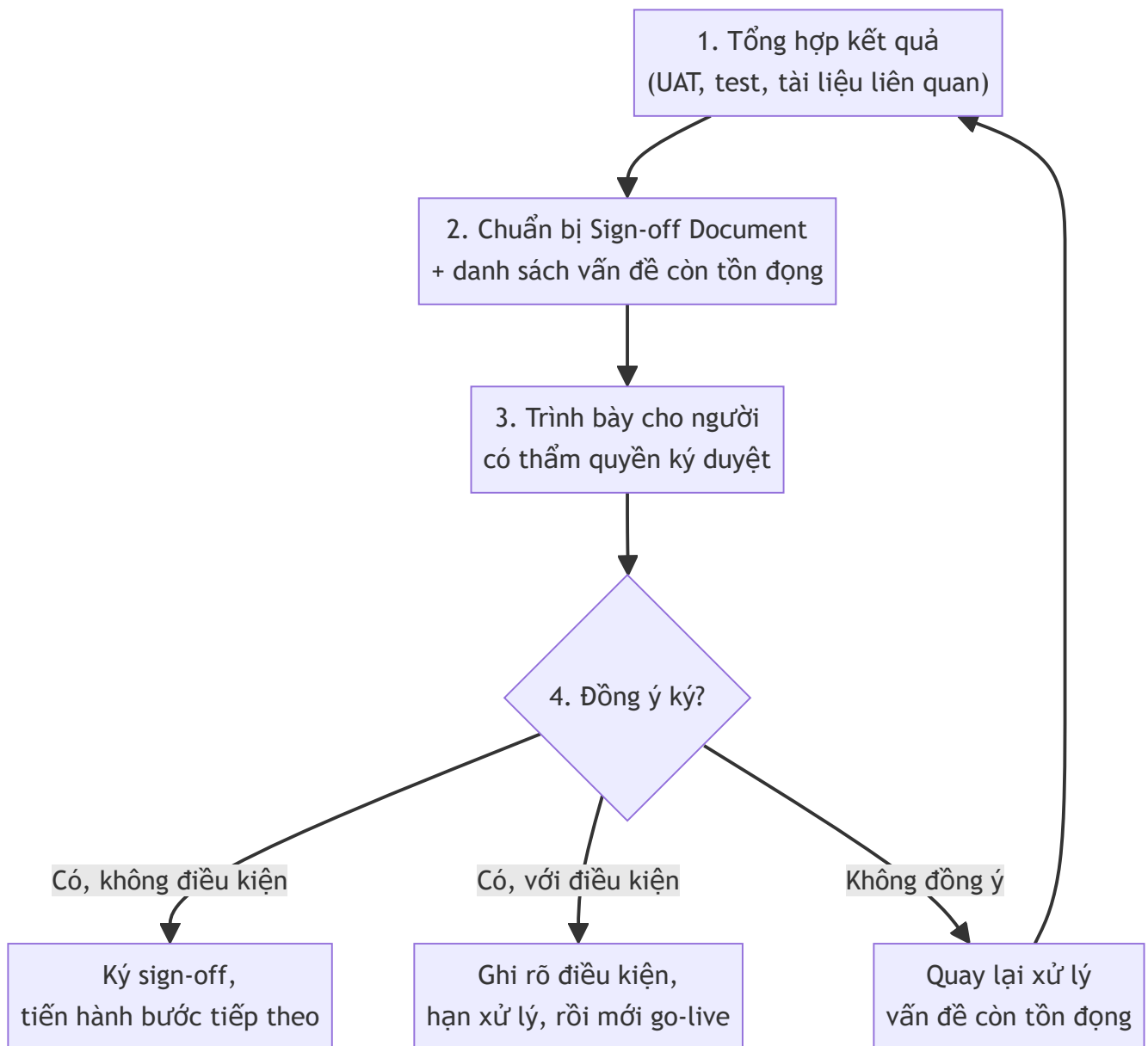
Vì sao cần thiết, kể cả trong Agile (nơi mọi thứ có vẻ "linh hoạt")?

- **Trách nhiệm giải trình rõ ràng:** nếu sau này phát sinh vấn đề, có bằng chứng ai đã xác nhận điều gì, khi nào — tránh tình trạng "tôi không nhớ đã đồng ý việc này".
- **Điểm dừng tâm lý quan trọng:** sign-off tạo ra một mốc rõ ràng để cả đội và stakeholder cùng thống nhất "đã xong, có thể tiến tiếp" — tránh tình trạng dự án kéo dài vô thời hạn vì không ai chính thức xác nhận hoàn thành.
- **Yêu cầu pháp lý/hợp đồng:** nhiều dự án (đặc biệt làm với nhà thầu bên ngoài) yêu cầu sign-off chính thức để làm căn cứ thanh toán theo từng giai đoạn.

35.2 Các loại Sign-off phổ biến

Loại	Khi nào	Ai ký
Sign-off BRD/SRS	Sau khi tài liệu yêu cầu hoàn tất, trước khi bắt đầu Design/Implementation (đặc biệt trong Waterfall — Chương 6)	Sponsor, các trưởng phòng liên quan
Sign-off UAT	Sau khi UAT (Chương 34) đạt yêu cầu, trước khi go-live	Đại diện người dùng, Product Owner/Sponsor
Sign-off giai đoạn/Increment	Sau mỗi Increment hoàn thành (Chương 28-29), xác nhận đủ điều kiện chuyển increment tiếp theo	PO, đại diện các bên liên quan đến increment đó
Sign-off đóng dự án (Project Closure)	Khi toàn bộ dự án hoàn thành, chuyển sang giai đoạn vận hành	Sponsor, PM

35.3 Quy trình tổ chức một buổi Sign-off



Bước 1-2: BA tổng hợp toàn bộ kết quả liên quan (ví dụ: kết quả UAT Checklist ở Chương 34), đặc biệt làm rõ **các vấn đề còn tồn đọng** (nếu có) — không che giấu để "dễ được ký" hơn, vì điều này gây rủi ro lớn hơn nhiều nếu vấn đề bộc lộ sau khi đã go-live.

Bước 3: Trình bày ngắn gọn theo nguyên tắc BLUF (Chương 32) — kết luận trước (đề xuất go-live hay chưa), sau đó giải thích căn cứ.

Bước 4: Ba khả năng xảy ra — đồng ý không điều kiện, đồng ý có điều kiện (ví dụ: "ký duyệt nhưng yêu cầu sửa lỗi X trong vòng 1 tuần sau go-live"), hoặc không đồng ý (cần quay lại xử lý vấn đề tồn đọng trước khi trình lại).

35.4 Xử lý khi có bên không đồng ý ký duyệt

Đây là tình huống nhạy cảm, cần xử lý khéo léo:

- **Không nên coi đây là "thất bại của BA"**: việc phát hiện vấn đề trước sign-off (thay vì sau go-live) chính là **giá trị của quy trình này đang hoạt động đúng** — nhớ lại nguyên lý chi phí sửa lỗi tăng theo giai đoạn (Chương 2).
- **Làm rõ lý do cụ thể không đồng ý**: dùng kỹ thuật elicitation (Chương 12) để hiểu chính xác vấn đề, tránh phản ứng phòng thủ.
- **Đề xuất phương án**: sign-off có điều kiện (nếu vấn đề nhỏ, không chặn go-live) hoặc hoãn sign-off đến khi xử lý xong (nếu vấn đề nghiêm trọng).
- **Ghi nhận đầy đủ vào biên bản**: dù kết quả thế nào, cần ghi chép lại quyết định và lý do để tham chiếu sau này.

35.5 Mẫu Sign-off Document

Mẫu Sign-off Document đầy đủ cho Increment 1 của FoodNow được lưu tại:

```
templates/sign-off/FoodNow-SignOff-Increment1.md
```

Bài tập

1. Giả sử kết quả UAT ở Chương 34 (mục 34.5, `templates/uat/FoodNow-UAT-Checklist.md`) có 2 kịch bản "Không đạt" (UAT-04 và UAT-09), trong đó UAT-04 là lỗi nghiêm trọng còn UAT-09 chỉ là vấn đề nhỏ về câu chữ thông báo lỗi. Đề xuất quyết định sign-off phù hợp (không điều kiện/có điều kiện/không đồng ý) và giải thích lý do.
2. Viết một đoạn hội thoại ngắn (giả định) giữa BA và Sponsor khi Sponsor ban đầu không đồng ý ký sign-off vì lo ngại tốc độ tải trang — áp dụng kỹ thuật elicitation để làm rõ mối lo ngại cụ thể.

Sai lầm thường gặp

- **Che giấu vấn đề tồn đọng để dễ được ký duyệt**: rủi ro rất lớn — vấn đề sẽ bộc lộ sau go-live với chi phí sửa cao hơn nhiều, và mất uy tín nghiêm trọng hơn khi bị phát hiện đã biết trước mà không báo cáo.
- **Coi sign-off là thủ tục hình thức, không thực sự trình bày đầy đủ căn cứ**: làm giảm giá trị của quy trình — người ký cần đủ thông tin để đưa ra quyết định có trách nhiệm, không phải ký cho có.

- **Không ghi chép rõ ràng khi sign-off có điều kiện:** dẫn đến tranh cãi sau này về việc điều kiện đã được thoả mãn hay chưa.
-

Tóm tắt & tiếp theo

Sign-off là bước xác nhận chính thức tạo trách nhiệm giải trình rõ ràng và điểm dừng tâm lý cho dự án — cần trình bày minh bạch cả kết quả tốt lẫn vấn đề tồn đọng, xử lý khéo léo khi có bên không đồng ý ký duyệt. Chương 36 sẽ khép lại Phần 5 với chủ đề đo lường thành công **sau khi** sản phẩm đã go-live: KPI, OKR, và thu thập phản hồi người dùng thực tế.

Chương 36: Đo lường thành công sau go-live: KPI, OKR, thu thập feedback

Mục tiêu học

- Phân biệt KPI và OKR — hai công cụ đo lường hay bị dùng lẫn lộn.
- Thiết kế được bộ KPI phù hợp để đo lường thành công của một tính năng/sản phẩm sau go-live.
- Xây dựng được vòng lặp thu thập phản hồi liên tục sau go-live, kết nối ngược lại vào backlog.

36.1 Vì sao công việc BA không kết thúc ở go-live?

Nhắc lại từ Chương 1 và Chương 2: nhiều BA mới nghĩ công việc kết thúc khi sản phẩm go-live thành công (sign-off — Chương 35). Thực tế, **go-live chỉ là điểm khởi đầu để kiểm chứng** liệu các mục tiêu kinh doanh đề ra từ BRD (Chương 19) có thực sự đạt được không — Business Requirements (như BO-01, BO-02, BO-03 của FoodNow) chỉ được xác nhận **sau một khoảng thời gian vận hành thực tế**, không phải ngay tại thời điểm ra mắt.

36.2 KPI vs OKR — khác nhau ở đâu?

	KPI (Key Performance Indicator)	OKR (Objectives and Key Results)
Bản chất	Chỉ số đo lường hiệu suất liên tục, thường theo dõi dài hạn	Khung đặt mục tiêu theo chu kỳ (thường theo quý), gồm Objective (định tính, truyền cảm hứng) + Key Results (định lượng, đo được)
Ví dụ	"Tỷ lệ chuyển đổi đặt hàng: 55%" (theo dõi liên tục)	"Objective: Trở thành kênh đặt hàng ưa thích nhất của khách hàng FoodNow. Key Result 1: Tỷ trọng đơn qua app riêng đạt 40%. Key Result 2: Điểm đánh giá app đạt 4.2/5"
Mục đích	Giám sát sức khỏe vận hành liên tục	Định hướng và tạo động lực cho một giai đoạn cụ thể, thường tham vọng hơn KPI thông thường
Quan hệ với BRD	KPI thường map trực tiếp từ "Chỉ số đo lường" trong Business Objectives (Chương 19, mục 2)	OKR có thể bao trùm nhiều KPI, đặt trong bối cảnh mục tiêu truyền cảm hứng hơn

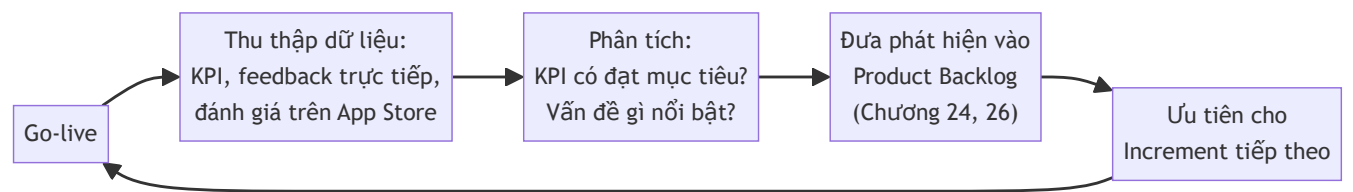
Trong thực tế, nhiều tổ chức dùng cả hai song song: **OKR** để định hướng ưu tiên theo quý (giúp trả lời "quý này tập trung vào điều gì"), **KPI** để giám sát liên tục sức khỏe vận hành hằng ngày.

36.3 Thiết kế bộ KPI cho FoodNow (sau go-live Increment 1)

KPI	Công thức/Cách đo	Mục tiêu	Nguồn dữ liệu
Tỷ lệ hoàn tất đặt hàng (Conversion rate)	Số đơn hoàn tất / Số lượt mở app đến bước giỏ hàng	≥ 60%	Analytics trong app
Thời gian trung bình hoàn tất đơn hàng	Thời gian từ mở app đến thanh toán xong, trung bình	< 2 phút	Analytics trong app
Tỷ lệ đơn hàng lỗi/khiếu nại	Số đơn có khiếu nại / Tổng số đơn	< 2%	Hệ thống hỗ trợ khách hàng
Tỷ lệ khách quay lại (Retention)	% khách đặt đơn thứ 2 trong vòng 30 ngày	≥ 35%	CRM/dữ liệu khách hàng
Tỷ trọng đơn qua app riêng	Số đơn qua app riêng / Tổng đơn online	≥ 40% (trong 6 tháng)	Đối soát doanh thu theo kênh

Các KPI này **truy vết trực tiếp** về Business Objectives (BO-01, BO-02, BO-03) và Success Criteria trong BRD (Chương 19) — một lần nữa áp dụng tư duy truy vết đã học ở Chương 16 (RTM), nhưng lần này áp dụng cho **giai đoạn sau go-live** thay vì giai đoạn phát triển.

36.4 Vòng lặp thu thập phản hồi liên tục



Đây là cách kết nối trực tiếp giữa "đo lường sau go-live" và "lập kế hoạch phát triển tiếp theo" — BA không chỉ báo cáo số liệu rồi dừng lại, mà cần **biến phát hiện thành hành động cụ thể** trong backlog. Ví dụ: nếu KPI "Tỷ lệ hoàn tất đặt hàng" chỉ đạt 45% (dưới mục tiêu 60%), BA cần điều tra nguyên nhân (có thể qua phỏng vấn khách hàng — Chương 12) và đề xuất User Story mới để cải thiện (ví dụ: đơn giản hoá luồng thanh toán nếu phát hiện khách hàng bỏ ngang ở bước đó).

36.5 Các nguồn thu thập phản hồi

- **Định lượng:** Analytics trong app (conversion, thời gian thao tác), đánh giá sao trên App Store/Google Play.
- **Định tính:** phỏng vấn/khảo sát khách hàng (Chương 12), phản hồi từ nhân viên chi nhánh trực tiếp vận hành hệ thống hằng ngày, bình luận trên App Store/mạng xã hội.

- **Bán định lượng:** Net Promoter Score (NPS) — hỏi khách hàng "Anh/chị có sẵn sàng giới thiệu app FoodNow cho bạn bè không, theo thang điểm 0-10?" — vừa đo lường được, vừa phản ánh cảm nhận thực.
-

Bài tập

1. Viết một OKR cho quý tiếp theo sau khi FoodNow hoàn thành cả 3 Increment (liên hệ lại BRD — Chương 19), theo cấu trúc Objective + 2-3 Key Results.
 2. Giả sử KPI "Tỷ lệ khách quay lại" chỉ đạt 20% (mục tiêu 35%). Đề xuất 2 hành động cụ thể (có thể liên quan đến chương trình khách hàng thân thiết — Epic C) để cải thiện, và giải thích cách bạn sẽ xác minh giả thuyết trước khi đầu tư phát triển.
-

Sai lầm thường gặp

- **Chỉ đo KPI mà không hành động dựa trên kết quả:** biến việc đo lường thành báo cáo hình thức, mất đi giá trị thực sự — luôn cần vòng lặp khép kín như mục 36.4.
 - **Đặt KPI không liên kết với Business Objectives ban đầu:** đo những con số "dễ đo" nhưng không phản ánh đúng mục tiêu kinh doanh thật sự của dự án.
 - **Coi go-live là điểm kết thúc công việc của BA:** bỏ lỡ giai đoạn quan trọng để chứng minh (hoặc phát hiện sớm nếu chưa đạt) giá trị thực sự mà dự án mang lại.
-

Tóm tắt & tiếp theo

KPI theo dõi liên tục sức khoẻ vận hành, OKR định hướng mục tiêu theo chu kỳ — cả hai cần truy vết về Business Objectives ban đầu, và phát hiện từ đo lường cần được đưa vào vòng lặp cải tiến backlog liên tục, không dừng lại ở việc báo cáo số liệu. Đây là chương khép lại Phần 5. Từ Chương 37, sách bước vào Phần 6 — Công cụ & kỹ năng nâng cao, bắt đầu với cách dùng Jira & Confluence hiệu quả trong công việc BA hằng ngày.

Phần 6 — Công cụ & kỹ năng nâng cao

Chương 37: Jira & Confluence cho BA: quản lý backlog, viết & liên kết tài liệu

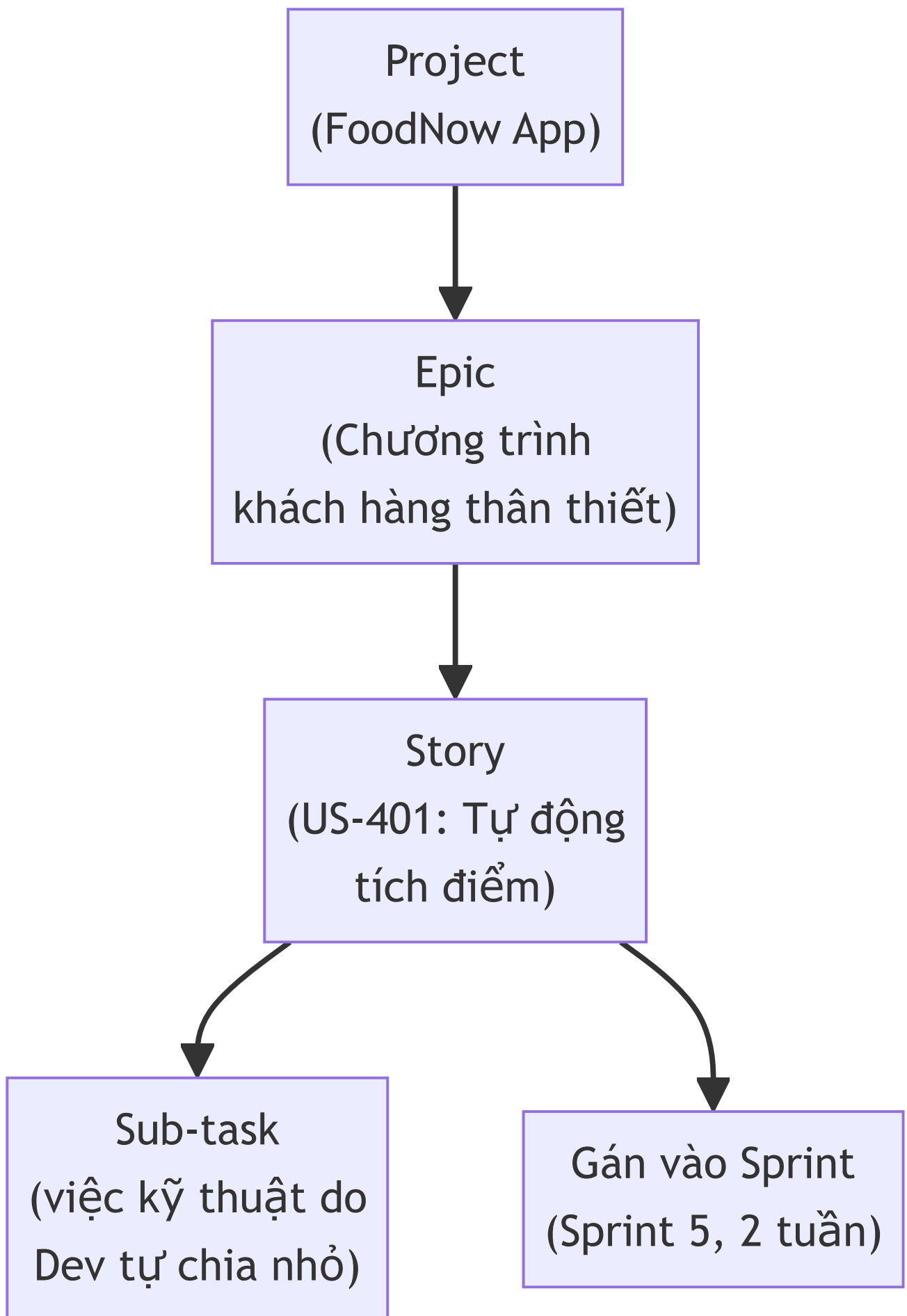
Mục tiêu học

- Hiểu cấu trúc dữ liệu cơ bản của Jira (Epic, Story, Task, Sprint) và cách nó ánh xạ vào những khái niệm đã học (Chương 8, 24).
 - Dùng Confluence để tổ chức tài liệu BA (BRD/SRS/User Story) liên kết với Jira.
 - Nắm được vài thao tác/thiết lập cơ bản BA cần biết — không đi sâu vào việc quản trị Jira.
-

37.1 Vì sao BA cần biết Jira & Confluence, ở mức nào?

Đây là bộ công cụ phổ biến nhất trong ngành phần mềm hiện nay để quản lý backlog (Jira) và viết/ lưu trữ tài liệu (Confluence). Như đã nói ở Chương 5, sách này **không hướng dẫn cấu hình/quản trị hệ thống** (đó là việc của PM/IT Admin) — chỉ tập trung vào **quy trình làm việc hằng ngày** mà một BA cần thành thạo.

37.2 Cấu trúc dữ liệu trong Jira, ánh xạ vào khái niệm đã học



Khái niệm Jira	Tương ứng với	Chương liên quan
Epic	Epic (nhóm User Story lớn)	Chương 24
Story	User Story	Chương 23-24
Sub-task	Task kỹ thuật (Dev tự quản lý)	Chương 24
Sprint	Sprint trong Scrum	Chương 8
Board (Scrum/Kanban)	Trực quan hoá Sprint Backlog hoặc luồng Kanban	Chương 8, 10
Story Points (custom field)	Ước lượng tương đối	Chương 30
Epic Link/Parent Link	Liên kết truy vết Story → Epic	Chương 16 (RTM)

Việc BA cần làm thành thạo trong Jira:

- Tạo Epic và Story đúng cấu trúc (vai trò - nhu cầu - lợi ích, kèm Acceptance Criteria trong phần mô tả hoặc trường riêng).
- Sắp xếp thứ tự ưu tiên trong Backlog view (kéo-thả theo độ ưu tiên MoSCoW/WSJF — Chương 26).
- Gắn nhãn (label)/thành phần (component) để lọc, báo cáo theo nhóm tính năng.
- Theo dõi Sprint Board trong Daily Scrum (Chương 8), cập nhật trạng thái khi cần trao đổi thêm.
- Dùng bộ lọc (JQL cơ bản hoặc filter có sẵn) để tra cứu nhanh: "tất cả Story thuộc Epic C chưa Done", "tất cả Story tôi đang phụ trách làm rõ yêu cầu".

37.3 Confluence — tổ chức tài liệu liên kết với Jira

Confluence là công cụ wiki nội bộ, nơi BA viết và lưu trữ BRD, SRS, biên bản họp, RAID Log... Điểm mạnh lớn nhất khi dùng chung với Jira: **liên kết hai chiều** — một trang Confluence (ví dụ SRS) có thể nhúng trực tiếp danh sách Story liên quan từ Jira (tự động cập nhật trạng thái), và ngược lại, một Story trong Jira có thể link ngược về trang Confluence chứa yêu cầu chi tiết.

Loại tài liệu	Nên lưu ở đâu	Lý do
BRD, SRS, Use Case Spec	Confluence	Văn bản dài, cần định dạng, versioning, review comment
User Story, Epic, Sprint Backlog	Jira	Cần trạng thái động, kéo-thả ưu tiên, theo dõi tiến độ
RAID Log	Confluence (dạng bảng) hoặc Excel/Jira issue riêng	Tuỳ quy mô — dự án nhỏ dùng Confluence table là đủ
Biên bản họp (meeting minutes)	Confluence	Dễ tìm kiếm lại, liên kết chéo với các trang liên quan

37.4 Ví dụ quy trình làm việc thực tế của BA với Jira + Confluence

1. Viết BRD/PRD/SRS đầy đủ trên Confluence (theo mẫu đã học Chương 19-21).
 2. Từ SRS, phân rã thành Epic + User Story (Chương 24), tạo trực tiếp trong Jira.
 3. Mỗi User Story trong Jira link ngược về mục tương ứng trong trang SRS trên Confluence (để Dev cần chi tiết hơn có thể bấm vào xem).
 4. Trong buổi Backlog Refinement (Chương 27), cập nhật trực tiếp Acceptance Criteria vào phần mô tả Story trong Jira.
 5. Sau Sprint Review, cập nhật kết quả/phản hồi vào biên bản trên Confluence, tạo Story mới trong Jira nếu có yêu cầu phát sinh (liên hệ Change Request — Chương 18).
-

37.5 Một vài mẹo thực dụng (không cần quyền admin)

- **Dùng Jira Filter đã lưu (Saved Filter)** để có view riêng theo dõi công việc của mình, tránh phải lọc lại thủ công mỗi ngày.
 - **Dùng template Confluence có sẵn** (nhiều tổ chức có sẵn template BRD/SRS/Meeting Notes) thay vì tạo trang trắng mỗi lần — giúp nhất quán định dạng giữa các tài liệu.
 - **Gắn nhãn phiên bản (version label)** trên các trang Confluence quan trọng để dễ theo dõi lịch sử thay đổi, đặc biệt hữu ích khi tài liệu qua nhiều lần Change Request (Chương 18).
-

Bài tập

1. Nếu bạn có tài khoản Jira/Confluence dùng thử (miễn phí cho cá nhân/nhóm nhỏ), thử tạo 1 Epic và 3 Story cho Epic C (Chương trình khách hàng thân thiết) của FoodNow, dựa theo nội dung ở `templates/user-story-epic/FoodNow-UserStories.md`.
 2. Thiết kế cấu trúc trang Confluence (chỉ cần liệt kê tên các trang và cách chúng liên kết nhau) cho toàn bộ tài liệu dự án FoodNow đã học từ Chương 19 đến Chương 31.
-

Sai lầm thường gặp

- **Trùng lặp thông tin giữa Jira và Confluence mà không liên kết:** dẫn đến hai nguồn thông tin không đồng bộ — chỉnh sửa ở một nơi nhưng quên cập nhật nơi kia.
- **Nhồi nhét toàn bộ chi tiết kỹ thuật vào mô tả Story trên Jira:** làm Story khó đọc — chi tiết dài nên để ở Confluence, Story trên Jira chỉ giữ đủ để Dev hiểu nhanh + link tham chiếu.
- **Không dùng Epic Link, để Story rời rạc không thuộc Epic nào:** mất khả năng theo dõi tiến độ tổng thể theo từng nhóm tính năng lớn.

Tóm tắt & tiếp theo

Jira quản lý backlog động (Epic/Story/Sprint), Confluence lưu trữ tài liệu tĩnh có cấu trúc (BRD/SRS/biên bản họp) — hai công cụ nên liên kết hai chiều để tránh trùng lặp và mất đồng bộ. Chương 38 sẽ học kỹ năng thường bị BA bỏ qua nhưng ngày càng cần thiết: đọc hiểu dữ liệu và viết SQL cơ bản.

Chương 38: Data & SQL cơ bản cho BA: đọc hiểu mô hình dữ liệu, SELECT query

Mục tiêu học

- Đọc hiểu được một sơ đồ ERD (Entity-Relationship Diagram) đơn giản.
- Viết được câu lệnh SELECT cơ bản để tự tra cứu dữ liệu, không cần nhờ Dev.
- Hiểu vì sao kỹ năng này ngày càng quan trọng với BA, dù không cần biết lập trình.

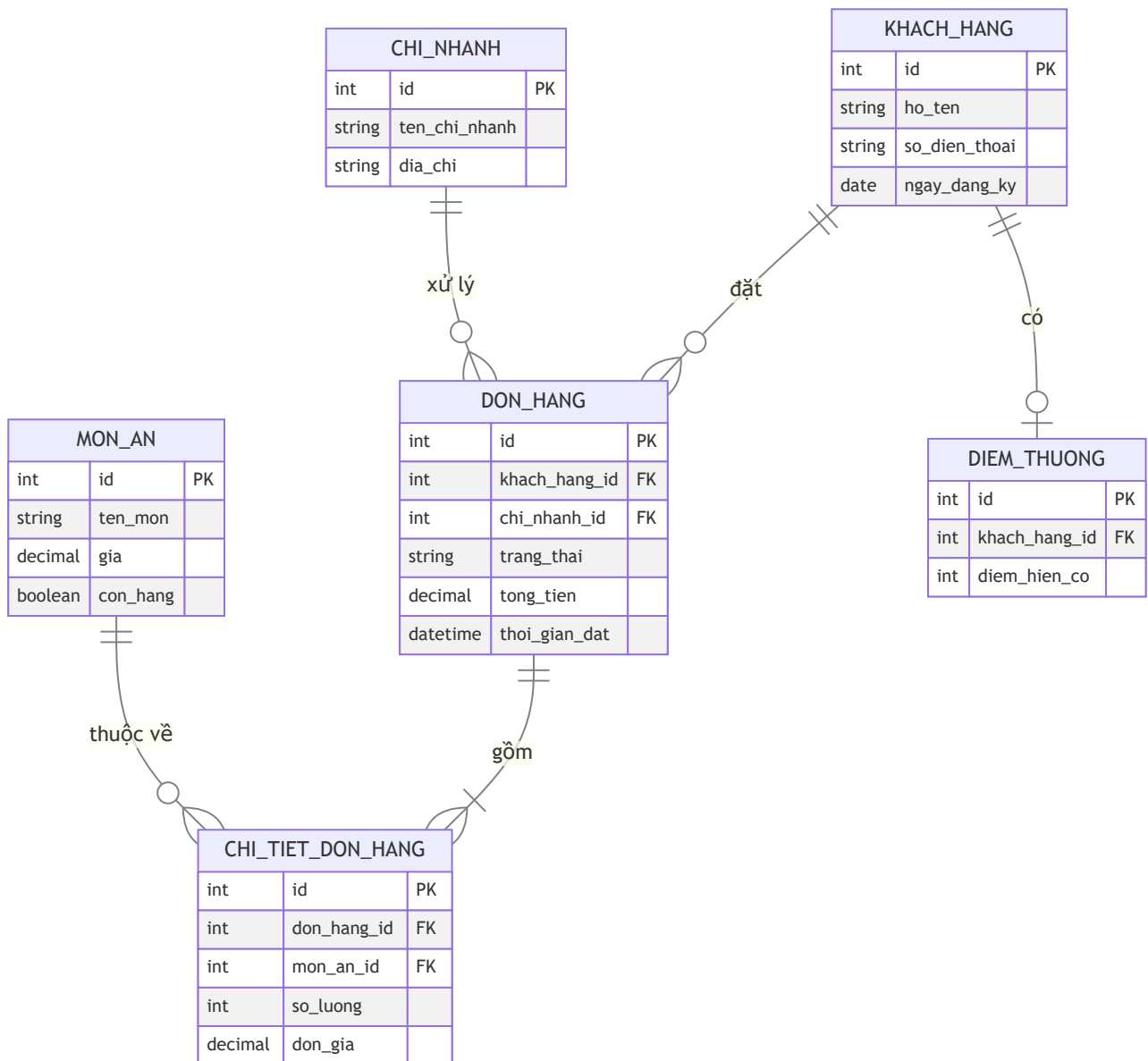
38.1 Vì sao BA cần biết SQL cơ bản?

Chương 5 đã nói: BA không cần biết lập trình. Nhưng **đọc hiểu dữ liệu** là một kỹ năng khác — ngày càng nhiều công việc BA đòi hỏi tự tra cứu số liệu thực tế thay vì luôn phải nhờ Dev/Data Analyst: kiểm tra một giả định trong RAID Log (Chương 31) có đúng không, xác minh một KPI (Chương 36) đang ở mức nào, hoặc chỉ đơn giản là hiểu rõ hơn cấu trúc dữ liệu khi viết yêu cầu (giúp yêu cầu chính xác và khả thi hơn).

Mức độ cần đạt: đọc hiểu ERD và viết được SELECT cơ bản (lọc, sắp xếp, nhóm) — **không cần** biết thiết kế database, viết stored procedure, hay tối ưu hiệu năng truy vấn (đó là công việc của Dev/DBA).

38.2 Đọc hiểu ERD (Entity-Relationship Diagram)

ERD mô tả các **bảng dữ liệu (entity)** và **mối quan hệ** giữa chúng — hữu ích để BA hiểu hệ thống lưu trữ thông tin gì, từ đó viết yêu cầu chính xác hơn.



Cách đọc ký hiệu quan hệ: **|—o{** nghĩa là "một-nhiều" (một khách hàng có thể có nhiều đơn hàng, nhưng mỗi đơn hàng chỉ thuộc về một khách hàng). Hiểu được mối quan hệ này giúp BA đặt đúng câu hỏi khi viết yêu cầu — ví dụ: "một đơn hàng có được phép chứa món từ nhiều chi nhánh khác nhau không?" (nhìn vào ERD, câu trả lời là KHÔNG — mỗi đơn hàng chỉ liên kết với một **chi_nhanh_id**).

38.3 Viết SELECT cơ bản

Truy vấn đơn giản: lấy dữ liệu từ một bảng

```
-- Lấy danh sách đơn hàng có tổng tiền trên 200.000đ, sắp xếp mới nhất trước
SELECT id, khách_hang_id, tong_tien, thoi_gian_dat
FROM DON_HANG
WHERE tong_tien > 200000
ORDER BY thoi_gian_dat DESC;
```

Truy vấn kết hợp nhiều bảng (JOIN)

```
-- Lấy tên khách hàng và tổng tiền các đơn hàng của họ trong tháng 3/2024
SELECT kh.ho_ten, dh.id AS ma_don, dh.tong_tien, dh.thoi_gian_dat
FROM DON_HANG dh
JOIN KHACH_HANG kh ON dh.khach_hang_id = kh.id
WHERE dh.thoi_gian_dat BETWEEN '2024-03-01' AND '2024-03-31'
ORDER BY dh.thoi_gian_dat;
```

Truy vấn tổng hợp (GROUP BY) — hữu ích để tự kiểm tra KPI

```
-- Đếm số đơn hàng và tổng doanh thu theo từng chi nhánh trong tháng 3/2024
SELECT cn.ten_chi_nhanh,
       COUNT(dh.id) AS so_don_hang,
       SUM(dh.tong_tien) AS tong_doanh_thu
FROM DON_HANG dh
JOIN CHI_NHANH cn ON dh.chi_nhanh_id = cn.id
WHERE dh.thoi_gian_dat BETWEEN '2024-03-01' AND '2024-03-31'
GROUP BY cn.ten_chi_nhanh
ORDER BY tong_doanh_thu DESC;
```

Câu lệnh trên chính là cách BA **tự kiểm tra** một phần dữ liệu cho KPI "Tỷ lệ hoàn tất đặt hàng theo chi nhánh" đã nói ở Chương 36, mà không cần chờ Dev/Data Analyst chạy báo cáo họ.

38.4 Bảng tổng hợp các mệnh đề SQL cơ bản cần biết

Mệnh đề	Chức năng	Ví dụ dùng cho FoodNow
<code>SELECT ... FROM</code>	Chọn cột, chọn bảng	Lấy danh sách đơn hàng
<code>WHERE</code>	Lọc điều kiện	Chỉ lấy đơn hàng có trạng thái "Đã giao"
<code>JOIN</code>	Kết hợp dữ liệu từ nhiều bảng	Lấy tên khách hàng cùng với đơn hàng của họ
<code>GROUP BY</code>	Nhóm dữ liệu để tổng hợp	Tổng doanh thu theo từng chi nhánh
<code>ORDER BY</code>	Sắp xếp kết quả	Sắp xếp đơn hàng mới nhất trước
<code>COUNT</code> , <code>SUM</code> , <code>AVG</code>	Hàm tổng hợp	Đếm số đơn, tính tổng/trung bình tiền

Bài tập

- Dựa vào ERD ở mục 38.2, viết một câu `SELECT` lấy danh sách 5 món ăn bán chạy nhất (theo tổng số lượng đã bán) trong tháng 3/2024.
- Giải thích bằng lời (không cần viết SQL): để trả lời câu hỏi "khách hàng nào đã tích lũy nhiều điểm thưởng nhất nhưng chưa từng đổi điểm lần nào", cần kết hợp (`JOIN`) những bảng nào trong ERD?

Sai lầm thường gặp

- Cố học sâu về tối ưu hiệu năng SQL, thiết kế database:** vượt quá phạm vi cần thiết cho BA — nên dừng ở mức đọc hiểu ERD và viết `SELECT` cơ bản, việc còn lại là chuyên môn của Dev/DBA.
- Chạy query trực tiếp trên database production mà không hỏi ý kiến Dev/DBA trước:** có thể ảnh hưởng hiệu năng hệ thống thật — nên dùng môi trường báo cáo/staging riêng nếu có, hoặc luôn hỏi trước khi truy vấn trên hệ thống thật.
- Không hiểu mối quan hệ một-nhiều/nhiều-nhiều trước khi viết yêu cầu liên quan đến dữ liệu:** dễ viết yêu cầu không khả thi hoặc mâu thuẫn với cấu trúc dữ liệu thực tế của hệ thống.

Tóm tắt & tiếp theo

BA nên biết đọc hiểu ERD và viết `SELECT` cơ bản (lọc, `JOIN`, `GROUP BY`) để tự tra cứu dữ liệu, hỗ trợ xác minh giả định và KPI mà không luôn phải nhờ Dev — không cần học sâu về thiết kế/tối ưu database. Chương 39 sẽ mở rộng góc nhìn sang **domain knowledge** — vì sao hiểu biết chuyên sâu về một ngành cụ thể (Banking, E-commerce, Insurance) tạo ra lợi thế cạnh tranh lớn cho BA.

Chương 39: Domain knowledge: BA trong Banking, E-commerce, Insurance

Mục tiêu học

- Hiểu vì sao domain knowledge (hiểu biết chuyên sâu về một ngành) là lợi thế cạnh tranh lớn của BA.
 - Nắm được đặc điểm nghiệp vụ đặc thù của 3 ngành phổ biến: Banking, E-commerce, Insurance.
 - Biết cách bắt đầu xây dựng domain knowledge cho một ngành mới khi chuyển việc/chuyển dự án.
-

39.1 Vì sao domain knowledge quan trọng?

Các kỹ thuật đã học xuyên suốt cuốn sách (elicitation, viết User Story, RTM...) là **kỹ năng tổng quát**, áp dụng được cho mọi ngành. Nhưng một BA có thêm **domain knowledge** — hiểu sâu về đặc thù nghiệp vụ, quy định, thuật ngữ riêng của một ngành cụ thể — sẽ:

- Elicitation hiệu quả hơn nhiều: biết trước câu hỏi nào quan trọng, tránh bỏ sót yêu cầu đặc thù ngành mà người ngoài ngành không biết để hỏi.
 - Được tin tưởng hơn từ stakeholder: nói đúng thuật ngữ, hiểu đúng ràng buộc, tạo cảm giác "đây là người hiểu việc của chúng tôi", không chỉ là "người biết viết tài liệu".
 - Phát hiện rủi ro/quy định tuân thủ mà BA thiếu domain knowledge để bỏ sót — hậu quả có thể rất nghiêm trọng ở các ngành có quy định pháp lý chặt (Banking, Insurance).
-

39.2 Banking (Ngân hàng)

Đặc điểm nghiệp vụ đặc thù:

- Quy định pháp lý rất chặt chẽ (Ngân hàng Nhà nước, chống rửa tiền — AML, xác thực khách hàng — KYC/eKYC).
- Yêu cầu bảo mật cực cao (giao dịch tài chính, dữ liệu nhạy cảm).
- Quy trình phê duyệt nhiều lớp (ví dụ: một khoản vay cần qua nhiều bước thẩm định, không chỉ 1 người quyết định).
- Thường ưu tiên mô hình Waterfall hoặc Hybrid (Chương 6, 11) cho các hệ thống lõi (core banking) vì yêu cầu tài liệu hoá đầy đủ phục vụ kiểm toán, nhưng có thể dùng Agile cho các kênh số (mobile banking, ứng dụng khách hàng) ít ràng buộc pháp lý hơn.

Thuật ngữ đặc thù BA cần biết: KYC (Know Your Customer), AML (Anti-Money Laundering), core banking system, giao dịch treo (pending transaction), hạn mức tín dụng (credit limit).

Ví dụ tình huống: khi phân tích yêu cầu "Cho phép khách hàng chuyển khoản qua app", BA có domain knowledge ngân hàng sẽ tự động hỏi thêm: hạn mức chuyển khoản/ngày là bao nhiêu? Có cần xác thực 2 lớp (OTP) cho giao dịch trên một ngưỡng nhất định không? Giao dịch có cần được ghi log phục vụ audit theo quy định không? — những câu hỏi này không phải ai cũng tự nghĩ ra nếu chưa quen ngành.

39.3 E-commerce (Thương mại điện tử)

Đặc điểm nghiệp vụ đặc thù:

- Quản lý tồn kho (inventory) phức tạp, đặc biệt khi bán trên nhiều kênh cùng lúc (đa kênh — omnichannel).
- Logistics và vận chuyển: nhiều đối tác giao hàng, tính phí ship theo khu vực/trọng lượng.
- Khuyến mãi/giá phức tạp: mã giảm giá, flash sale, giá theo số lượng mua, chương trình thành viên.
- Thường ưu tiên Agile/Scrum (Chương 8) vì thị trường thay đổi nhanh, cần phản hồi liên tục.

Thuật ngữ đặc thù BA cần biết: SKU (Stock Keeping Unit — mã quản lý từng biến thể sản phẩm), cart abandonment (bỏ giỏ hàng giữa chừng), fulfillment (quy trình xử lý đơn từ lúc đặt đến lúc giao), omnichannel.

Liên hệ trực tiếp: dự án FoodNow xuyên suốt sách này chính là một ví dụ e-commerce dạng đặt đồ ăn (food delivery) — các khái niệm như quản lý tồn kho món ăn theo chi nhánh (FR-CART-04, Chương 20), tính phí/bán kính giao hàng (FR-DELIVERY-02) đều là ví dụ domain knowledge đặc thù ngành này.

39.4 Insurance (Bảo hiểm)

Đặc điểm nghiệp vụ đặc thù:

- Quy trình thẩm định rủi ro (underwriting) phức tạp trước khi phát hành hợp đồng bảo hiểm.
- Quy trình xử lý bồi thường (claims processing) — nhiều bước xác minh, có thể liên quan điều tra gian lận.
- Sản phẩm bảo hiểm thường có nhiều điều khoản, điều kiện loại trừ (exclusion) phức tạp cần mô tả chính xác trong hệ thống.
- Thường là mô hình Hybrid: core system (tính phí, quản lý hợp đồng) theo Waterfall vì cần chính xác pháp lý cao; các kênh bán hàng số/ứng dụng chăm sóc khách hàng theo Agile.

Thuật ngữ đặc thù BA cần biết: underwriting (thẩm định rủi ro), premium (phí bảo hiểm), claim (yêu cầu bồi thường), policy (hợp đồng bảo hiểm), exclusion (điều khoản loại trừ).

39.5 Bảng so sánh nhanh 3 ngành

	Banking	E-commerce	Insurance
Mức độ ràng buộc pháp lý	Rất cao	Trung bình (bảo vệ dữ liệu, quyền lợi người tiêu dùng)	Rất cao
Mô hình phổ biến	Waterfall/Hybrid cho core, Agile cho kênh số	Agile/Scrum	Hybrid
Độ phức tạp nghiệp vụ đặc trưng	Quy trình phê duyệt nhiều lớp	Quản lý tồn kho, logistics, khuyến mãi	Thẩm định rủi ro, xử lý bồi thường
Rủi ro nếu BA thiếu domain knowledge	Vi phạm quy định pháp lý, lỗ hổng bảo mật	Sai lệch tồn kho, tính phí ship sai	Sai điều khoản hợp đồng, xử lý bồi thường sai

39.6 Cách xây dựng domain knowledge khi vào ngành mới

1. **Đọc tài liệu quy định/chuẩn ngành** (nếu có) — ví dụ thông tư của Ngân hàng Nhà nước cho Banking, quy định bảo vệ người tiêu dùng cho E-commerce.
2. **Học từ tài liệu nội bộ của dự án trước đó** (BRD/SRS cũ, nếu có quyền truy cập) — cách nhanh nhất để thấy ngôn ngữ/quy trình thực tế công ty đang dùng.
3. **Chủ động hỏi chuyên gia nghiệp vụ (Subject Matter Expert - SME)**: đừng ngại hỏi "cơ bản" khi mới vào ngành — SME thường sẵn lòng giải thích nếu thấy bạn thực sự muốn hiểu sâu.
4. **Tự trải nghiệm sản phẩm với tư cách người dùng** (nếu có thể): tự mở tài khoản ngân hàng, tự mua hàng online, tự tìm hiểu mua một gói bảo hiểm — trải nghiệm trực tiếp giúp hiểu nhanh hơn đọc tài liệu.
5. **Tích lũy dần qua nhiều dự án cùng ngành**: domain knowledge sâu thường cần thời gian, không có đường tắt hoàn toàn — đây cũng là lý do nhiều BA chọn "chuyên sâu" một ngành thay vì nhảy ngành liên tục (liên hệ lộ trình sự nghiệp — Chương 4).

Bài tập

1. Chọn một ngành khác (không phải Banking/E-commerce/Insurance) mà bạn quan tâm, liệt kê 3 thuật ngữ đặc thù và 1 đặc điểm nghiệp vụ riêng của ngành đó mà một BA mới vào cần biết.
2. Áp dụng bước 3 ở mục 39.6 (hỏi SME): viết ra 3 câu hỏi bạn sẽ hỏi một chuyên gia vận hành nhà hàng thực tế nếu được phỏng vấn để hiểu sâu hơn về domain "chuỗi nhà hàng/F&B" cho dự án FoodNow.

Sai lầm thường gặp

- **Nghĩ kỹ năng phân tích tổng quát là đủ, không cần đầu tư domain knowledge:** dễ dẫn đến bỏ sót yêu cầu đặc thù ngành quan trọng, đặc biệt nguy hiểm ở ngành có ràng buộc pháp lý cao.
- **Ngại hỏi vì sợ bị đánh giá "không biết gì":** SME thường đánh giá cao người chủ động hỏi để hiểu đúng, hơn là người tự đoán sai rồi làm sai.
- **Áp dụng nguyên xi kinh nghiệm từ ngành cũ sang ngành mới mà không kiểm chứng:** mỗi ngành có đặc thù riêng — kinh nghiệm cũ là nền tảng tốt nhưng cần xác minh lại, không áp dụng máy móc.

Tóm tắt & tiếp theo

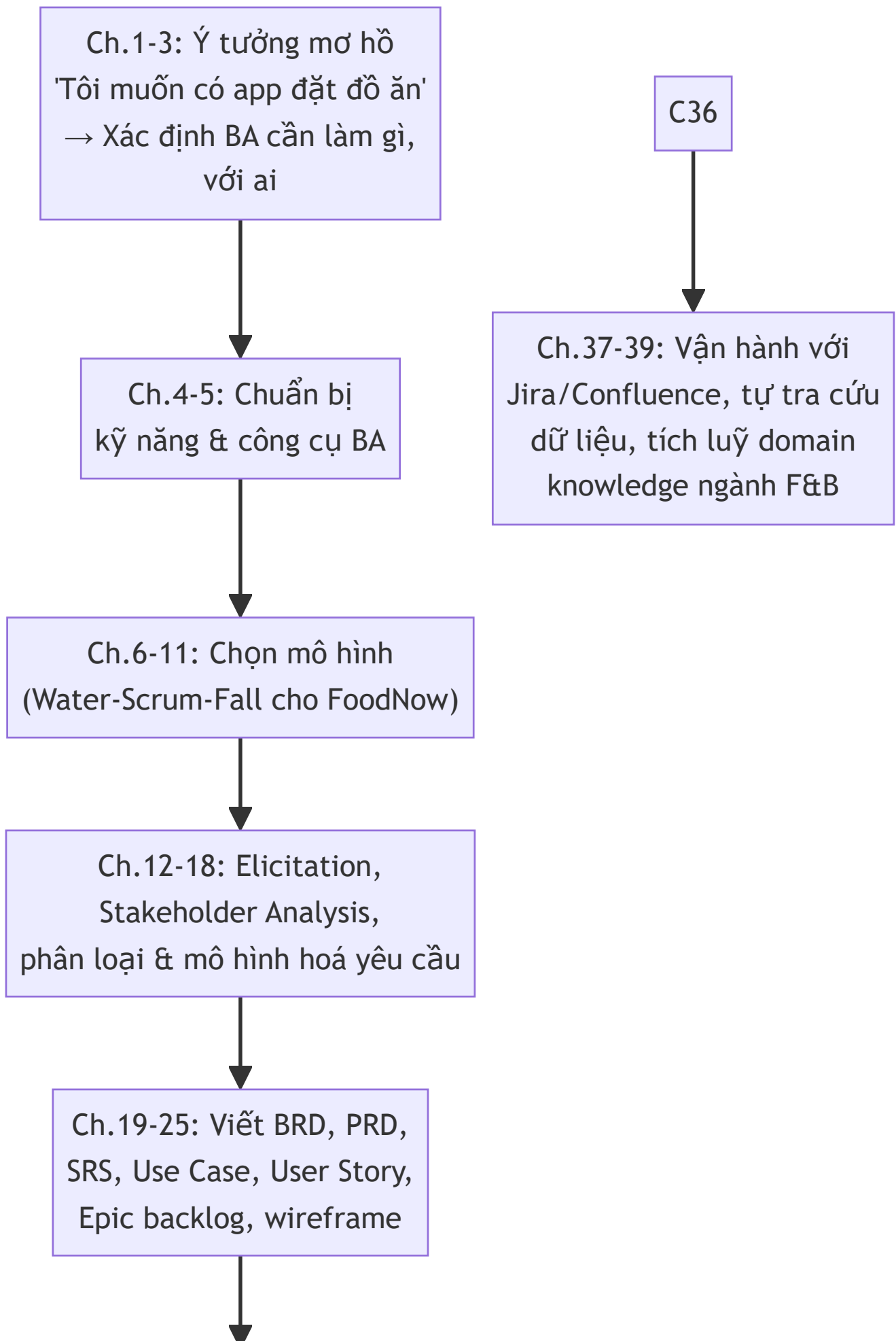
Domain knowledge tạo lợi thế cạnh tranh lớn cho BA — mỗi ngành (Banking, E-commerce, Insurance...) có đặc điểm nghiệp vụ, thuật ngữ, mô hình quản lý dự án phổ biến riêng, và xây dựng domain knowledge cần thời gian tích lũy qua tài liệu, SME, và trải nghiệm thực tế. Chương 40 — chương cuối cùng của phần nội dung chính — sẽ tổng kết toàn bộ sách qua một case study xuyên suốt: nhìn lại toàn bộ vòng đời dự án FoodNow từ góc nhìn BA, từ ý tưởng ban đầu đến sau go-live.

Chương 40: Case study tổng hợp "FoodNow": nhìn lại toàn bộ vòng đời dự án

Mục tiêu học

- Nhìn lại toàn bộ hành trình dự án FoodNow qua lăng kính BA, kết nối lại tất cả khái niệm đã học.
- Tự đánh giá lại bản thân theo bảng kỹ năng đã lập ở Chương 4.
- Có một bức tranh tổng thể để tham chiếu khi bắt đầu dự án thực tế đầu tiên của chính bạn.

40.1 Hành trình FoodNow từ đầu đến cuối



Ch.26-31: Ưu tiên hoá,
Sprint Planning, Incremental
Delivery Plan, Estimation,
RAID Log



Ch.32-36: Giao tiếp, workshop,
UAT, Sign-off, đo lường
KPI/OKR sau go-live

40.2 Tổng hợp bộ tài liệu FoodNow đã tạo ra xuyên suốt sách

Tài liệu	Chương giới thiệu	Vị trí file mẫu
BRD (Business Requirements Document)	Chương 19	<code>templates/brd-prd/FoodNow-BRD.md</code>
PRD (Product Requirements Document)	Chương 19	<code>templates/brd-prd/FoodNow-PRD.md</code>
SRS (Software Requirements Specification)	Chương 20	<code>templates/srs/FoodNow-SRS.md</code>
Use Case Specification	Chương 22	<code>templates/use-case/FoodNow-UC01-DatMonThanhToan.md</code>
Product Backlog (Epic + User Story)	Chương 23-24	<code>templates/user-story-epic/FoodNow-UserStories.md</code>
Incremental Delivery Plan	Chương 28-29	<code>templates/incremental-delivery/FoodNow-IncrementalDeliveryPlan.md</code>
RAID Log	Chương 31	<code>templates/raid-log/FoodNow-RAIDLog.md</code>
UAT Checklist	Chương 34	<code>templates/uat/FoodNow-UAT-Checklist.md</code>
Sign-off Document	Chương 35	<code>templates/sign-off/FoodNow-SignOff-Increment1.md</code>

Nhận xét quan trọng: đây **không phải các tài liệu rời rạc** — mỗi tài liệu đều truy vết được ngược lên tài liệu trước đó (BRD → PRD → SRS → User Story → Test Case → UAT → Sign-off), đúng tinh thần Requirements Traceability đã học ở Chương 16. Khi bạn làm dự án thực tế, hãy giữ nguyên tắc này: mọi tài liệu đều nên "kể tiếp câu chuyện" của tài liệu trước, không phải là các mảnh ghép tách biệt.

40.3 Những quyết định quan trọng đã đưa ra trong hành trình FoodNow — và bài học tương ứng

Quyết định	Chương	Bài học tổng quát
Chọn mô hình Water-Scrum-Fall thay vì Waterfall/Scrum thuần	11	Mô hình lai (Hybrid) phổ biến hơn "thuần" trong thực tế
Chỉ cho áp dụng 1 mã giảm giá/đơn (Simple Design)	9	Đơn giản hoá hợp lý khi chưa chắc chắn cần độ phức tạp cao hơn
Chia 3 Increment theo tính năng + phân khúc người dùng	28-29	Giao hàng từng phần giảm rủi ro so với giao hàng một lần
Sign-off có điều kiện cho Increment 1 (chấp nhận lỗi nhỏ, không chặn go-live)	35	Không phải mọi vấn đề đều cần chặn tiến độ — cần phân loại mức độ nghiêm trọng
Đặt KPI truy vết trực tiếp về Business Objectives trong BRD	36	Đo lường sau go-live phải liên kết với lý do dự án tồn tại từ đầu

40.4 Nếu bạn bắt đầu một dự án thực tế đầu tiên — checklist tổng hợp

Dựa trên toàn bộ hành trình FoodNow, đây là checklist tổng hợp khi bạn bắt đầu vai trò BA cho một dự án mới:

- ☐ Xác định rõ mô hình đội đang áp dụng (Chương 11) — quan sát cách làm thực tế, không chỉ tên gọi.
- ☐ Làm Stakeholder Analysis + RACI trước khi elicitation sâu (Chương 13).
- ☐ Dùng kỹ thuật "5 Whys" để tìm vấn đề gốc rễ đằng sau mọi yêu cầu ban đầu (Chương 12).
- ☐ Viết BRD/PRD/SRS (hoặc User Story + Acceptance Criteria nếu Agile) đạt tiêu chí Verification (Chương 17) và luôn Validate lại với đúng người có thẩm quyền.
- ☐ Xây RTM ở mức phù hợp quy mô dự án (Chương 16), duy trì cập nhật liên tục.
- ☐ Lập kế hoạch giao hàng Incremental nếu có thể (Chương 28-29), tránh đặt cược vào một lần ra mắt duy nhất.
- ☐ Duy trì RAID Log, rà soát định kỳ (Chương 31).
- ☐ Chuẩn bị và điều phối UAT nghiêm túc trước go-live, không làm hình thức (Chương 34).
- ☐ Đặt KPI/OKR truy vết về mục tiêu kinh doanh ban đầu, duy trì vòng lặp phản hồi sau go-live (Chương 36).
- ☐ Đầu tư domain knowledge của ngành bạn đang làm việc, song song với kỹ năng tổng quát (Chương 39).

40.5 Tự đánh giá lại (quay lại bảng Chương 4)

Quay lại bảng tự đánh giá ở Chương 4, mục 4.3, chấm điểm lại (1-5) cho từng kỹ năng — so sánh với lần chấm đầu tiên trước khi đọc sách. Ghi chú lại những kỹ năng còn yếu để tiếp tục thực hành thêm, đặc biệt qua các dự án cá nhân/thực tập/công việc đầu tiên.

Bài tập tổng hợp cuối sách

1. Viết một bản tóm tắt (dưới 300 từ) kể lại toàn bộ hành trình FoodNow theo góc nhìn của bạn — như thể bạn đang giải thích cho một người bạn chưa biết gì về BA, dựa trên những gì đã học.
 2. Chọn một sản phẩm/dự án thực tế bạn quan tâm (có thể là ý tưởng khởi nghiệp riêng, hoặc một sản phẩm bạn quen dùng), áp dụng checklist ở mục 40.4 để phác thảo bước đầu tiên bạn sẽ làm với tư cách BA cho dự án đó.
-

Tóm tắt & lời kết

Xuyên suốt 40 chương, dự án FoodNow đã đi từ một câu nói mơ hồ của ban giám đốc đến một bộ tài liệu đầy đủ, được kiểm thử, ký duyệt, ra mắt theo từng giai đoạn, và đo lường thành công dựa trên đúng mục tiêu kinh doanh ban đầu — đây chính là bản chất công việc của một Business Analyst: không phải người ghi chép yêu cầu, mà là người đảm bảo **cả một đội hiểu đúng, hiểu giống nhau, và cùng đi đúng hướng** xuyên suốt vòng đời một sản phẩm phần mềm. Phần Phụ lục (A-F) tiếp theo tổng hợp lại toàn bộ các mẫu tài liệu đầy đủ để bạn tiện tra cứu và tái sử dụng cho công việc thực tế.

Phụ lục

Phụ lục A: Mẫu BRD & PRD đầy đủ (dự án FoodNow)

Nội dung dưới đây in lại đầy đủ từ file mẫu gốc, dùng được ngay để sao chép/điều chỉnh cho dự án thật của bạn: `templates/brd-prd/FoodNow-BRD.md` và `templates/brd-prd/FoodNow-PRD.md`. Xem giải thích cấu trúc và cách viết từng phần tại Chương 19.

Business Requirements Document (BRD)

FoodNow — Ứng dụng đặt đồ ăn trực tuyến

Mã tài liệu	FN-BRD-001
Phiên bản	1.0
Ngày soạn thảo	2024-02-01
Người soạn thảo	Nguyễn Văn A — Business Analyst
Trạng thái	Đã phê duyệt

Lịch sử phiên bản

Phiên bản	Ngày	Người sửa	Mô tả thay đổi
0.1	2024-01-15	Nguyễn Văn A	Bản nháp đầu tiên
0.2	2024-01-25	Nguyễn Văn A	Cập nhật sau workshop với quản lý 12 chi nhánh
1.0	2024-02-01	Nguyễn Văn A	Bản chính thức, đã Ban giám đốc phê duyệt

1. Bối cảnh & vấn đề nghiệp vụ

Chuỗi nhà hàng FoodNow hiện có 12 chi nhánh tại TP.HCM, nhận đặt hàng qua điện thoại (tổng đài nội bộ) và các ứng dụng giao đồ ăn của bên thứ ba (chiếm khoảng 65% tổng số đơn hàng online). Việc phụ thuộc vào app bên thứ ba gây ra 3 vấn đề: chi phí hoa hồng cao (20-25% giá trị mỗi đơn hàng), không sở hữu dữ liệu khách hàng, và không kiểm soát được trải nghiệm thương hiệu.

2. Mục tiêu kinh doanh

ID	Mục tiêu	Chỉ số đo lường	Hạn hoàn thành
BO-01	Giảm phụ thuộc vào app bên thứ ba	Tỷ trọng đơn hàng qua app riêng đạt tối thiểu 40% tổng đơn online trong 6 tháng sau go-live	6 tháng sau go-live
BO-02	Tăng biên lợi nhuận trên mỗi đơn hàng	Biên lợi nhuận trung bình/đơn tăng tối thiểu 15% so với đơn qua app bên thứ ba	12 tháng sau go-live
BO-03	Xây dựng dữ liệu khách hàng riêng	Thu thập được thông tin và lịch sử mua hàng của tối thiểu 10.000 khách hàng	6 tháng sau go-live

3. Phạm vi

Trong phạm vi (In scope)

- Ứng dụng di động (iOS + Android) cho khách hàng đặt món, thanh toán, theo dõi đơn hàng.
- Hệ thống quản trị (web) cho nhân viên chi nhánh xác nhận và xử lý đơn hàng.
- Tích hợp cổng thanh toán online (thẻ, ví điện tử) và thanh toán khi nhận hàng (COD).
- Chương trình khách hàng thân thiết cơ bản (tích điểm, đổi ưu đãi).
- Thông báo trạng thái đơn hàng theo thời gian thực.

Ngoài phạm vi (Out of scope) — cho phiên bản đầu tiên

- Tính năng đặt bàn trước để ăn tại quán.
- Giao hàng liên tỉnh/ngoài TP.HCM.
- Đa ngôn ngữ (chỉ tiếng Việt ở phiên bản đầu).
- Tích hợp với hệ thống kế toán/ERP hiện có.
- Chương trình giới thiệu bạn bè (referral program).

4. Đối tượng sử dụng (Stakeholders)

Nhóm	Nhu cầu tổng quan
Khách hàng	Đặt món nhanh, theo dõi đơn hàng, tích điểm thành viên
Nhân viên chi nhánh	Nhận và xử lý đơn hàng hiệu quả, không thao tác thủ công qua điện thoại
Người giao hàng	Nhận thông tin đơn hàng rõ ràng, cập nhật trạng thái giao hàng dễ dàng
Ban giám đốc	Theo dõi số liệu kinh doanh tổng thể, biên lợi nhuận theo kênh bán hàng
Phòng Marketing	Quản lý chương trình khách hàng thân thiết, gửi khuyến mãi
Phòng Kế toán	Đối soát doanh thu, dữ liệu giao dịch chính xác

5. Yêu cầu nghiệp vụ chi tiết

ID	Yêu cầu	Nguồn
BR-01	Khách hàng phải đặt và hoàn tất thanh toán trong một luồng liên tục, không quá 2 phút với đơn hàng thông thường	Ban giám đốc, workshop khách hàng
BR-02	Nhân viên chi nhánh phải nhận được thông báo đơn hàng mới ngay lập tức, không cần thao tác kiểm tra thủ công	Quản lý chi nhánh
BR-03	Hệ thống phải ghi nhận và tích lũy điểm thưởng cho khách hàng theo giá trị đơn hàng	Phòng Marketing
BR-04	Ban giám đốc phải xem được báo cáo doanh thu, số đơn hàng theo từng chi nhánh, theo ngày/tuần/tháng	Ban giám đốc
BR-05	Hệ thống phải cho phép nhân viên chi nhánh từ chối đơn hàng (hết món, quá tải) kèm lý do rõ ràng gửi lại khách hàng	Quản lý chi nhánh

6. Giả định & ràng buộc

Giả định: toàn bộ 12 chi nhánh có kết nối Internet ổn định; khách hàng mục tiêu chủ yếu dùng smartphone (iOS/Android).

Ràng buộc: ngân sách phát triển giai đoạn 1 tối đa 1.5 tỷ VNĐ; ra mắt phiên bản đầu trong vòng 6 tháng; tuân thủ quy định bảo vệ dữ liệu cá nhân khách hàng hiện hành.

7. Rủi ro sơ bộ

Rủi ro	Mức độ	Biện pháp giảm thiểu sơ bộ
Khách hàng quen dùng app bên thứ ba, ngại chuyển sang app mới	Cao	Chương trình khuyến mãi ra mắt, ưu đãi độc quyền trên app riêng
Nhân viên chi nhánh khó thích nghi hệ thống mới	Trung bình	Đào tạo kỹ trước go-live
Trễ tiến độ do tích hợp cổng thanh toán phức tạp hơn dự kiến	Trung bình	Ưu tiên tích hợp cổng thanh toán ở sprint đầu để phát hiện rủi ro sớm

(Xem RAID Log đầy đủ tại Phụ lục còn lại và Chương 31.)

8. Tiêu chí thành công

- Đạt được BO-01, BO-02, BO-03 trong khung thời gian đề ra.
- Ứng dụng đạt điểm đánh giá trung bình tối thiểu 4.2/5 trên App Store/Google Play sau 3 tháng.
- Tỷ lệ đơn hàng lỗi/khiếu nại dưới 2% tổng số đơn.

9. Phê duyệt

Vai trò	Họ tên	Chữ ký/Xác nhận	Ngày
Sponsor (Giám đốc điều hành)	Trần Thị B	Đã phê duyệt	2024-02-01
Trưởng phòng Marketing	Lê Văn C	Đã phê duyệt	2024-02-01
Trưởng phòng Vận hành	Phạm Thị D	Đã phê duyệt	2024-02-01

Product Requirements Document (PRD)

FoodNow App — Phiên bản 1.0

Mã tài liệu	FN-PRD-001
Phiên bản	1.0
Ngày soạn thảo	2024-02-10
Người soạn thảo	Nguyễn Văn A — Business Analyst
Tài liệu gốc liên quan	FN-BRD-001

1. Tóm tắt sản phẩm

FoodNow App là ứng dụng di động cho phép khách hàng của chuỗi nhà hàng FoodNow đặt món, thanh toán, và theo dõi đơn hàng trực tiếp — không qua nền tảng giao đồ ăn bên thứ ba, đi kèm hệ thống quản trị web cho nhân viên chi nhánh xử lý đơn hàng.

2. Đối tượng người dùng & Persona

Persona 1 — "Chị Hương" (Khách hàng thân thiết): 32 tuổi, nhân viên văn phòng, đặt đồ ăn trưa 3-4 lần/tuần. Ưu tiên tốc độ đặt hàng, theo dõi đơn, tích điểm đổi ưu đãi.

Persona 2 — "Anh Tùng" (Nhân viên chi nhánh): 24 tuổi, thu ngân kiêm xử lý đơn hàng tại chi nhánh Quận 1. Ưu tiên nhận đơn rõ ràng, nhanh, không phải vừa nghe điện thoại vừa ghi chép tay.

3. Danh sách tính năng (theo Epic)

Epic A — Đặt món & Thanh toán: xem thực đơn, giỏ hàng, chọn giao nhận, thanh toán online/COD, áp dụng mã giảm giá.

Epic B — Theo dõi đơn hàng: trạng thái real-time, thông báo đẩy, lịch sử đơn hàng.

Epic C — Chương trình khách hàng thân thiết: tích điểm tự động, xem điểm/lịch sử, đổi điểm lấy ưu đãi.

Epic D — Quản trị đơn hàng: nhận thông báo đơn mới, xác nhận/từ chối, cập nhật trạng thái, báo cáo doanh thu.

4. Yêu cầu phi chức năng (tổng quan)

Nhóm	Yêu cầu tổng quan
Hiệu năng	Thao tác chính phản hồi dưới 2 giây
Bảo mật	Dữ liệu thanh toán/cá nhân được mã hoá, tuân thủ quy định bảo vệ dữ liệu
Khả năng mở rộng	Chịu tải gấp 3 lần hiện tại khi mở rộng chi nhánh

5. Ưu tiên hoá tính năng (MoSCoW)

Tính năng	Mức ưu tiên
Đặt món, thanh toán (Epic A)	Must have
Theo dõi đơn hàng real-time (Epic B)	Must have
Quản trị đơn hàng (Epic D)	Must have
Chương trình khách hàng thân thiết (Epic C)	Should have
Đặt bàn trước tại quán	Won't have (phiên bản 1.0)
Đa ngôn ngữ	Won't have (phiên bản 1.0)

6. Lộ trình phát hành sơ bộ

Giai đoạn	Nội dung	Thời gian dự kiến
Increment 1	Epic A — thử nghiệm 2 chi nhánh	Tháng 1-2
Increment 2	Epic B + D — mở rộng 12 chi nhánh	Tháng 3-4
Increment 3	Epic C — ra mắt công khai đầy đủ	Tháng 5-6

7. Chỉ số đo lường thành công của sản phẩm

- Tỷ lệ hoàn tất đặt hàng $\geq 60\%$.
- Thời gian trung bình hoàn tất đơn hàng < 2 phút.
- Tỷ lệ khách hàng quay lại đặt đơn thứ 2 trong vòng 30 ngày $\geq 35\%$.

Phụ lục B: Mẫu SRS đầy đủ theo IEEE 830 (dự án FoodNow)

Nội dung dưới đây in lại đầy đủ từ file mẫu gốc: `templates/srs/FoodNow-SRS.md`. Xem giải thích cấu trúc IEEE 830 và nguyên tắc viết yêu cầu chất lượng cao tại Chương 20.

Software Requirements Specification (SRS)

FoodNow App — Module: Đặt món & Thanh toán

Mã tài liệu	FN-SRS-001
Phiên bản	1.0
Ngày soạn thảo	2024-02-20
Người soạn thảo	Nguyễn Văn A — Business Analyst
Tài liệu tham chiếu	FN-BRD-001, FN-PRD-001

1. Giới thiệu

Mục đích: đặc tả chi tiết yêu cầu chức năng và phi chức năng cho module "Đặt món & Thanh toán" của FoodNow App, làm cơ sở cho đội phát triển hiện thực và đội kiểm thử xây dựng test case.

Phạm vi: bao gồm xem thực đơn, quản lý giỏ hàng, chọn phương thức giao nhận, thanh toán, áp dụng mã giảm giá. Không bao gồm theo dõi đơn hàng (FN-SRS-002) hay chương trình khách hàng thân thiết (FN-SRS-003).

Định nghĩa & thuật ngữ: Giỏ hàng (Cart) — danh sách món chưa thanh toán; COD — Cash On Delivery; Đơn hàng (Order) — giỏ hàng sau khi hoàn tất thanh toán.

2. Mô tả tổng quan

Module vận hành trên ứng dụng di động (iOS/Android). Khách hàng tương tác trực tiếp; hệ thống tương tác với cổng thanh toán bên thứ ba (VNPay) và hệ thống quản trị chi nhánh.

3. Yêu cầu chức năng

3.1 Xem thực đơn & Giỏ hàng

- **FR-CART-01:** Hệ thống PHẢI hiển thị danh sách món ăn theo danh mục (Khai vị, Món chính, Tráng miệng, Đồ uống), mỗi món hiển thị tên, ảnh, giá, mô tả ngắn.
- **FR-CART-02:** Hệ thống PHẢI cho phép khách hàng thêm một món vào giỏ hàng với số lượng tùy chọn (tối thiểu 1, tối đa 20 một loại món).
- **FR-CART-03:** Hệ thống PHẢI cho phép khách hàng sửa số lượng hoặc xóa một món khỏi giỏ hàng bất kỳ lúc nào trước khi thanh toán.
- **FR-CART-04:** Nếu một món trong giỏ hàng hết hàng tại chi nhánh được chọn, hệ thống PHẢI thông báo cho khách hàng và yêu cầu xóa hoặc thay thế món đó trước khi tiếp tục thanh toán.

3.2 Chọn phương thức giao nhận

- **FR-DELIVERY-01:** Hệ thống PHẢI cho phép khách hàng chọn giao hàng tận nơi hoặc lấy tại quán.
- **FR-DELIVERY-02:** Nếu chọn giao hàng tận nơi, hệ thống PHẢI tự động xác định chi nhánh gần nhất trong bán kính phục vụ tối đa 5km; nếu không có chi nhánh phù hợp, hệ thống PHẢI thông báo rõ và không cho phép tiếp tục đặt hàng.

3.3 Thanh toán

- **FR-PAY-01:** Hệ thống PHẢI cho phép thanh toán bằng thẻ nội địa/quốc tế hoặc ví điện tử (MoMo, ZaloPay) qua cổng thanh toán VNPay.
- **FR-PAY-02:** Hệ thống PHẢI cho phép thanh toán COD, chỉ áp dụng cho giao hàng tận nơi.
- **FR-PAY-03:** Nếu giao dịch online thất bại, hệ thống PHẢI hiển thị lỗi rõ ràng, giữ nguyên giỏ hàng, cho phép thử lại hoặc chọn phương thức khác.
- **FR-PAY-04:** Hệ thống PHẢI cho phép nhập một mã giảm giá hợp lệ, tối đa 1 mã/đơn hàng.
- **FR-PAY-05:** Sau khi thanh toán thành công (hoặc xác nhận COD), hệ thống PHẢI tạo đơn hàng và gửi thông báo đến hệ thống quản trị chi nhánh trong vòng dưới 5 giây.

4. Yêu cầu giao diện bên ngoài

- **EXT-01:** Tích hợp cổng thanh toán VNPay qua API REST.
- **EXT-02:** Gửi dữ liệu đơn hàng mới đến Hệ thống quản trị chi nhánh qua API nội bộ.

5. Yêu cầu phi chức năng

ID	Yêu cầu	Ghi chú
NFR-01	Màn hình thực đơn tải xong dưới 2 giây với kết nối 4G	Đo bằng công cụ kiểm thử hiệu năng chuẩn
NFR-02	Thông tin thẻ thanh toán không lưu trực tiếp trên hệ thống FoodNow, chỉ qua tokenization của cổng thanh toán	Yêu cầu bảo mật bắt buộc
NFR-03	Hệ thống chịu tải tối thiểu 500 đơn hàng đồng thời trong giờ cao điểm	Kiểm thử tải trước go-live

6. Phụ lục kỹ thuật

Sơ đồ Activity Diagram cho luồng đặt món & thanh toán: xem Chương 15, mục 15.3.

Phụ lục C: Mẫu Epic + User Story backlog đầy đủ (dự án FoodNow)

Nội dung dưới đây in lại đầy đủ từ file mẫu gốc: `templates/user-story-epic/FoodNow-UserStories.md`. Xem cách phân rã Epic thành User Story tại Chương 24, cách viết Acceptance Criteria tại Chương 23, và cách ưu tiên hoá tại Chương 26.

Epic A: Đặt món & Thanh toán

ID	User Story	Ưu tiên
US-101	Là khách hàng, tôi muốn xem thực đơn theo danh mục, để dễ dàng tìm món tôi muốn ăn	Must have
US-102	Là khách hàng, tôi muốn thêm/sửa/xoá món trong giỏ hàng, để tôi kiểm soát đúng đơn hàng trước khi thanh toán	Must have
US-103	Là khách hàng, tôi muốn chọn giao tận nơi hoặc lấy tại quán, để phù hợp với nhu cầu của tôi lúc đó	Must have
US-104	Là khách hàng, tôi muốn thanh toán bằng thẻ hoặc ví điện tử, để tôi không cần chuẩn bị tiền mặt	Must have
US-105	Là khách hàng, tôi muốn thanh toán khi nhận hàng (COD), để tôi yên tâm khi chưa quen với thanh toán online	Should have
US-106	Là khách hàng, tôi muốn áp dụng mã giảm giá khi thanh toán, để tôi tiết kiệm chi phí	Must have

Acceptance Criteria mẫu cho US-106 (xem đầy đủ tại Chương 23, mục 23.3):

Scenario 1: Áp dụng mã giảm giá hợp lệ

Given giỏ hàng có tổng giá trị 200.000đ

And mã giảm giá "FOODNOW10" đang có hiệu lực, giảm 10%

When khách hàng nhập mã "FOODNOW10" và bấm Áp dụng

Then tổng tiền thanh toán giảm còn 180.000đ

And hệ thống hiển thị thông báo "Đã áp dụng mã giảm giá FOODNOW10"

Scenario 2: Nhập mã giảm giá đã hết hạn

Given mã giảm giá "TET2024" đã hết hạn sử dụng

When khách hàng nhập mã "TET2024" và bấm Áp dụng

Then hệ thống hiển thị thông báo lỗi "Mã giảm giá đã hết hạn"

And tổng tiền thanh toán không thay đổi

Scenario 3: Nhập mã giảm giá thứ hai sau khi đã áp dụng một mã

Given khách hàng đã áp dụng thành công mã "FOODNOW10"

When khách hàng nhập thêm mã "SUMMER5" và bấm Áp dụng

Then hệ thống hiển thị thông báo "Chỉ được áp dụng 1 mã giảm giá cho mỗi đơn hàng"

And mã "FOODNOW10" vẫn được giữ nguyên hiệu lực

Epic B: Theo dõi đơn hàng

ID	User Story	Ưu tiên
US-201	Là khách hàng, tôi muốn xem trạng thái đơn hàng theo thời gian thực, để tôi biết khi nào đồ ăn sẽ đến	Must have
US-202	Là khách hàng, tôi muốn nhận thông báo đẩy khi trạng thái đơn hàng thay đổi, để tôi không cần liên tục mở app kiểm tra	Must have
US-203	Là khách hàng, tôi muốn xem lịch sử các đơn hàng đã đặt, để tôi có thể đặt lại nhanh chóng	Should have
US-204	Là khách hàng, tôi muốn huỷ đơn hàng trong vòng 2 phút sau khi đặt, để tôi có thể sửa sai nếu đặt nhầm	Should have

Epic C: Chương trình khách hàng thân thiết

ID	User Story	Ưu tiên
US-401	Là khách hàng, tôi muốn được tự động tích điểm sau mỗi đơn hàng thành công, để tôi tích lũy dần	Must have
US-402	Là khách hàng, tôi muốn xem số điểm hiện có, để biết mình còn bao nhiêu điểm	Must have
US-403	Là khách hàng, tôi muốn xem lịch sử tích/đổi điểm, để kiểm tra lại giao dịch	Should have
US-404	Là khách hàng, tôi muốn đổi điểm lấy mã giảm giá, để tiết kiệm chi phí đơn hàng tiếp theo	Must have
US-405	Là khách hàng, tôi muốn nhận thông báo khi điểm sắp hết hạn, để không bỏ lỡ cơ hội dùng	Could have
US-406	Là Ban giám đốc, tôi muốn xem báo cáo tổng điểm đã phát hành/đã đổi theo tháng, để đánh giá hiệu quả	Should have

Epic D: Quản trị đơn hàng (dành cho nhân viên chi nhánh)

ID	User Story	Ưu tiên
US-301	Là nhân viên chi nhánh, tôi muốn nhận thông báo ngay khi có đơn hàng mới, để tôi xử lý kịp thời	Must have
US-302	Là nhân viên chi nhánh, tôi muốn xác nhận hoặc từ chối đơn hàng kèm lý do, để khách hàng biết trạng thái đơn của họ	Must have
US-303	Là nhân viên chi nhánh, tôi muốn cập nhật trạng thái xử lý đơn hàng, để khách hàng theo dõi được	Must have
US-304	Là quản lý chi nhánh, tôi muốn xem báo cáo doanh thu và số đơn theo ngày/tuần/tháng, để theo dõi hiệu quả kinh doanh	Should have

Phụ lục D: Mẫu Incremental Delivery Plan đầy đủ (dự án FoodNow)

Nội dung dưới đây in lại đầy đủ từ file mẫu gốc: `templates/incremental-delivery/FoodNow-IncrementalDeliveryPlan.md`. Xem giải thích khái niệm Incremental/Iterative Delivery tại Chương 28 và cách trình bày kế hoạch tại Chương 29.

1. Tổng quan

Kế hoạch chia việc phát triển FoodNow App thành 3 increment, mỗi increment mang lại giá trị sử dụng được ngay, kết hợp chiến lược "theo tính năng" và "theo phân khúc người dùng" (mở rộng dần từ 2 chi nhánh thí điểm ra toàn bộ 12 chi nhánh).

2. Bảng chi tiết các Increment

Increment 1 — Nền tảng đặt món & thanh toán (thí điểm)

- Nội dung: Epic A (US-101 đến US-106).
- Đối tượng: khách hàng tại 2 chi nhánh thí điểm (Quận 1, Quận 3).
- Thời gian: Tháng 3-4.
- Tiêu chí chuyển Increment 2: conversion $\geq 50\%$; không có critical bug tồn đọng quá 48 giờ.
- Rủi ro chính: tích hợp cổng thanh toán VNPAY phức tạp hơn dự kiến.

Increment 2 — Theo dõi đơn hàng & Quản trị chi nhánh (mở rộng toàn bộ)

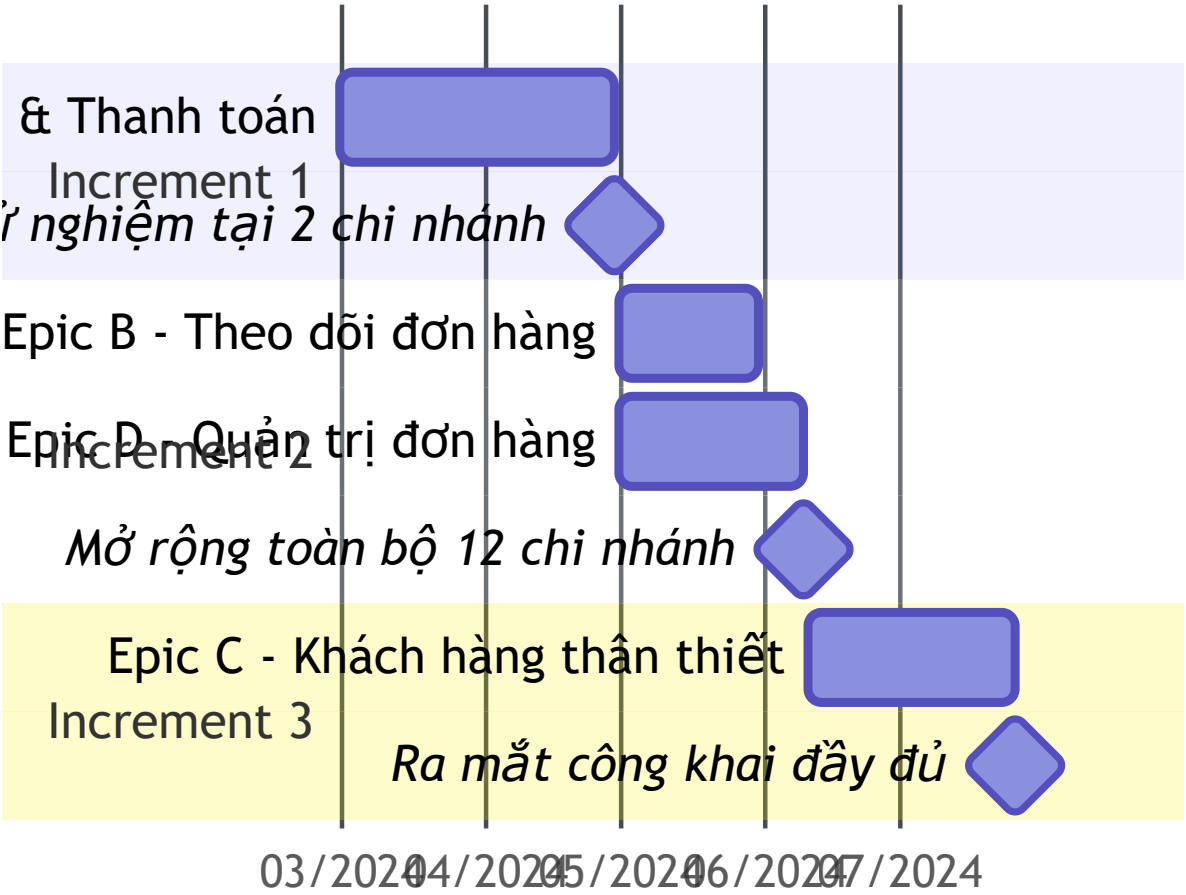
- Nội dung: Epic B (US-201 đến US-204), Epic D (US-301 đến US-304).
- Đối tượng: toàn bộ 12 chi nhánh.
- Thời gian: Tháng 5 đến giữa tháng 6.
- Tiêu chí chuyển Increment 3: 12/12 chi nhánh vận hành ổn định ≥ 2 tuần liên tục; khiếu nại $< 2\%$.
- Rủi ro chính: 10 chi nhánh còn lại cần thời gian đào tạo.

Increment 3 — Chương trình khách hàng thân thiết (ra mắt công khai đầy đủ)

- Nội dung: Epic C (US-401 đến US-406).
- Đối tượng: toàn bộ khách hàng, công khai trên App Store/Google Play.
- Thời gian: Giữa tháng 6 đến cuối tháng 7.
- Tiêu chí thành công: đạt các chỉ số BO-01, BO-02, BO-03 trong BRD.
- Rủi ro chính: khách hàng chưa quen thao tác tích/đổi điểm.

3. Timeline trực quan

Incremental Delivery Plan - FoodNow AI



4. Kế hoạch truyền thông/đào tạo đi kèm

Increment	Đối tượng đào tạo	Nội dung	Thời điểm
1	Nhân viên 2 chi nhánh thí điểm	Sử dụng hệ thống quản trị đơn hàng cơ bản	Tuần cuối trước go-live Increment 1
2	Nhân viên 10 chi nhánh còn lại	Sử dụng hệ thống quản trị đơn hàng đầy đủ	2 tuần trước go-live Increment 2
2	Toàn bộ khách hàng hiện tại	Giới thiệu app mới, hướng dẫn tải và đặt hàng lần đầu	Trùng go-live Increment 2
3	Toàn bộ khách hàng	Giới thiệu chương trình tích điểm, cách đổi ưu đãi	Trùng go-live Increment 3

5. Điều kiện điều chỉnh kế hoạch

Mọi thay đổi về nội dung/thứ tự increment phải qua quy trình Change Request (Chương 18), phê duyệt bởi Ban giám đốc (thay đổi phạm vi/ngân sách tổng thể) hoặc Product Owner (điều chỉnh thứ tự ưu tiên trong ngân sách đã duyệt).

Phụ lục E: Bảng thuật ngữ BA (Glossary) tra cứu nhanh

Tổng hợp toàn bộ thuật ngữ chính đã xuất hiện xuyên suốt sách, sắp xếp theo alphabet, kèm chương giải thích chi tiết để tra cứu nhanh.

Thuật ngữ	Giải thích ngắn gọn	Chương
Acceptance Criteria	Điều kiện cụ thể xác nhận một User Story đã hiện thực đúng	23
AML (Anti-Money Laundering)	Chống rửa tiền — quy định đặc thù ngành Banking	39
BABOK	Business Analysis Body of Knowledge — tài liệu nền tảng của IIBA	4
BLUF (Bottom Line Up Front)	Nguyên tắc trình bày: nêu kết luận trước, giải thích sau	32
BPMN	Business Process Model and Notation — chuẩn ký hiệu mô tả quy trình liên phòng ban	15
BRD	Business Requirements Document — tài liệu yêu cầu nghiệp vụ cấp cao	19
CBAP/CCBA/ECBA	Các chứng chỉ BA do IIBA cấp, theo cấp độ kinh nghiệm	4
Change Request (CR)	Đề xuất thay đổi yêu cầu, cần qua quy trình đánh giá và phê duyệt	18
Cost of Delay	Chi phí trì hoãn — thành phần dùng để tính WSJF	26
Definition of Done (DoD)	Điều kiện để một User Story được coi là hoàn thành thực sự	24
Definition of Ready (DoR)	Điều kiện để một User Story đủ điều kiện vào Sprint Planning	24
Elicitation	Quá trình chủ động thu thập/khơi gợi yêu cầu từ stakeholder	12
Epic	Yêu cầu lớn, cần nhiều sprint để hoàn thành, được phân rã thành User Story	24
ERD (Entity-Relationship Diagram)	Sơ đồ mô tả các bảng dữ liệu và mối quan hệ giữa chúng	38
FRD	Functional Requirements Document — tài liệu tập trung yêu cầu chức năng	21
Gherkin (Given-When-Then)	Cú pháp viết Acceptance Criteria/test case	23
INVEST	Bộ tiêu chí đánh giá chất lượng User Story	23
IEEE 830	Chuẩn cấu trúc cho tài liệu SRS	20
Kano Model	Mô hình phân loại tính năng theo tác động đến sự hài lòng khách hàng (Basic/Performance/Excitement)	26
KPI	Key Performance Indicator — chỉ số đo lường hiệu suất liên tục	36
MoSCoW	Kỹ thuật ưu tiên hoá: Must/Should/Could/Won't have	26
Non-functional Requirements	Yêu cầu về chất lượng: hiệu năng, bảo mật, độ tin cậy...	21
OKR	Objectives and Key Results — khung đặt mục tiêu theo chu kỳ	36
PRD	Product Requirements Document — tài liệu yêu cầu ở mức sản phẩm/tính năng	19

Thuật ngữ	Giải thích ngắn gọn	Chương
RACI	Ma trận phân định trách nhiệm: Responsible/Accountable/Consulted/Informed	13
RAID Log	Công cụ theo dõi Risk/Assumption/Issue/Dependency	31
RTM (Requirements Traceability Matrix)	Bảng theo dõi liên kết yêu cầu xuyên suốt vòng đời dự án	16
Scope Creep	Phạm vi dự án phình to không kiểm soát, không qua phê duyệt chính thức	18
Scrum	Framework Agile phổ biến, tổ chức công việc theo Sprint	8
Sign-off	Xác nhận chính thức của người có thẩm quyền cho một mốc/tài liệu	35
SRS	Software Requirements Specification — đặc tả chi tiết yêu cầu phần mềm	20
Stakeholder Analysis	Phân tích các bên liên quan theo mức độ quyền lực/quan tâm	13
Story Point	Đơn vị ước lượng độ phức tạp tương đối trong Agile	30
T-shirt Sizing	Kỹ thuật ước lượng nhanh, tổng quan (XS/S/M/L/XL)	30
UAT (User Acceptance Testing)	Vòng kiểm thử cuối cùng bởi người dùng thực tế trước go-live	34
Use Case Specification	Đặc tả chi tiết một luồng tương tác actor-hệ thống	22
User Story	Mô tả yêu cầu ngắn gọn theo góc nhìn người dùng (vai trò-nhu cầu-lợi ích)	23
Validation	Kiểm tra "có đúng vấn đề nghiệp vụ cần giải quyết không"	17
Verification	Kiểm tra "tài liệu có viết đúng cách, rõ ràng, đầy đủ không"	17
Velocity	Tốc độ trung bình (story point/sprint) của một đội Scrum	30
WIP Limit (Work In Progress)	Giới hạn số công việc đang làm cùng lúc trong Kanban	10
Wireframe	Bản phác thảo bố cục/luồng màn hình, chưa đi vào chi tiết thẩm mỹ	25
WSJF (Weighted Shortest Job First)	Kỹ thuật ưu tiên hoá định lượng: $\text{Cost of Delay} \div \text{Job Size}$	26
XP (Extreme Programming)	Framework Agile tập trung thực hành kỹ thuật (TDD, pair programming...)	9

Phụ lục F: Tài liệu tham khảo & nguồn học thêm

Danh sách nguồn tham khảo và hướng học thêm cho từng chủ đề chính trong sách. Đây là điểm bắt đầu gợi ý, không phải danh sách đầy đủ — khuyến khích người đọc tự tìm hiểu sâu hơn theo hướng công việc/ngành cụ thể của mình.

Nền tảng BA & BABOK

- **A Guide to the Business Analysis Body of Knowledge (BABOK Guide)** — IIBA. Tài liệu nền tảng chính thức, nguồn gốc của nhiều khái niệm trong sách này (elicitation, requirements classification, traceability, stakeholder analysis).
- Trang chủ IIBA (International Institute of Business Analysis) — thông tin về chứng chỉ ECBA/CCBA/CBAP, cộng đồng BA quốc tế.
- Trang chủ PMI (Project Management Institute) — thông tin về chứng chỉ PMI-PBA.

Agile, Scrum, XP, Kanban

- **Agile Manifesto** (agilemanifesto.org) — nguồn gốc chính thức của 4 giá trị và 12 nguyên tắc Agile đã trình bày ở Chương 7.
- **Scrum Guide** — tài liệu chính thức, ngắn gọn, định nghĩa đầy đủ vai trò/sự kiện/artifact của Scrum (Chương 8), cập nhật định kỳ bởi những người đồng sáng lập Scrum.
- **Extreme Programming Explained** (Kent Beck) — nguồn gốc các thực hành kỹ thuật XP (Chương 9).
- Tài liệu về Kanban Method (David J. Anderson) — nguyên lý luồng liên tục và giới hạn WIP (Chương 10).

Viết yêu cầu & tài liệu

- **IEEE 830** — chuẩn cấu trúc SRS (Chương 20). Bản chuẩn chính thức có phí, nhưng nhiều tài liệu tóm tắt cấu trúc miễn phí có sẵn trực tuyến.
- Sách/tài liệu về **User Story Mapping** (Jeff Patton) — kỹ thuật trực quan hoá backlog mở rộng thêm cho kỹ thuật phân rã Epic/User Story đã học ở Chương 24.
- Tài liệu về cú pháp **Gherkin** và Behavior-Driven Development (BDD) — mở rộng thêm cho Acceptance Criteria đã học ở Chương 23.

Quản lý dự án & giao hàng

- Tài liệu về **SAFe (Scaled Agile Framework)** và **LeSS (Large-Scale Scrum)** — các framework mở rộng Agile cho tổ chức lớn, nhắc đến ở Chương 11 và Chương 26 (WSJF).
- Tài liệu về **PMBOK Guide** (PMI) — góc nhìn quản lý dự án truyền thống, hữu ích để hiểu công việc của PM mà BA thường phối hợp cùng (Chương 3).

Công cụ

- Tài liệu hướng dẫn chính thức của **Jira** và **Confluence** (Atlassian) — Chương 37 chỉ giới thiệu quy trình làm việc, tài liệu chính thức có hướng dẫn chi tiết từng thao tác.
- Tài liệu học SQL cơ bản (nhiều khoá học trực tuyến miễn phí) — mở rộng thêm cho Chương 38.

Domain knowledge theo ngành

- Với Banking: tìm hiểu quy định của Ngân hàng Nhà nước Việt Nam về giao dịch điện tử, chống rửa tiền (AML), định danh khách hàng điện tử (eKYC).
- Với E-commerce: tìm hiểu Luật Bảo vệ quyền lợi người tiêu dùng, quy định về thương mại điện tử hiện hành tại Việt Nam.
- Với Insurance: tìm hiểu Luật Kinh doanh bảo hiểm hiện hành, các khái niệm cơ bản về underwriting và claims processing từ tài liệu đào tạo của các công ty bảo hiểm.

Lưu ý khi tự học thêm

Ngành phần mềm và phương pháp luận quản lý dự án thay đổi liên tục — các framework mới (hoặc biến thể của framework cũ) xuất hiện thường xuyên. Nguyên tắc chọn lọc tài liệu tự học: ưu tiên **hiểu bản chất vấn đề** (vì sao một kỹ thuật ra đời, giải quyết vấn đề gì) thay vì học thuộc lòng quy trình theo từng bước — đây cũng là tinh thần xuyên suốt cách trình bày của cuốn sách này.